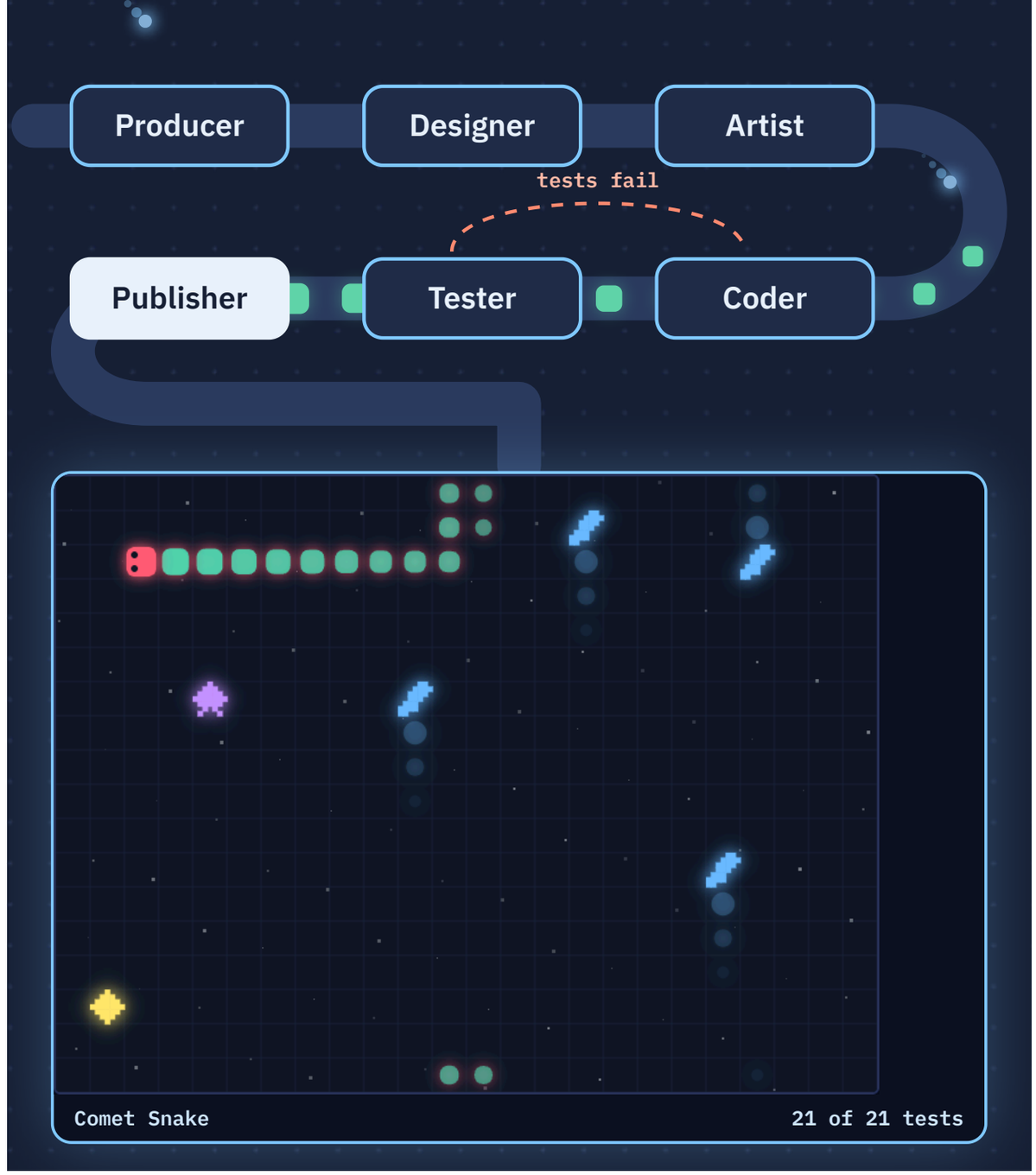
langgraph

# Multi Agents with LangGraph

Patterns, Communication and Deep Agents




SpiralTrain
developer courses

SpiralTrain
developer courses

# General Course Remarks

- Course times
- Drinks
- Meals
- Toilettes
- Smoking
- Mobile Phones
- Manual
- Software

*Everything should be made as simple as possible, but not simpler*

**Albert Einstein**

*Ja, was ich hier geschrieben habe, macht im Einzelnen überhaupt nicht den Anspruch auf Neuheit; und darum gebe ich auch keine Quellen an, weil es mir gleichgültig ist, ob das was ich gedacht habe, vor mir schon ein anderer gedacht hat.*

**Ludwig Wittgenstein**

# From LLM to Agent


**From LLM to Agent**

Introduction to LangGraph

Graph Fundamentals

State Management

Building Agents

Multi-Agent Patterns

Agent Communication

Advanced Workflows

Deep Agents

Production Deployment

Real-World Applications

Comet Arcade

What is an LLM?

Tokens

The Next Token

Sampling and Temperature

A Model Call

No Memory Between Calls

What is Context?

The Context Window

What Context Costs

System Prompt

What is Tool Calling?

Tool Definitions

What is an Agent?

The Agent Loop

Tokens Across a Run

How an Agent Stops

Agents and Workflows

What is a Harness?

Solo Agent and Full Harness

Context Engineering

Compaction

Verification

Objective Metric and Optimizer

Four Ways a Loop Breaks

The Outer Loop

Why Several Agents?

A Crew of Thirty Agents

Ten Thousand Agents on Navier-Stokes

How the Swarm Was Organised

Status of the Proof

A Compiler from Sixteen Agents

A Browser from Hundreds of Agents

A Game Maker from Three Agents

A Research Team of Agents

Recursive Language Models

The Bill for a Swarm

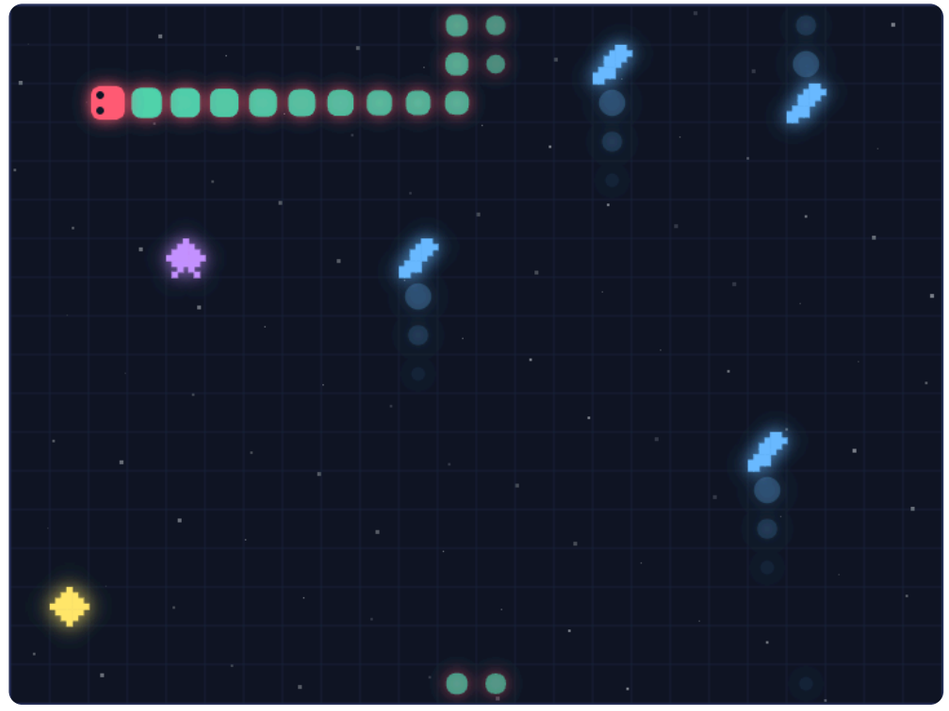
Patterns Behind the Examples

Summary : From LLM to Agent

SpiralTrain
developer courses

# Comet Arcade

- Our example is a made-up game studio
- AI programs do the work : they design, draw, code and test
- A person approves each release
- Their game is Comet Snake : Snake in space, where you eat stars and dodge comets



SpiralTrain developer courses

# What is an LLM?

- **A large language model is a program trained on a huge amount of text :**
  - Given text, it predicts what comes next, one piece at a time
  - A reply is a prediction and can be wrong even when it sounds confident
- **ChatGPT is an application built around such a model :**
  - The application draws the chat window and keeps the conversation
  - The model turns text into more text
- **The course reaches the model `gpt-6-luna` through OpenRouter :**
  - OpenRouter is a service that forwards requests to the companies that run models, called providers

SpiralTrain developer courses

# Tokens

- **A token is the piece of text a model reads and writes :**
  - A common word is often one token, a long or rare word several
  - As a rule of thumb, about 4 characters of English make one token
- **Prices and size limits are counted in tokens**

| text | tokens | pieces |
|---|---|---|
| `the` | 1 | `the` |
| `star` | 1 | `star` |
| `CS-SPR-03` | 5 | `CS` , `-SP` , `R` , `-` , `03` |
| `Comet` | 2 | `Com` , `et` |
| `kometenstaart` | 4 | `kom` , `eten` , `sta` , `art` |

**demo01 · tokens**

SpiralTrain developer courses

# Text Becomes Tokens

| Is | sprite | CS | -SP | R | - | 03 | of |
|---|---|---|---|---|---|---|---|
| 3031 | 36767 | 15903 | 115754 | 49 | 12 | 3659 | 328 |

| Com | et | Snake | final | , | and | does | it |
|---|---|---|---|---|---|---|---|
| 1390 | 292 | 86554 | 1721 | 11 | 326 | 2226 | 480 |

| pass | the | contrast | rule | ? |
|---|---|---|---|---|
| 1853 | 290 | 20663 | 10652 | 30 |

| 77 | 14 | 21 |
|---|---|---|
| CHARACTERS | WORDS | TOKENS |

Rare text costs more tokens : the sprite id is 5 tokens and the word Comet 2. Prices and size limits are counted in tokens.

demo01 · tokens

SpiralTrain developer courses

# The Next Token

- **A model writes one token at a time :**
  - It gives every token it knows a probability
  - One token is drawn from those probabilities
  - The model adds that token to the text and repeats the step
- **A special end token or a length limit stops the writing**
- **The same prompt can give a different reply each time :**
  - Five tries with one prompt gave three different replies

demo02 · next-token

# One Token at a Time

PROMPT

Complete in a few words : The snake stopped because the comet

REPLY SO FAR

**The snake stopped because the comet passed.**

FIVE MORE CALLS, SAME PROMPT

The snake stopped because the comet passed overhead.

blocked its path.

The snake stopped because the comet passed.

Five more tries with one prompt gave three different replies. Code around a model has to expect that.

SpiralTrain developer courses

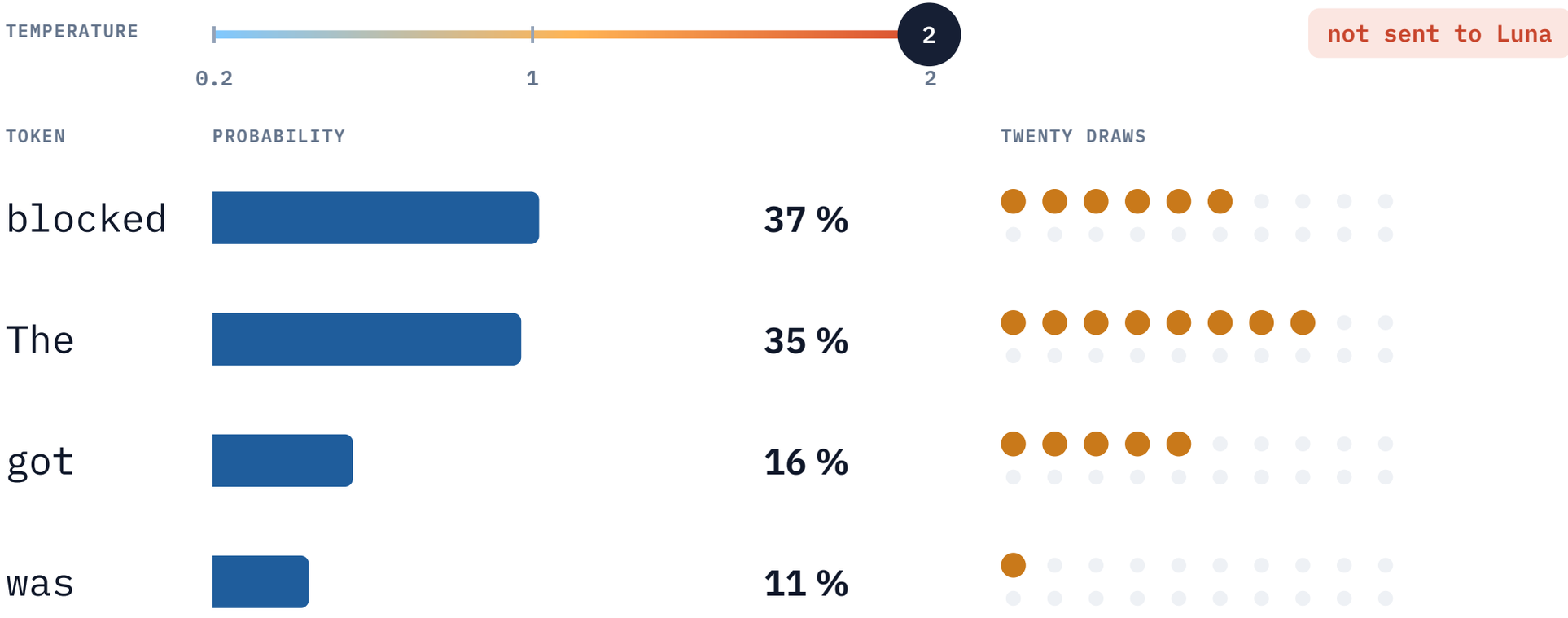
# Sampling and Temperature

- **Sampling draws the next token from the probabilities**
- **Temperature changes the odds before the draw :**
  - A low temperature favours the likeliest token
  - A high temperature gives rarer tokens a chance
- **`gpt-6-luna` ignores `temperature` through OpenRouter ; code computes this table from the first token of demo02**

```
def with_temperature(probs, temperature):
    scaled = {t: math.log(p) / temperature for t, p in probs.items()}
    total = sum(math.exp(s) for s in scaled.values())
    return {t: round(100 * math.exp(s) / total) for t, s in scaled.items()}
```

| temperature | `blocked` | `The` | `got` | `was` |
|---|---|---|---|---|
| 0.2 | 59 % | 41 % | 0 % | 0 % |
| 1 | 45 % | 42 % | 9 % | 4 % |
| 2 | 37 % | 35 % | 16 % | 11 % |

# Temperature and the Odds




Code on the laptop computed this table, because gpt-6-luna ignores temperature through OpenRouter. Luna's replies still vary per call.

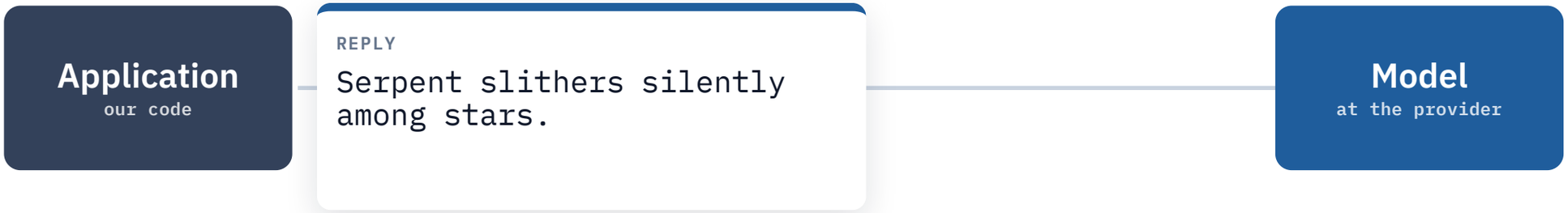
# A Model Call

- **A model call is one request and one reply :**
  - The application sends text and settings
  - The provider runs the model and returns text and a token count
- **`make_llm` is the course helper that connects to the model ; `invoke` makes the call**

```
llm = make_llm(max_tokens=40)
reply = llm.invoke("In five words, describe snake in space.")
print(reply.content)
print(reply.usage_metadata)
```

```
Serpent slithers silently among stars.
{'input_tokens': 15, 'output_tokens': 13, 'total_tokens': 28, ...}
```

# One Request and One Reply




**15** INPUT TOKENS

**13** OUTPUT TOKENS

**28** TOTAL TOKENS

**$0.000008** AT LIST PRICE

The reply returns with a token count : 15 tokens read and 13 written, $0.000008 at list price.

SpiralTrain developer courses

# No Memory Between Calls

- **Each call is independent :**
  - The model keeps nothing after it replies
- **A chat application keeps the conversation and resends it with every call :**
  - The application keeps the history of a chat
  - Each call is larger than the last, because it includes all the earlier messages

| messages sent | input tokens | reply |
|---|---|---|
| 1 | 22 | noted |
| 3 | 46 | noted |
| 5 | 78 | Your asset ID is CS-SPR-03, and it is too dark. |

demo03 · no-memory

# Resending the Conversation

APPLICATION · ONE REQUEST

My sprite is CS-SPR-03. Reply with one word : noted.

noted

It is too dark. Reply with one word : noted.

noted

Which asset id did I give you, and what is wrong with it? Reply in one sentence.



**Model**

keeps nothing between calls

Your asset ID is CS-SPR-03, and it is too dark.

INPUT TOKENS **78**

Call 3 sends five messages : 78 input tokens. The application keeps the history.

demo03 · no-memory

SpiralTrain developer courses

# What is Context?

- **Context is everything the model can see on this call :**
  - The model sees nothing else
  - The model can use a file, a database or an earlier call only after the application puts it in the context

| in the context | Comet Arcade example |
|---|---|
| Instructions | `You are the studio assistant of Comet Arcade.` |
| Tool descriptions, the actions on offer | `game_info` , `assets_for_game` |
| Conversation so far | `Which asset is at fault?` |
| Tool results, what those actions returned | `{"asset_id": "CS-SPR-03", "colour": "#3a4a7a"}` |

# The Context Window

- **The context window is the limit on how much fits in one call :**
  - Instructions, tools, history, tool results and the reply share it
- **`gpt-6-luna` has a window of 1,050,000 tokens, and one reply can use up to 128,000 of them**
- **Quality can drop well before the limit :**
  - Answers get less accurate as the context grows ; Anthropic calls this context rot

# What Fills the Window

CONTEXT WINDOW OF GPT-6-LUNA

1,050,000 tokens

demo05 run, 0.1 % of the window



- **19** system prompt
- **195** tool definitions
- **48** the task
- **253** model replies and tool calls
- **523** tool results

Tool results are the largest part : 523 of 1,038 tokens, counted locally. Long tool results fill a window first.

SpiralTrain developer courses

# What Context Costs

- **A provider bills every token of every call :**
  - `gpt-6-luna` costs $0.10 per million input tokens and $0.50 per million output tokens
  - Input means all the context, sent again on each call

| what | tokens | at list price |
|---|---|---|
| One question answered in 4 calls, demo05 | 1,900 in, 167 out | $0.000273 |
| One crew run that builds Comet Snake | about 2,860 in, 1,620 out | about $0.001 |
| One crew run with no cap on its replies, 5 calls | 71,934 in, 68,338 out | $0.041 |
| One call with a full window | 1,050,000 in | $0.105 |

SpiralTrain developer courses

# System Prompt

- **The system prompt contains standing instructions from the application :**
  - It goes ahead of the conversation on every call
  - The user does not type it
- **One model plays every crew member ; the system prompts differ :**

| crew member | opening of the system prompt |
|---|---|
| Designer | `Turn the brief into rules. Pick the one twist from the menu ...` |
| Artist | `You pick the colours and draw sprites for a space game.` |
| Coder | `You write the pure game logic of a one-file canvas game.` |
| Tester | `File one short bug per failed test ... Do not write the fix.` |

- **A brief is the request for a game ; a twist is one extra rule from a menu of 4**

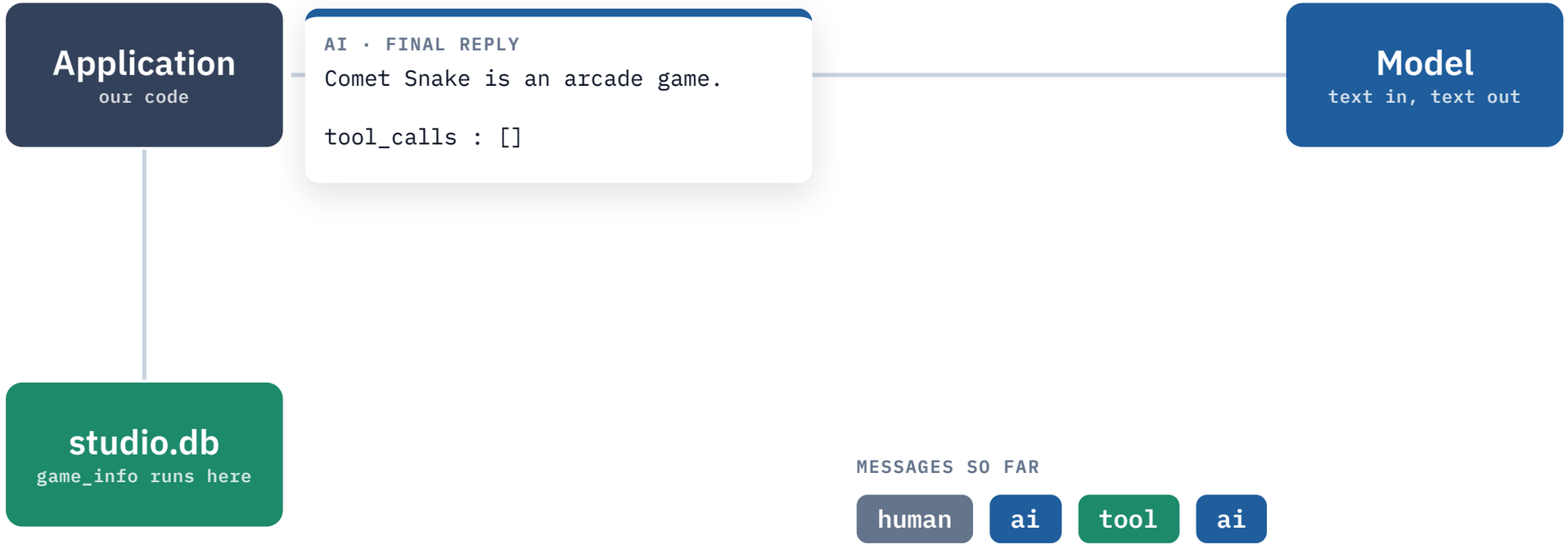
SpiralTrain developer courses

# What is Tool Calling?

- **A model on its own can only write text**
- **Tool calling lets it ask the application to act :**
  - The model writes a request in a fixed format
  - The application runs the request
  - The result goes back to the model in a new call
- **The model never runs anything itself**
- **A tool is a piece of code the application offers, such as one that reads a database or runs the game's tests**

demo04 · tool-call-by-hand

# A Tool Call Round Trip




The reply has no tool call, which ends the exchange. Two model calls produced one answer.

demo04 · tool-call-by-hand

SpiralTrain developer courses

# Tool Definitions

- **A tool definition tells the model what a tool does and what it takes :**
  - The model picks a tool by its name and description
  - The model never sees the body of the function

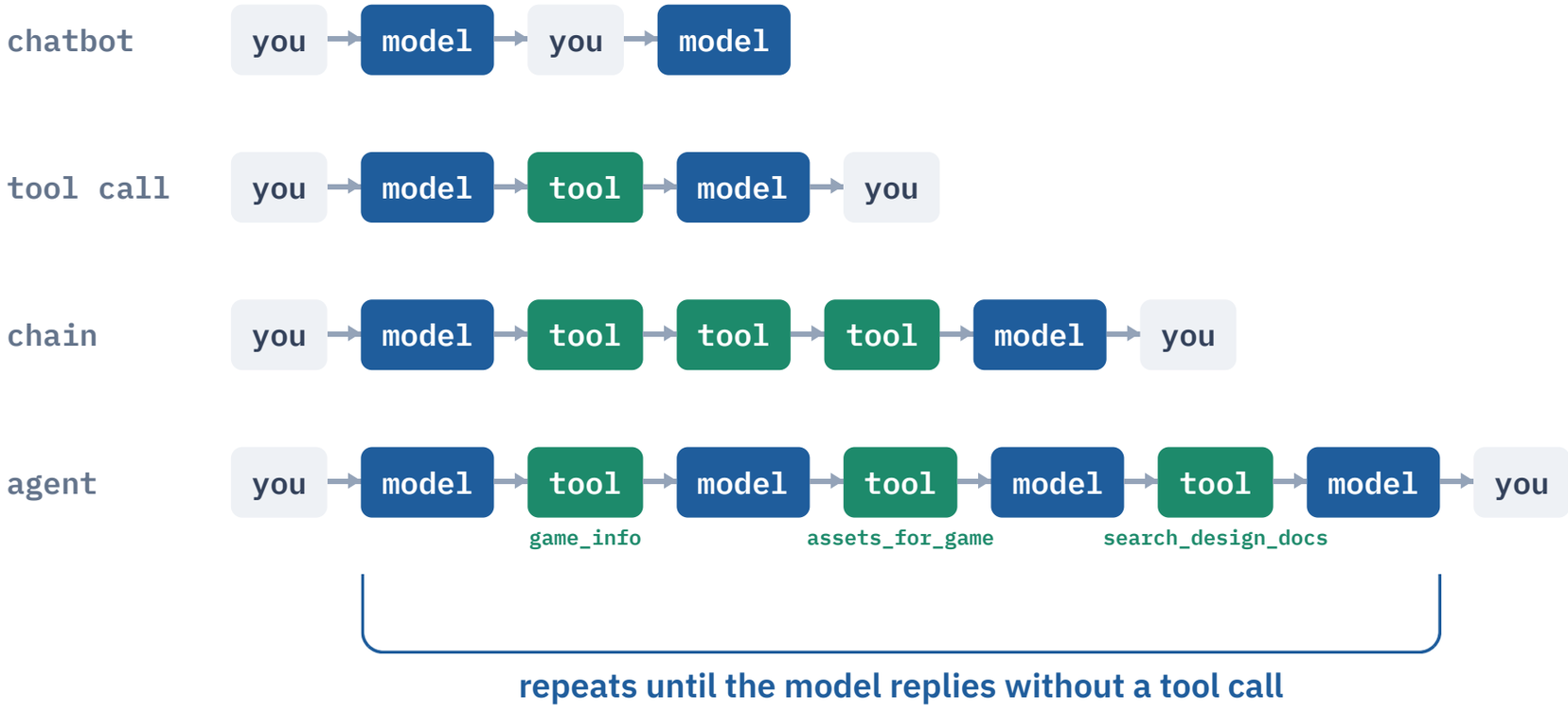
```
@tool
def game_info(title: str) -> str:
    """Look up a Comet Arcade game by its title or id. Returns id, title, genre, status and
pitch."""
```

- **`@tool` comes from LangChain, a Python library for model applications, and builds the definition from the function :**

```
{"name": "game_info",
 "description": "Look up a Comet Arcade game by its title or id. Returns id, title, ...",
 "parameters": {"properties": {"title": {"type": "string"}}, "required": ["title"]}}
```

SpiralTrain developer courses

# From Chatbot to Agent




The loop has no fixed length. The model reacting to each result makes this an agent, however few or many the tool calls.

SpiralTrain developer courses

# What is an Agent?

- **An agent is a model calling tools in a loop until the task is done :**
  - The model chooses the next step from what the last step returned
- **A question for the studio assistant :**
  - `Playtesters lose Comet Snake to comets they cannot see. Which asset is at fault?`
  - The agent looks up the game, then its assets, then the style guide
  - It names the comet sprite `CS-SPR-03` and the fix, the colour `#ff6b5e`

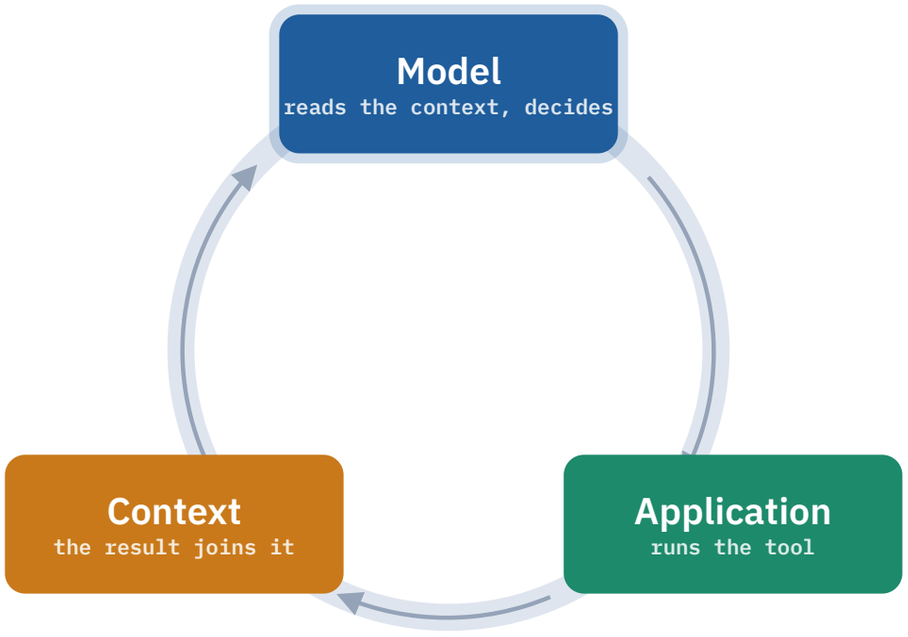
SpiralTrain developer courses

# The Agent Loop

- **Every pass through the loop has three steps :**
  - The model reads the context and chooses what to do
  - The application runs the tool the model asked for
  - The application adds the result to the context
- **The loop ends when the model replies without a tool call, or when a cap, a fixed limit, stops it**
- **The loop is a few lines of code, a Python `while` loop that repeats until told to stop :**
  - The model supplies the decisions
  - The code around it runs the tools and resends the messages

demo05 · agent-loop-by-hand

# One Lap at a Time




TASK

Playtesters lose Comet Snake to comets they cannot see. Which asset is at fault, and what should the Artist change?

PASSES

1. `game_info`
   Comet Snake is CS-01, an arcade game
2. `assets_for_game`
   7 assets, among them comet CS-SPR-03
3. `search_design_docs`
   Contrast rule : at least 4.5 to 1
4. `no tool call`
   answer, and the loop ends

The model replies without a tool call, and the loop ends : CS-SPR-03 is at fault, change it to #ff6b5e.

# The Loop as a `while` Statement

```python
messages = [system, task]
while True:
    reply = model.invoke(messages)
    messages.append(reply)
    if not reply.tool_calls:
        break
    for call in reply.tool_calls:
        result = by_name[call["name"]].invoke(call["args"])
        messages.append(ToolMessage(result, tool_call_id=call["id"]))
print(reply.content)
```

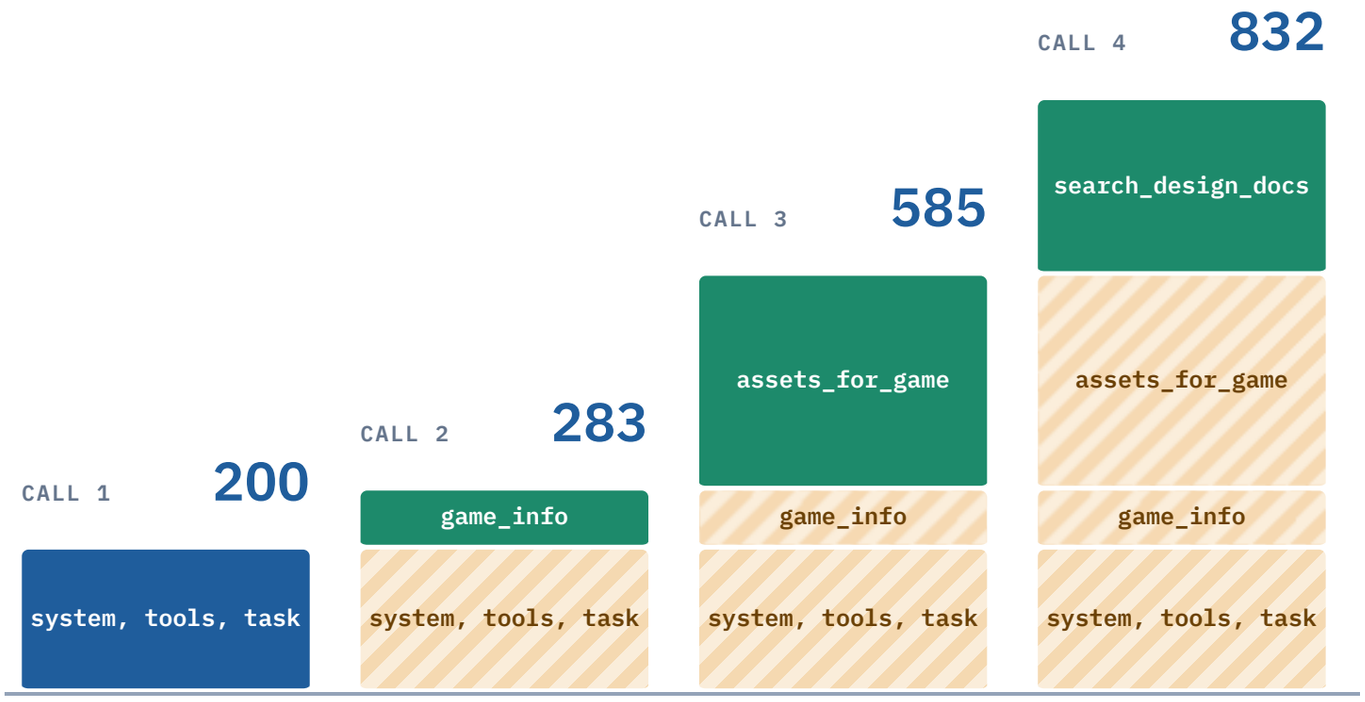
MESSAGES

`call 4 reads 832 tokens`

| | |
|---|---|
| **system** | You are the studio assistant of … |
| **human** | Playtesters lose Comet Snake to … |
| **ai** | call game_info |
| **tool** | {"game_id": "CS-01", … |
| **ai** | call assets_for_game |
| **tool** | [{"asset_id": "CS-SPR-01", … |
| **ai** | call search_design_docs |
| **tool** | # Comet Arcade style guide … |
| **ai** | The comet sprite (CS-SPR-03) is at fault … |

The list now has nine messages. Four model calls produced one answer.

# Resending the Context Each Call




BILLED INPUT SO FAR

**1,900**

| | |
|---|---|
| call 1 | + 200 |
| call 2 | + 283 |
| call 3 | + 585 |
| call 4 | + 832 |

new on this call
sent again

The last call read about 830 tokens, and the run billed about 1,900. The striped blocks were paid for more than once.

demo05 · agent-loop-by-hand

SpiralTrain developer courses

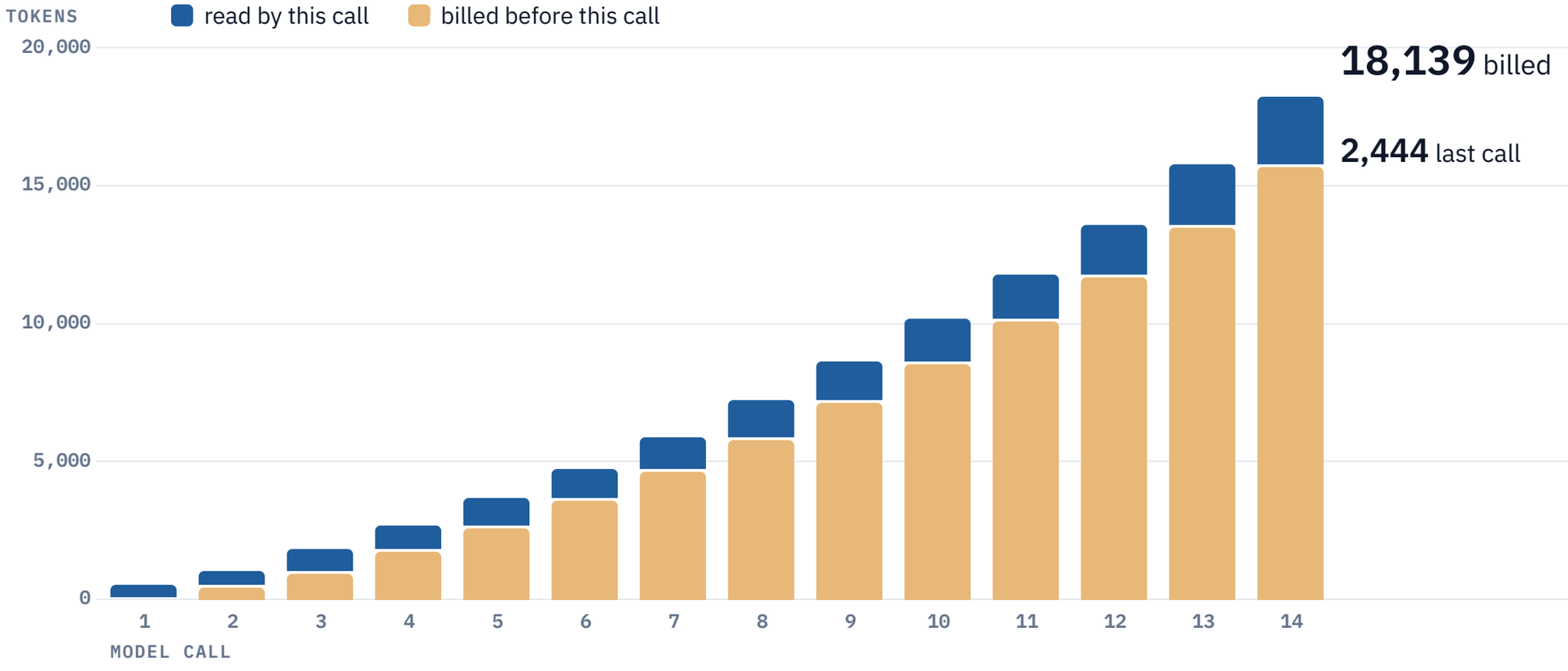
# Tokens Across a Run

- **Two numbers describe a run :**
  - Context is what one call reads
  - Billed input is the sum over all the calls
- **Billed input grows much faster than context, because each call is billed for the whole context again**

| run | calls | context of the last call | billed input |
|---|---|---|---|
| demo05, one question | 4 | about 830 | about 1,900 |
| `Can Comet Snake ship?` | 14 | 2,444 | 18,139 |

- **The longer run billed 7 times its last context**

# Tokens Over Fourteen Calls




The last call read 2,444 tokens, and the run billed 18,139 : 7 times the last context. Calls cost more as a run grows.

SpiralTrain developer courses

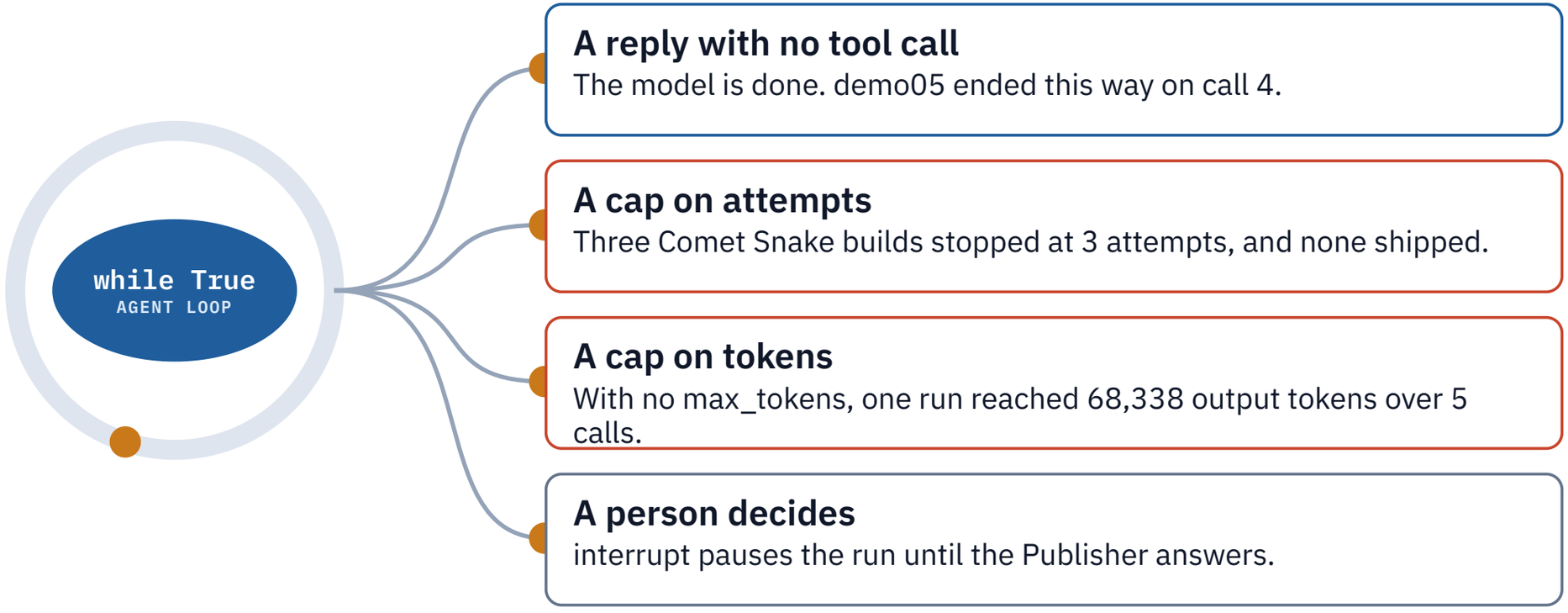
# How an Agent Stops

- **A model reply without a tool call ends the loop**
- **A `while` loop has no other exit ; the code around it adds caps :**
  - A cap on turns, the passes through the loop, stops a run that never finishes
  - A cap on tokens stops one long reply
- **A person can pause a run and decide**
- **A build that stops at the cap with failing tests does not ship**

```
llm = make_llm(max_tokens=8000)
for turn in range(10):
    reply = model.invoke(messages)
```

SpiralTrain developer courses

# Four Ways Out of the Loop




A person can pause a run. The run stays saved until the answer comes, and it goes on from there.

SpiralTrain developer courses

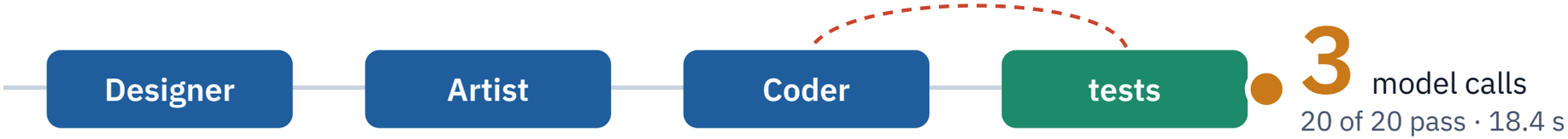
# Agents and Workflows

- **A workflow follows a path fixed in code :**
  - After the Designer comes the Artist, after the Artist the Coder
  - The model fills in each step
- **An agent chooses its own path :**
  - The Producer decides who works next, from the last result
- **Both finish a game that passes its tests on the course model :**

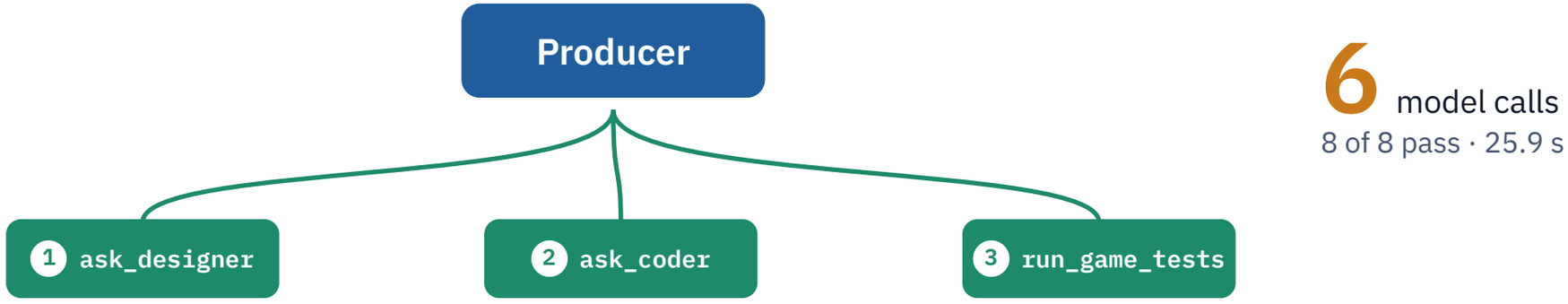
| pattern | runs passed | model calls | seconds, median |
|---|---|---|---|
| Workflow, fixed path | 20 of 20 | 3 | 18.4 |
| Agent, Producer with the crew as tools | 8 of 8 | 6 | 25.9 |

# Fixed Path and Chosen Path

WORKFLOW · PATH FIXED IN CODE




AGENT · PATH CHOSEN BY THE MODEL




Both built a passing game. With a known path, the agent spent twice the calls on the same result.

SpiralTrain developer courses

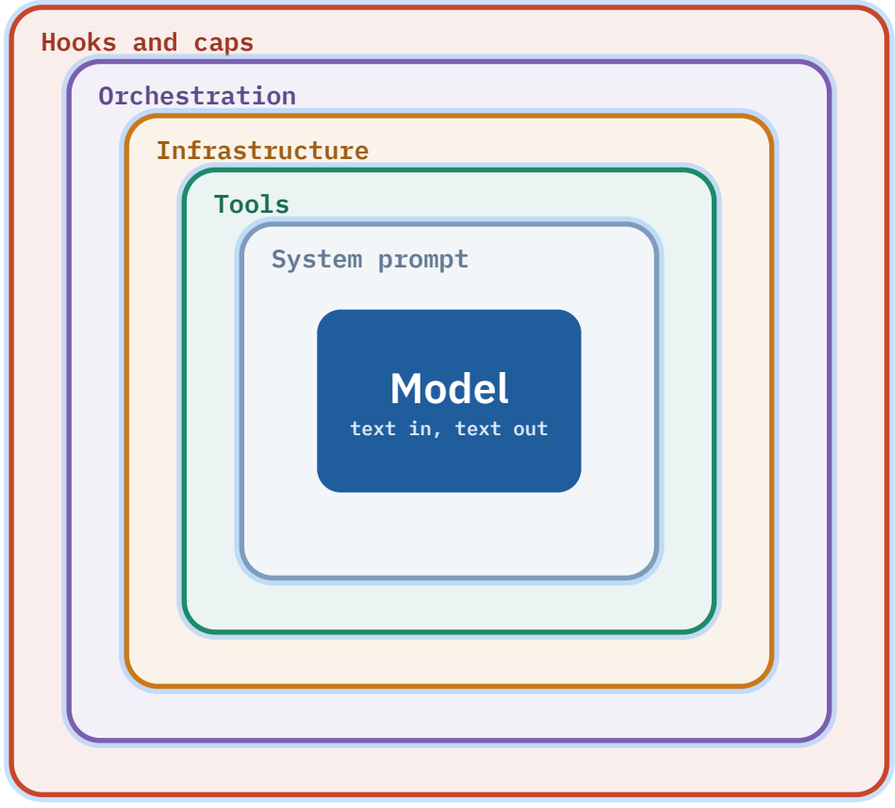
# What is a Harness?

- **Agent = Model + Harness**
- **A harness is**

```
every piece of code, configuration, and execution logic that isn't the model itself
```

- **The model decides ; the harness acts :**
  - It runs the tools and keeps the files
  - It trims the context
  - It runs the tests and stops the loop
- **Three harnesses around a model : ChatGPT, the coding tool Claude Code, and Deep Agents from LangChain**

# Harness Layers Around the Model




**System prompt**
`You are the Tester at Comet Arcade.`

**Tools**
`game_info, run_tests, play_bot`

**Infrastructure**
`files, QuickJS to run the game, a browser`

**Orchestration**
`the crew graph, handoffs, a model per job`

**Hooks and caps**
`cap on attempts, max_tokens, compaction`

**Agent = Model + Harness**

Agent = Model + Harness. Everything outside the blue box is the harness, which takes most of the engineering.

SpiralTrain developer courses

# Solo Agent and Full Harness

- **Anthropic gave one model the same task twice : build a retro game maker**
- **The harness split the work over a planner, a generator and an evaluator :**
  - The evaluator clicked through the running app

| | solo agent | full harness |
|---|---|---|
| time | 20 minutes | 6 hours |
| cost | $9 | $200 |
| result | `the actual game was broken` | `I was actually able to move my entity and play the game` |

- **The harness cost over 20 times as much, and only its game played**

SpiralTrain developer courses

# Context Engineering

- Context engineering decides what goes into the window at each step and what stays out
- Each crew member reads only what its job needs :

| crew member | reads | input tokens |
|---|---|---|
| Artist | the game title and the extra object | 404 |
| Designer | the brief and the twist menu | 574 |
| Tester | the test report and the code | 1,426 |
| Coder | the rules and the logic block of the kit | 1,702 |

- Tools return short answers, because large tool results fill a window fastest

SpiralTrain developer courses

# Compaction

- **A long run fills the window with old tool results**
- **Compaction replaces the older messages with a summary :**
  - Writing the summary takes one more model call
  - The run goes on from the summary
- **Details can be lost :**
  - Facts that must survive go into a file or the crew’s shared project board
- **The harness runs compaction ; the model only writes the summary**

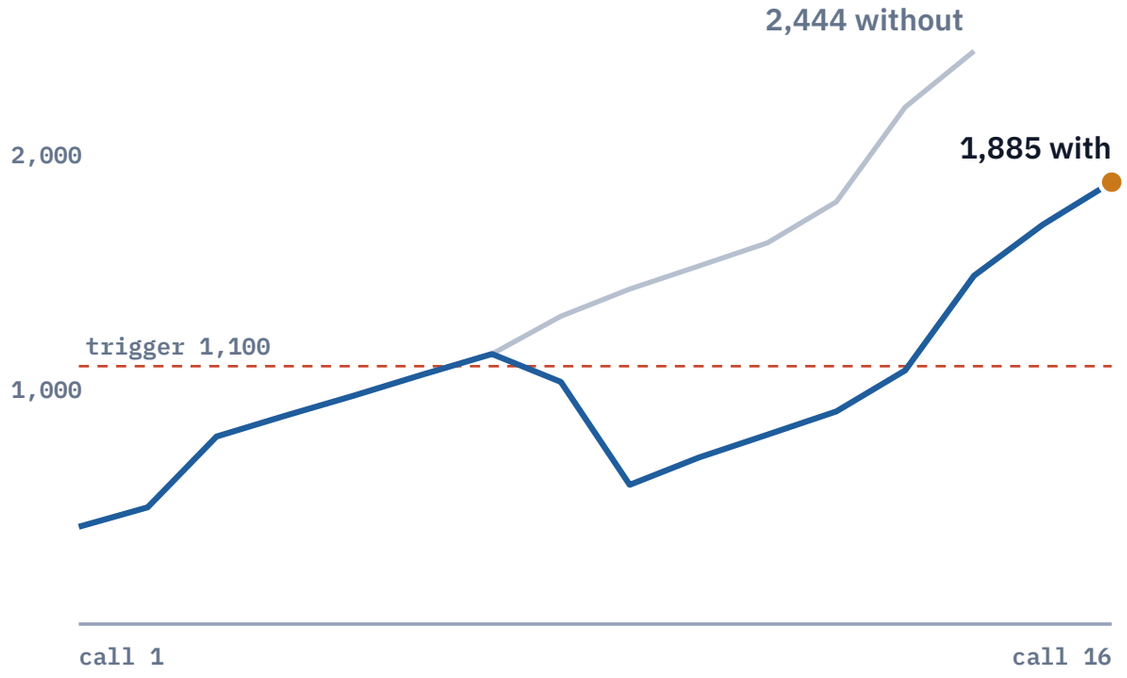
# Compaction in a Run

WHAT THE NEXT CALL READS

system, tools, task

**summary** ids of done work, facts, what is left

bug_details
bug_details
bug_details
builds_for_game
playtests_for_game
search_design_docs
search_design_docs




Without compaction the same task ended at 2,444 tokens and billed 18,139. With compaction : 1,885 and 16,003.

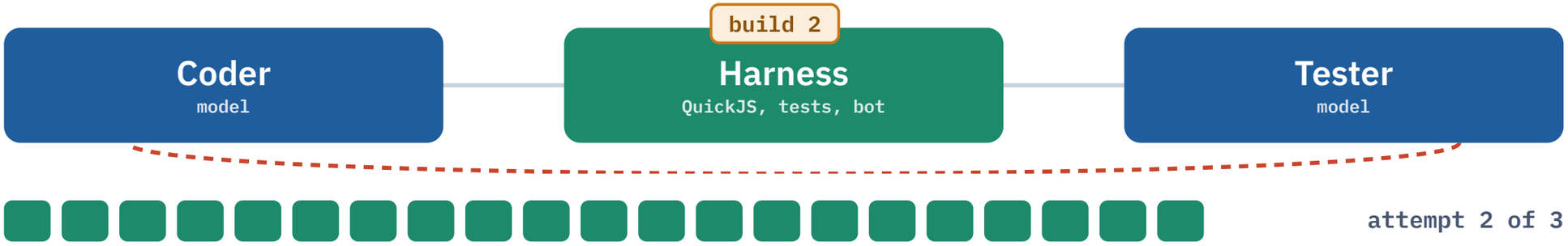
SpiralTrain developer courses

# Verification

- **A model stops when the work looks done**
- **Verification lets the agent find out whether it is done :**
  - The agent runs something that says pass or fail
  - It reads the result and tries again
  - A checker other than the writer decides
- **An agent can work unattended for as long as something else can tell it what is wrong**
- **At Comet Arcade the verdict comes from real tests and a bot :**
  - The model only writes the bug report

SpiralTrain developer courses

# A Verification Loop




**21 of 21** rule tests pass

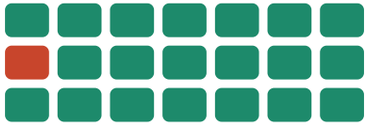
**66.0** stars, bot average

The harness runs the new build. 21 of 21 rule tests pass, and the loop exits with a playable game.

SpiralTrain developer courses

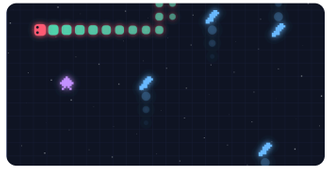
# Giving the Loop Senses

### Run it
QuickJS runs the game logic with no browser



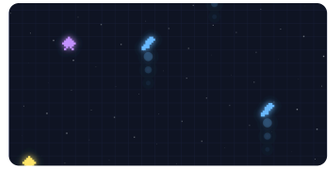
20 of 21 rule tests pass

### Play it
A bot plays five games



bot : 66.0 stars on average

### Look at it
Chromium opens the file and takes a screenshot



score 13, no page errors

### Count it
The provider reports tokens and cost



6,390 input tokens, $0.002067

**Agent**
CONTEXT

- the code it wrote
- 20 of 21 rule tests pass
- bot : 66.0 stars on average
- score 13, no page errors
- 6,390 input tokens, $0.002067

Tokens and cost per game are a sense too. Each check a person does by hand is a sense the loop still lacks.

SpiralTrain developer courses

# Objective Metric and Optimizer

- **A loop that improves work has three parts :**

| part | what it is | at Comet Arcade |
|---|---|---|
| Objective | the goal | `eat stars, dodge comets, every fifth star speeds up` |
| Metric | a number that stands in for the goal | rule tests and the bot’s score |
| Optimizer | what takes the next step | the Coder reads the bugs and changes the code |

- **The metric only approximates the goal and can be gamed :**
  - A bot could score well by exploiting a bug
- **Writing down each failure lets the next attempt read it**

SpiralTrain developer courses

# Four Ways a Loop Breaks

| failure | what happens | at Comet Arcade |
|---|---|---|
| Blind | the loop cannot see the result | tests passed on a blank page |
| Wrong proxy | the metric drifts from the goal | a bot that scores by exploiting a bug |
| Stuck | no attempt gets closer | the Coder chased a wrong diagnosis for 3 attempts |
| Model limit | the model cannot take the step | narrow the task, or use a stronger model |

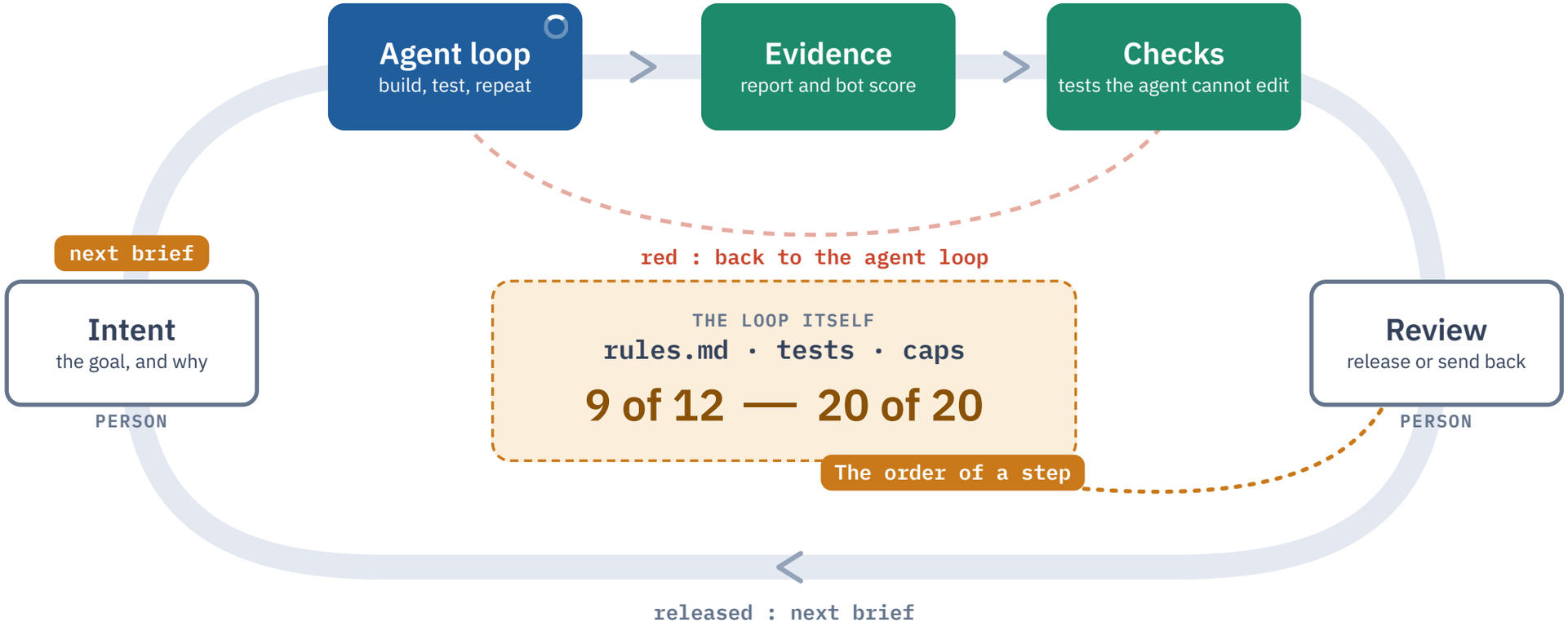
- **Improving the loop comes before trying another prompt**

SpiralTrain developer courses

# The Outer Loop

- **The agent loop sits inside a larger loop with a person at both ends :**
  - Intent : a person states the goal and why
  - Agent loop : the agent works and checks
  - Evidence : the agent passes on its results and the proof
  - Checks : automatic tests that the agent cannot change
  - Review : a person accepts the work or sends it back
- **A failure in the outer loop calls for a better loop :**
  - Better checks, a better rule or a better tool

# Outer Loop at Comet Arcade




After the same bug came back in later runs, the rules got a section on the order of a step. First-try passes went from 9 of 12 to 20 of 20.

SpiralTrain developer courses

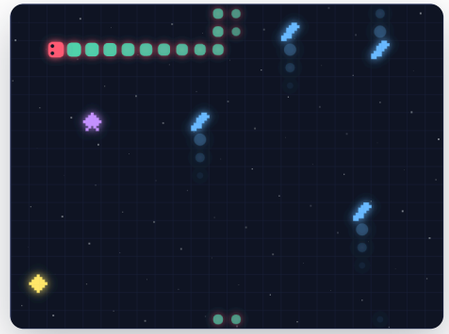
# Why Several Agents?

- **One agent has everything in one window, and the context grows with every job**
- **Several agents split the work :**
  - Each reads only the context its job needs
  - The writer and the checker stay separate
  - Work can run side by side
- **A second agent costs calls and coordination :**
  - Start with one agent, and add another when a measurement says so
- **The Comet Arcade crew is the course’s running example of this split**

# One Window or a Crew

ONE WINDOW

rule tester | bot tester — bugs



21 of 21 · bot 66.0 stars

writer and checker

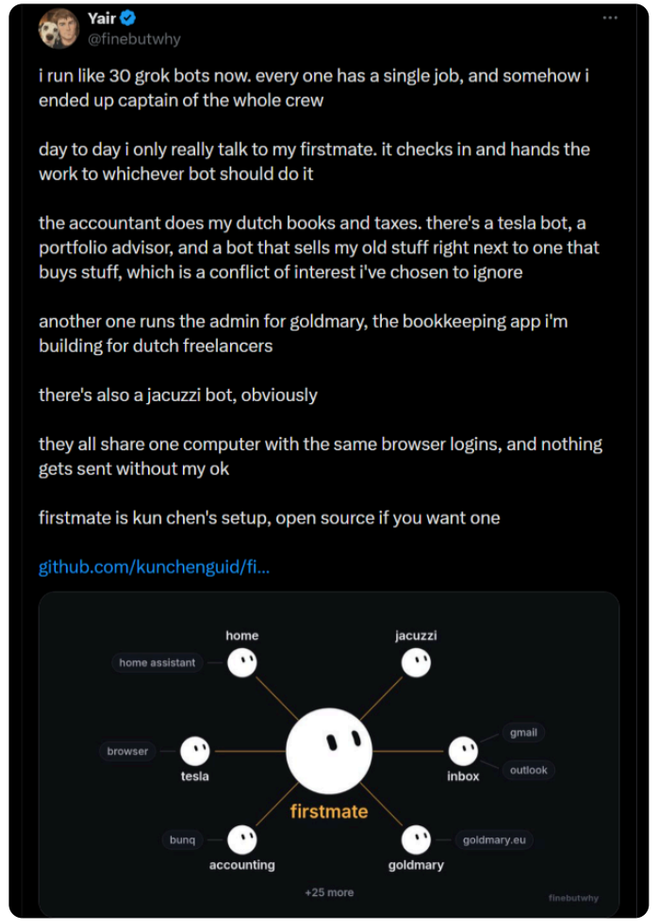
| Designer | Artist | Coder | Tester |
|---|---|---|---|
| brief and twist menu | title and extra object | rules and logic block | test report and code |
| 574 | 404 | 1,702 | 1,426 |

The crew finishes a real, playable game, at the cost of more calls and more coordination. A second agent has to be worth that cost.

SpiralTrain developer courses

# A Crew of Thirty Agents

- **A personal setup, October 2026, built on `firstmate`, an open-source project :**
  - About 30 agents running on Grok, an AI model, each with a single job
  - One agent, the first mate, takes every request and passes it to the right agent
- **The jobs include :**
  - Dutch bookkeeping and taxes against a bunq account
  - An inbox over Gmail and Outlook, Home Assistant, a Tesla
  - The admin of goldmary.eu, the owner's bookkeeping app
  - A seller and a buyer, side by side
- **The agents share one computer and one set of browser logins**
- **"nothing gets sent without my ok"**

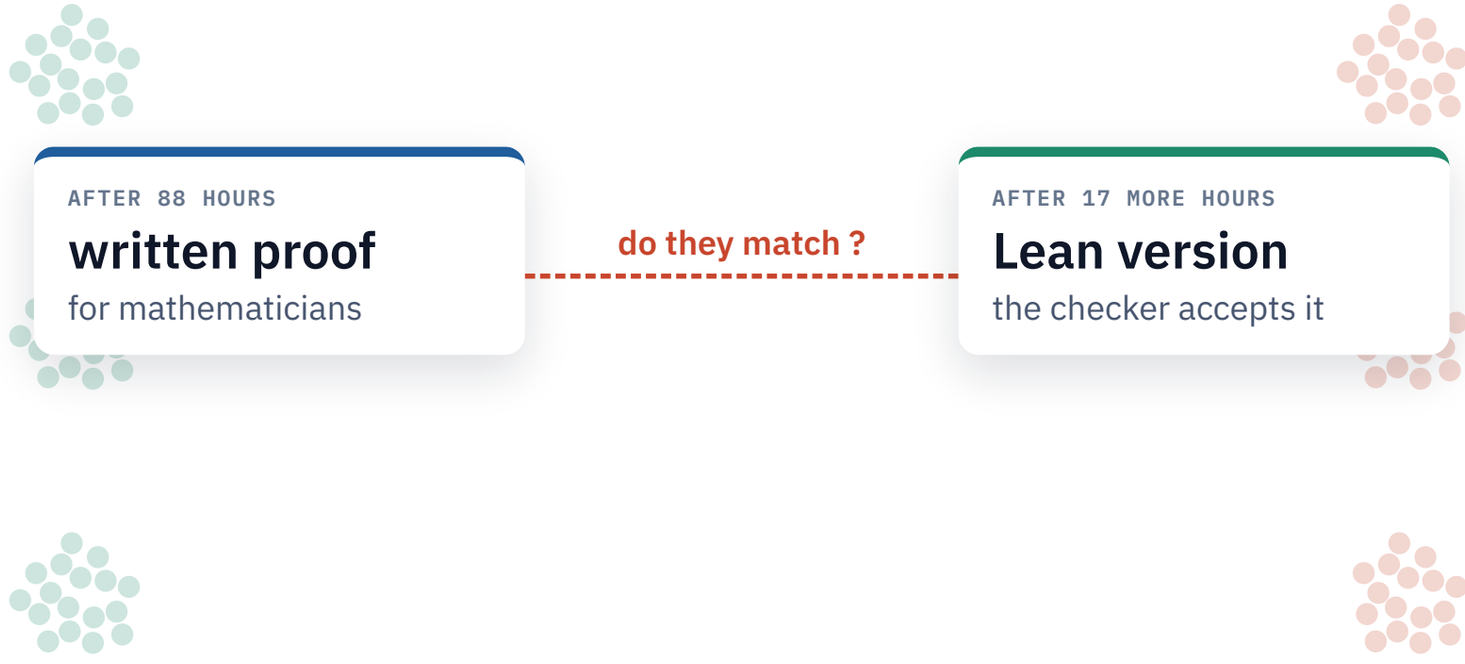


SpiralTrain developer courses

# Ten Thousand Agents on Navier-Stokes

- **OpenAI proposed a solution to a Millennium Prize Problem, a famous open problem in mathematics, on 8 September 2026 :**
  - The Navier-Stokes equations describe how fluids move
  - The claim : a fluid at rest, pushed by a smooth force, reaches a point in finite time where the equations break down
- **Found by a system of coordinating agents on an unreleased internal model :**
  - On the order of 10,000 concurrent agents in the group that found it
  - 88 hours to the proof, then 17 hours to rewrite it in Lean, a language in which a program checks every step
  - 2.7 million messages and about 130 billion output tokens
- **Not confirmed and not refuted on 30 October 2026**
- **OpenAI does not claim the prize**

# The Swarm in Groups




Lean is a language a program can check. 17 hours turned the proof into Lean, and a preprint of 6 October says the two do not match.

# How the Swarm Was Organised

- **Agents worked in groups and messaged only within their own group**
- **Separate groups received different variants of the problem :**
  - Some aimed at a proof of smoothness, others at a blowup
- **Codex merged the most useful findings of each group into new prompts**
- **Agents moved to Navier-Stokes after about 100 agents resolved a related Euler problem**
- **The model was replaced by a further-trained version during the run**
- **Lean checked the final proof :**
  - The proof checker accepts no step that does not follow
  - Lean checks the formal statement, and a preprint of 6 October says it differs from the written proof

SpiralTrain developer courses

# Status of the Proof

| date | what happened |
|---|---|
| 8 September | OpenAI publishes the proof and a Lean formalisation |
| 11 September | Clay Mathematics Institute : the review is "deliberately unhurried" |
| 6 October | A preprint argues that the Lean proof does not match the written proof |
| 27 October | King’s College London : this "does not yet establish that OpenAI’s original proof is incorrect" |
| 30 October | Clay lists Navier-Stokes under Active ; only Poincaré is Solved |

- **Lean is a language in which a program checks every step of a proof**
- **The checker tests only the Lean text it receives ; the translation from the paper needs its own check**

SpiralTrain developer courses

# A Compiler from Sixteen Agents

- **Anthropic, February 2026 : 16 agents wrote a C compiler in Rust :**
  - A compiler turns source code into a program a computer can run
  - About 100,000 lines that build Linux 6.9 on x86, ARM and RISC-V
  - Nearly 2,000 sessions over two weeks, just under $20,000
- **Each agent has its own container, a sealed work area, and its own copy of one shared repository**
- **An agent claims a task by writing a lock file ; git, the version control tool, rejects a second claim**
- **Tests steer the crew : "the task verifier is nearly perfect, otherwise Claude will solve the wrong problem"**
- **Limits remain : GCC, an existing compiler, still does the 16-bit step of booting Linux, and the demo used its assembler and linker**

SpiralTrain developer courses

# A Browser from Hundreds of Agents

- **Cursor, January 2026 : "Hundreds of workers run concurrently" on one branch of the code :**
  - "close to a week" of work
  - "over 1 million lines of code across 1,000 files"
- **Three roles keep them apart :**
  - Planners explore the code and create tasks
  - Workers finish one task each
  - A judge decides whether another cycle runs
- **A setup with no roles failed first : "Twenty agents would slow down to the effective throughput of two or three"**
- **The project’s README says "Not recommended for production use"**

SpiralTrain developer courses

# A Game Maker from Three Agents

- **Anthropic, March 2026 : one prompt for a retro game maker, run by one agent and by a harness of three**
- **The three agents :**
  - The planner turns the short prompt into a full product spec
  - The generator builds the app in short rounds, called sprints
  - The evaluator tests the running app with Playwright, a tool that drives a browser
- **The evaluator’s brief : "click through the running application the way a user would"**
- **One failed check fails the sprint, and detailed feedback goes back to the generator**

# One Prompt in Two Runs

THE PROMPT "Create a 2D retro game maker with features including a level editor ..."

SOLO

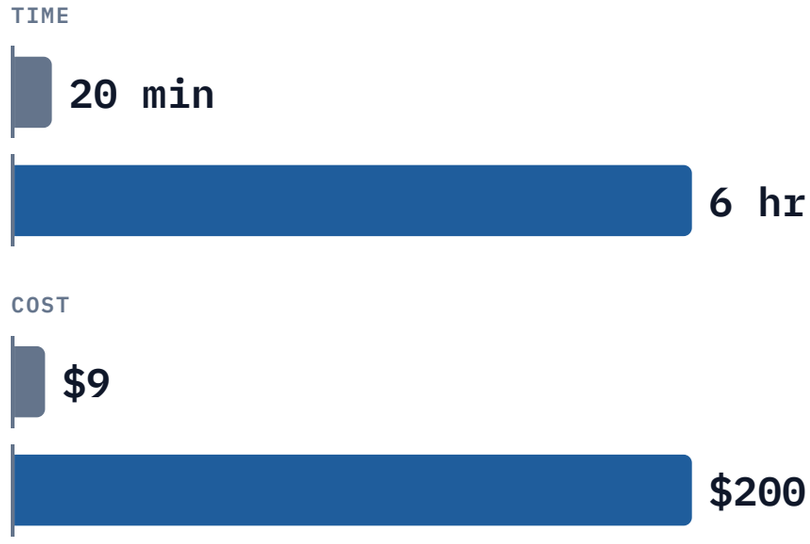
one agent

"the actual game was broken"

FULL HARNESS

planner → generator → evaluator

"I was actually able to move my entity and play the game"




The harness took 6 hours and $200, over 20 times the cost, and the game plays. The author reports rough physics and a working core.

SpiralTrain developer courses

# A Research Team of Agents

- **Anthropic, June 2025 : a lead agent starts helper agents, called subagents, in parallel :**
  - 90.2 % better than a single agent on the Opus 4 model, on Anthropic’s internal evaluation
- **The cost, from the same report :**
  - Agents use about 4 times the tokens of a chat ; multi-agent systems about 15 times
  - Token use alone explained 80 % of the differences in performance
- **A poor fit, from the same report :**
  - Tasks where every agent needs the same context
  - Most coding tasks, which have fewer parallel parts than research

SpiralTrain developer courses

# Recursive Language Models

- **A model that treats a long input as an environment to explore in code :**
  - The input is in a Python REPL instead of the prompt
  - The model writes code that splits it and calls fresh copies of itself on the parts
  - Only the sub-calls use tools ; the root model sees their results
- **Zhang, Kraska and Khattab (MIT, December 2025) report inputs about two orders of magnitude beyond the context window**
- **Prime Intellect called it the paradigm of 2026**
- **Alex Zhang describes it as a harness whose only tool is code**
- **Deep Agents implements the idea as dynamic subagents**

SpiralTrain developer courses

# The Bill for a Swarm

- **Every extra agent reads context and writes messages**
- **The bill for a large group of agents, a swarm, runs from a tenth of a cent to millions of dollars :**
  - One Comet Snake build : 3 model calls, about $0.001
  - The C compiler : just under $20,000
- **Measured in this course on the two-part question of module 05, with `gpt-6-luna` :**
  - A single agent answered in 2 model calls ; each multi-agent pattern needed 4 to 6
  - A Deep Agent read 28 times the input tokens of the single agent
- **Alex Zhang, an MIT researcher, suspects most of a large swarm's work is wasted**
- **Cognition, maker of the Devin coding agent, reports that several reviewers on one change work, and several writers on one change still fail (April 2026)**
- **The LangChain docs say a single agent with the right tools often does as well**

SpiralTrain developer courses

# Patterns Behind the Examples

| example | what happens | module |
|---|---|---|
| a first mate and its crew | one agent passes work to the others | 05 |
| groups that message only internally | teams with their own channel | 05, 06 |
| proof and disproof groups | two sides work on one problem | 05 |
| Codex merging each group’s findings | one agent combines parallel results | 05, 06 |
| planners, workers and a judge | a boss, a pool and a stop decision | 05 |
| a seller and a buyer for one owner | two agents with opposite goals | 06 |
| "nothing gets sent without my ok" | a person approves the action | 07 |
| Lean, tests, an evaluator | a checker other than the writer | 05, 07 |
| a model that splits its input and calls copies of itself | agents started from code | 08 |

SpiralTrain developer courses

# Summary : From LLM to Agent

- An LLM predicts the next token and keeps nothing between calls
- Context is everything the model sees on one call, and every call bills all of it
- A tool call is a request ; the application runs it
- An agent is a model calling tools in a loop, and the loop grows the context
- A harness is everything around the model : tools, files, caps, compaction and checks
- Verification lets an agent find out whether the work is done
- A person sets the intent and reviews the result ; each failure leads to a better check, rule or tool
- Crews of agents have built compilers, browsers and game makers ; their results still need a checker

# Introduction to LangGraph


From LLM to Agent

**Introduction to LangGraph**

Graph Fundamentals

State Management

Building Agents

Multi-Agent Patterns

Agent Communication

Advanced Workflows

Deep Agents

Production Deployment

Real-World Applications

Comet Arcade

How Far the Crew Gets

What is LangGraph?

What the Runtime Adds

Runtime and Harness Layers

Agent Frameworks Overview

Framework Capabilities

LangGraph Control Flow

Model APIs and Agent Runtimes

Why Multi-Agent?

Context Engineering

One Agent with More Tools

When One Agent is Enough

How Multi-Agent Systems Fail

Graph-Based Architecture

Conditional Routing

A First Graph

A First Multi-Agent System

A StateGraph by Hand

create_agent

create_deep_agent

Three Levels Compared

Agents versus Workflows

What Survived in 2026

The firstmate Orchestrator

The Service Desk Case

The Capstone in Five Stages

The Three Days

Summary : Introduction to LangGraph

SpiralTrain developer courses

# Comet Arcade

- **The crew of module 00 builds Comet Snake, one HTML file that opens in any browser**

| crew member | job |
|---|---|
| Producer | takes the brief, assigns the work, decides when the game ships |
| Designer | picks a twist from the menu and copies its rules |
| Artist | picks colours and draws sprites |
| Coder | writes the game logic |
| Tester | reports the bugs that real tests and a bot find |
| Publisher | a person who approves a release |

# The Crew Builds Comet Snake

project board — attempts 2

BRIEF Playtesters love Comet Snake but rage-quit when one comet ends a long run. Give them one way to survive.

RULES twist : shield

- Every fourth star places a shield
- The head on a shield : shield true
- A comet hit uses up the shield
- A second hit ends the game

THEME

TESTS 21 of 21

bot : 66 stars, best 80

BUG LIST · 1 · FIXED

Self-collision is not detected when the head lands on the body.

```
if (!state.over) {
  // The tail has already been removed when the snake did not grow, so
  // every remaining segment after the head is a body cell to check.
  if (state.snake.slice(1).some(segment => same(segment, head))) {
    state.over = true;
    state.reason = 'self';
```




**5** model calls

**6,390** tokens in

**2,976** tokens out

**40.5 s** wall time

**$0.002** at list price

The finished game, running on this slide : the crew's rules and colours, played by the kit's bot.

SpiralTrain developer courses

# How Far the Crew Gets

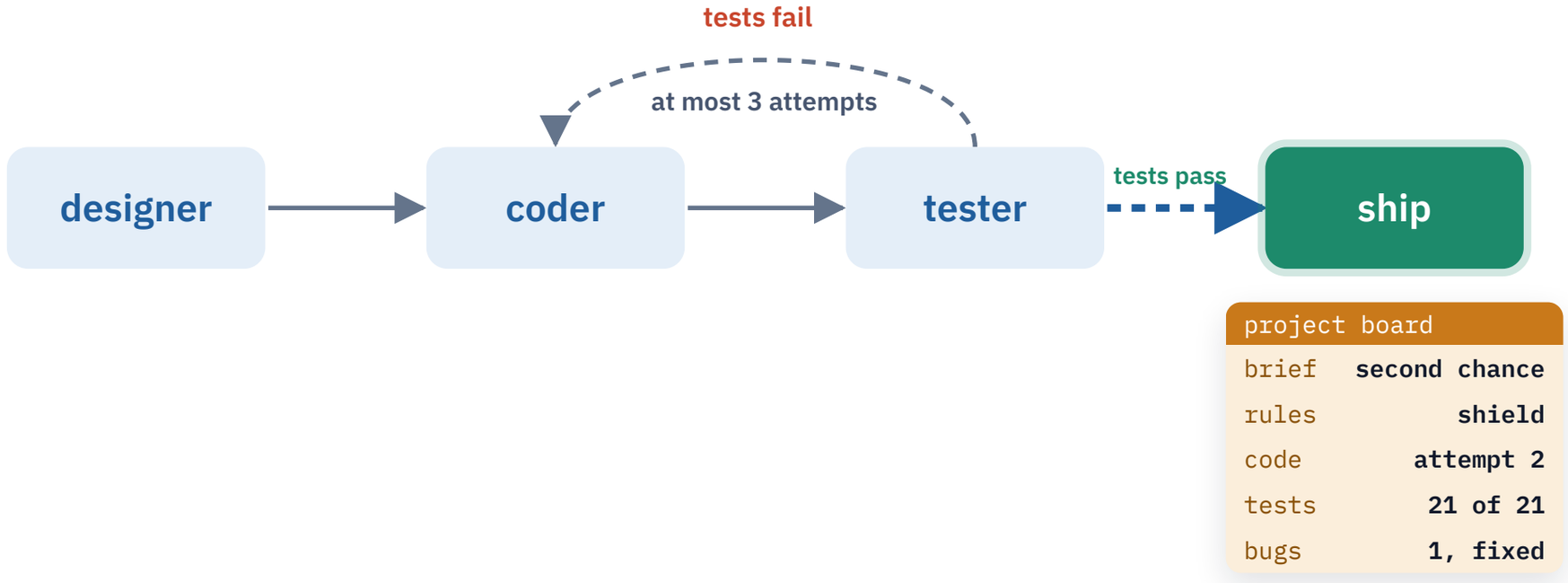
- **The current kit passes 20 of 20 runs, and all 20 play in a browser**
- **One run takes 3 model calls and about 19 seconds**
- **The kit does much of the work :**
  - It contains the page, the drawing, the keyboard, the bot and the tests
  - The Designer picks 1 of 4 twists from a menu
  - The Coder writes 132 to 165 lines of game logic
- **The verdict comes from real tests and a real bot, and the model writes the bug report**
- **A twist off the menu has no tests**
- **Tests prove the rules ; only opening the file proves that it plays :**
  - Without the kit, 8 of 8 whole files passed the tests and 1 of 8 was a blank page

SpiralTrain
developer courses

# What is LangGraph?

- LangGraph is a Python library that runs a program drawn as a graph of boxes and arrows
- A node is one box : one step of work, such as a lookup, a model call or a whole agent
- An edge is one arrow : after the Designer finishes, an edge starts the Coder
- State is the data every node shares : the crew’s project board
- One graph can mix fixed steps with model decisions
- LangChain agents and Deep Agents run on LangGraph

# One Run Through a Graph



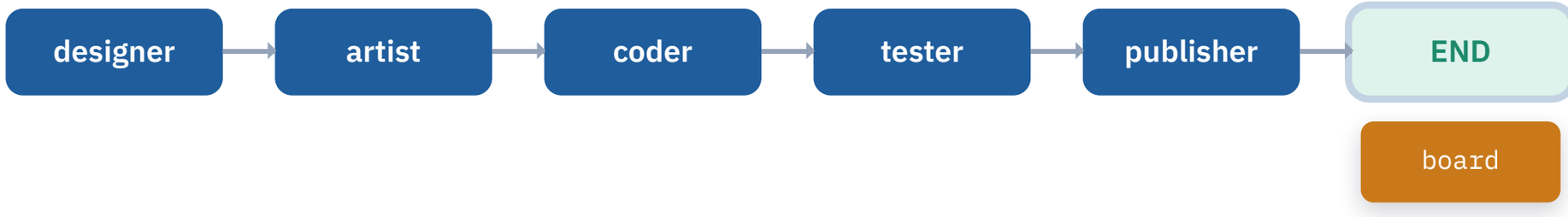

The second attempt passes all 21 tests, and the same conditional edge now leads to ship.

SpiralTrain developer courses

# What the Runtime Adds

- **A long agent run can crash, wait for a person or take minutes to finish**
- **LangGraph is the runtime, the software that runs the graph and looks after it through these three cases :**
  - Checkpoints : after every step the project board is saved, so a crash at the Tester restarts at the Tester
  - Interrupts : the run stops at the Publisher until a person approves the release
  - Streaming : each step and the model’s words appear while the run continues
- **Module 03 teaches checkpoints, and module 07 teaches interrupts**

# A Crash and a Pause




CHECKPOINTER : ONE SAVED BOARD PER STEP

step 1 step 2 step 3 step 4 step 5

THE PUBLISHER, A PERSON

**Release Comet Snake ?**

```
Command(resume="yes")
```

The Publisher answers yes. The run continues from the pause, and Comet Snake ships.

# Three Layers Under One Crew

HARNESS
**Deep Agents**
**files, a helper subagent, summaries**
Comet Arcade : a long build that keeps its notes in files

FRAMEWORK
**LangChain**
**create_agent, tools, middleware**
Comet Arcade : the Producer, an agent loop with the crew as tools

RUNTIME
**LangGraph**
**state, checkpoints, interrupts, streaming**
Comet Arcade : saves the project board after every step

**Python** your functions **Model** text in, text out




Each layer runs on the one below : create_agent and create_deep_agent both return a LangGraph graph. Higher means more ready-made, lower means more control.

# Runtime and Harness Layers

| layer | provides | package |
|---|---|---|
| runtime | state, checkpoints, interrupts, streaming | LangGraph 1.2.17 |
| framework | model interfaces, tools, agent loop, middleware | LangChain 1.4.6 |
| harness | filesystem, delegation, context management | Deep Agents 0.7.30 |

- **A framework supplies building blocks ; the application assembles them**
- **A harness supplies a ready agent with tools and instructions around its loop**
- **The runtime leaves the most control to the application ; the harness the least**

# Agent Frameworks Overview

- **Provider-led SDKs integrate with their ecosystems :**
  - OpenAI Agents SDK, Google ADK and Claude Agent SDK
  - Support for other models varies by SDK and feature
- **Model-agnostic frameworks work across providers :**
  - LangChain, CrewAI and Pydantic AI
  - LangGraph exposes the graph runtime directly
- **Microsoft Agent Framework succeeds AutoGen and Semantic Kernel**
- **Two requirements decide the choice :**
  - Who controls the flow, the code or the model
  - Where the agent runs and who operates it

# Framework Capabilities

| package | PyPI, 30 October 2026 | main abstraction |
|---|---|---|
| CrewAI | 1.15.33 | agents, crews, tasks and flows |
| OpenAI Agents SDK | 0.24.0 | agent loop, tools and handoffs |
| Google ADK | 2.13.0 | agents and workflow composition |
| Microsoft Agent Framework | 1.23.0 | agents and graph workflows |
| Claude Agent SDK | 0.2.175 | Claude Code harness as an SDK |
| Pydantic AI | 2.63.0 | typed dependencies and validated output |

- **LangGraph needs no agent loop ; a graph of plain Python functions runs as it is**

# LangGraph Control Flow

- **LangGraph starts from state and transitions :**
  - A node runs a function, a model call or a whole agent
  - The arrows are drawn in advance : when only one leaves the Designer, the Coder always comes next
- **Agent SDKs start from an agent loop and its tools**
- **Other frameworks add explicit flows next to the loop :**
  - CrewAI provides Flows
  - ADK and Microsoft Agent Framework provide workflow composition
- `create_agent` **supplies a standard loop for an agent that needs no custom graph**

# Model APIs and Agent Runtimes

| interface | who runs the tool loop | role |
|---|---|---|
| Chat Completions | application | course OpenRouter model endpoint |
| Responses API | application, with hosted tools available | OpenAI model interface |
| Agents SDK | SDK in the application | agent loop, handoffs and tracing |
| Managed Deep Agents | provider-hosted harness | hosted sessions and execution |

- **OpenAI-compatible names an API shape ; each provider supports its own subset**
- **A model only requests a tool call ; the runtime executes it and returns the result**
- **LangGraph keeps the application's state between model calls**

SpiralTrain developer courses

# Why Multi-Agent?

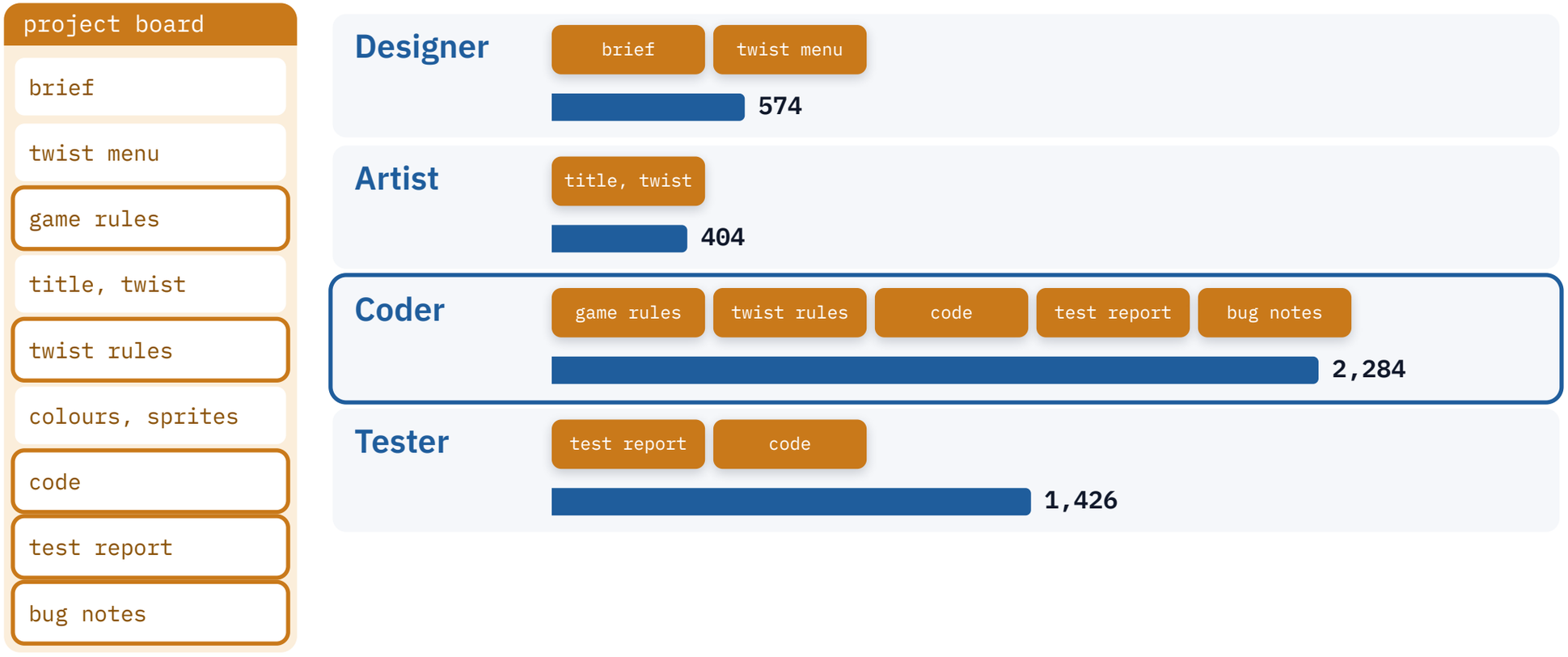
- **Module 00 gave the first reasons : each agent reads less, and the writer stays apart from the checker**
- **Separate teams can own separate agents**
- **The LangChain docs add three reasons to split :**
  - Too many tools lead the model to pick the wrong one
  - Specialised knowledge takes up much of the context
  - Steps in a fixed order unlock tools in stages, such as a release tool after the tests pass

SpiralTrain
developer courses

# Context Engineering

- **Context engineering decides which information each agent sees**
- **The multi-agent patterns of module 05 differ in that choice :**
  - Subagents : a helper agent sees only the task the Producer wrote for it
  - Handoffs : one agent passes the work and the conversation to the next
  - Skills : one agent loads a pack of instructions when it needs them
  - Router : a dispatcher sends each agent the part of the input meant for it
- **Modules 05 and 06 compare calls and tokens across patterns**

# What Each Crew Member Sees




The Coder's second attempt also gets the report and the bug notes : 2,284 input tokens. No crew member saw the whole board.

SpiralTrain developer courses

# One Agent with More Tools

- **One agent with the right tools is the baseline every split is measured against**
- **Every model call includes every tool description**
- **An experiment keeps 9 studio tools and 6 questions, and adds 51 tools that return no records :**
  - Input tokens per call grow from 564 at 9 tools to 1,672 at 60
  - The fact check passes 6 of 6 at 9, 20, 40 and 60 tools
  - No added tool is called in the saved run

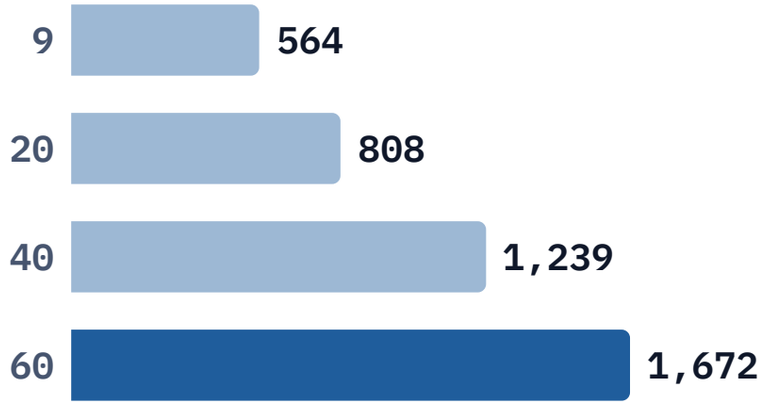
demo01 · one-agent-limits

# The Tool List Grows

SENT TO THE MODEL ON EVERY CALL — 60 tools

| | | | |
|---|---|---|---|
| list_games | daily_players | press_contacts | accessibility_options |
| game_info | session_length | trailer_status | save_file_format |
| assets_for_game | retention_curve | screenshot_library | cloud_save_usage |
| asset_info | crash_analytics | event_calendar | server_status |
| bugs_for_game | leaderboard_top | booth_plan | patch_notes |
| bug_details | achievement_list | localisation_status | release_calendar |
| builds_for_game | achievement_rates | translator_list | team_directory |
| playtests_for_game | player_reviews | font_library | meeting_notes |
| search_design_docs | forum_threads | music_tracks | office_calendar |
| storefront_page | forum_moderation | sound_library | hardware_inventory |
| store_price | discord_mentions | voice_actors | license_keys |
| store_discounts | social_posts | controller_support | backup_status |
| wishlist_count | newsletter_drafts | platform_rules | build_server_queue |
| sales_by_region | streamer_list | platform_submissions | competitor_games |
| refund_rate | press_kit | age_ratings | genre_trends |

INPUT TOKENS PER CALL, AVERAGE




6 of 6 at every size

0 detours

At every size the agent answered 6 of 6, in 14 to 16 calls, and never called another system's tool. Here the long list cost tokens and nothing else.

# When One Agent is Enough

- **The LangChain docs warn before they teach the patterns :**
  - "a single agent with the right (sometimes dynamic) tools and prompt can often achieve similar results"
- **One agent suits a task whose context fits and whose tools it picks correctly**
- **An agent can load a tool only when a step needs it, which keeps its list short**
- **A fixed process fits a workflow with model calls only where judgment is needed**
- **A second agent adds model calls : 2 for one agent, 4 to 6 per pattern in the module 05 experiment**
- **A split needs a measured failure that one agent cannot fix**

# How Multi-Agent Systems Fail

- **MAST (Cemri et al., UC Berkeley, 2025) studies more than 1,600 traces from 7 frameworks**
- **14 failure modes in three categories :**
  - System design issues, such as step repetition or a missed termination condition
  - Inter-agent misalignment, such as information withholding or reasoning-action mismatch
  - Task verification, such as premature termination or incorrect verification
- **Cognition, April 2026, "writes stay single-threaded" :**
  - Extra agents review or advise ; one agent acts
  - "Our original observations still hold today for parallel-writer swarms"
- **Modules 05 to 07 cover each category through loops, handoffs and judges**

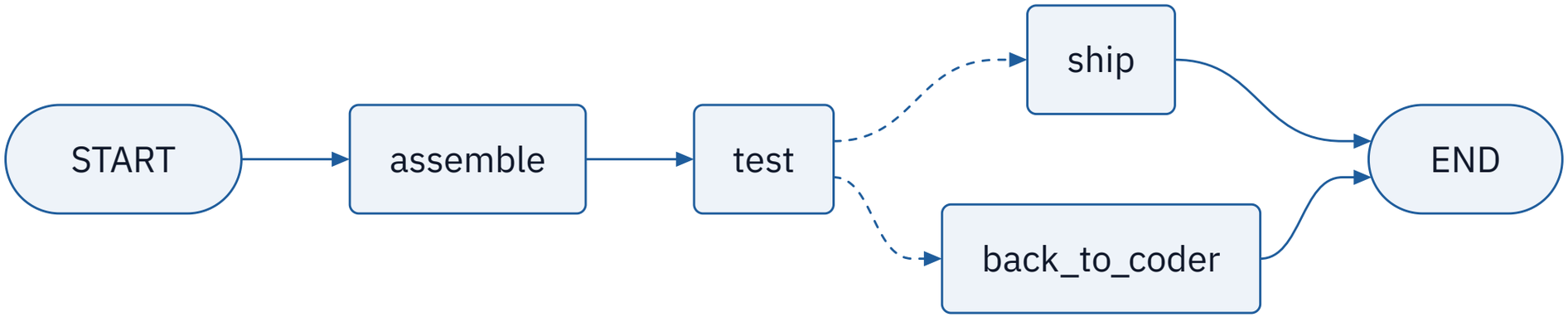
# Graph-Based Architecture

- A `StateGraph` runs steps over one shared state, such as a build and its test result
- Each node receives the state and returns the keys it changed
- `add_edge` sets a fixed next step
- `add_conditional_edges` lets a function choose the next step from the state
- A cycle lets a step run again, such as the Coder after a failed test
- `compile` turns the definition into a graph that runs
- `START` and `END` mark the entry and the exit

# Conditional Routing

- A routing function reads the state and returns the next node's name
- `Literal` in the return type names the possible targets
- The graph diagram shows a dashed edge to each possible target

```
def route(state: Build) -> Literal["ship", "back_to_coder"]:
    return "ship" if state["passed"] else "back_to_coder"
```



SpiralTrain developer courses

# A First Graph

```
builder = StateGraph(Build)
builder.add_node("assemble", assemble)
builder.add_node("test", test)
builder.add_node("ship", lambda state: {"verdict": "ship"})
builder.add_node("back_to_coder", lambda state: {"verdict": "back to the Coder"})
builder.add_edge(START, "assemble")
builder.add_edge("assemble", "test")
builder.add_conditional_edges("test", route)
builder.add_edge("ship", END)
builder.add_edge("back_to_coder", END)
graph = builder.compile()
```

- The graph calls no model. It puts a build in the kit, runs the real tests and picks the next node
- Three real builds : the plain kit, 10 of 19 tests ; the first attempt, 20 of 21 ; the fix, 21 of 21

demo02 · first-look

SpiralTrain developer courses

# A First Multi-Agent System

- **The Producer is one agent, and the crew members are its tools :**
  - `ask_designer`, `ask_artist` and `ask_coder` each make one model call in a fresh conversation
  - `run_game_tests` runs plain code : the tests and the bot
- **Each crew member receives only what the Producer passes in**
- **The code stays on the board, so the Producer never reads it**
- **The saved run takes 8 model calls and passes 21 of 21 tests**
- **The last cell shows the game in the notebook, and the file opens in any browser**

`demo02b · first-multi-agent`

SpiralTrain developer courses

# A StateGraph by Hand

- **A `StateGraph` is a LangGraph graph written by hand, here for one bug question :**
  - A model extracts the bug id
  - Python looks up the bug, the game and the latest build
  - A model writes the answer from those facts
- **State stores the bug id, the facts and the answer**
- **Code decides which lookups run and in what order**
- **The saved run makes 2 model calls, one to extract and one to answer**

demo03 · three-levels

SpiralTrain developer courses

# create_agent

- `create_agent` builds a tool-calling loop on LangGraph
- The model receives the prompt and the tool descriptions
- Each tool result goes back to the model for its next step
- The model decides which lookups to run and when to answer
- The bug question is the same as in the graph by hand, and the lookups are now tools
- The saved run takes 3 model calls against 2 for the graph by hand
- The result is a LangGraph graph, so its nodes and edges can be listed

demo03b · create-agent

SpiralTrain developer courses

# create_deep_agent

- `create_deep_agent` **adds a harness around the agent loop :**
  - Filesystem tools move large results out of the context
  - A general-purpose subagent, a helper agent, takes on delegated work
  - Summarisation shortens a long conversation history
- **The prompt and tools stay the same**
- **Built-in tools and instructions raise input tokens from 1,433 to 4,797 on the bug question**
- **Todo planning is opt-in in version 0.7.30**

demo03c · deep-agent

# Who Picks the Next Step

THE TASK BUG-104 is open on Comet Snake. How serious is it, and how many tests does the latest build pass ?

### StateGraph by hand

the code picks every step

extract look_up answer

243

2 model calls · input tokens

### create_agent

the model picks the steps

bug_details game_info builds_for_game

1,433

3 model calls · input tokens

### create_deep_agent

the model picks, inside a harness

bug_details builds_for_game file tools, subagent, longer instructions

4,797

2 model calls · input tokens

create_deep_agent adds its harness. It picked two tools in 2 calls, and every call includes the harness tools and instructions : 4,797 input tokens.

# Three Levels Compared

| level | who picks the steps | calls | input tokens | expected facts |
|---|---|---|---|---|
| `StateGraph` by hand | the code | 2 | 243 | pass |
| `create_agent` | the model | 3 | 1,433 | pass |
| `create_deep_agent` | the model, inside a harness | 2 | 4,797 | pass |

- One run each on `gpt-6-luna`, 30 October 2026, on the BUG-104 question
- All three report a major bug and 19 passing tests
- The Deep Agent used none of its harness tools on this question
- Module 08 compares the harness on a longer task

# Agents versus Workflows

- **Module 00 told the two apart : a workflow follows a path in code, and an agent chooses its next step**
- **Here both receive the same four playtester reports :**
  - A fixed workflow gives no answer to a report without a bug id, and passes it to the Producer
  - An agent can search the bugs and assets instead
- **Production systems often combine fixed steps with agent steps**

demo04 · workflow-or-agent

# Four Reports on Two Paths

REPORTS

**BUG-104**
shield does nothing

**BUG-105**
sound plays twice

**BUG-202**
blocked slide counts

**no id**
comets I cannot see

WORKFLOW : THE PATH IS IN THE CODE

parse — look_up — write_note

escalate no id

AGENT : THE MODEL PICKS THE PATH

model — tools

| | | |
|---|---|---|
| BUG-104 | **4 calls** | 1,942 in |
| BUG-105 | **3 calls** | 1,634 in |
| BUG-202 | **3 calls** | 1,546 in |
| no id | **4 calls** | 1,919 in |

WORKFLOW
**3 calls**
557 in
3 of 4 answered

AGENT
**14 calls**
7,041 in
4 of 4 answered

The agent answered four of four, in 14 model calls and 7,041 input tokens. The workflow answered three, in 3 calls and 557 tokens. Each is a single run.

# What Survived in 2026

- **Anthropic's research system and `firstmate` both use an orchestrator with isolated workers :**
  - One agent is responsible for the task and combines results
  - Each worker receives a bounded brief and its own context
- **`firstmate` implements this design around coding harnesses :**
  - Each task has its own git worktree and session
  - A watcher wakes the orchestrator on events
  - Briefs and state persist in files
- **Each worker's result still goes through review before it is merged**

# The firstmate Orchestrator

- `firstmate` supplies instructions, skills, scripts and persistent state for coding harnesses
- The MIT-licensed project supports Claude Code, Codex and Pi

| `firstmate` | in this course |
|---|---|
| a first mate dispatching to crewmates | subagents behind one orchestrator, module 05 |
| one git worktree and session per task | context isolation and backends, module 08 |
| a watcher that wakes the first mate on events | event systems and async subagents, modules 06 and 09 |
| briefs and state files on disk | checkpointers and the `Store`, module 03 |
| merges escalated to the human | interrupts and approval flows, module 07 |

- Yair reports using `firstmate` for his coding fleet

SpiralTrain developer courses

# The Service Desk Case

- **Aurelia Imaging Systems is a fictional imaging-equipment manufacturer**
- **The capstone, the final project of the course, builds a service desk over its supplied data :**
  - Manuals and service bulletins
  - Parts, equipment and service records in SQLite
  - Tickets and an evaluation set
- **Specialist agents handle manuals, parts and scheduling**
- **Part orders pause for approval**
- **Three made-up worlds run through the course :**
  - Modules 00 to 04 : Comet Arcade, the game studio
  - Modules 05 to 10 : Veenhaven Bus, 10 buses at 5 depots
  - Capstone : Aurelia Imaging Systems

# The Capstone in Five Stages

| stage | after | Aurelia service desk |
|---|---|---|
| 1 | module 03 | triage, which sorts a ticket, a `service.db` lookup and an answer ; state that survives a restart |
| 2 | module 05 | specialists for parts, manuals and scheduling behind a supervisor, an agent that assigns the work |
| 3 | module 07 | diagnoses side by side with a judge ; the manuals agent over A2A, a protocol between agents ; part-order approval inside a subagent |
| 4 | module 08 | the same desk as a Deep Agent, compared with stage 3 |
| 5 | module 09 | running the desk as a service with `langgraph dev`, a background diagnosis subagent and evaluation per agent |

- **Evaluation compares answer quality, model calls and tokens as the desk gains capabilities**

SpiralTrain developer courses

# The Three Days

| day | modules | builds |
|---|---|---|
| Monday | 01 to 04 | graphs, state and single agents in plain LangGraph |
| Tuesday | 05 to 07 | the multi-agent patterns, communication between agents, control |
| Wednesday | 08 to 10 | the same system as a Deep Agent, then served, evaluated and applied |

- **Three ideas run through every module :**
  - Multi-agent is context engineering
  - One agent first ; add an agent when a measurement says so
  - Build each pattern by hand, then use the harness

SpiralTrain developer courses

# Summary : Introduction to LangGraph

- LangGraph runs a program as nodes, edges and shared state
- LangChain supplies ready-made agents ; Deep Agents adds a harness
- A workflow follows a path set in code ; an agent picks its next step from the last result
- Multi-agent design decides what each agent sees
- One agent sets the baseline for quality and cost
- A measurement decides whether a second agent is worth its cost

# Graph Fundamentals

From LLM to Agent
Introduction to LangGraph
**Graph Fundamentals**
State Management
Building Agents
Multi-Agent Patterns
Agent Communication
Advanced Workflows
Deep Agents
Production Deployment
Real-World Applications

Why LangGraph?
LangChain vs. LangGraph
Thinking in LangGraph
Directed Graphs
DAGs and Cycles
State Machines
Model Transitions
LangGraph Concepts
Node Functions
StateGraph Nodes and Edges
Hello LangGraph
START and END
Conditional Edges

Routing with path_map
Loops and Cycles
GraphRecursionError
Command
Fan-out and Fan-in
Parallel Branches and Send
Deferred Nodes
Compiling a Graph
Visualising a Graph
Functional API
Graph API or Functional API
A Model in a Node
Summary : Graph Fundamentals

SpiralTrain developer courses

# Why LangGraph?

- **In the agent loop of module 00 the model picks every next step**
- **A game build needs some of those steps fixed :**
  - After the Designer finishes, the Coder starts
  - When tests fail, the build goes back to the Coder
  - When tests pass, the game ships
- **LangGraph writes those steps as code :**
  - Each step is a function, called a node
  - A project board passes from node to node and contains the work so far
- `create_agent` **from module 01 builds such a graph, and so do Deep Agents**

# LangChain vs. LangGraph

- **LCEL is LangChain's pipe syntax : `prompt | model | parser`, one straight line**

| Concern | LCEL runnables | LangGraph |
|---|---|---|
| Composition | Sequences, parallel calls and branches | Nodes and edges over state |
| State | Values passed between runnables | Explicit state returned from nodes |
| Repetition | Application code or custom runnables | Cycles and conditional edges |
| State updates | Application-managed values | Explicit updates returned from nodes |

- **`create_agent` returns a compiled LangGraph graph**
- **A graph of its own suits logic specific to the application, such as a fix loop**

SpiralTrain developer courses

# Thinking in LangGraph

- **Designing a graph starts from the process, with one question for each part :**

| Part | Question | Comet Snake |
|---|---|---|
| Node | What are the steps? | Designer, Coder, Tester |
| State | What must every step see? | The project board : brief, rules, code, bugs |
| Edge | What runs after what? | After the Designer, the Coder |
| Conditional edge | What gets decided on the way? | Tests fail : back to the Coder |
| Control point | Where does a mistake cost most? | The Publisher approves a release |

- **A step that needs judgment calls a model, such as writing the game logic**
- **A step with a fixed answer is a plain function, such as running the tests**

SpiralTrain developer courses

# A Graph Drawn from Code

```
builder = StateGraph(Board)

builder.add_node("designer", designer)

builder.add_node("coder", coder)

builder.add_node("tester", tester)

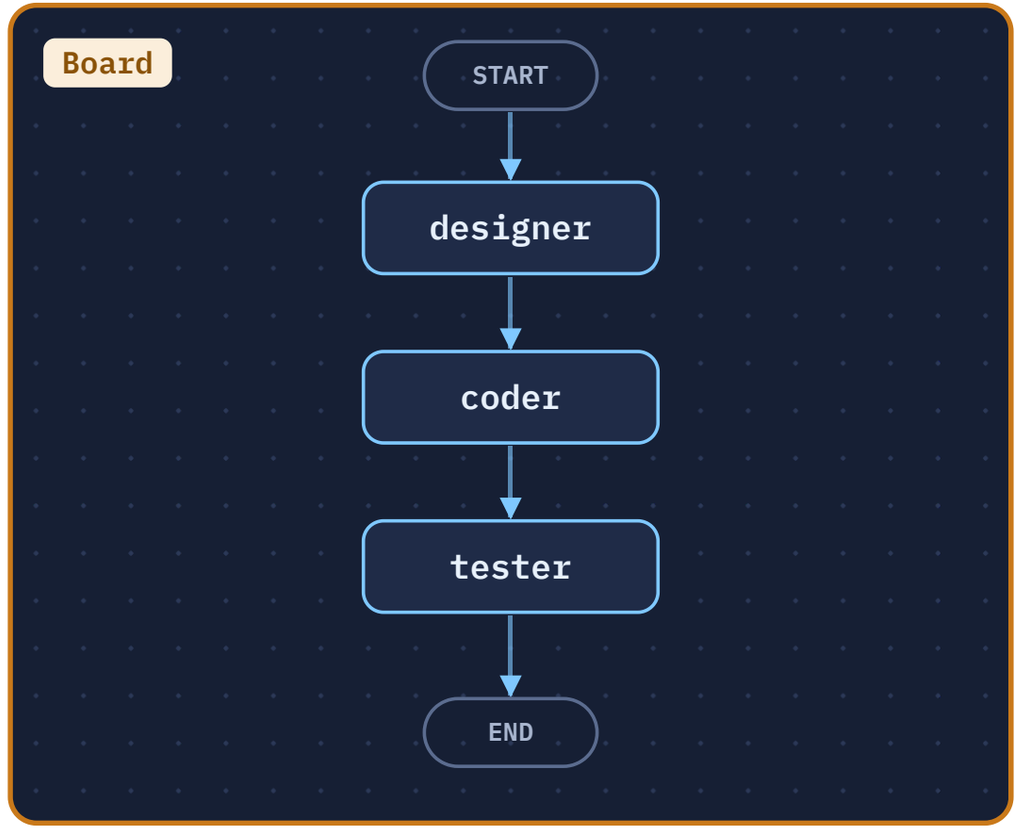
builder.add_edge(START, "designer")

builder.add_edge("designer", "coder")

builder.add_edge("coder", "tester")

builder.add_edge("tester", END)
```

```
graph = builder.compile()
```




`compile` checks the drawing and returns a graph that can run. The dot follows the only path through it.

SpiralTrain developer courses

# Directed Graphs

- **A directed graph is a set of steps joined by one-way arrows :**
  - A step is a node, such as `designer` or `coder`
  - An arrow is an edge : after `designer` comes `coder`
- **A run enters at `START`, goes `designer`, `coder`, `tester`, and stops at `END`**
- **An arrow back to an earlier node makes a loop, called a cycle : from `tester` back to `coder`**

SpiralTrain developer courses

# DAGs and Cycles

- A path is one route through a graph, such as `designer`, `coder`, `tester`
- DAG stands for directed acyclic graph : no path leads back to an earlier node
- A graph with only that path is a DAG
- An edge from `tester` back to `coder` adds a cycle
- Graph theory calls a node a vertex, and LangGraph uses node
- LangGraph accepts both shapes, and a recursion limit stops a cycle that never ends

SpiralTrain developer courses

# State Machines

- **A state machine describes a process as states and transitions :**
  - A state records where the build stands, such as `tests fail`
  - A transition moves the build on and selects the next step
- **A plain node applies a fixed transition, such as `tests pass` to `shipped`**
- **A model can choose a transition when the next step needs judgment**
- **Demo01 b prints one build as it moves through its states :**
  - `build 1`, `tests fail`, `build 2`, `tests pass`, `shipped`, after 2 attempts
  - A build that never passes stops at 3 attempts, with `tests fail`

SpiralTrain developer courses

# Model Transitions

- A plain node changes the board the same way on every run
- A model node supplies a judgment, such as whether a brief is an arcade game or a puzzle
- A routing function maps that label to a node, such as `puzzle`
- The other steps stay plain functions

# LangGraph Concepts

| Term | Meaning | Comet Arcade |
|---|---|---|
| State | The data of one run, in named fields called keys | The project board |
| Node | A function that reads the state and returns updates | `coder` writes `code` |
| Edge | An arrow : which node starts when this one finishes | After `designer` , `coder` |
| Conditional edge | A function that reads the board and picks the next node | Tests fail : `coder` |
| `START` , `END` | Where a run enters and where it stops | `START` to `designer` |
| Builder | The graph while it is being defined ; `compile` makes it runnable | `builder = StateGraph(Board)` |

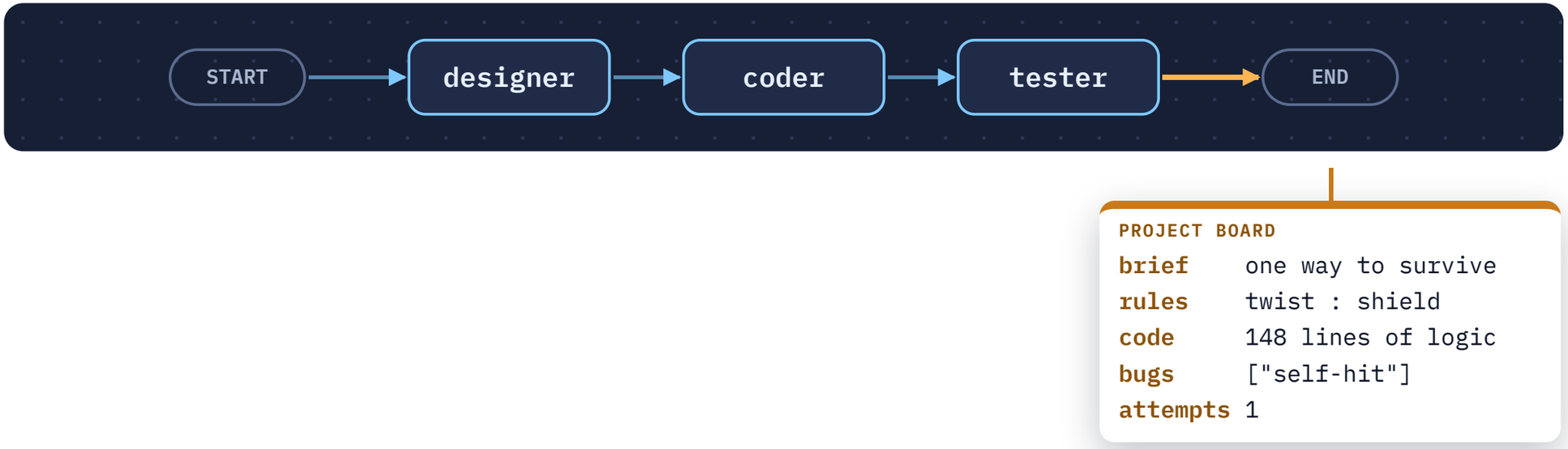
SpiralTrain developer courses

# Node Functions

```
def coder(board: Board) -> dict:
    return {"code": "game.html"}
```

- A node receives the current state as its input
- A node returns only the keys it changed ; LangGraph writes them onto the board
- The keys it leaves out stay as they were
- One graph can be a node inside another graph

# The Project Board




LAST BOARD three nodes wrote to it, and no node read the bug

---

Only one arrow leaves the Tester, so this run stops at `END`, with the bug still on the board.

SpiralTrain developer courses

# StateGraph Nodes and Edges

- `StateGraph` **builds a graph around a state schema, the list of keys on the board :**
  - `add_node` adds a step : a function or a whole graph
  - `add_edge` adds an arrow : after `designer`, always `coder`
  - `add_conditional_edges` adds a decision : a function picks the next node
- `compile` **checks the definition and returns a graph that can run**
- **A compiled graph exposes** `invoke` **and** `stream` **:**
  - `invoke` takes the first board and returns the last one
  - `stream` returns results while the run goes on

demo02 · nodes-and-edges

SpiralTrain developer courses

# Hello LangGraph

```python
from typing import TypedDict
from langgraph.graph import END, START, StateGraph

class Board(TypedDict):
    brief: str
    rules: str

def designer(board: Board) -> dict:
    return {"rules": f"rules for {board['brief']}"}

builder = StateGraph(Board)
builder.add_node("designer", designer)
builder.add_edge(START, "designer")
builder.add_edge("designer", END)
graph = builder.compile()
print(graph.invoke({"brief": "snake in space", "rules": ""}))
```

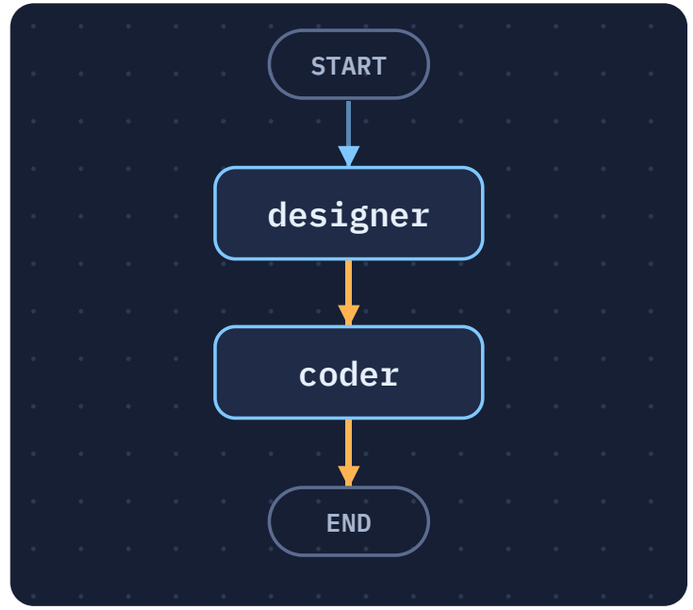
- `Board` lists the keys of the state, and `invoke` runs the graph once

# Adding a Second Node

```
def designer(board: Board) -> dict:
    return {"rules": f"rules for {board['brief']}"}

def coder(board: Board) -> dict:
    return {"code": f"game from {board['rules']}"}

builder = StateGraph(Board)
builder.add_node("designer", designer)
builder.add_node("coder", coder)
builder.add_edge(START, "designer")
builder.add_edge("designer", "coder")
builder.add_edge("coder", END)
```




Two edges change, and the Coder runs after the Designer.

SpiralTrain developer courses

# START and END

- `START` and `END` are two fixed names from `langgraph.graph`
- `add_edge(START, "designer")` makes the Designer the first node of every run
- `add_edge("tester", END)` stops the run after the Tester
- A graph with no edge from `START` fails at `compile` : `Graph must have an entrypoint`
- `set_entry_point` and `set_finish_point` are older equivalents ; the docs use `add_edge`

demo02b · start-end-entry

SpiralTrain developer courses

# Conditional Edges

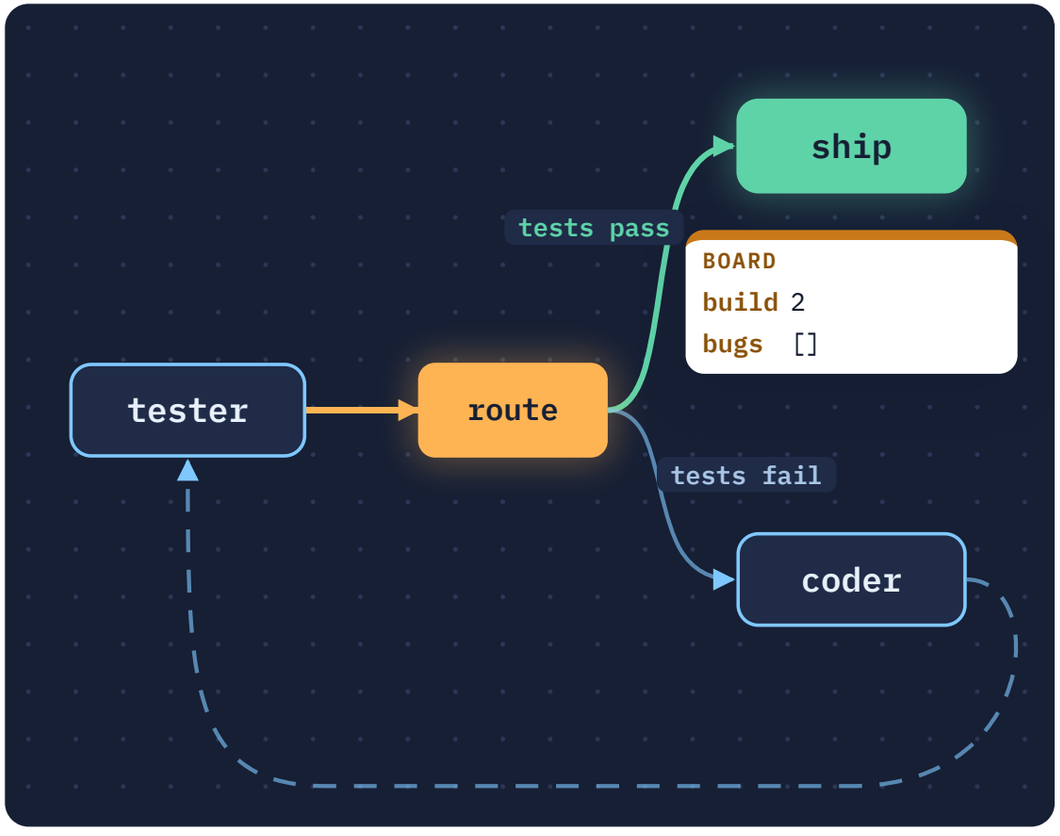
- A routing function reads the board and returns the name of the next node :

```
def route(board: Board) -> Literal["ship", "coder"]:
    return "coder" if board["bugs"] else "ship"

builder.add_conditional_edges("tester", route)
```

- LangGraph calls `route` when `tester` has finished
- The returned text is the name of a node, as `"ship"` is here
- `Literal` in the return type lists the allowed values, so the graph drawing can show each route

# Tests Decide the Branch




RUN_TESTS, BUILD 2

```
21 of 21 rule tests pass

bot : 66.0 stars on average, best score 80
```

ROUTING FUNCTION

```
def route(board: Board) -> Literal["ship", "coder"]:
    return "coder" if board["bugs"] else "ship"
```

RETURNS `"ship"`

The same `route` reads an empty list and returns `"ship"`. A different board takes the other branch.

SpiralTrain developer courses

# Routing with path_map

- `path_map` maps route labels to graph node names :

```
builder.add_conditional_edges(
    "tester",
    lambda board: board["tests"],
    {"pass": "ship", "fail": "coder"},
)
```

- A dictionary maps labels to destinations
- A list declares possible destinations when labels match node names
- A label missing from the map, such as `"flaky"`, raises `KeyError`

demo03b · path-map

SpiralTrain developer courses

# Loops and Cycles

- A cycle sends the build back to an earlier node : tests fail, back to the Coder
- The router decides whether to go round again, and it reads a cap from the board :

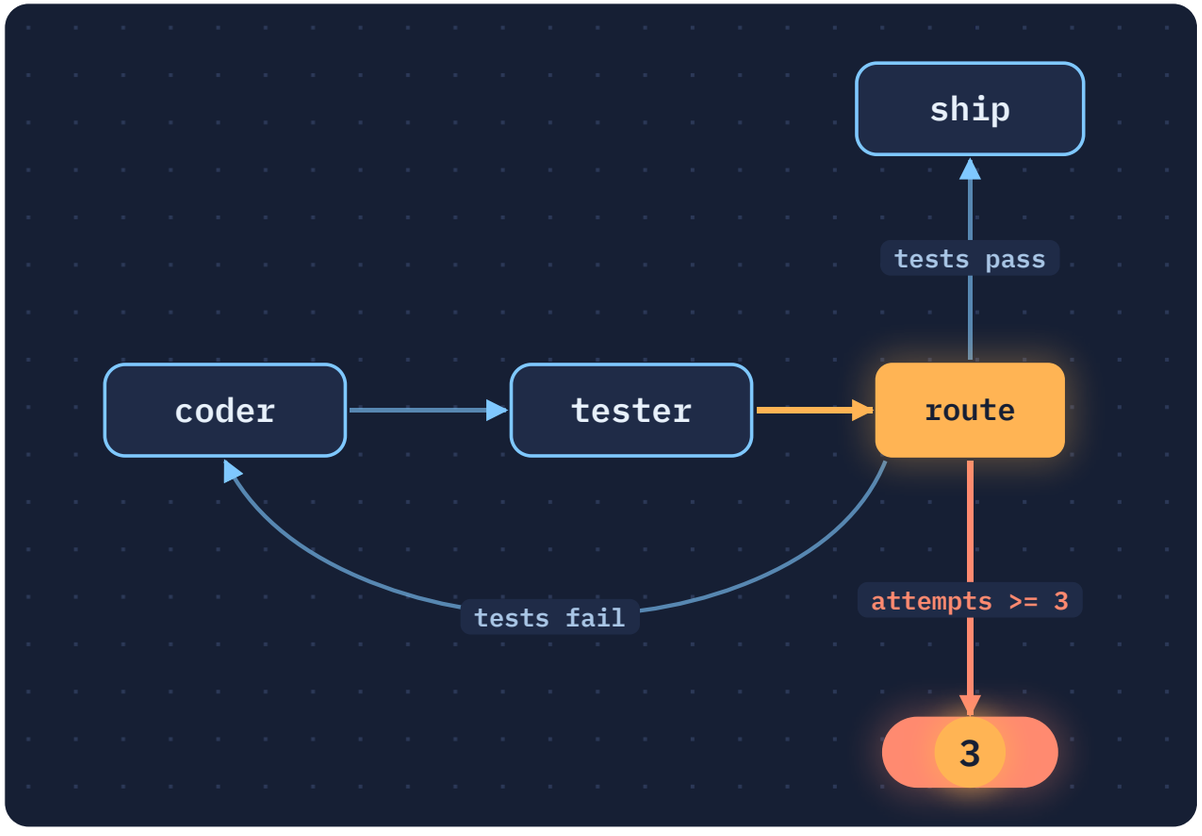
```
def route(board: Build) -> Literal["ship", "coder", "__end__"]:
    if board["status"] == "tests pass":
        return "ship"
    if board["attempts"] >= 3:
        return END
    return "coder"
```

- Returning `END`, the text `"__end__"`, leaves the graph, so a build at the cap never ships

demo01b · fix-loop

SpiralTrain developer courses

# The Fix Loop




RUN 2, SEED 12

**3** attempts

cap

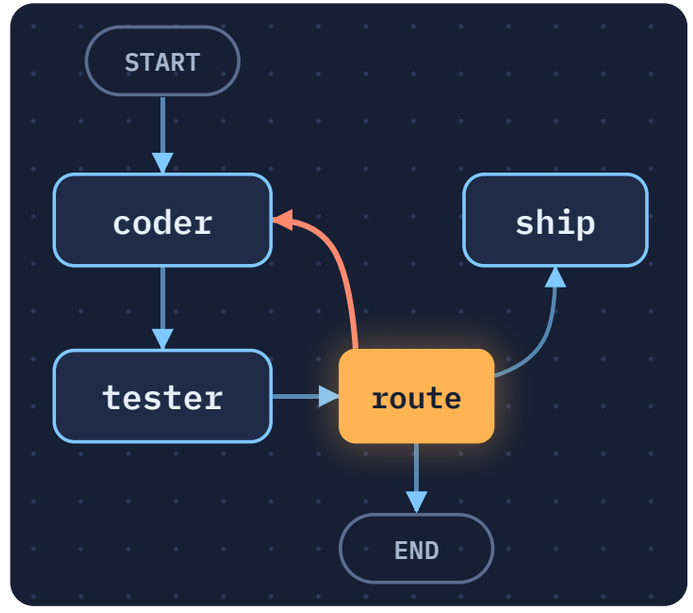
| | |
|---|---|
| 1 | tests fail |
| 2 | tests fail |
| 3 | tests fail<br>more-comets, bot-score |

The router returns `END`. The run stops with its failing tests, and the build never reaches `ship`.

# Reading the Fix Loop

```python
def route(board: Build) -> Literal["ship", "coder", "__end__"]:
    if board["status"] == "tests pass":
        return "ship"
    if board["attempts"] >= 3:
        return END
    return "coder"

loop = StateGraph(Build)
loop.add_node("coder", coder)
loop.add_node("tester", tester)
loop.add_node("ship", ship)
loop.add_edge(START, "coder")
loop.add_edge("coder", "tester")
loop.add_conditional_edges("tester", route)
loop.add_edge("ship", END)
```




The back edge has no `add_edge`. It exists only in a return value.

SpiralTrain developer courses

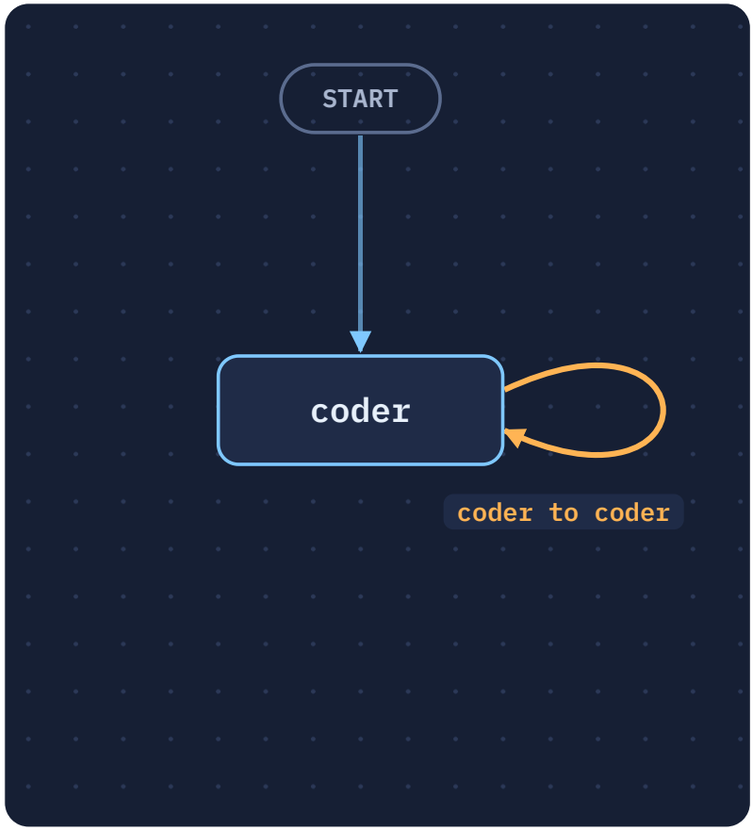
# GraphRecursionError

- **LangGraph counts the steps of every run and stops at a limit :**
  - The default is 10,007 steps on LangGraph 1.2.17
  - `create_agent` and `create_deep_agent` set 9,999
- **Passing the limit stops the run with a `GraphRecursionError`**
- **A node with an edge back to itself runs until that error**
- **`recursion_limit` sets the limit for one run : `invoke(board, {"recursion_limit": 40})`**
- **A loop around a model needs a cap of its own, or the limit arrives after thousands of model calls**

demo03c · recursion-limit

SpiralTrain developer courses

# A Loop with No Exit




STEPS IN ONE RUN

**10,007**

**GraphRecursionError**
Recursion limit of 10007 reached without hitting a stop condition

**3** attempts, then the router returned `END`

The fix loop reads a cap from the board. A build that never passes stopped after 3 attempts.

SpiralTrain developer courses

# Command

- A node can return a `Command` to write to the state and pick the next node in one step :

```
from typing import Literal
from langgraph.types import Command

def triage(state: Bug) -> Command[Literal["coder", "artist"]]:
    target = "artist" if state["kind"] == "sprite" else "coder"
    return Command(update={"assigned": target}, goto=target)
```

- `update` lists the keys to write ; `goto` names the next node
- A node returning `Command` needs no conditional edge
- The `Command[Literal[...]]` return type adds its edges to the drawing

demo04 · command-and-send

SpiralTrain developer courses

# Fan-out and Fan-in

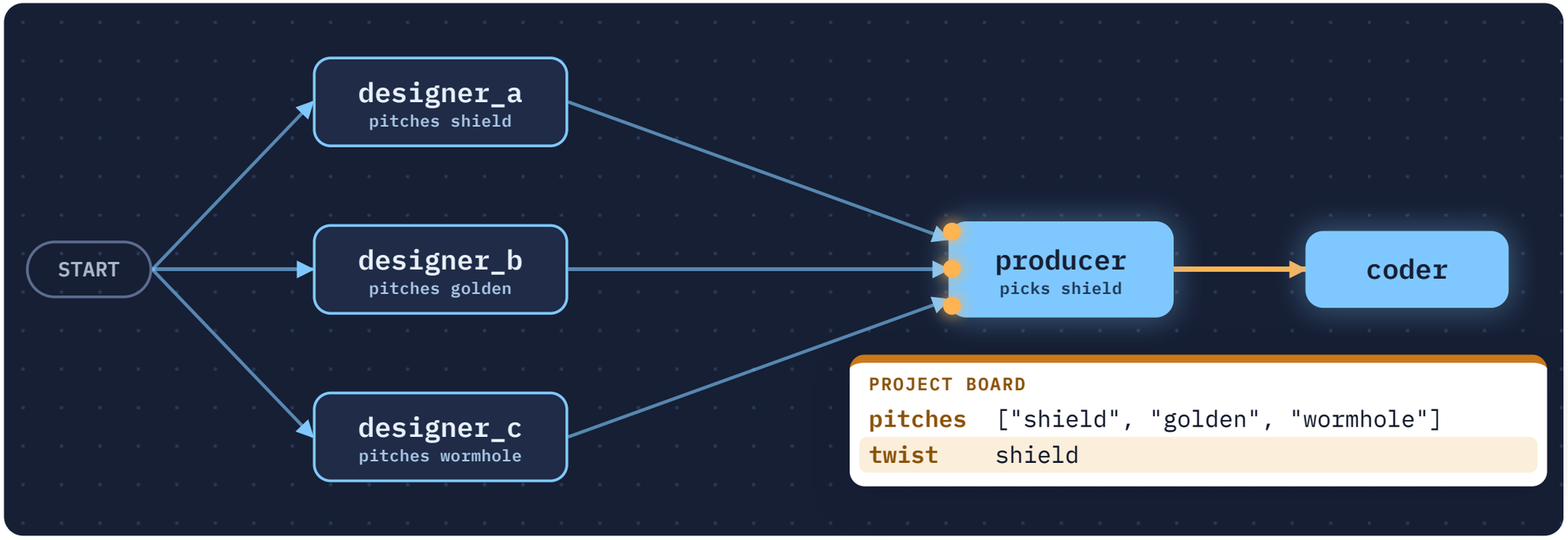
- Fan-out : one node starts several nodes at the same time
- Fan-in : those nodes meet at one node, called the join
- The join below waits for both Testers :

```
builder.add_edge(["rule_tester", "bot_tester"], "gate")
```

- Two nodes that write one key need a reducer, a function that combines the writes :

```
bugs: Annotated[list, operator.add]
```

# Three Designers and a Join

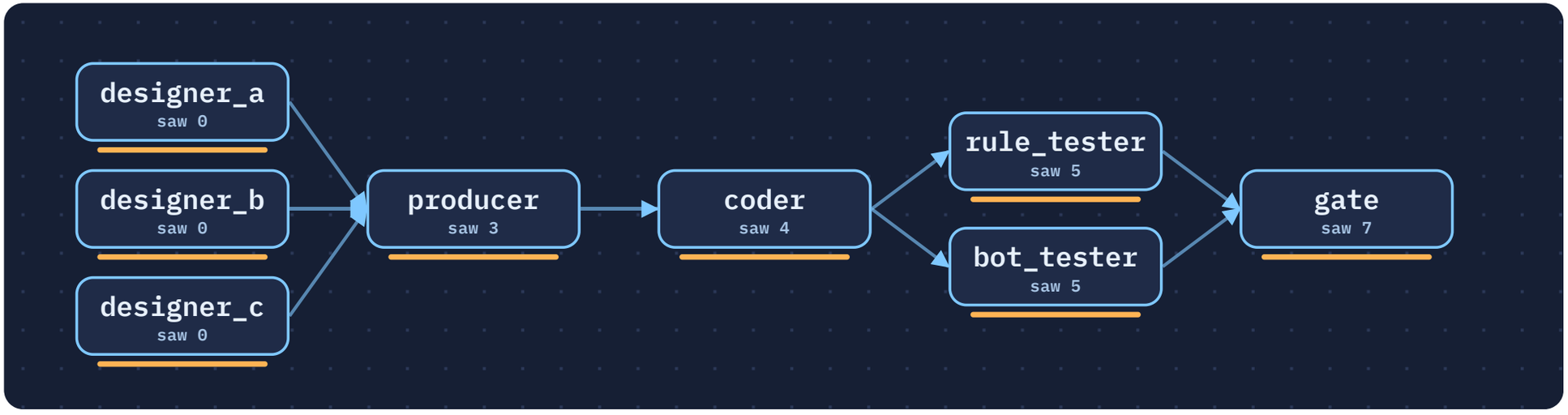



```
builder.add_edge(["designer_a", "designer_b", "designer_c"], "producer")
pitches: Annotated[list, operator.add]
```

The Producer runs once, picks `shield` and sends one board on to the Coder.

SpiralTrain developer courses

# Supersteps




LOG ON THE BOARD — 8 entries

designer_a | designer_b | designer_c | producer | coder | rule_tester | bot_tester | gate

5 supersteps | 8 nodes | 1.5 s, one node takes 0.3 s

---

8 nodes, each waiting 0.3 seconds, took 1.5 seconds : 5 rounds, one after the other.

demo04d · supersteps

SpiralTrain developer courses

# Parallel Branches and Send

- `Send` creates branches at run time, each with its own input :

```python
from langgraph.types import Send

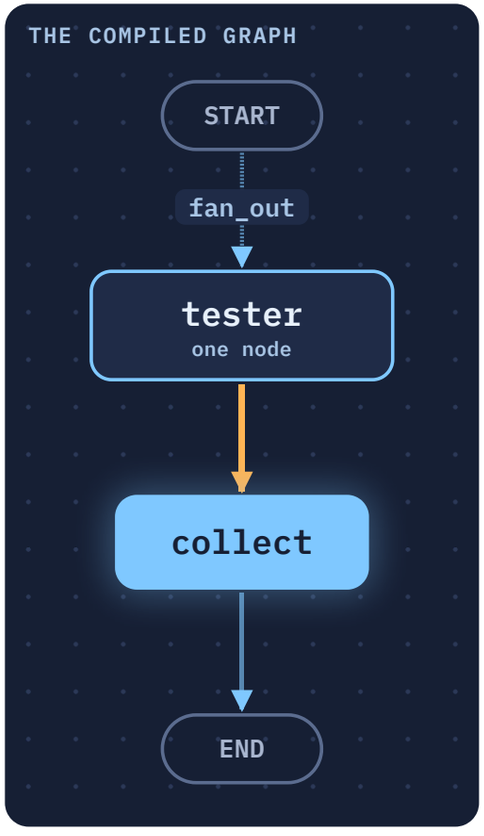
def fan_out(state):
    return [Send("tester", {"level": level}) for level in state["levels"]]
```

- A level here is one seed, a start number : `init(seed)` lays out a different game per seed
- A `collect` node sums up the reports as `3 levels tested`
- Map-reduce : many runs of one node, then one step that combines the results

demo04b · send-map-reduce

SpiralTrain developer courses

# One Tester per Level




THIS RUN

PROJECT BOARD

levels [1, 2, 3, 4, 5]

summary 5 levels tested

level 1: 56 stars

level 2: 66 stars

level 3: 65 stars

level 4: 80 stars

level 5: 63 stars

The drawing on the left has not moved. The list on the board set the number of testers.

SpiralTrain developer courses

# Deferred Nodes

- **A deferred node waits until the other scheduled work is complete**

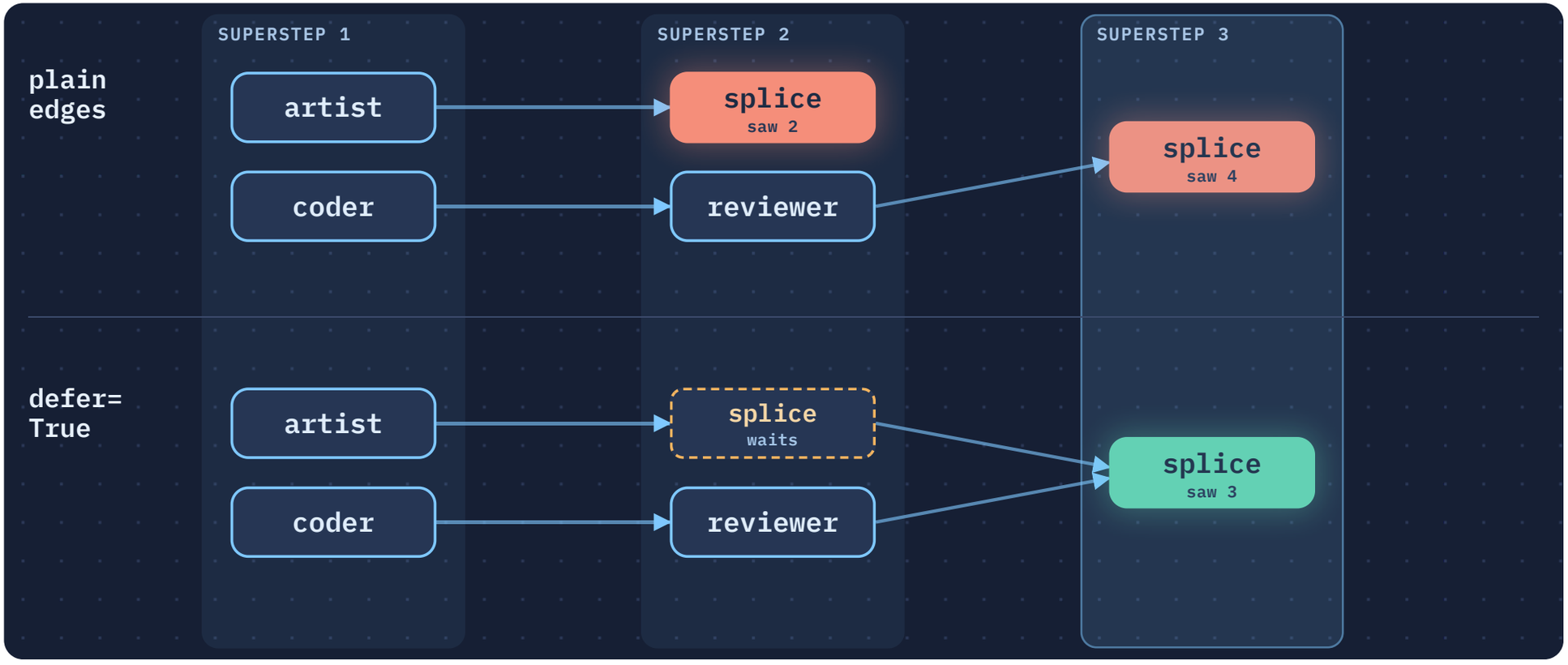
```
builder.add_node("splice", splice, defer=True)
```

- **One branch runs `artist`, the other `coder` then `reviewer`; both lead to `splice`:**
  - Plain: `splice` runs twice, first seeing 2 entries, then 4
  - Deferred: `splice` runs once, after `reviewer`, and sees 3
- **Without `defer`, a node behind uneven branches runs once per arriving branch**

demo04c · deferred-nodes

SpiralTrain developer courses

# Uneven Branches




Superstep 3 : the plain `splice` runs a second time. The deferred one runs once, with no other work left, and sees the work of all three.

SpiralTrain developer courses

# Compiling a Graph

- `compile` checks the builder and returns a graph that can run
- `compile` rejects an edge to a missing node : `Found edge ending at unknown node`
- A checkpointer is attached at compile time ; it saves the board after every step
- Options for one run, such as `recursion_limit`, go into each `invoke`
- One compiled graph runs many times : demo04 b ran one graph on 3 levels, then on 5

SpiralTrain developer courses

# Visualising a Graph

- A compiled graph returns a Mermaid description, a text format for diagrams :

```
print(graph.get_graph().draw_mermaid())
```

- A Markdown viewer that knows Mermaid draws the text as a diagram
- The text comes from the graph itself, so the drawing matches the code
- No network service is needed

# Functional API

- The Functional API expresses a workflow with decorated functions :

```python
from langgraph.func import entrypoint, task

@task
def normalise(text: str) -> str:
    return text.strip().title()

@entrypoint()
def workflow(text: str) -> str:
    return normalise(text).result()
```

- Tasks can run in parallel and return values to the entrypoint

demo05b · functional-api

SpiralTrain developer courses

# Graph API or Functional API

- **Graph API :**
  - Explicit nodes and edges can be drawn and inspected
  - Branches and loops appear directly in the graph definition
- **Functional API :**
  - Ordinary Python control flow expresses the sequence
  - Tasks mark work that can run and resume independently
- **Both APIs use the LangGraph runtime**
- **A task can call a compiled graph**

SpiralTrain developer courses

# A Model in a Node

- **A model call is one node among plain functions :**
  - `classify` asks the model whether a brief is an arcade game or a puzzle
  - A routing function sends the brief to the `arcade` or the `puzzle` node
- **The model’s reply can vary between runs**
- **Every other step runs the same way each time**

demo06 · a-model-in-a-node

SpiralTrain developer courses

# Summary : Graph Fundamentals

- A directed graph is nodes joined by one-way edges, from `START` to `END`
- The project board is the state : each node returns the keys it changed
- `StateGraph` builds the graph : a fixed edge always leads to one node, a conditional edge lets a function choose
- A fix loop is a cycle, and a cap on the board ends it
- Nodes whose turn has come run together in one round, a superstep, and a join waits for all of them
- `Send` creates branches at run time from what is on the board
- `compile` checks the builder and returns a graph that can run and draw itself

# State Management

From LLM to Agent
Introduction to LangGraph
Graph Fundamentals
**State Management**
Building Agents
Multi-Agent Patterns
Agent Communication
Advanced Workflows
Deep Agents
Production Deployment
Real-World Applications

What is State?
TypedDict State
Dataclass State
Pydantic State

Input and Output Schemas
Private State
Reducers
Merging State Updates

MessagesState
add_messages

Custom Reducers
Overwriting a Reduced Key
Concurrent Updates
Checkpointers and Persistence

Durability Modes
Threads
SqliteSaver
PostgreSQL Checkpoint Storage

Restoring Saved State
Inspecting Thread State

State History
Time Travel
Forking a Checkpoint
DeltaChannel

Long-Term Memory
Streaming Modes
Debugging State
The v2 Stream Format

v3 Event Projections
Summary : State Management

SpiralTrain developer courses

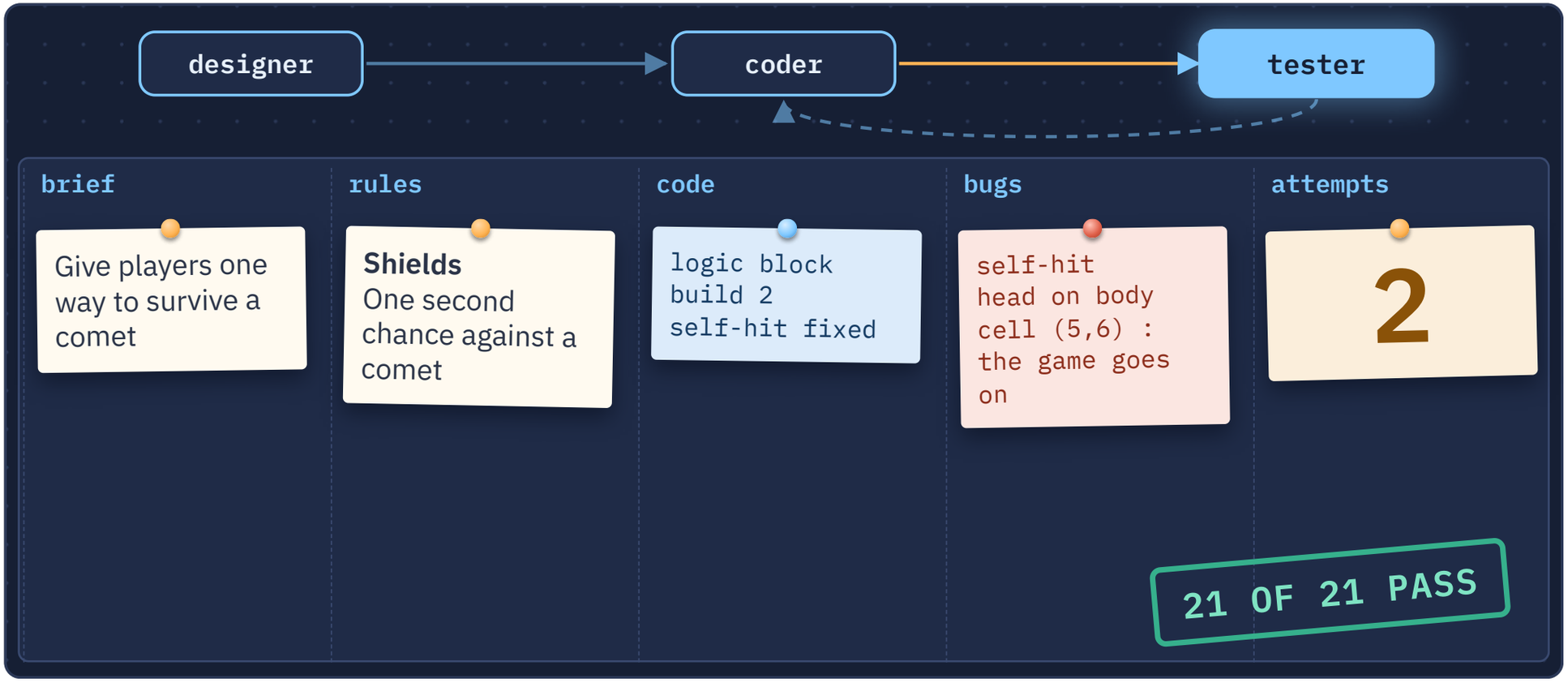
# What is State?

- State is one dictionary, a set of named values called keys, that a graph passes from node to node
- Comet Arcade keeps it as the project board of a game :

| key | contains | written by |
|---|---|---|
| `brief` | what the game must do | the input |
| `rules` | the rules of the game | Designer |
| `code` | the game logic | Coder |
| `bugs` | what the tests found | Tester |
| `attempts` | builds tried so far | Coder |

- Each node gets the whole board and returns only the keys it changed ; the model inside sees only what the node passes to it

# The Project Board




The tests run on the new code : 21 of 21 rule tests pass. All three crew members worked from one board.

SpiralTrain developer courses

# TypedDict State

- A state schema lists the keys of the board and the type of each
- `TypedDict` describes a dictionary with named keys

```
class Board(TypedDict):
    brief: str
    rules: str
    code: str
    bugs: list[str]
    attempts: int

def designer(board: Board):
    return {"rules": "Eat stars. Dodge comets."}
```

- The Designer returns one key ; the other four keep their values
- Python does not check these types at run time : a wrong type passes

demo01 · typed-dict

# Dataclass State

- A dataclass gives a key a default, so the input can leave that key out
- Nodes read `board.attempts` and still return a dictionary

```
@dataclass
class BoardDefaults:
    brief: str
    attempts: int = 0

def count_attempt(board: BoardDefaults):
    return {"attempts": board.attempts + 1}
```

- Dataclass annotations do not enforce the types either

SpiralTrain developer courses

# Pydantic State

- Pydantic checks a value when a node receives it
- Here `le=3` rejects a fourth attempt

```
class CheckedBoard(BaseModel):
    brief: str
    attempts: int = Field(ge=0, le=3)

def next_attempt(board: CheckedBoard):
    return {"attempts": board.attempts + 1}
```

- A bad write fails at the next node that reads it
- A bad last write passes : no node reads it, and `invoke` returns a plain dictionary
- Validation adds run-time work compared with `TypedDict`

SpiralTrain developer courses

# Input and Output Schemas

- **A graph can hide its working keys from the caller :**
  - `input_schema` lists the keys a caller passes in
  - `output_schema` lists the keys `invoke` returns

```
public = StateGraph(
    InternalBoard,
    input_schema=BriefIn,
    output_schema=VerdictOut,
)
```

- **The caller sends a brief and receives only a verdict**
- **The checkpoints still contain every key, `code` included**

SpiralTrain developer courses

# Private State

- A private key passes a note between two nodes and stays outside the public schema
- The node that reads the note declares the key in the type of its argument

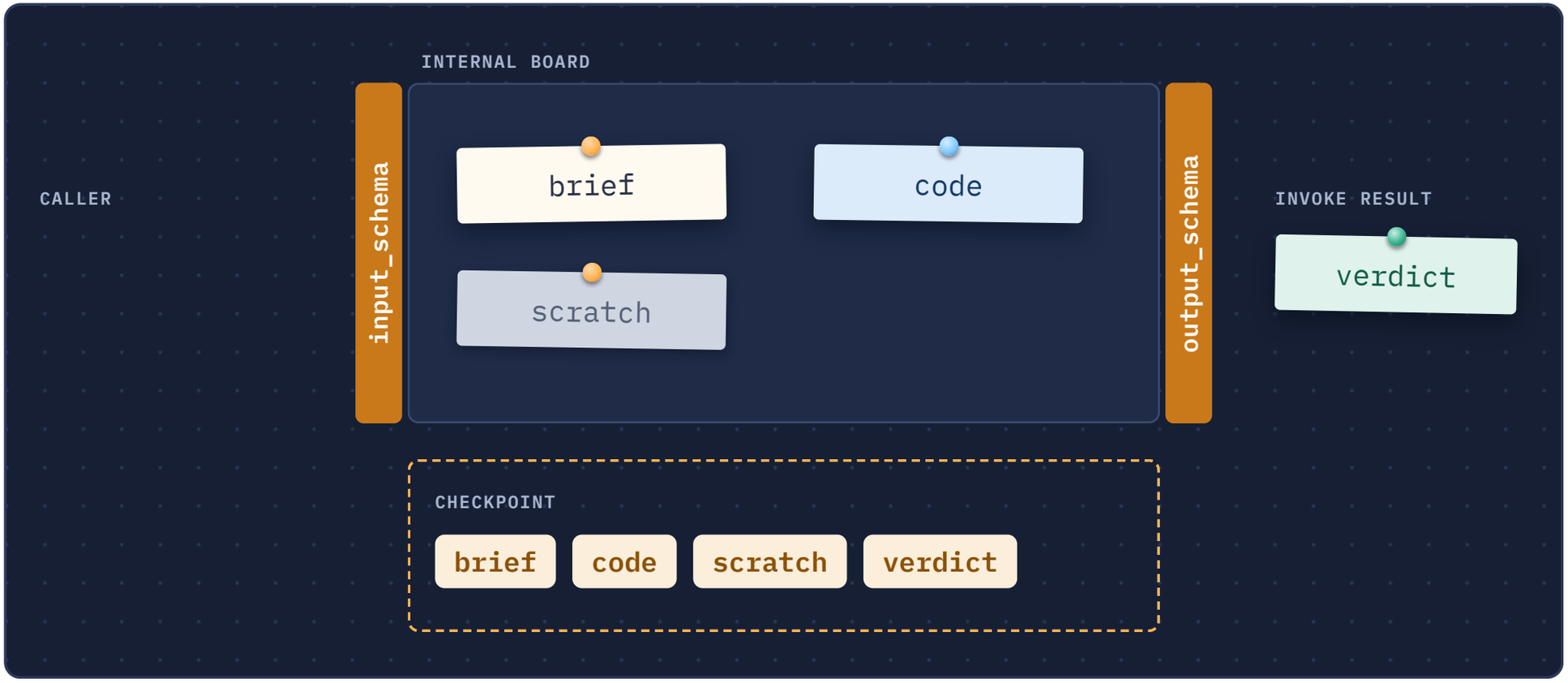
```
class Scratch(TypedDict):
    scratch: str

def tester(board: Scratch):
    return {"verdict": "retest : " + board["scratch"]}
```

- `scratch` stays out of the `invoke` result
- `scratch` still appears in checkpoints and in streamed output
- Hiding a key from the output gives no access control

demo01d · private-state

# Keys at the Edge of a Graph




The checkpoint saved all four keys, and streaming can show them. The filter changes what `invoke` returns and protects nothing.

SpiralTrain developer courses

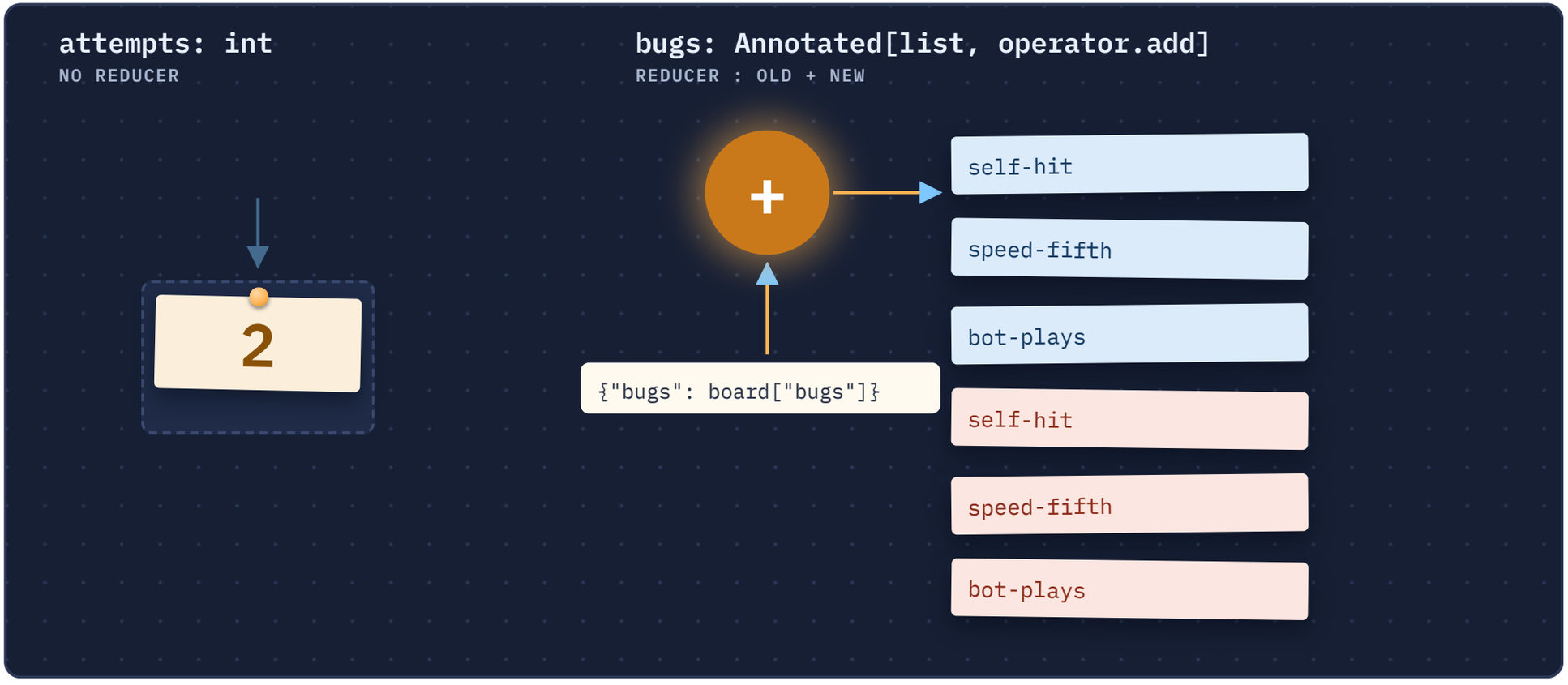
# Reducers

- **A reducer decides what happens when a node writes a key that already has a value :**
  - Without a reducer the new value replaces the old one
  - With a reducer a function combines the old value and the new one
- `Annotated` **attaches the function to the key**

```
class Collect(TypedDict):
    bugs: Annotated[list[str], operator.add]
    status: str
```

- `operator.add` **joins the saved list and the incoming list into one longer list**
- **A node returns only its new bugs**
- **A node that returns the whole list again adds every bug a second time**

# Append and Overwrite




A node that returns the whole list goes through the same reducer. Every bug is now on the list twice.

# Merging State Updates

- Each key has its own reducer, or none :

| key | reducer | saved | update | result |
|---|---|---|---|---|
| `status` | none | `"rules done"` | `"bot done"` | `"bot done"` |
| `bugs` | `operator.add` | `["self-hit"]` | `["bot-score"]` | `["self-hit", "bot-score"]` |

- The list keeps the reports of two testers, even when the reports disagree
- Deciding which report is right takes a judge or a review

SpiralTrain developer courses

# MessagesState

- A chat model needs the whole conversation on every call, so the board has a list of messages
- `MessagesState` is a ready schema with one key, `messages`, and its reducer `add_messages`
- A subclass adds extra keys, such as `game`

```
from langgraph.graph import MessagesState

class CrewState(MessagesState):
    game: str
```

- A node returns only its new message, and the reducer keeps the earlier ones
- `game` has no reducer, so a write replaces it

SpiralTrain developer courses

# add_messages

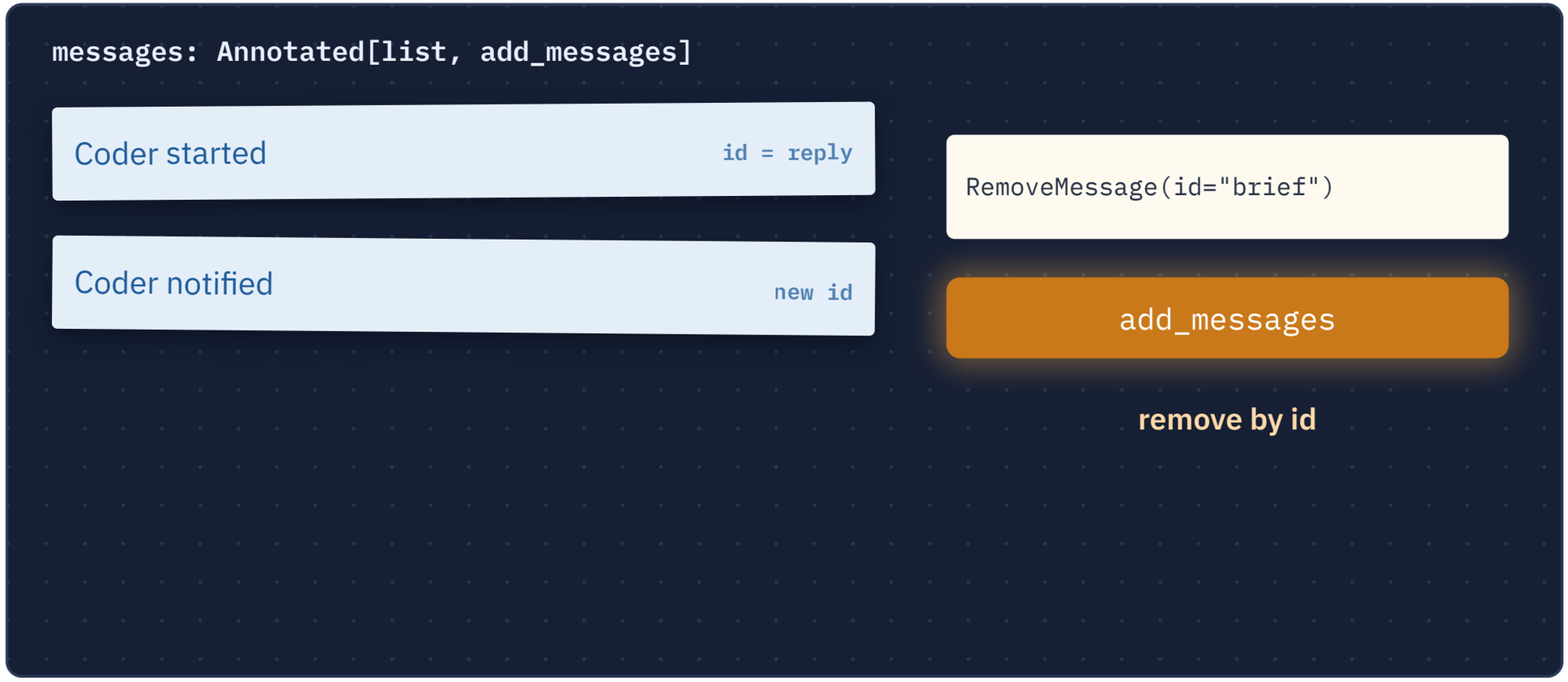
- `add_messages` **matches messages by id :**
  - A new id appends the message
  - An existing id replaces that message
  - `RemoveMessage(id=...)` deletes it

```
history = add_messages(history, [
    AIMessage("Coder started", id="reply")
])
history = add_messages(history, [RemoveMessage(id="brief")])
```

- **A message without an id gets a new one, so it appends**
- **Message dictionaries and role-content tuples become message objects**

**demo02 · add-messages**

# Messages on the Board




`RemoveMessage` deletes the matching entry, and the list closes up. The next model call receives this list.

SpiralTrain developer courses

# Custom Reducers

- A custom reducer is a function of the current value and the incoming update
- Its return value becomes the next value of the key

```
def merge_bugs(current: list[str], incoming: list[str]):
    return sorted(set(current) | set(incoming))

class Bugs(TypedDict):
    bugs: Annotated[list[str], merge_bugs]
```

- Set union keeps one copy of a bug that two testers both report
- Sorting gives a stable order
- A reducer returns a new value and leaves its arguments unchanged

demo02b · custom-reducers

# Overwriting a Reduced Key

- An ordinary update to `bugs` passes through the reducer, so an empty list clears nothing
- A new build needs an empty list : `Overwrite` skips the reducer

```
from langgraph.types import Overwrite

reset.add_node("clear", lambda state: {
    "bugs": Overwrite(value=[])
})
```

- Two `Overwrite` values for one key in one step raise `InvalidUpdateError`
- An `Overwrite` drops other branches' ordinary writes to that key in the same step, with no error

demo02c · overwriting

SpiralTrain developer courses

# Concurrent Updates

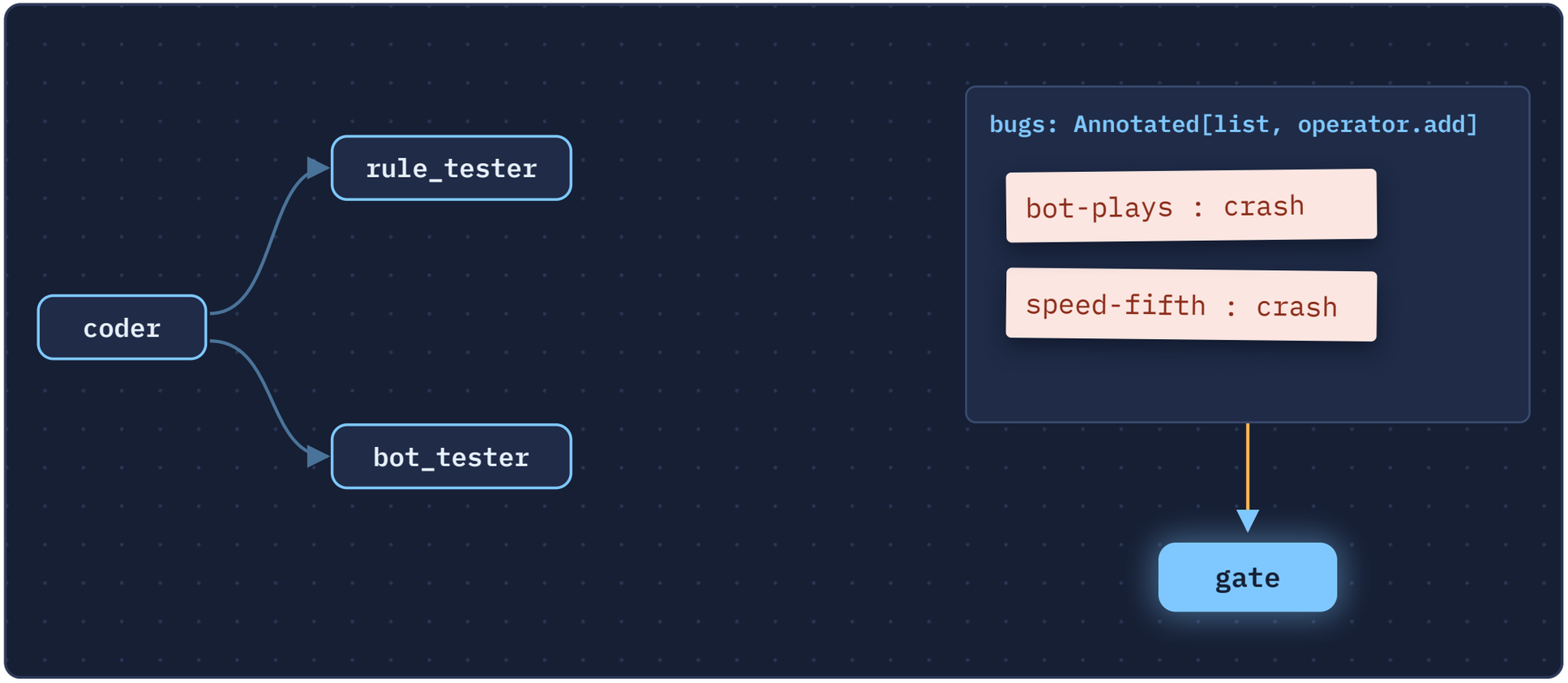
- **Nodes scheduled together run in one superstep, one round of the graph :**
  - Each reads the board as it stood when the round began
  - Their updates land on the board when all of them have finished
- **Two writes to one key without a reducer raise `InvalidUpdateError`**

```
parallel.add_edge(START, "rule_tester")
parallel.add_edge(START, "bot_tester")
```

- **A reducer lets both testers add to one bug list**
- **Separate keys suit branches that produce different results**
- **Picking the right bug from the list is a later step**

demo02d · concurrent-updates

# Two Testers in One Step




The next step reads the whole list. The reducer kept both bugs ; a later node decides what they mean.

# Checkpointers and Persistence

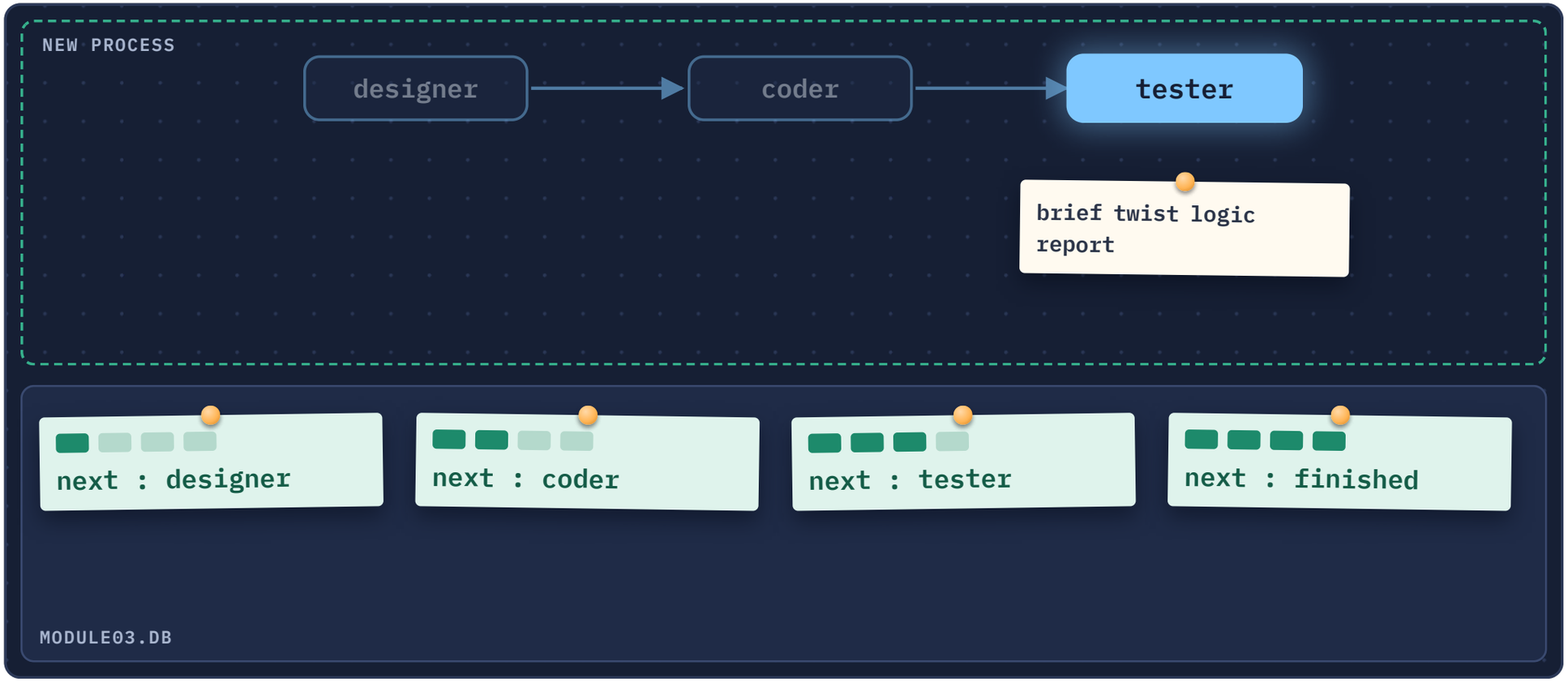
- **Without a checkpointer the board is gone when `invoke` returns**
- **A checkpointer saves the board after every step. One save is a checkpoint :**
  - The value of every key
  - The nodes that run next
  - An id and details such as the step number
- **Saves let a graph remember between calls, wait for a person and survive a restart**

```
from langgraph.checkpoint.memory import InMemorySaver

graph = builder.compile(checkpointer=InMemorySaver())
```

- **`InMemorySaver` keeps saves in memory and loses them when the process ends**

# A Crash at the Tester




`invoke(None, config)` runs the Tester and saves the result. The Designer and the Coder do not run again, so their model calls are not paid twice.

SpiralTrain developer courses

# Durability Modes

- `durability` on `invoke` sets when each save is written
- A hard kill of the process during the Tester, on a three-node graph with SQLite :

| `durability` | a save is written | saves left of 4 | resumes at |
|---|---|---|---|
| `"sync"` | before the next step starts | 4 | Tester |
| `"async"`, the default | in the background | 4 after 50 ms ; 1 at once | Tester ; restart |
| `"exit"` | when the run ends | 0 | restart |

- A model call takes seconds, so the default keeps up in a normal run
- `"exit"` leaves no history for time travel

SpiralTrain developer courses

# Threads

- A thread is the history of one game : its own line of saved boards
- `thread_id` names the thread. The same id loads that board before the new input
- A new id starts with an empty board

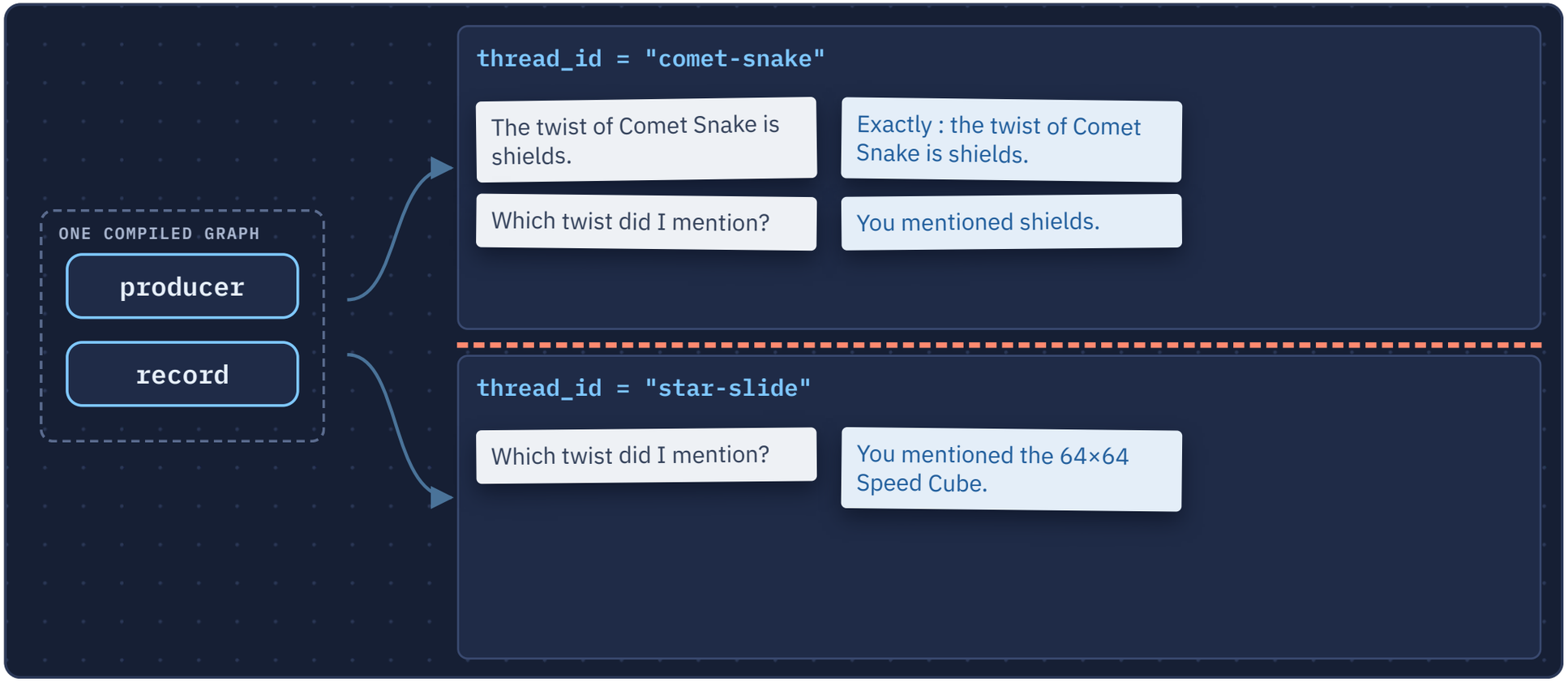
```
snake = {"configurable": {"thread_id": "comet-snake"}}
slide = {"configurable": {"thread_id": "star-slide"}}
first = {"messages": [("user", "The twist of Comet Snake is shields.")]}
question = {"messages": [("user", "Which twist did I mention?")]}

graph.invoke(first, snake)
graph.invoke(question, snake)
graph.invoke(question, slide)
```

- A thread id gives no access control ; the application decides who may read a thread
- One user can own many threads

demo03 · threads

# One Thread per Game




One graph can serve both games, and neither board can leak into the other. A thread id gives no access control : the application decides who may use an id.

SpiralTrain developer courses

# SqliteSaver

- `SqliteSaver`, from `langgraph-checkpoint-sqlite`, writes saves to a file
- The file is kept after the process ends, so a new process finds the board again
- The `with` block opens and closes the connection

```
from langgraph.checkpoint.sqlite import SqliteSaver

with SqliteSaver.from_conn_string(str(database)) as saver:
    crew = builder.compile(checkpointer=saver)
    crew.invoke(inputs, config)
```

- Every graph call that uses the saver belongs inside the `with` block
- SQLite suits local development and small applications

demo03b · sqlite-saver

# PostgreSQL Checkpoint Storage

- `langgraph-checkpoint-postgres` **provides a PostgreSQL backend**
- **Several application processes can share one PostgreSQL checkpoint store**
- **Production operation includes :**
  - Connection management and schema setup
  - Backups and checkpoint retention
  - Access controls for thread data
- **The checkpointer saves execution state within a thread**
- **Cross-thread application memory uses a Store**

# Restoring Saved State

- A new process compiles the same graph on the same file
- `get_state` shows the saved board ; `next` names the node that was due
- `invoke(None, config)` runs that node and continues to the end

```
with SqliteSaver.from_conn_string(str(database)) as saver:
    crew = builder.compile(checkpointer=saver)
    print(crew.get_state(config).next)    # ('tester',)
    crew.invoke(None, config)
```

- The config has the same thread id, and the graph code must still match the save
- New input in place of `None` applies a new update

# Inspecting Thread State

- `get_state` returns a `StateSnapshot`, one saved board, and runs no node

```
snapshot = graph.get_state(snake)
print(snapshot.values)
print(snapshot.next)
print(snapshot.config["configurable"]["checkpoint_id"])
```

- `values` contains the saved keys
- `next` names the nodes due next ; an empty `next` means the run finished
- A config with a `checkpoint_id` selects an older save

SpiralTrain developer courses

# State History

- `get_state_history` lists the saves of a thread, newest first
- Each snapshot has its own config and points to its parent save

```
history = list(graph.get_state_history(config))
for snapshot in history:
    print(snapshot.metadata["step"], snapshot.next)

before_coder = next(
    snapshot for snapshot in history
    if snapshot.next == ("coder",)
)
```

- The config of a snapshot selects the point to go back to

demo04 · state-history

SpiralTrain developer courses

# Time Travel

- Time travel runs a graph again from an older save
- Replay passes `None` and the config of that save

```
replayed = graph.invoke(None, before_coder.config)
```

- The nodes before the save keep their saved results
- The nodes after it run again : the Coder is a new model call, and the code can differ
- A node with an effect outside the graph repeats that effect : a replayed release ships twice
- A save with an empty `next` has nothing to replay

demo04b · replay

SpiralTrain developer courses

# Forking a Checkpoint

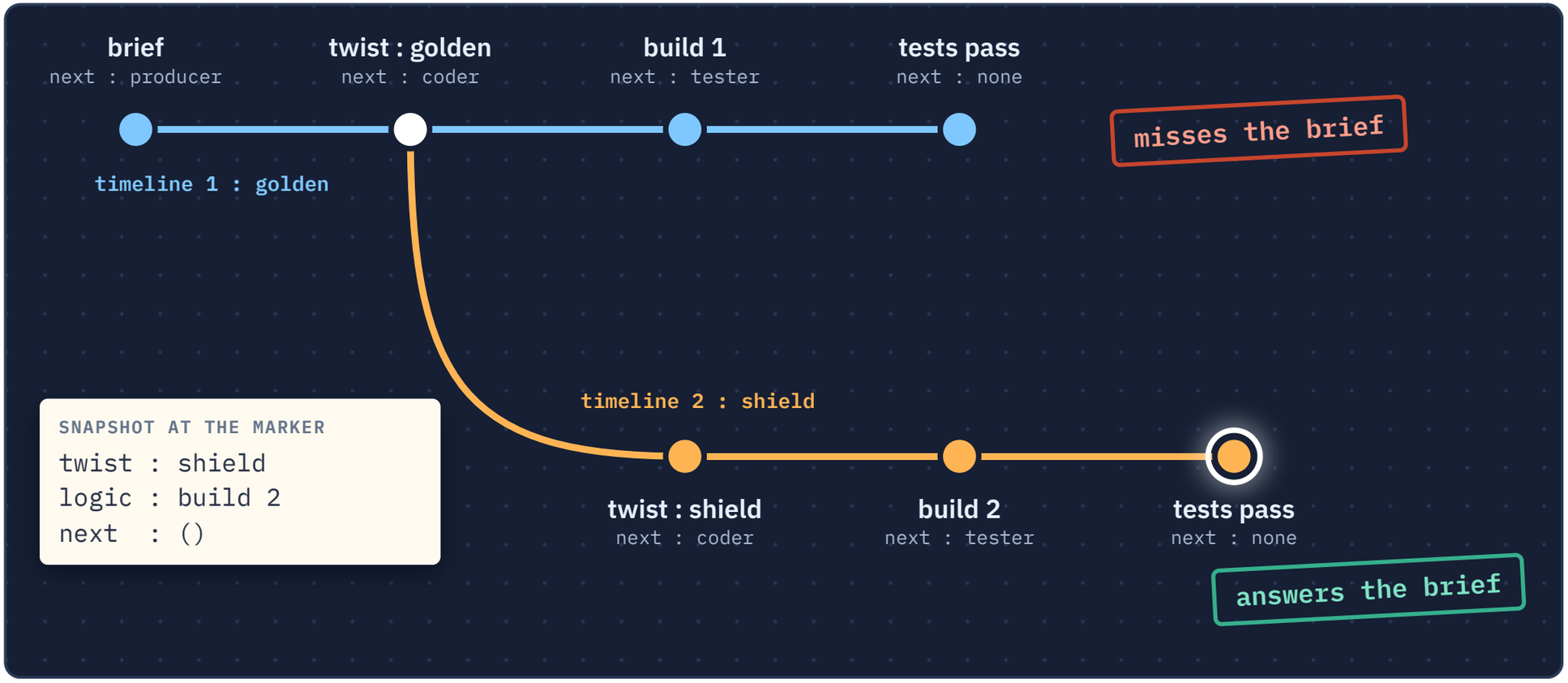
- A fork changes one value at an older save and runs on from there
- `update_state` writes a new checkpoint next to the old one
- `as_node` says which node the change counts as, and so what runs next

```
forked = graph.update_state(
    before_coder.config, {"twist": "shield"}, as_node="producer")
alternative = graph.invoke(None, forked)
```

- Updates pass through the same reducers as node updates
- The original save stays, and the old timeline can still be read
- A fork does not recall a game that the old timeline already released

demo04c · forking

# Rolling Back a Build




Two timelines on one thread. The golden build can still be read, and anything it did outside the graph, such as a release, stays done.

# Time Travel in Code

```
history = list(graph.get_state_history(config))
before_coder = next(snapshot for snapshot in history
                    if snapshot.next == ("coder",))
forked = graph.update_state(before_coder.config, {"twist": "shield"},
                            as_node="producer")
alternative = graph.invoke(None, forked)
```

List the saves of the thread, newest first.
Find the save where the Coder is the next node.
Write the new twist as a new save, as if the Producer had picked it.
**Run on from the new save : the Coder and the Tester run again.**

# DeltaChannel

- `DeltaChannel` is beta in LangGraph 1.2.17
- Between full snapshots, LangGraph rebuilds a growing value from the saved writes
- A delta reducer receives the accumulated value and a batch of writes

```
def merge_batches(current, writes):
    return current + [item for batch in writes for item in batch]

class LongThread(TypedDict):
    entries: Annotated[list[str], DeltaChannel(merge_batches)]
```

- Reducers must be deterministic and give one result however the writes are batched
- A full snapshot is stored every 1,000 updates (`snapshot_frequency`) or 5,000 supersteps
- Delta storage does not shorten the model's context

SpiralTrain developer courses

# Long-Term Memory

- A checkpoint belongs to one thread, so a new game starts on an empty board
- A Store keeps data across threads, such as the studio style guide
- A namespace, a path of labels, groups documents ; a key names one document

```
store = InMemoryStore()
style = ("studio", "comet-arcade", "style")
store.put(style, "danger", {"text": "Danger is warm : use #ff6b5e."})
print(store.get(style, "danger").value)
```

- `put` with the same namespace and key replaces the document ; `delete` removes it
- `search` returns up to 10 items ( `limit` ) under a namespace prefix
- Labels cannot contain `.` , so an e-mail address used as a user id needs encoding first
- A node reaches the Store through `runtime.store` , and the application picks the namespaces it may read

demo05 · the-store

# Short-Term and Long-Term Memory




The danger sprite of Moon Miner comes out in the studio colour. In the demo the model replied `#ff6b5e` on both games.

SpiralTrain developer courses

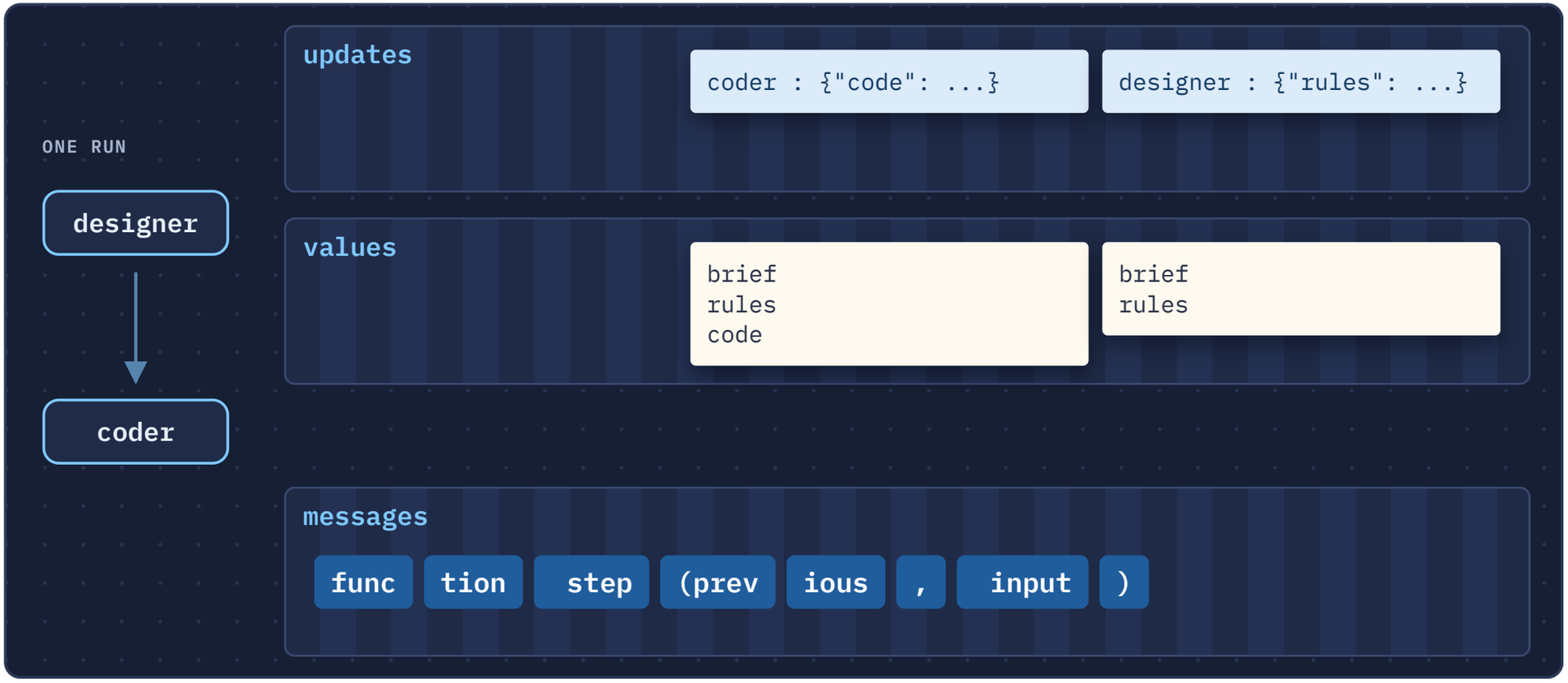
# Streaming Modes

- `stream` **returns what a run produces while it runs**
- `stream_mode` **chooses what comes out :**
  - `updates` : the name of a node and the keys it returned
  - `values` : the whole board after each step
  - `messages` : the model's reply, token by token
  - `custom` : anything a node sends through `get_stream_writer`

```
for part in graph.stream(inputs, config,
                         stream_mode=["values", "updates"], version="v2"):
    print(part["type"], part["data"])
```

demo06 · stream-modes

# One Run in Stream Modes




`updates` shows who wrote what. `values` shows the resulting board. `messages` gives a person something to read while the run goes on.

SpiralTrain developer courses

# Debugging State

- `debug` **combines checkpoint and task events :**
  - Checkpoint values and scheduled work
  - Task starts and completions, including results and errors
- `checkpoints` **and** `tasks` **select those events separately**
- **Checkpoint events require a checkpointer**
- `updates` **shows which node wrote a value**
- `values` **shows the result after reducers merged updates**
- **Studio shows the graph and its state in a visual interface**

# The v2 Stream Format

- `stream(..., version="v2")` emits every part in one dictionary format
- `type` names the stream mode ; `data` is its payload
- `ns` identifies the graph namespace

```
for part in graph.stream(inputs, config,
                         stream_mode=["custom", "updates"], version="v2"):
    print(part["type"], part["ns"], part["data"])
```

- Root graph events use an empty namespace tuple
- Values events also contain interrupt information
- The default v1 format varies with modes and subgraph settings

# v3 Event Projections

- `stream_events(..., version="v3")` **returns a** `GraphRunStream`
- **Experimental on 1.2.17 ; the docs recommend it for new application code**
- **Projections select typed views of one run :**
  - `values` yields state snapshots
  - `messages`, `lifecycle` and `subgraphs` expose other views

```
run = small_graph.stream_events({"attempts": 0}, version="v3")
for state in run.values:
    print(state)
print(run.output)
```

- **Iteration drives the synchronous run without a background thread**
- **v3 sets stream modes and subgraph streaming itself, so** `stream_events` **takes neither as an argument**

SpiralTrain developer courses

# Summary : State Management

- The board is one dictionary ; a schema names its keys and a node returns only what it changed
- A reducer combines a new write with the saved value ; without one the write replaces it
- Parallel writes to one key need a reducer or separate keys
- A checkpointer saves the board after every step, so a crash at the Tester restarts at the Tester
- A thread keeps one game’s saves apart from every other game
- Time travel reruns from an older save ; a fork changes a value there and leaves the old timeline in place
- A Store keeps memory across threads, such as the studio style guide
- Stream modes show who wrote what while the run goes on

exercise02 · State for Two Agents

# Building Agents

From LLM to Agent
Introduction to LangGraph
Graph Fundamentals
State Management
**Building Agents**
Multi-Agent Patterns
Agent Communication
Advanced Workflows
Deep Agents
Production Deployment
Real-World Applications

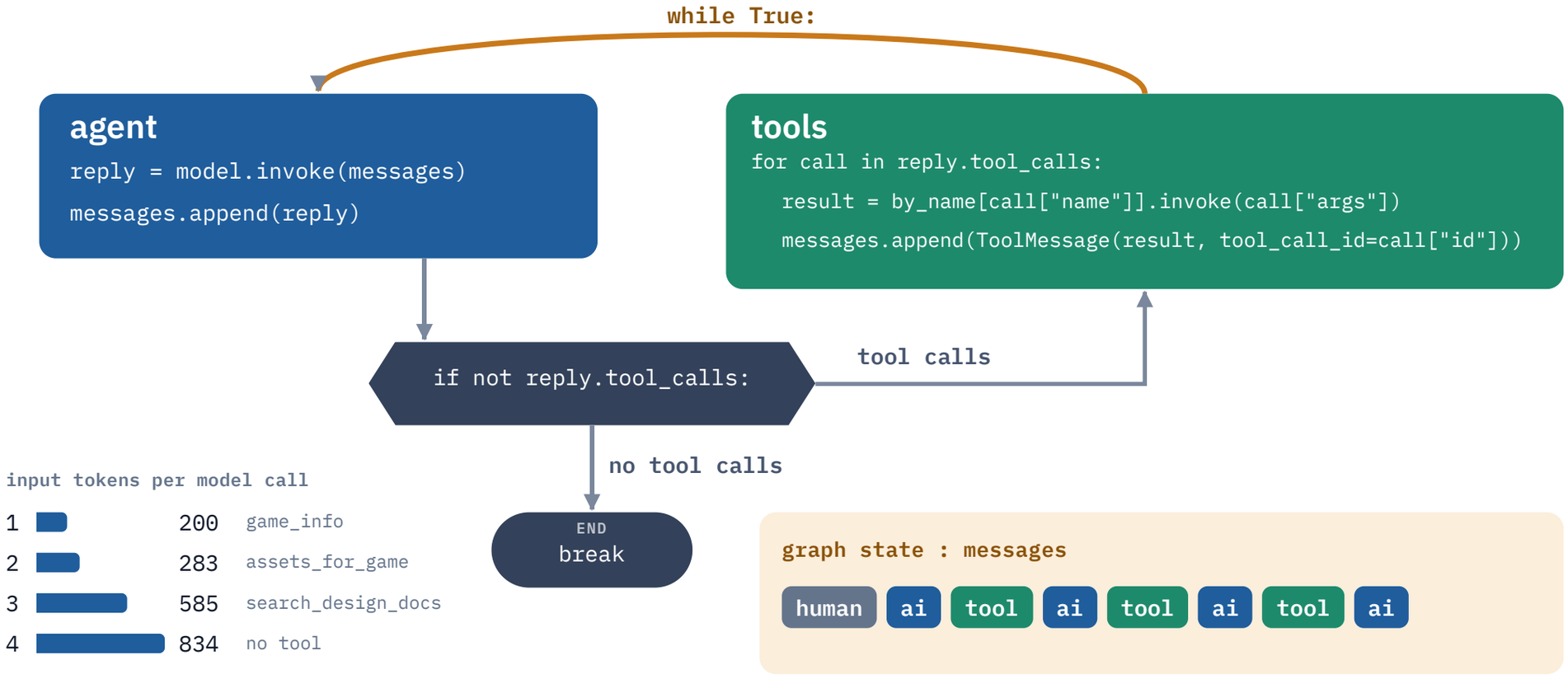
Tool-Calling Agents
Models and Messages
Defining Tools
Agent Single Pass
Agent Loop
ToolNode
tools_condition
ReAct Agent
create_agent
An Agent as a Node
Custom Agent Logic
Custom Agent State
A Crew as Tools
Middleware Hooks
Writing Logging Middleware
Failure Modes of an Agent
Fault Tolerance
Node Run Timeout
Node Idle Timeout
Node Error Handler
Tool Errors
Model Fallback
Model Call Limit
Tool Call Limit
LangSmith Tracing
Testing Strategies
Unit Tests with a Fake Model
Testing Memory
Running pytest
Best Practices
Summary : Building Agents

SpiralTrain developer courses

# Tool-Calling Agents

- **Module 00 wrote an agent as a `while` loop around a model :**
  - The model reads the messages and the tool list, then asks for a tool or answers
  - Python runs the tool and adds the result to the messages
  - A reply without a tool call ends the loop
- **A graph runs the same loop, one node per job :**
  - A crash in the tool step restarts at the tool step, from the saved state
  - Checkpoints from module 03 and the human approval of module 07 work on the loop without new code
- **One question runs through this module, the one from module 00 :**
  - Playtesters lose Comet Snake to comets they cannot see. Which asset is at fault?

# From Loop to Graph




Calls 2 and 3 fetch the assets and the style guide. Call 4 reads about 830 tokens, asks for no tool and ends the run.

# Models and Messages

- **A chat model accepts a list of messages and returns an `AIMessage`**
- **Module 00 used four kinds of message :**
  - `SystemMessage` contains the standing instructions
  - `HumanMessage` contains the request
  - `AIMessage` contains the model's text and any `tool_calls`
  - `ToolMessage` contains a result, linked to its request by `tool_call_id`
- **`llm.bind_tools(tools)` sends the tool descriptions with every call**
- **Binding runs nothing : the reply only asks for a tool**

```
bound = llm.bind_tools(tools)
message = bound.invoke([("human", "What genre is Comet Snake?")])
print(message.content, message.tool_calls)
```

SpiralTrain developer courses

# Defining Tools

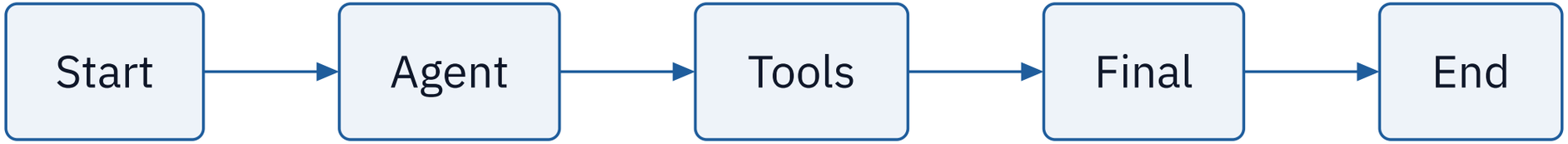
- `@tool` **turns a Python function into the description the model reads :**
  - The function name becomes the tool name
  - Type hints describe the arguments
  - The docstring tells the model when to use it
- **The `?` in the SQL keeps a title from being read as SQL**
- **A tool returns `Unknown game` when no game matches, so the model has nothing to guess**

```
@tool
def game_info(title: str) -> str:
    """Look up a Comet Arcade game by its title."""
    with sqlite3.connect(DB) as conn:
        row = conn.execute(
            "SELECT game_id, genre, status FROM games WHERE title = ?",
            (title,)).fetchone()
    return json.dumps(dict(zip(["game_id", "genre", "status"], row))) if row else "Unknown game"
```

SpiralTrain developer courses

# Agent Single Pass

- **A graph with no way back gives the model one chance to choose tools**
- **The lookups of the Comet Snake question depend on each other :**
  - `game_info` returns the game id, `CS-01`
  - `assets_for_game` needs that id
  - The style guide search comes after the assets
- **The saved run calls `game_info`, then has to answer**
- **Its answer names the asset and style guide lookups as missing**



SpiralTrain developer courses

# Agent Loop

- An edge from `tools` back to `agent` gives the model a turn after each result
- The model reads each result before it picks the next tool
- A routing function reads the last message :
  - Tool calls present : go to `tools`
  - No tool calls : end the graph
- The saved run of the loop makes 4 model calls, as the `while` loop did, and names the comet sprite, `CS-SPR-03`

```
loop.add_node("agent", call_model)
loop.add_node("tools", run_tools)
loop.add_edge(START, "agent")
loop.add_conditional_edges("agent", route)
loop.add_edge("tools", "agent")
result = loop.compile().invoke(question, {"recursion_limit": 16})
```

demo01 · react-as-a-graph

SpiralTrain developer courses

# ToolNode

- `ToolNode` **is the** `run_tools` **function of the loop, ready-made :**
  - Finds the registered tool for each call in the last message
  - Checks the arguments against the tool's description
  - Returns one `ToolMessage`, the message type for a tool result, per call, with the same call id
- **Several calls in one reply can run in parallel**
- **A tool can return a** `Command` **to update custom state or route the graph**
- **An exception, Python's signal for an error, stops the run unless a handler catches it**

```
from langgraph.prebuilt import ToolNode

builder.add_node("tools", ToolNode(tools))
```

SpiralTrain developer courses

# tools_condition

- `tools_condition` **is the** `route` **function of the loop, ready-made :**
  - Tool calls in the last message : returns `"tools"`
  - No tool calls : returns `"__end__"`
- **The default route expects a node named** `tools`
- **A path map sends the route to a node with another name**

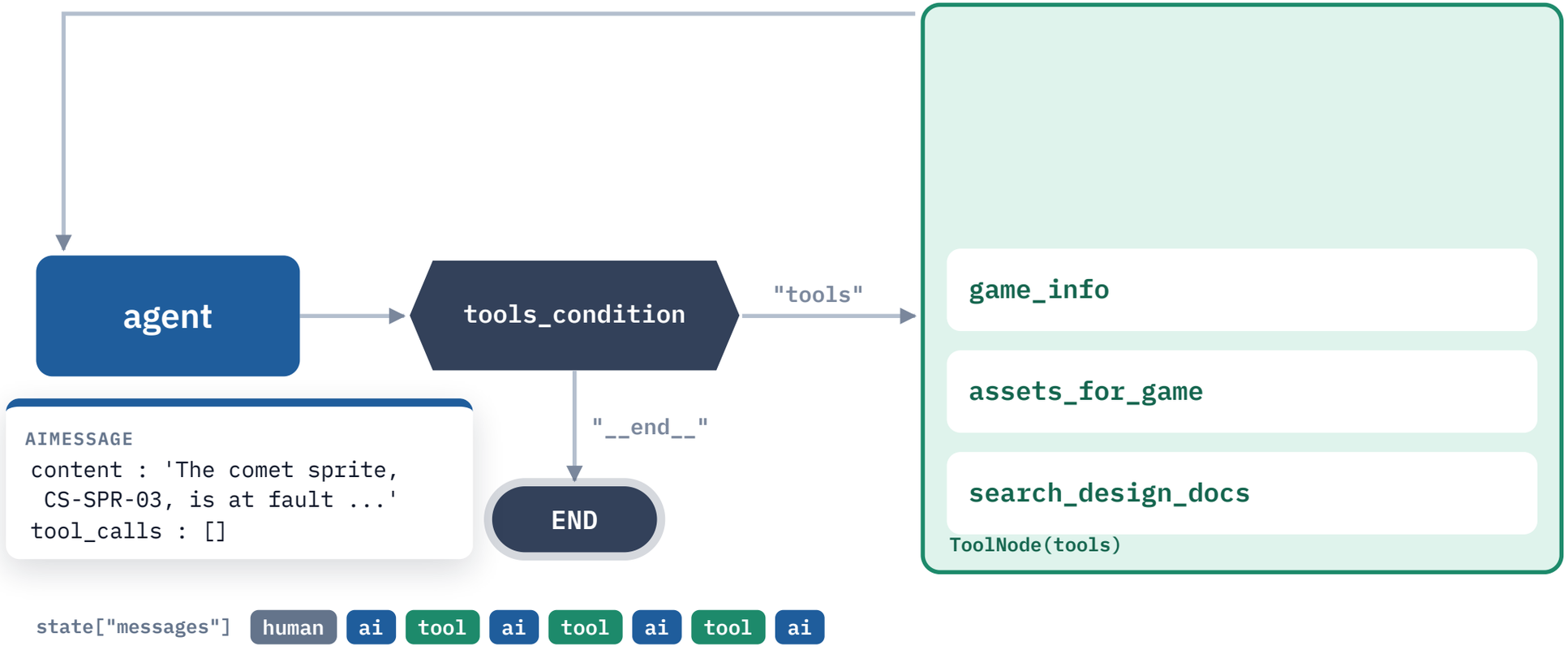
```python
from langgraph.prebuilt import tools_condition

builder.add_conditional_edges("agent", tools_condition)
builder.add_edge("tools", "agent")
graph = builder.compile()
```

demo01b · toolnode-and-tools-condition

SpiralTrain developer courses

# Routing a Tool Call

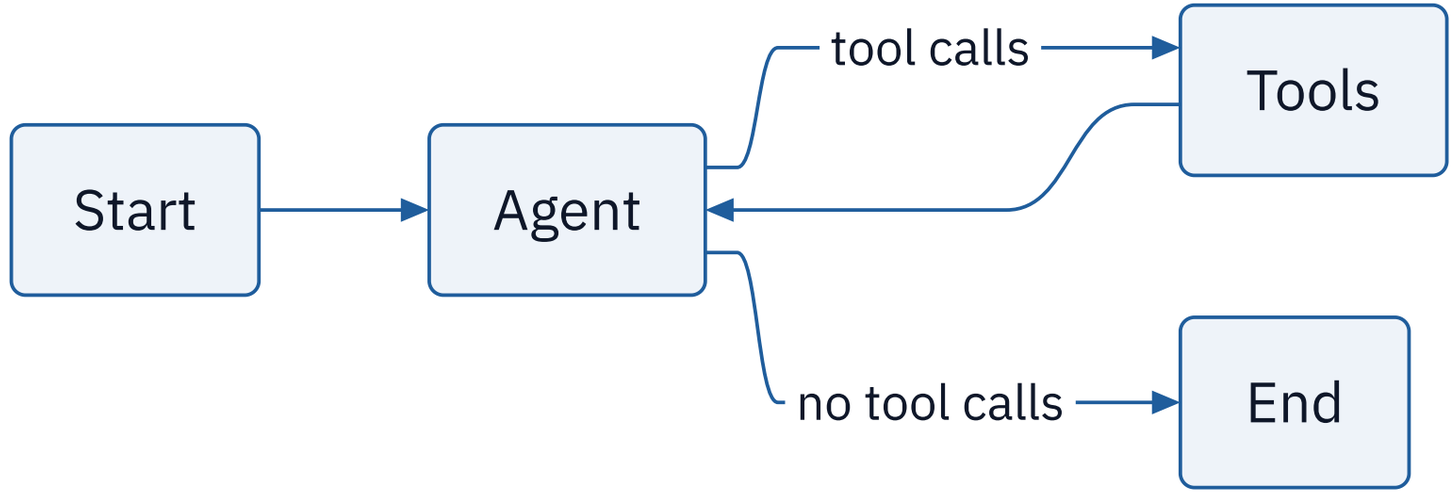



The run ends with 4 model calls. Neither prebuilt part ever looks at the text of the answer.

SpiralTrain developer courses

# ReAct Agent

- **ReAct is the 2022 paper behind this loop : reason, act, observe, repeat**
- **A tool-calling agent follows the same order :**
  - The model selects a tool
  - The tool returns an observation
  - The model uses the observation in its next decision
- **The graph records each action and observation as messages**
- **A custom node can force an order, such as the game lookup before the assets**



SpiralTrain developer courses

# create_agent

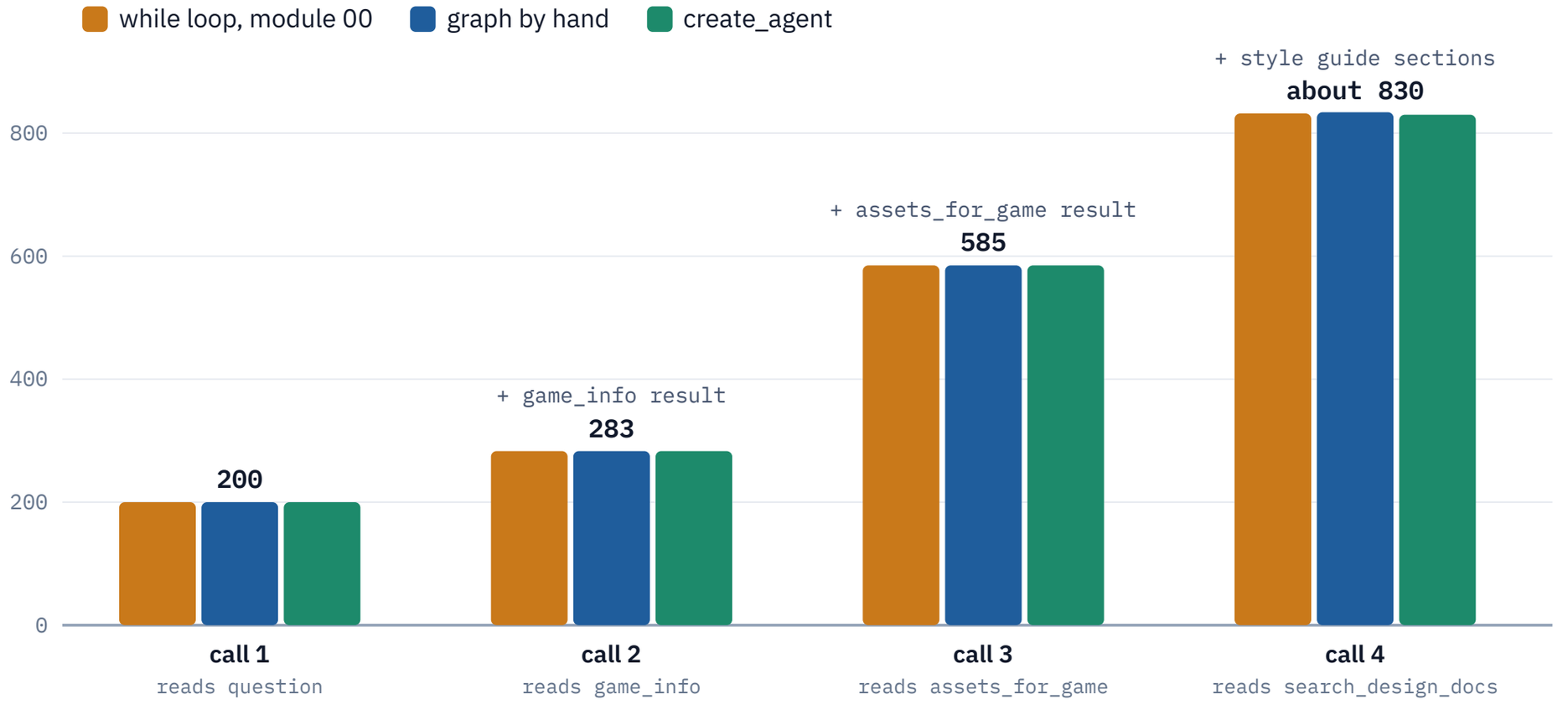
- `create_agent` **builds the loop from a model and a tool list :**
  - A node named `model` chooses tools or answers
  - A node named `tools` runs them
  - Middleware, extra code the application supplies, runs around both
- **The result is a compiled LangGraph graph**
- `invoke` **returns the final state, with every message**

```
from langchain.agents import create_agent

agent = create_agent(llm, tools=tools,
    system_prompt="You are the studio assistant of Comet Arcade. Use the tools.")
result = agent.invoke(question, {"recursion_limit": 16})
print(result["messages"][-1].content)
print(list(agent.get_graph().nodes))
```

demo02 · create-agent

# One Loop in Three Forms




Each step adds its tool result to the context, and every call pays for all of it again : about 1,900 input tokens for one question.

SpiralTrain developer courses

# An Agent as a Node

- A compiled agent can be one node of a larger graph
- Keys the parent and the agent share, such as `messages`, pass in and out
- The parent decides what happens around the agent :
  - The `handover` node always runs after the agent answers
  - It writes the note for the Artist, with no model involved
- An agent with different keys needs a small function that translates between the two

```
parent = StateGraph(StudioState)
parent.add_node("studio_agent", agent)
parent.add_node("handover", handover)
parent.add_edge(START, "studio_agent")
parent.add_edge("studio_agent", "handover")
parent.add_edge("handover", END)
```

demo02b · agent-as-a-node

SpiralTrain developer courses

# Custom Agent Logic

- **A custom model node decides the prompt and the messages the model sees**
- **A prompt states the goal and the allowed moves :**
  - Look up the game before its assets
  - Report a missing record as missing
  - Leave the release decision to the Publisher
- **An edge guarantees a step ; a prompt only asks for it :**
  - The prompt asks for the game lookup first
  - Only an edge makes the lookup happen every time

```
def call_model(state: MessagesState):
    return {"messages": [bound.invoke([prompt, *state["messages"]])]}
```

# Custom Agent State

- `AgentState` **contains the messages, and a subclass can add fields :**
  - `game_id` , the game this team works on
  - `model_calls` , a counter that middleware updates
- **Middleware reads and updates those fields**
- `ToolRuntime` **passes state to a tool, and the model never sees that argument**

```
class GameState(AgentState):
    game_id: str
    model_calls: NotRequired[int]

game_agent = create_agent(llm, tools=[open_bugs],
    state_schema=GameState, middleware=[game_prompt, count_calls])
```

demo02c · custom-agent-state

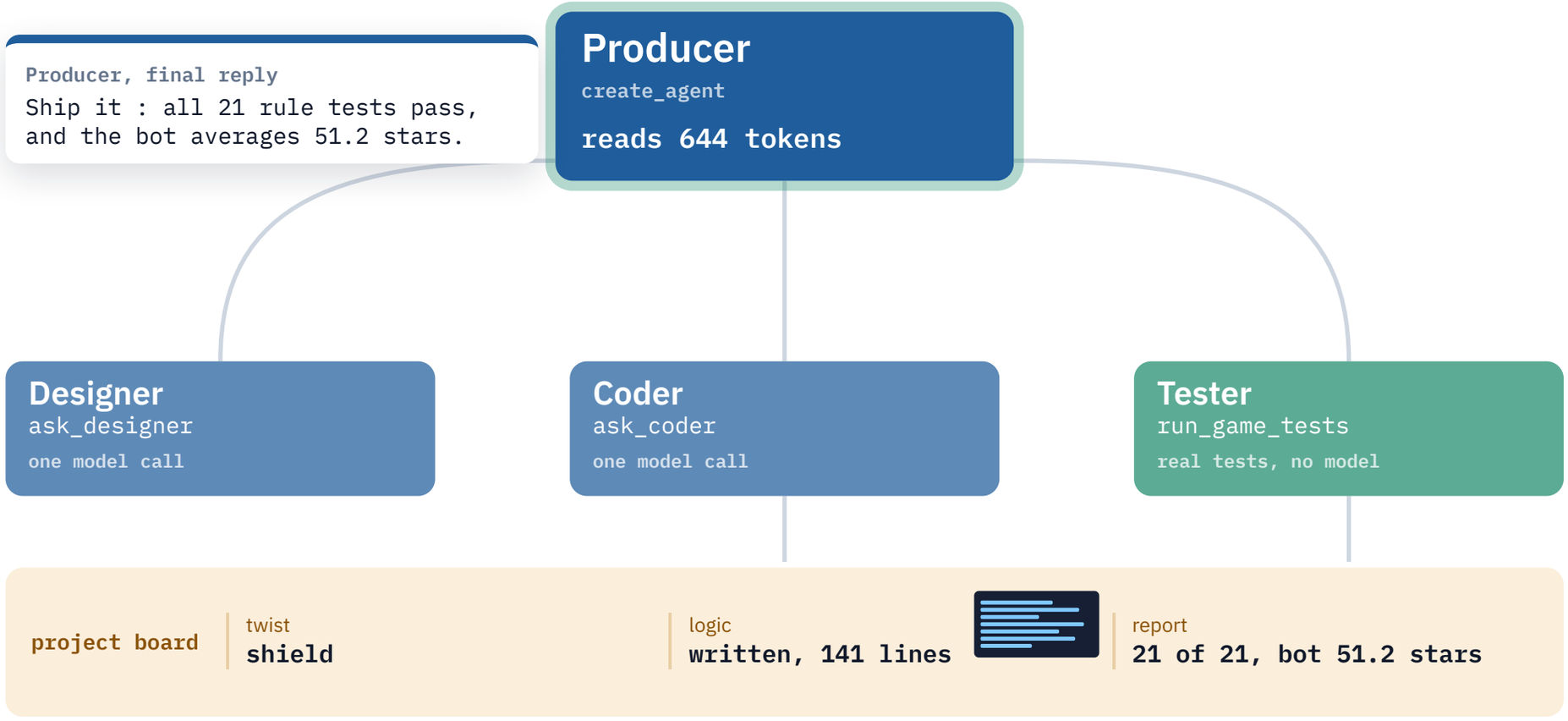
# A Crew as Tools

- **The Producer of module 01 is one `create_agent` :**
  - `ask_designer` and `ask_coder` each make one fresh model call
  - `run_game_tests` runs the rule tests and the bot, with no model
- **8 of 8 measured runs finished a game that passed its tests, in 6 model calls**

```
@tool
def ask_designer(brief: str) -> str:
    """Send a brief to the Designer. Returns the chosen twist and its rules."""
    reply = crew_llm.invoke([SystemMessage(DESIGNER), HumanMessage(brief)])
    return reply.content

producer = create_agent(crew_llm, tools=[ask_designer, ask_coder, run_game_tests],
                        system_prompt=PRODUCER)
```

# The Producer at Work




6 model calls : 4 for the Producer, 2 for the crew. The Producer stops at 644 tokens, because the code never entered its context.

SpiralTrain developer courses

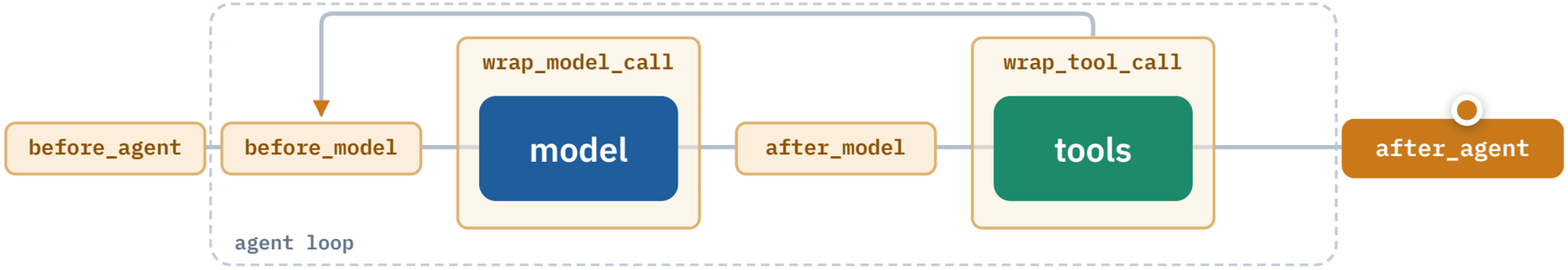
# Middleware Hooks

- Middleware is code that runs at fixed points of the agent loop, called hooks

| hook | runs |
|---|---|
| `before_agent` | once, before the loop |
| `before_model` | before each model call |
| `wrap_model_call` | around each model call |
| `after_model` | after each model call |
| `wrap_tool_call` | around each tool call |
| `after_agent` | once, after the loop |

- A `before` or `after` hook can read the state and return updates
- A `wrap` hook decides whether the call happens at all

# Hooks Around the Model Call




demo03 log

```
agent: start
model input messages: 1
requested tools: ['game_info']
tool: game_info {'title': 'Comet Snake'}
result: {"game_id": "CS-01", "title": "Comet Snake", ...
model input messages: 3
... two more laps ...
requested tools: []
agent: end
```

`after_agent` fires once. Six hooks, and the two nodes of the agent never changed.

# Writing Logging Middleware

- A middleware class can print at every model turn
- A decorated wrapper can print tool arguments and results
- The saved log shows `game_info`, `assets_for_game`, `search_design_docs`, in order
- A tool argument can contain personal data, so check what a log keeps
- Each hook adds graph steps : a `recursion_limit` of 16 stopped this agent, so it gets 30

```
class Logger(AgentMiddleware):
    def before_model(self, state, runtime):
        print("model input messages:", len(state["messages"]))

@wrap_tool_call
def log_tool(request, handler):
    print("tool:", request.tool_call["name"], request.tool_call["args"])
    result = handler(request)
    print("result:", result.content[:100])
    return result
```

demo03 · middleware-hooks

SpiralTrain developer courses

# Failure Modes of an Agent

| what goes wrong | Comet Arcade example | what handles it |
|---|---|---|
| A node crashes | The asset store drops the connection | `RetryPolicy` , `error_handler` |
| A tool crashes | The playtest server times out | `ToolErrorMiddleware` |
| The model is down | The provider refuses every call | `ModelFallbackMiddleware` |
| The loop never ends | `build_status` keeps saying `still building` | `ModelCallLimitMiddleware` |

- **A retry suits a failure that passes by itself**
- **A tool error goes to the model as a message, so the model can report it**
- **A limit stops a run that no error would ever stop**

SpiralTrain developer courses

# Fault Tolerance

- A retry policy runs a failed node again
- `max_attempts` includes the first attempt
- `retry_on` selects the exceptions worth a retry
- The retries come first ; the node error handler runs after the last one fails
- The demo node fails twice and fetches the sprites on attempt 3
- A retried upload must not publish the build twice

```
from langgraph.types import RetryPolicy

builder.add_node("fetch_sprites", fetch_sprites, retry_policy=RetryPolicy(
    max_attempts=3, initial_interval=0.01,
    jitter=False, retry_on=ConnectionError))
```

demo04 · node-failures

SpiralTrain developer courses

# Node Run Timeout

- Node `timeout` limits one async attempt
- A number sets a run timeout in seconds
- `TimeoutPolicy.run_timeout` caps the wall-clock time of the attempt
- Progress does not extend the run limit
- A timeout raises `NodeTimeoutError` and drops the writes of that attempt
- Compiling a graph rejects a timeout on a synchronous node

```
await timed(slow_fetch, TimeoutPolicy(run_timeout=0.08))
```

SpiralTrain developer courses

# Node Idle Timeout

- `idle_timeout` limits the time without visible progress
- The refresh policy selects what counts as progress :
  - `auto` accepts graph and LangChain callback progress
  - `heartbeat` accepts explicit `runtime.heartbeat()` calls
- A heartbeat refreshes the idle limit and leaves the run limit alone
- With both limits set, the first one reached cancels the attempt

```
await timed(progressing_fetch,
    TimeoutPolicy(run_timeout=0.08, idle_timeout=0.08,
                  refresh_on="heartbeat"))
```

demo04b · node-timeout

SpiralTrain developer courses

# Node Error Handler

- `error_handler` runs after the retries are used up, or when no retry applies
- The handler receives the state and a typed `NodeError` :
  - `error.node` names the failed node
  - `error.error` is the exception
- A returned `Command` updates state and picks the next node
- The demo builds with placeholder sprites when the asset store stays down
- An error raised by the handler fails the run

```
def on_error(state: BuildState, error: NodeError) -> Command:
    return Command(update={"status": "placeholder sprites"},
                   goto="placeholders")

builder.add_node("fetch_sprites", unavailable,
    error_handler=on_error, retry_policy=RetryPolicy(max_attempts=2,
        retry_on=ConnectionError, initial_interval=0.01, jitter=False))
```

demo04c · error-handler

SpiralTrain developer courses

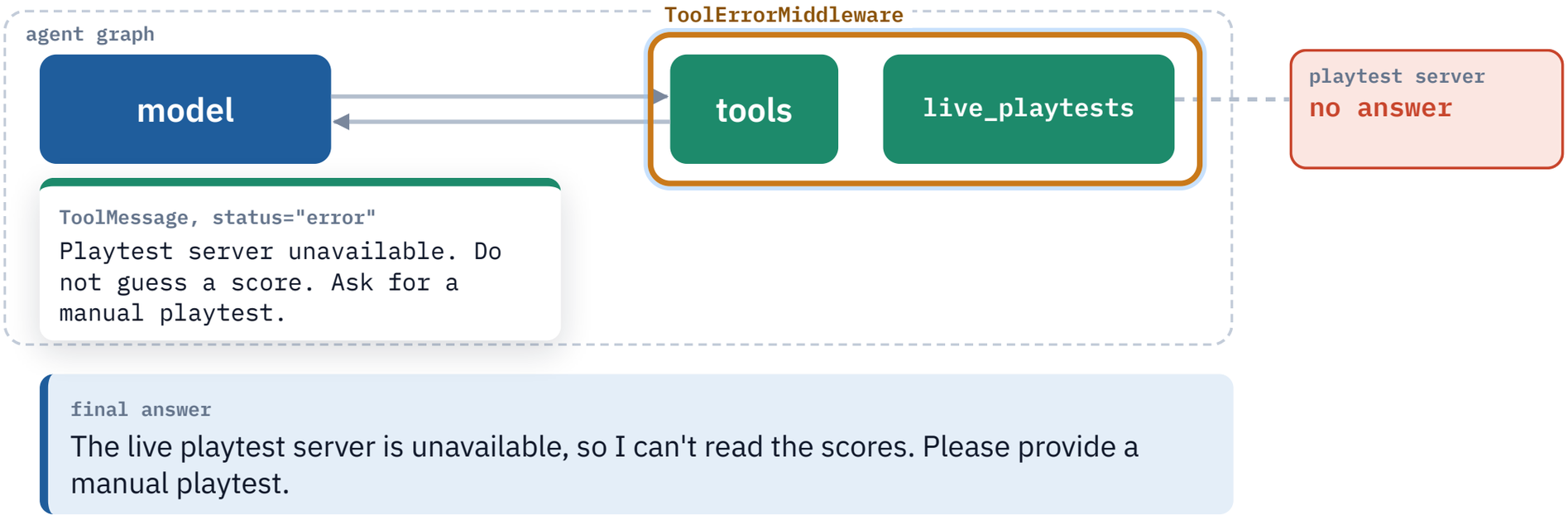
# Tool Errors

- **An exception inside a tool stops the whole agent**
- **`ToolErrorMiddleware` turns chosen exceptions into an error `ToolMessage`:**
  - Text returned by `on_error` goes to the model as the tool result
  - `None` returned by `on_error` lets the exception through
- **The model reads the error and reports it in its answer**
- **`ToolNode` handles wrong tool arguments before this middleware is reached**

```
def on_error(exc, request):
    if isinstance(exc, TimeoutError):
        return "Playtest server unavailable. Do not guess a score. Ask for a manual playtest."
    return None

safe = create_agent(llm, tools=[live_playtests],
                    middleware=[ToolErrorMiddleware(on_error)])
```

# A Tool Error Reaches the Model




The model reports the outage and invents no score. The error text told it not to guess.

SpiralTrain developer courses

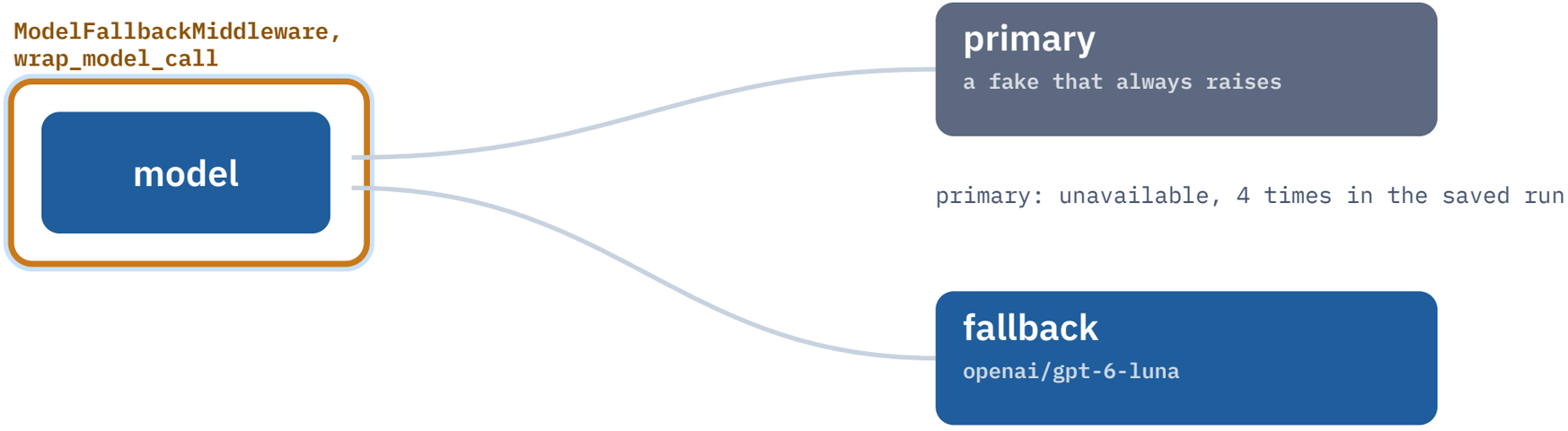
# Model Fallback

- `ModelFallbackMiddleware` tries the next model when a model call raises
- `create_agent` receives the primary model ; the middleware receives the fallbacks in order
- The primary is tried again on every turn of the loop
- If every model fails, the last exception reaches the caller
- A fallback needs tool calling and the same request settings as the primary

```
agent = create_agent(primary, tools=tools,
    middleware=[ModelFallbackMiddleware(llm)],
    system_prompt="You are the studio assistant of Comet Arcade. Use the tools.")
out = agent.invoke(question, {"recursion_limit": 16})
```

demo05b · model-fallback

# A Fallback Model Takes Over




turns of the loop

| | | | |
|---|---|---|---|
| 1 primary failed<br>luna : game_info | 2 primary failed<br>luna : assets_for_game | 3 primary failed<br>luna : search_design_docs | 4 primary failed<br>luna : answer |

A fallback catches exceptions only. If the last model raises too, the exception reaches the caller, and a wrong answer passes straight through.

SpiralTrain developer courses

# Model Call Limit

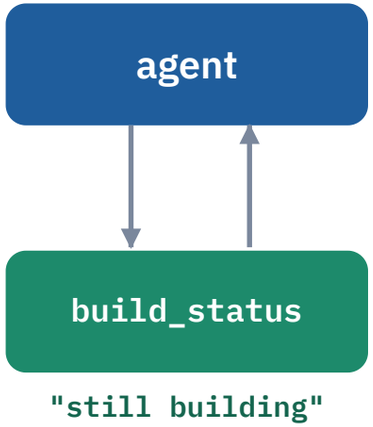
- `ModelCallLimitMiddleware` **counts model calls :**
  - `run_limit` applies to one invocation
  - `thread_limit` adds up over a thread, in saved state
- `exit_behavior="end"` **stops with a limit message**
- `exit_behavior="error"` **raises an exception**
- `create_agent` **has no call limit of its own : its step budget is 9,999 graph steps**

```
middleware = ModelCallLimitMiddleware(run_limit=1, exit_behavior="end")
agent = create_agent(repeated_lookup(), tools=[game_info],
                     middleware=[middleware])
```

demo05c · call-limits

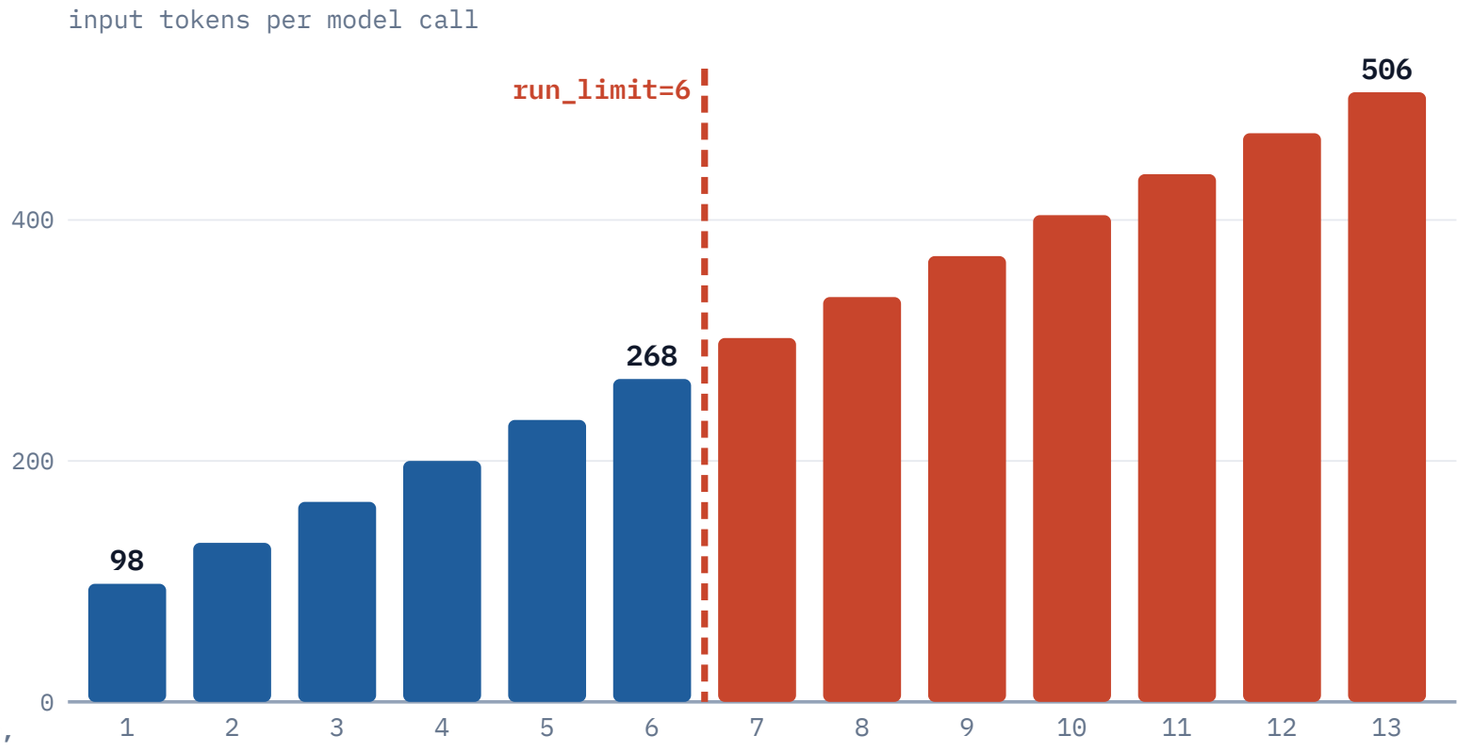
demo05d · runaway-agent

# Stopping a Runaway Agent




**1,098**
tokens, stopped by the limit, with a message

**3,926**
tokens, stopped by an exception, with no answer




About twice the calls cost more than three times the tokens. `create_agent` alone allows 9,999 graph steps.

# Tool Call Limit

- `ToolCallLimitMiddleware` **counts tool calls per run or per thread**
- `tool_name` **limits the counter to one tool**
- **The exit behaviour decides what happens at the limit :**
  - `continue` returns an error for the blocked call and the agent runs on
  - `end` stops the agent
  - `error` raises an exception
- **With** `continue` **, an agent can keep asking for the blocked tool, so a model call limit is still needed**

```
middleware = ToolCallLimitMiddleware(tool_name="game_info",
                                     run_limit=1, exit_behavior="end")
```

SpiralTrain developer courses

# LangSmith Tracing

- LangSmith is LangChain’s service for storing and viewing traces
- A trace records one run as a tree of model and tool calls :
  - Inputs, outputs and errors
  - Timing and the token counts the provider reports
- `tracing_context` switches tracing on for one block of code
- Traces go to the EU region set in `LANGSMITH_ENDPOINT`
- A project name groups related traces
- A tracing failure prints a line, and the agent still answers

```
with tracing_context(enabled=True, client=client, project_name=PROJECT):
    out = agent.invoke(question)
client.flush()
```

demo06 · tracing-with-langsmith

SpiralTrain developer courses

# Testing Strategies

| layer | model | what it proves |
|---|---|---|
| Unit test | none | A tool or a middleware does its job |
| Scripted agent test | fake | The graph runs the tool and passes the result on |
| Evaluation | real | The model picks a suitable tool on questions with known answers |
| Monitoring | real | Failures on live traffic, outside those questions |

- **A scripted tool call proves the wiring, and says nothing about the model's choice**
- **A failure in production becomes a new test**

SpiralTrain developer courses

# Unit Tests with a Fake Model

- `GenericFakeChatModel` returns scripted messages, with no API call
- An agent passes its tools to the model through `bind_tools`, so the fake needs its own version
- A scripted tool call runs the real tool and the real agent wiring
- A scripted final answer proves nothing about how a model reads the result

```
class FakeToolModel(GenericFakeChatModel):
    def bind_tools(self, tools, **kwargs):
        return self

fake = FakeToolModel(messages=iter([
    call("CS-SPR-03"), AIMessage("CS-SPR-03 is still a draft.")]))
```

SpiralTrain developer courses

# Testing Memory

- `InMemorySaver` keeps graph state within a process
- A thread id selects the conversation
- A memory test reads the saved state and ignores the scripted answer text :
  - The earlier tool result is in the saved history
  - A different thread starts without that history
- A new agent and saver per test keep the tests apart
- A production database needs its own integration tests

```
config = {"configurable": {"thread_id": "first"}}
saved = agent.get_state(config).values["messages"]
assert any(m.type == "tool" and m.content == "draft" for m in saved)
```

# Running pytest

- **pytest, Python's usual test runner, runs a test file without the notebook**
- **The tests check what can be observed :**
  - Tool result content and call id
  - `Unknown asset` for an asset that does not exist
  - Saved history and thread isolation
- `check_returncode` **fails the cell when a test fails**

```
run = subprocess.run([sys.executable, "-B", "-m", "pytest", "-p",
                      "no:cacheprovider", "-q", str(test_path)],
                     capture_output=True, text=True)
print(run.stdout)
run.check_returncode()
```

demo07b · pytest

SpiralTrain
developer courses

# Best Practices

- One agent with a short tool list comes first
- An edge guarantees a step ; a prompt only asks for it
- A tool returns `Unknown asset`, so the model has nothing to guess
- Every agent gets a model call limit and every crew model a `max_tokens`
- Retries are for short outages of operations that are safe to repeat
- Traces show the path ; tokens per call show what the path costs
- A scripted test proves the wiring ; an evaluation tests the model’s choices
- A human makes the release decision : the Publisher, in module 07

SpiralTrain developer courses

# Summary : Building Agents

- An agent is the `while` loop of module 00, drawn as two nodes and a conditional edge
- `ToolNode` and `tools_condition` replace the hand-written tool step and router
- `create_agent` builds the same loop, and it can be a node or call a crew as tools
- Middleware adds code before, after and around each model and tool call
- A tool error goes to the model, a fallback replaces a failing model, a limit stops a runaway loop
- Traces show what ran ; scripted tests check the wiring
- Real-model evaluations judge the model’s choices and answers

# Multi-Agent Patterns

From LLM to Agent
Introduction to LangGraph
Graph Fundamentals
State Management
Building Agents
**Multi-Agent Patterns**
Agent Communication
Advanced Workflows
Deep Agents
Production Deployment
Real-World Applications

Multi-Agent Concepts
Examples in This Module
Agent Roles and Responsibilities
Choosing a Pattern
Pattern Capabilities
Documentation Call Counts

Subagents
Subagent Context
Writing a Task Description
Manager-Worker
A Supervisor Node
Hierarchical Teams

Router
Router Code
Delegation and Handoff
Handoffs
Fixed Handoffs
Handoff Middleware
Skills

Specialized Agents
Custom Workflow
Sequential Pattern
Parallel Agents
Orchestration in Code
Collaborative Agents

Maker-Checker
Checkers Written in Code
Debate with a Judge
Generate and Filter

Orchestration Shapes
Evaluating the Boundaries
Supervisor and Swarm Packages

The Earlier Pattern Probe
What Each Pattern Costs
Folding Subagents Back
When One Agent Is Enough
When a Crew Earns Its Cost
Summary : Multi-Agent Patterns

# Multi-Agent Concepts

- **A multi-agent system splits one job across agents, each with its own context :**
  - The context is everything a model can see on one call
  - The Artist reads the asset list, the Tester reads the bug list
  - The Producer reads only what those two report back
- **Multi-agent design decides what each agent sees and what comes back**
- **Three measured problems justify another agent :**
  - With too many tools, the model picks the wrong one
  - Instructions for different specialities clash in one prompt
  - Tool output crowds the request out of the context
- **One agent comes first, as the baseline for calls, tokens, seconds and quality**
- **Each pattern is built from LangGraph nodes before a harness hides it**

SpiralTrain developer courses

# Examples in This Module

- **Pictures and explanations use the Comet Arcade crew ; the demos use Veenhaven Bus, a made-up bus maintenance company**
- **Both ask one question that needs two specialists :**

| | Comet Arcade | Veenhaven Bus demos |
|---|---|---|
| Question | Sprite status and open blocker | Part stock and error guidance |
| Main agent | Producer | Coordinator |
| First specialist | Artist, `assets_for_game` | Parts worker, `part_stock` |
| Second specialist | Tester, `bugs_for_game` | Manuals worker, `fault_guidance` |
| Key identifiers | CS-01, CS-SPR-03, BUG-101 | CE-CF-02, F-CE-110 |

- **Code on the slides comes from the demo notebooks**
- **Each number is labelled with the example it was measured on**

SpiralTrain developer courses

# Agent Roles and Responsibilities

- **Each agent gets one job, stated in its prompt :**
  - The Artist answers questions on sprites from the asset list
  - The Tester answers questions on bugs from the bug list
  - The Producer delegates and writes the final answer
- **Each agent gets only the tools its job needs**
- **A role prompt states the scope and when to stop**
- **A tool description tells the Producer when to call each specialist**
- **A delegated task includes the identifiers and says when the work is done**
- **A fact missing from the answer is traced to the worker’s reply first, then to the Producer’s summary**
- **A different model per role needs an evaluation that shows the gain**

SpiralTrain developer courses

# Choosing a Pattern

- **The LangChain documentation names five patterns :**

| Pattern | Control | Suitable task |
|---|---|---|
| Subagents | Main agent calls workers as tools | Specialist work that needs its own context |
| Handoffs | Active agent passes the conversation on | A conversation that moves through stages |
| Skills | One agent loads instructions when needed | Many specialities in one conversation |
| Router | A classify step sends work to specialists | Input that falls into clear categories |
| Custom workflow | The graph fixes the order in code | A process with known steps and rules |

- **One pattern can sit inside another, such as a router or a subagent in a workflow node**
- **A subagent and a skill can both serve the Tester’s job ; only the subagent gets its own context**

SpiralTrain developer courses

# Pattern Capabilities

- LangChain’s documentation rates the patterns by capability :

| Pattern | Separate teams | Parallel | Multi-hop | User interaction |
|---|---|---|---|---|
| Subagents | High | High | High | Low |
| Handoffs | Unrated | Unrated | High | High |
| Skills | High | Medium | High | High |
| Router | Medium | High | Unrated | Medium |

- Separate teams means different groups can own and change each part
- Multi-hop means one request passes through several specialists in turn
- A custom workflow gets its capabilities from the graph the developer writes
- A rating says nothing about answer quality or cost on a given question

SpiralTrain developer courses

# Documentation Call Counts

- LangChain illustrates these calls and processed tokens :

| Pattern | One request | Repeat turn | Three domains |
|---|---|---|---|
| Subagents | 4 calls | 4 calls | 5 calls, about 9K tokens |
| Handoffs | 3 calls | 2 calls | 7+ calls, about 14K+ tokens |
| Skills | 3 calls | 2 calls | 3 calls, about 15K tokens |
| Router | 3 calls | 3 calls | 5 calls, about 9K tokens |

- A repeat turn is cheaper when the active agent or the loaded skill stays in place
- Loaded instructions remain in every later call to the same agent
- The counts come from LangChain’s own examples ; measured counts follow in demo09

SpiralTrain developer courses

# Subagents

- A subagent is an agent wrapped as a tool : the main agent calls it and receives text
- Each worker’s model and tool loop is its own LangGraph graph
- demo01 on Veenhaven Bus, with a parts worker as the tool :

```
workers = specialists(llm)

@tool
def call_parts(question: str) -> str:
    """Ask the parts specialist about stock and location."""
    result = workers["parts"].invoke(ask(question), LIMIT)
    return last_text(result)

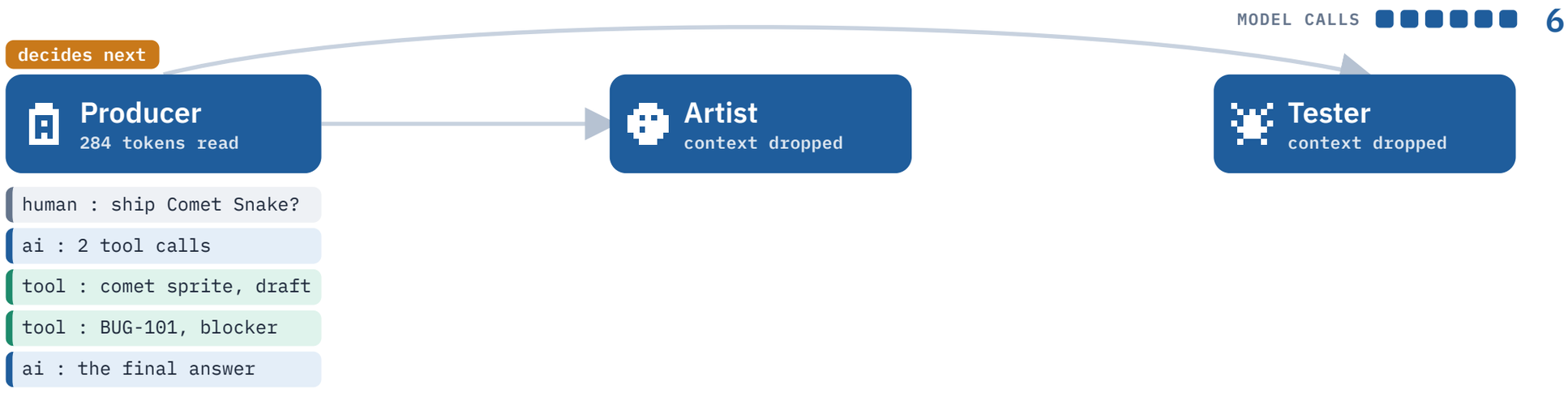
coordinator = agent_by_hand(llm, [call_parts, call_manuals],
    "Delegate both data questions, then answer briefly.")
```

- The main agent sees the worker’s final answer as a tool result and none of its tool calls

demo01 · subagents

SpiralTrain developer courses

# A Subagent at Work




FINAL ANSWER

The comet sprite is in draft status. The open blocker is BUG-101: "Comet hard to see on the dark background."

The Producer answers from two sentences. Its last call reads 284 tokens ; the Artist's last call read 408. Saved run, Comet Arcade.

SpiralTrain developer courses

# Subagent Context

- The Artist starts with two things : its prompt and the task
- Everything it reads stays in its own message list
- The Producer keeps the task and the reply
- A fact crosses the boundary only in the task or in the reply :
  - A task without game id CS-01 sends the Artist to the wrong asset list
  - A reply without the status leaves the Producer guessing
- Each call to a worker starts empty ; it keeps nothing from its last task

demo01b · subagent-context

SpiralTrain developer courses

# Writing a Task Description

- **The Producer knows a worker only from its tool name and description**
- **The worker knows the request only from the task text**
- **A task description states :**
  - The objective, with identifiers such as CS-01 and CS-SPR-03
  - The expected output, such as the status on one line
  - The tools and sources to use
  - The boundaries, such as no design advice
- **Veenhaven Bus, demo01 c, one need and two tasks :**
  - "Is the cooling fan for this bus in stock?" sends `part_stock` the argument `cooling fan` and returns no count
  - The specified task returns "6 at Rietpoort"

SpiralTrain developer courses

# Manager-Worker

- A manager owns splitting the job and writing the final answer
- Workers own one bounded task each and return findings
- The Producer is a manager ; the Artist and the Tester are workers
- Workers can be tools of an agent or nodes of a graph
- Code sets which workers exist and how many rounds the manager gets
- The manager decides whether a finding needs more work
- A lookup with known inputs, such as the status of CS-SPR-03, stays an ordinary function

SpiralTrain developer courses

# A Supervisor Node

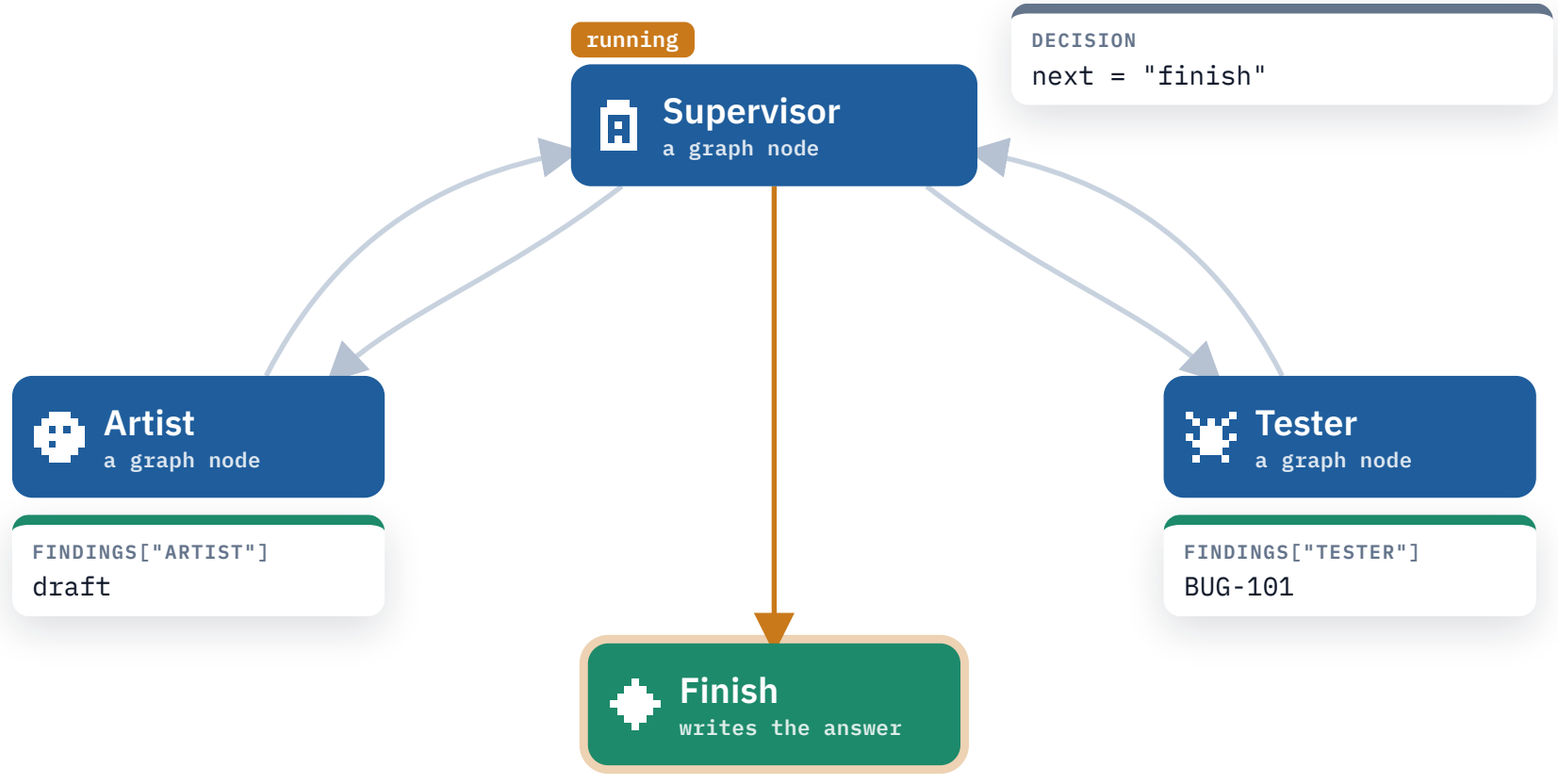
- A supervisor node picks an unfinished worker and writes its task
- The worker receives that task in a separate context
- After each worker, an edge leads back to the supervisor
- Findings are stored per worker ; code caps the loop at two worker rounds
- demo02 on Veenhaven Bus :

```
graph.add_edge(START, "supervisor")
for name in workers:
    graph.add_node(name, worker(name))
    graph.add_edge(name, "supervisor")
graph.add_conditional_edges("supervisor", lambda s: s["next"],
                            ["parts", "manuals", "finish"])
graph.add_edge("finish", END)
```

demo02 · supervisors

SpiralTrain developer courses

# A Supervisor Loop




With both findings in the state the supervisor picks finish. A supervisor that picks finish too early leaves a fact out.

SpiralTrain developer courses

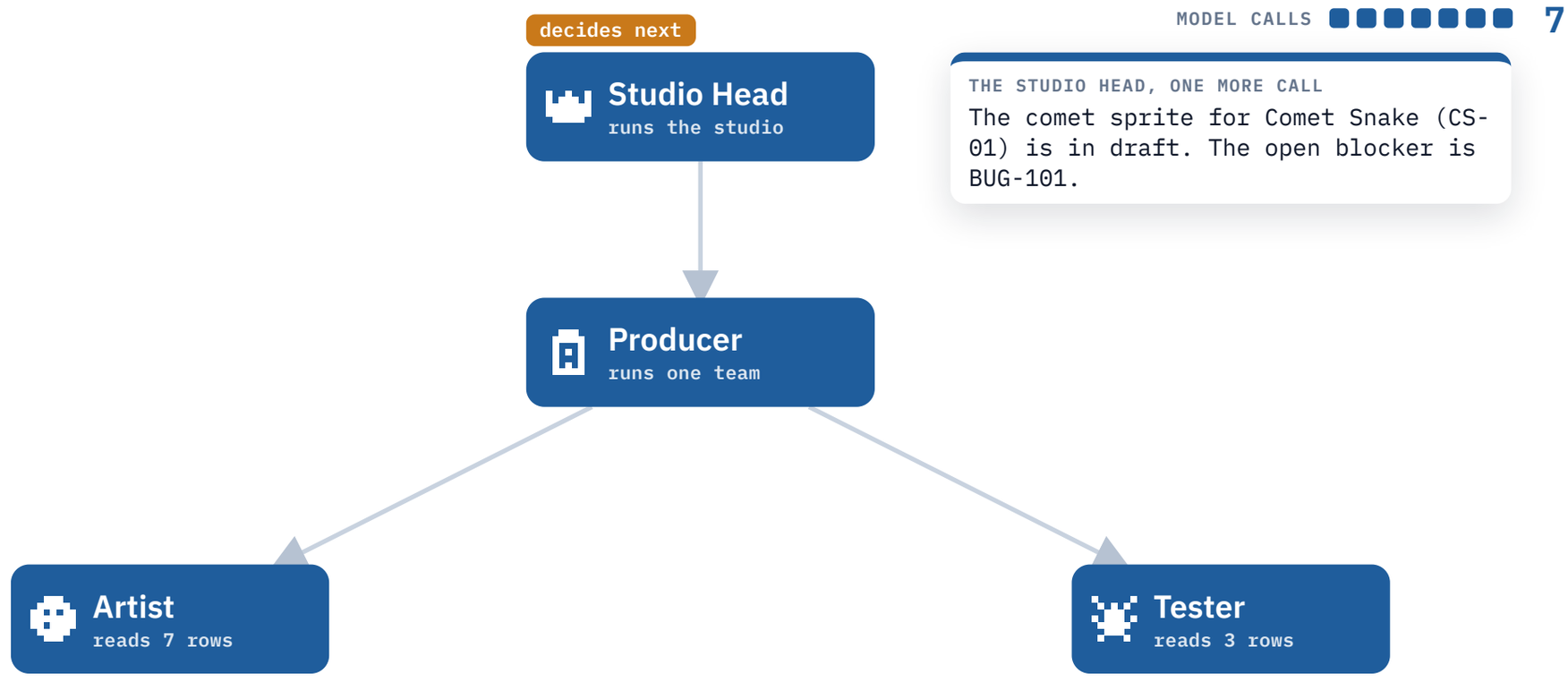
# Hierarchical Teams

- In a hierarchy one team sits inside another, and a manager manages managers
- The Studio Head reads the Producer’s report ; the Producer reads the crew’s replies
- Each level adds a context boundary and a model call :
  - Comet Arcade, 7 calls against 6 for the team alone
  - Veenhaven Bus, demo02 b, 8 calls against 7
- A fact can be lost at either report
- demo02 b, the outer manager calls the compiled team as one node :

```
def manage(state):
    report = team.invoke(team_input(state["question"]), LIMIT)
    return {"report": report["answer"]}

manager.add_node("maintenance_team", manage)
manager.add_edge("maintenance_team", "manager")
```

# Reports Through Two Levels




The extra level costs one more call : 7 against 6 here, 8 against 7 in the Veenhaven Bus demo. The level adds no fact, and its report can lose one.

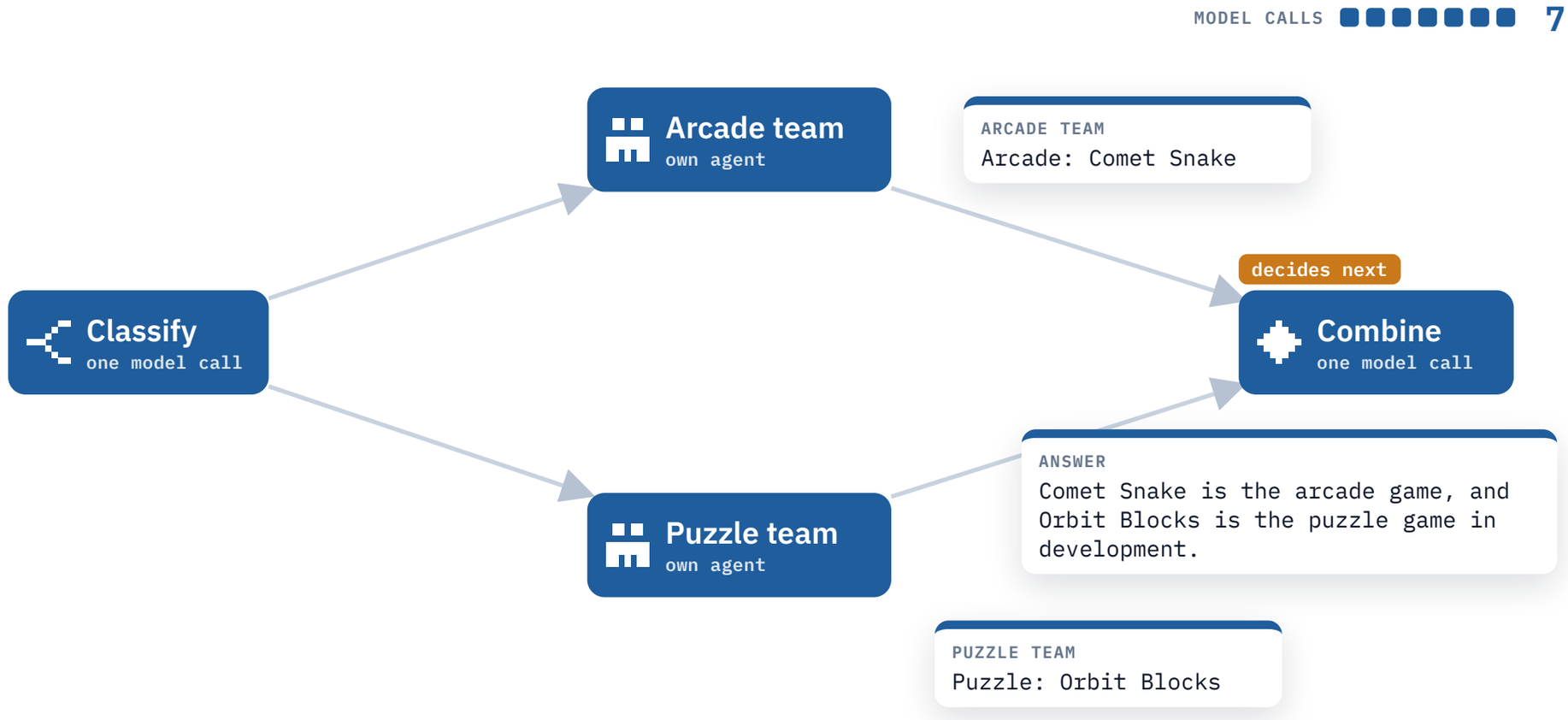
SpiralTrain developer courses

# Router

- A router is a classify step at the entry of the graph
- The classifier picks the specialists and writes each one’s task
- Dispatch starts only the selected branches
- A final node combines the answers
- A brief about puzzle games reaches the puzzle team alone ; a brief about both reaches both teams
- A follow-up needs a surrounding conversation or a checkpointer, because classification runs once per request

demo03 · router

# A Router Fan-Out




Combine answers for both : seven calls. The first, looser prompt sent the puzzle brief to both teams in 3 of 4 runs, so the classifier needs its own test.

# Router Code

- `Send` gives each selected branch its own input
- demo03 b on Veenhaven Bus :

```
def dispatch(state):
    return [Send(r["agent"], {"question": r["query"]})
            for r in state["routes"]]

class RouterState(TypedDict):
    question: str
    routes: list[dict]
    answers: Annotated[list[str], operator.add]
    final: str

graph.add_conditional_edges("classify", dispatch,
                            ["parts", "manuals"])
```

- Each branch appends one answer through the reducer on `answers`
- The combining node runs after every selected branch has finished

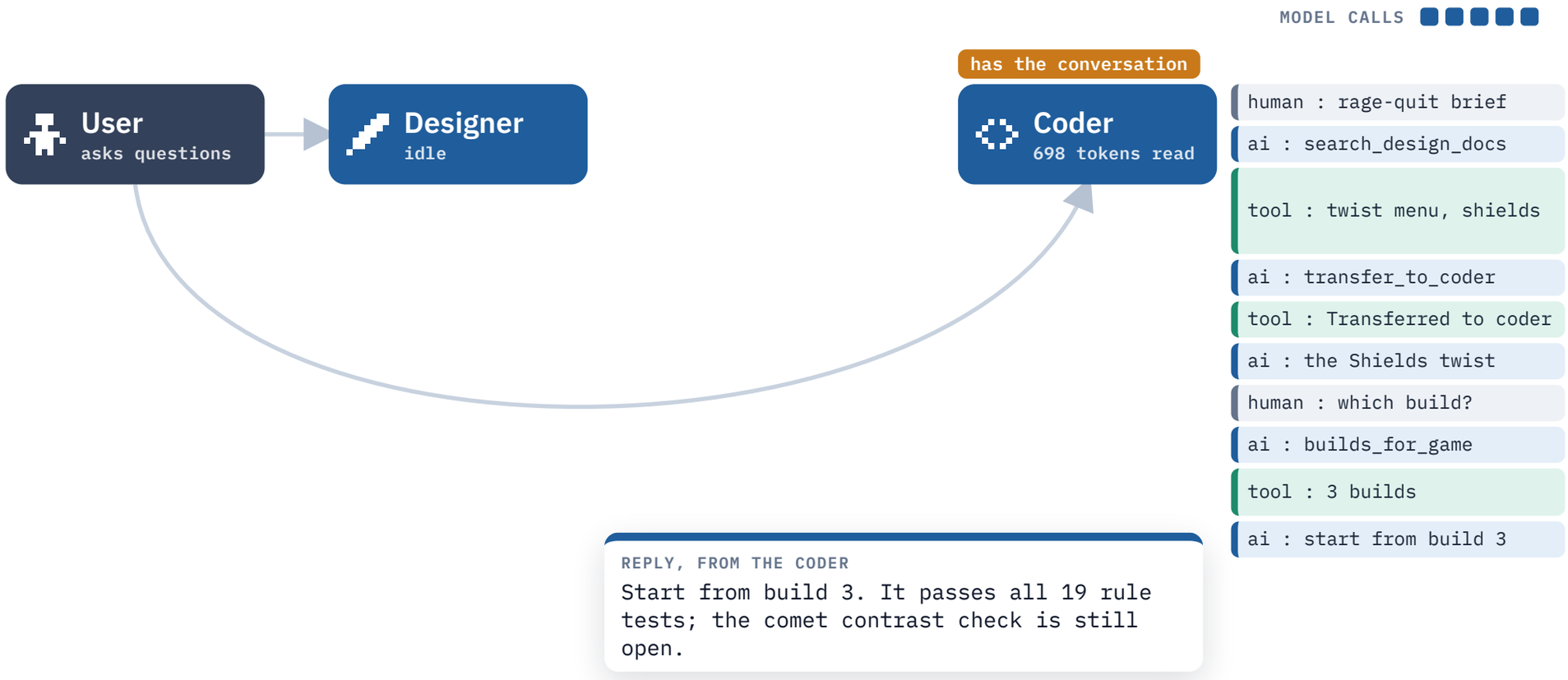
demo03b · router-parallel

# Delegation and Handoff

| Property | Delegation | Handoff |
|---|---|---|
| Caller | Waits for the task result | Passes on the active role |
| Worker output | Returns to the caller | Continues the conversation |
| Typical context | One bounded task | The conversation so far |
| Control | Caller chooses the next action | Active agent chooses it |

- A status lookup for the comet sprite suits delegation
- Choosing a twist with the Designer and then planning the build with the Coder suits a handoff
- Either choice still has to settle how much history the receiver gets

# Passing the Baton




Five calls. Everything the Designer read stays in the message list, so each later call reads it again : 698 tokens. Saved run, Comet Arcade.

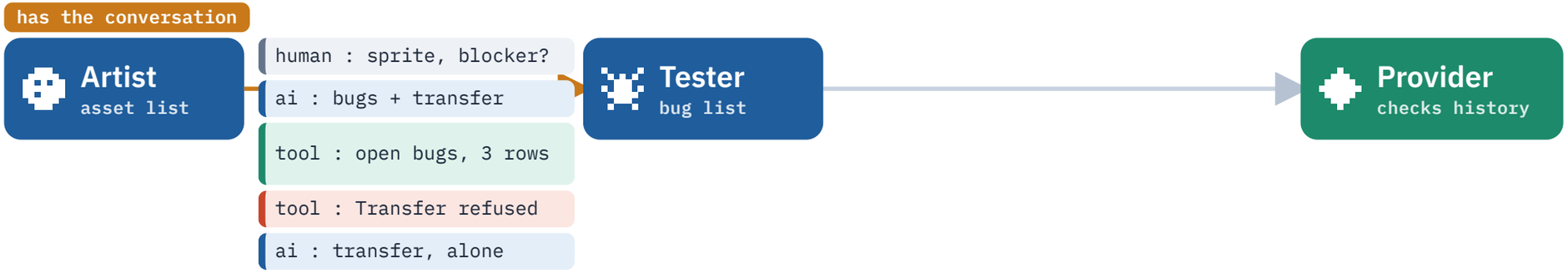
SpiralTrain developer courses

# Handoffs

- A transfer is a tool call that changes the active agent through `Command.PARENT`
- The documented example forwards only the last assistant message
- The page says the example assumes no parallel tool calls
- Veenhaven Bus, demo04 a, shows two faults in that transfer :
  - A stock result from the same turn is dropped, and the provider rejects the history
  - An earlier lookup is lost, and the receiver reports stock unavailable
- Both faults repeat on Comet Arcade with a seeded history and a live model

demo04 · handoffs

# What a Transfer Can Lose




ONE RUN OF THREE, THE ARTIST ANSWERS

The comet sprite is in draft status. I've handed off the open-bug question to the tester.

The fix protects the history. The agent still decides : in 1 of 3 runs the Tester passed the baton straight back, and half the answer was lost.

SpiralTrain developer courses

# Fixed Handoffs

- **The transfer tool refuses a turn that has other tool calls :**
  - Every other call in the turn still gets its result
  - The model transfers alone on its next turn
- **The tool enforces the rule because OpenRouter drops `parallel_tool_calls=False` for the course model**
- **The transfer forwards the full history and a matching receipt**

```
messages = runtime.state["messages"]
if len(messages[-1].tool_calls) > 1:
    return "Transfer refused : call the transfer on its own, after the other tool results."
return Command(goto=target, graph=Command.PARENT, update={
    "active_agent": target, "messages": [*messages,
        ToolMessage("Transferred to " + target,
                    tool_call_id=runtime.tool_call_id)]})
```

demo04b · handoffs-fixed

SpiralTrain developer courses

# Handoff Middleware

- One agent keeps the conversation and the active step in state
- `record_bus` in demo04 c validates the fleet number and updates the step
- Middleware picks the prompt and the tools for each model call :

```
@wrap_model_call
def apply_step(request, handler):
    if request.state.get("current_step", "triage") == "triage":
        return handler(request.override(
            system_prompt="Collect the fleet number.",
            tools=[record_bus]))
    return handler(request.override(
        system_prompt="Answer using the tools.",
        tools=[part_stock, fault_guidance]))
```

- A checkpointer keeps the step across user turns

demo04c · handoffs-one-agent

SpiralTrain developer courses

# Skills

- A skill is a pack of instructions that the agent loads into its own history
- The prompt contains only the names and descriptions of the packs
- demo05 on Veenhaven Bus :

```
@tool
def load_skill(name: str) -> str:
    """Load task instructions. Names: parts or manuals."""
    return SKILLS.get(name, "Unknown skill.")

skilled = agent_by_hand(llm,
    [load_skill, part_stock, fault_guidance],
    "Load each matching skill before using its tools.")
```

- The same agent keeps control, and later calls read the loaded pack
- The prompt asks for the skill first, and nothing stops a tool call without it
- Agent Skills is an open standard at the Agentic AI Foundation since October 2026

demo05 · skills

# A Skill Loaded When Needed

SpiralTrain developer courses

MODEL CALLS 6

decides next

**Studio assistant**
1,036 tokens read

ANSWER, TURN 3

Replacing the comet list after spawning a comet throws away the newly spawned comet.

prompt : 3 pack names

human : open bugs of CS-01?

ai : bugs_for_game, then a reply

human : head through body?

ai : load_skill(canvas-games)

tool : canvas-games pack, 293 tokens

ai : tail removed twice

human : lost comet?

ai : load_skill, a second time

tool : the same pack again

ai : comet list replaced

**Skill packs**
files, not in context

style-guide

testing

canvas-games

The pack stays in the history, and every later call reads it. A follow-up could reuse it, yet in 2 of 3 runs Luna loaded it again.

SpiralTrain developer courses

# Specialized Agents

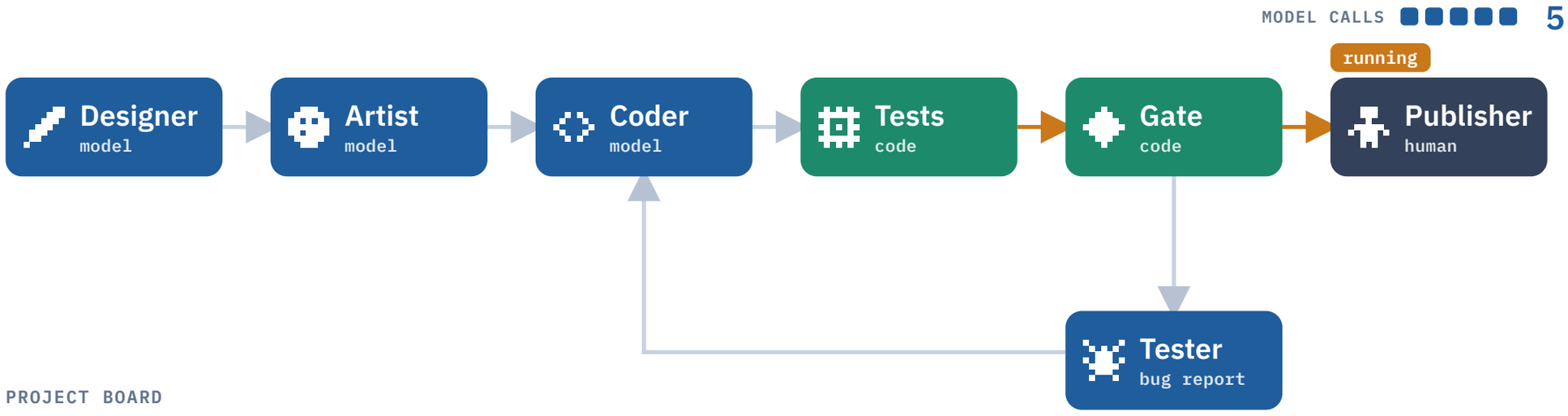
- **Specialisation can use a subagent or a skill**
- **A subagent isolates its work :**
  - The worker keeps its own tool calls
  - The caller receives a short result
- **A skill keeps the work in the conversation :**
  - Follow-ups can reuse the loaded pack and the findings
  - Every later call reads all of it again
- **Input tokens on one question, subagents against skills :**
  - Veenhaven Bus, demo09 : 942 against 1,139, with 435 for a single agent
  - Comet Arcade : 1,341 against 3,920, with 789 for a single agent

SpiralTrain developer courses

# Custom Workflow

- A custom workflow is a LangGraph graph that mixes agent steps with ordinary code
- The developer fixes the order of the steps in edges
- Model steps handle writing and judgement : the Designer picks a twist, the Coder writes the logic
- Code steps handle known rules : the tests run, a gate reads the result
- A workflow node can contain a subagent, a router or a handoff
- Each node’s input and update can be inspected in the state

# The Crew as a Workflow




The tests pass, so the gate sends the build to the Publisher, a human, through interrupt. Five model calls, in an order that no model chose.

# Sequential Pattern

- demo06 on Veenhaven Bus : a customer ticket is classified, gets a priority, gets a reply
- Structured output restricts the model to the permitted labels
- A lookup table sets the priority without a model

```
def prioritise(state):
    return {"priority": {
        "complaint": "high", "question": "normal", "praise": "low"
    }[state["category"]]}

graph.add_edge(START, "categorise")
graph.add_edge("categorise", "prioritise")
graph.add_edge("prioritise", "respond")
graph.add_edge("respond", END)
```

- The prompt forbids a repair date ; the saved reply gives none

SpiralTrain developer courses

# Parallel Agents

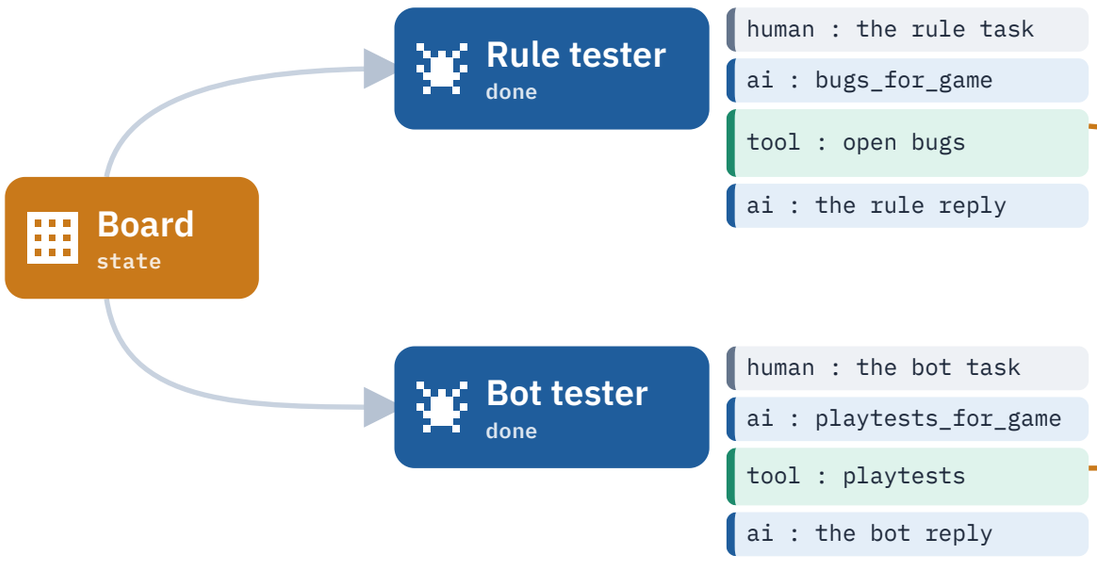
- A rule tester and a bot tester can run at once because neither needs the other’s result
- Separate state keys keep their results apart
- demo06 b on Veenhaven Bus :

```
graph.add_edge(START, "parts")
graph.add_edge(START, "manuals")
graph.add_edge(["parts", "manuals"], "compose")
graph.add_edge("compose", END)
```

- The join edge waits for both branches
- `Send` and a reducer do the same for a number of branches known only at run time
- A parallel run saves no tokens, because tokens add up across the branches
- At 0.4 requests per second the parallel graph takes 11.9 seconds and the sequential one 13.5

demo06b · workflows-parallel

# Shared and Isolated Context




input tokens and seconds, 5 calls each, median of 3 runs

| | | |
|---|---|---|
| sequential, shared | 1,467 | 10.2 s |
| sequential, isolated | 1,034 | 10.3 s |
| parallel, isolated | 1,050 | 10.1 s |

The join edge starts the gate when both are done, and a reducer merges the findings. Time did not fall, because the rate limiter spaces every call.

SpiralTrain developer courses

# Orchestration in Code

- When the identifiers are known, no model has to choose a tool
- Code runs both lookups, and one model call writes the answer
- One call against five for the sequential agents : Veenhaven Bus, demo06 c, and Comet Arcade alike
- demo06 c :

```
def fetch(state):
    return {
        "stock": part_stock.invoke({"part": "CE-CF-02"}),
        "guidance": fault_guidance.invoke({"error_code": "F-CE-110"})
    }

graph.add_edge(START, "fetch")
graph.add_edge("fetch", "compose")
graph.add_edge("compose", END)
```

- Code guarantees the lookups run ; the written answer still needs checking

demo06c · orchestration-in-code

SpiralTrain developer courses

# Collaborative Agents

- Collaborative agents work on one result together
- Maker-checker separates writing from reviewing : the Coder writes, the Tester reviews
- Debate gives a judge two proposals, such as the shield twist or the clock twist
- Generate and filter keeps only the candidates that pass the tests
- Every shape needs the same evidence : the rules, the tests, the data
- Code sets the completion rule and the cap on attempts
- The test result decides, because two models that agree can both be wrong

SpiralTrain developer courses

# Maker-Checker

- A maker writes, a checker returns approval and specific feedback
- A rejected draft returns to the maker with that feedback
- demo07 on Veenhaven Bus, a parts note from database evidence :

```
graph.add_edge(START, "make")
graph.add_edge("make", "check")
graph.add_conditional_edges("check", lambda s:
    END if s["approved"] or s["rounds"] >= 2 else "make",
    ["make", END])
```

- Reaching the cap can leave the draft unapproved
- The saved model checker approves a correct note ; the code checker rejects it for lacking the digit 0
- A model checker costs a call : 2 calls against 1 for the code checker in demo07

demo07 · maker-checker

SpiralTrain developer courses

# Checkers Written in Code

- Code can check any fact that a query can return
- The Comet Arcade tests are such a checker : they run the game, and the Coder reads what failed
- A check returns specific findings, and the maker receives them on the next attempt
- demo07 b on Veenhaven Bus starts from a draft that marks CE-DA-03 available ; its stock is 0
- A format rule can reject a correct paraphrase
- A pass proves only what the check tests :
  - Comet Arcade, no kit : 8 of 8 whole-file games passed the tests, and 1 was a blank page
  - The tests read the logic and never opened the page
- Findings still open at the cap stay visible

demo07b · checker-in-code

SpiralTrain developer courses

# Debate with a Judge

- Two agents argue different actions, such as adding a shield or adding a clock
- Both receive the same evidence
- A judge reads the proposals and the evidence, then returns a choice and a reason
- demo08 on Veenhaven Bus, inspect the fan or replace it :
  - Six fans in stock do not justify a replacement
  - The manual replaces the fan only below 80 percent airflow
  - The saved judge selects inspection and keeps the airflow condition
- The judge’s extra call changed no decision, because both saved proposals already put inspection first

demo08 · debate-and-selection

# Generate and Filter

- Independent calls write several candidates, such as three twist pitches
- Code checks each candidate, and a rejected one keeps its findings
- Selection applies a fixed rule ; demo08 b on Veenhaven Bus keeps the shortest passing parts note
- A stable label breaks ties between equal candidates
- When no candidate passes, the run returns an explicit failure
- More candidates add calls and tokens without guaranteeing a pass

```
return [Send("generate", {"task": style}) for style in
        ["Compact bullets.", "A compact table."]]
```

demo08b · generate-and-filter

# Orchestration Shapes

- **Named shapes describe how work runs and how a result is picked :**

| Shape | At Comet Arcade |
|---|---|
| Classify and act | Send a brief to the right team |
| Fan-out and synthesise | Combine the Artist’s and the Tester’s findings |
| Adversarial verification | A second Tester challenges a passing build |
| Generate and filter | Keep the twist pitches that pass the checks |
| Tournament | Compare surviving pitches in rounds |
| Loop until done | Fix until tests pass or attempts run out |

- **Code defines coverage, joins and the stopping rule**
- **Module 08 covers Deep Agents, which package delegation and context management**

SpiralTrain developer courses

# Evaluating the Boundaries

- **Each boundary gets its own check :**
  - The specialist’s answer against the asset list
  - The delegated task for game id CS-01 and the sprite asked for
  - The reply for facts lost on the way back
  - The final answer against the whole request
- **The comparison always includes one agent with both tools**
- **A keyword check passes a sentence that contradicts itself**
- **Failures are read before a prompt changes or an agent is added**

# Supervisor and Swarm Packages

- **Two earlier prebuilt packages supplied multi-agent graphs :**
  - `langgraph-supervisor` centralised worker coordination
  - `langgraph-swarm` switched between active agents
- **Older tutorials and applications still contain them**
- **Neither package has had a release in 11 months**
- **The documented patterns give the same control choices :**
  - A supervisor uses agents as tools or worker nodes
  - A swarm-style transfer uses handoffs and an active-agent key
- **Both shapes need only nodes, conditional edges and `Command`**

# The Earlier Pattern Probe

- Luna medians on a small in-memory bus fixture, 13 October 2026 :

| Pattern | Correct | Calls | Input | Output | Seconds |
|---|---|---|---|---|---|
| Single agent | 3/3 | 2 | 399 | 96 | 6.7 |
| Subagents | 3/3 | 6 | 781 | 204 | 20.3 |
| Handoffs, fixes applied | 2/3 | 4 | 1,165 | 99 | 13.5 |
| Skills | 3/3 | 4 | 1,036 | 131 | 14.6 |
| Router | 3/3 | 6 | 622 | 184 | 20.6 |
| Custom workflow | 3/3 | 5 | 472 | 129 | 17.0 |
| Deep Agent, module 08 | 3/3 | 6 | 11,044 | 214 | 20.6 |

- Seconds include rate limiting at 0.3 requests per second

# What Each Pattern Costs

- Luna medians on the Veenhaven Bus records, 26 October 2026 :

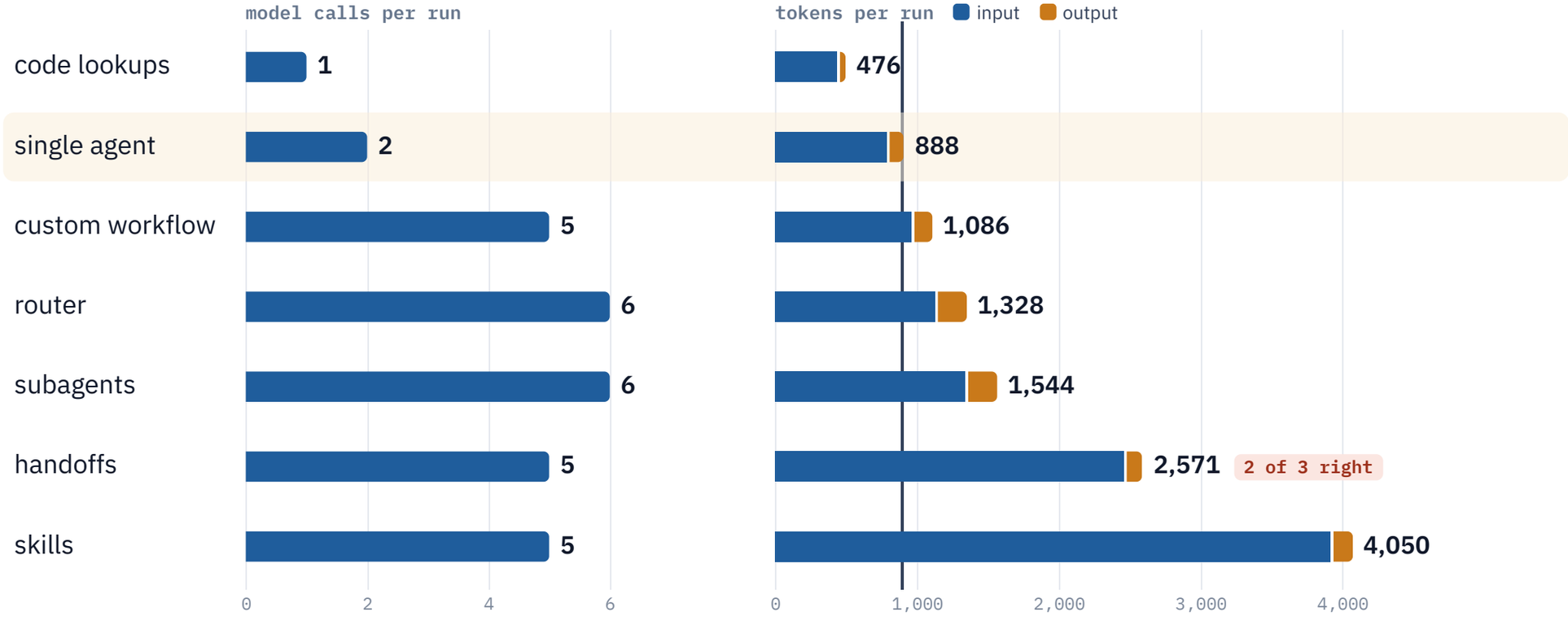
| Pattern | Correct | Calls | Input | Output | Seconds |
|---|---|---|---|---|---|
| Single agent | 3/3 | 2 | 435 | 120 | 5.3 |
| Subagents | 3/3 | 6 | 942 | 260 | 15.4 |
| Handoffs, fixes applied | 3/3 | 4 | 947 | 121 | 10.2 |
| Skills | 3/3 | 4 | 1,139 | 159 | 10.2 |
| Router | 3/3 | 6 | 825 | 246 | 15.2 |
| Custom workflow | 3/3 | 5 | 632 | 181 | 12.8 |
| Code lookups | 3/3 | 1 | 129 | 47 | 2.6 |

- Seconds include the shared limiter at 0.4 requests per second
- Code lookups use the known part number and error code

demo09 · what-each-pattern-costs

# Measured Cost per Pattern

Comet Arcade : the comet sprite and the open blocker




The two cheapest stay the same : code, then one agent. Handoffs got 2 of 3 right here ; every other row got 3 of 3.

# Folding Subagents Back

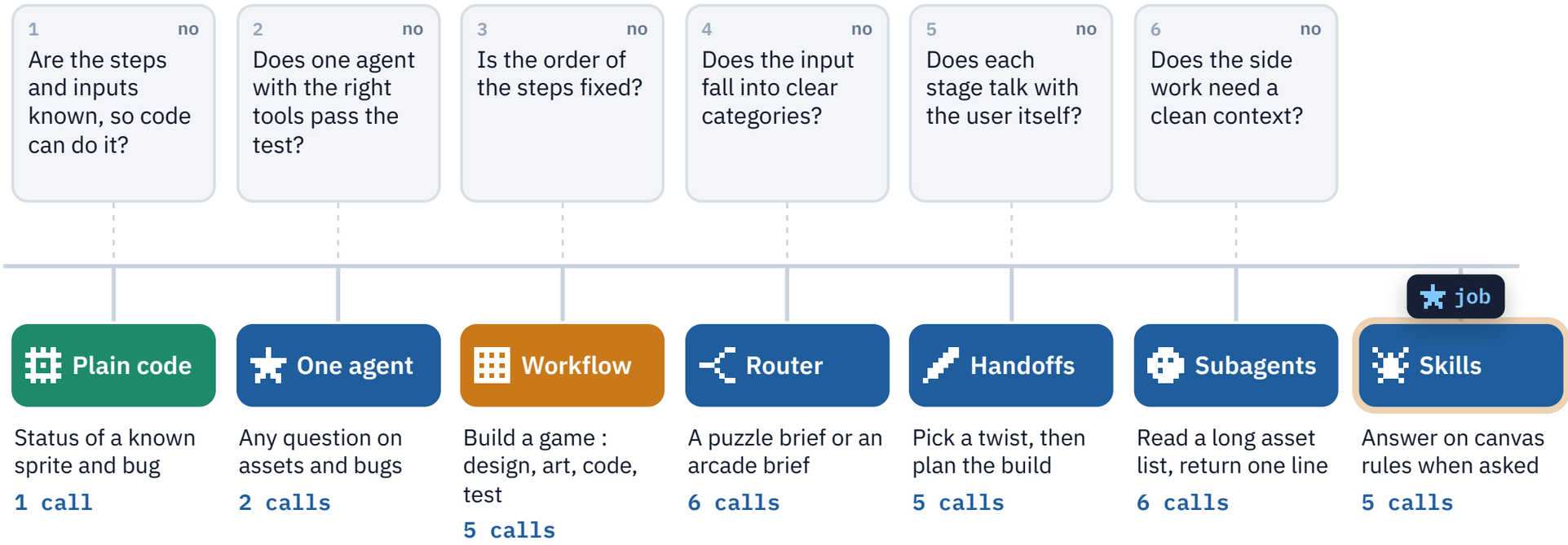
- **Anthropic reported in October 2026 that one agent with skills often beat both one large prompt and a set of subagents**
- **Subagents stay where one loop measurably fails :**
  - Context bloat degrades the single loop
  - Parallel tasks need fresh contexts that are discarded afterwards
  - The single agent picks the wrong tools, or its instructions conflict
- **A subagent that stays is called as a tool and returns a compact result**
- **Migration moves one subagent at a time and runs an end-to-end eval more than once**
- **demo09 agrees on its question : 2 calls for one agent, 4 to 6 for each pattern**

SpiralTrain developer courses

# When One Agent Is Enough

- **One agent is enough while it passes the test on the real job**
- **LangChain : "a single agent with the right (sometimes dynamic) tools and prompt can often achieve similar results"**
- **Comet Arcade, one question, three runs each :**
  - Single agent : 2 calls, 3 of 3 right
  - Every multi-agent pattern : 5 to 6 calls
- **A fixed order needs no manager :**
  - Plain graph building a game : 3 model calls, 20 of 20 runs pass
  - Same crew under a Producer agent : 6 calls, 8 of 8 pass
- **A failure that can be shown justifies another agent : wrong tool, clashing instructions, a crowded context**

# Which Pattern for Which Job




calls : median on one Comet Arcade question, three runs

A canvas rule is needed now and then, in the same conversation, so one agent loads a skill pack when the question comes up.

# When a Crew Earns Its Cost

- **Anthropic built a retro game maker twice, published 24 March 2026 :**
  - One agent : about 20 minutes, $9, and "the actual game was broken"
  - Planner, generator and evaluator : about 6 hours, $200, and a game that played
- **The evaluator clicks through the running app and grades each sprint**
- **The separate checker found what the builder did not see**
- **The harness cost over 20 times as much, and only a working game justified that**
- **The Comet Arcade crew keeps a checker for the same reason : the Tester’s tests run the game**

SpiralTrain developer courses

# Summary : Multi-Agent Patterns

- Multi-agent design controls what each agent sees and what comes back
- One agent is the baseline for any architecture decision
- Subagents isolate work ; skills keep it in one conversation
- Supervisors and managers coordinate bounded tasks
- A hierarchy adds another context boundary and another report
- Routers classify and dispatch ; handoffs pass the conversation on
- Workflows fix the order in code, and joins wait for parallel branches
- Collaborative patterns need evidence, checks and a cap
- Code handles known steps and deterministic checks
- Calls, tokens, seconds and answer quality decide the trade-off

# Agent Communication


From LLM to Agent

Introduction to LangGraph

Graph Fundamentals

State Management

Building Agents

Multi-Agent Patterns

**Agent Communication**

Advanced Workflows

Deep Agents

Production Deployment

Real-World Applications

Message Passing

Shared State

Full History

Agent Context

Handoffs

Parent Message Update

Shared Keys

Transformed Input and Output

Communication Protocols

The Agentic AI Foundation

Agent Protocol

What is MCP?

MCP for Tools

Served Agents

What is A2A?

Agent Cards

A2A Tasks

Two Agents over A2A

Event Systems

Subgraph Streaming

v3 Projections

Custom Events

State Broadcasting

Synchronization

Conflict Resolution

Conflicting Writes

Voting

Judge and Tie-Break

Seller and Buyer

Least Privilege

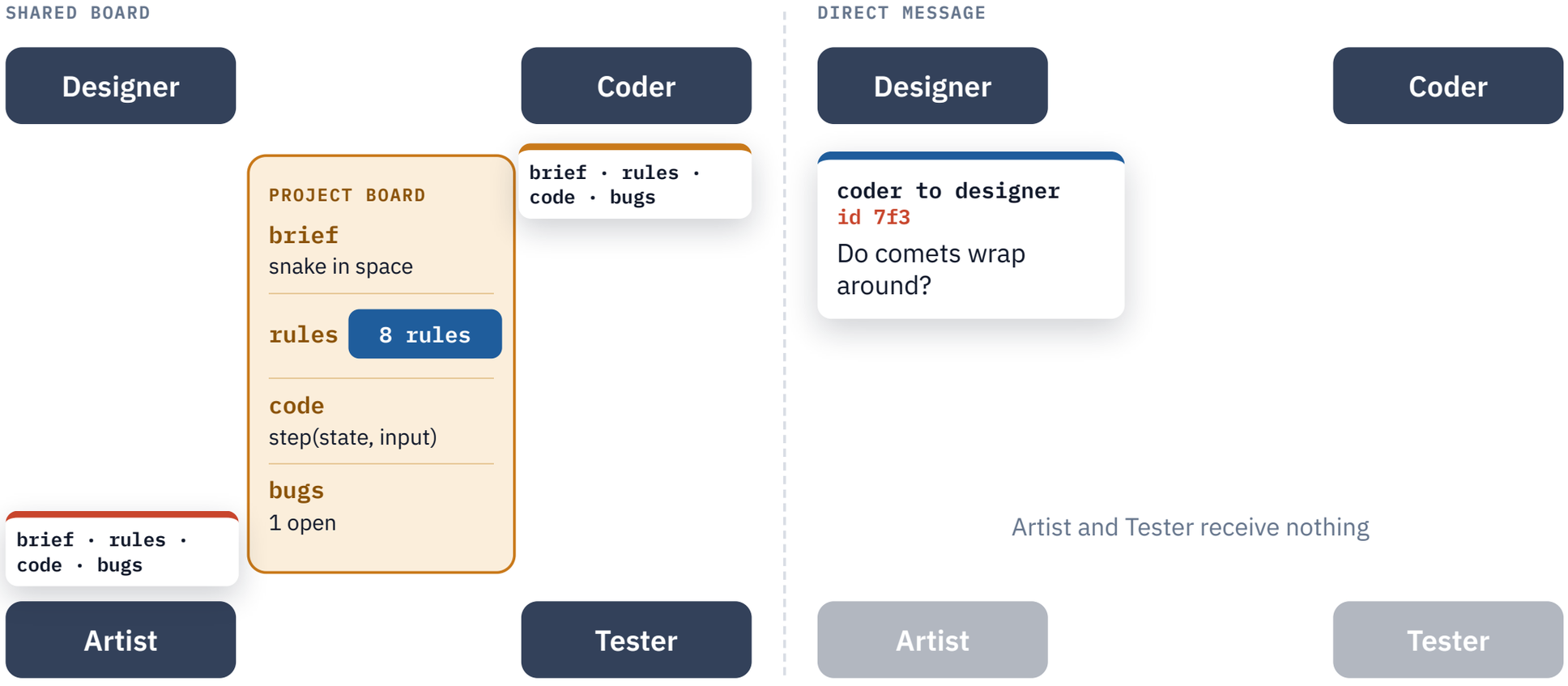
Summary : Agent Communication

SpiralTrain developer courses

# Message Passing

- **Agents move facts in two ways :**
  - A direct message goes to one named agent and contains only what that agent needs
  - A shared board has every key, and every agent can read all of it
- **The Designer can message the rules to the Coder, while the Testers write on the project board**
- **A direct message names its sender, its recipient and a correlation id, a code that ties a reply to its request**
- **Inside one Python process a message is a plain object and crosses no network**

# Board or Direct Message




The Coder answers with a question. The correlation id 7f3 ties that reply to the message it answers.

SpiralTrain developer courses

# Shared State

- The project board is the shared state of a graph, a set of keys every node can read
- The rule Tester and the bot Tester run side by side and both report to the board
- The gate starts only after both have finished :

```
builder.add_edge("coder", "rule_tester")
builder.add_edge("coder", "bot_tester")
builder.add_edge(["rule_tester", "bot_tester"], "gate")
```

- Bus depot demo : `parts` writes stock, `manuals` writes the error guidance, and the report reads both

SpiralTrain developer courses

# Full History

- Full history gives the next agent every message the caller has
- A Coder given the whole chat reads the brief, the pitches that lost and every old bug report
- Each new message makes the next call larger
- A later agent can repeat a lookup that already sits in that history
- Bus depot demo : the receiver had the stock note, and its saved answer still left out the stock count

SpiralTrain developer courses

# Agent Context

- **Four choices for what the next agent receives :**
  - Full history : every message so far
  - Task only : one selected payload, such as the rules
  - Fork : a copy of the history plus a new task
  - Summary : the agent works apart and returns selected facts
- **Task only leaves earlier facts out on purpose**
- **A returned summary can drop a condition the caller needed**

demo01b · full-history-task-or-summary

# What Each Agent Sees

SpiralTrain developer courses

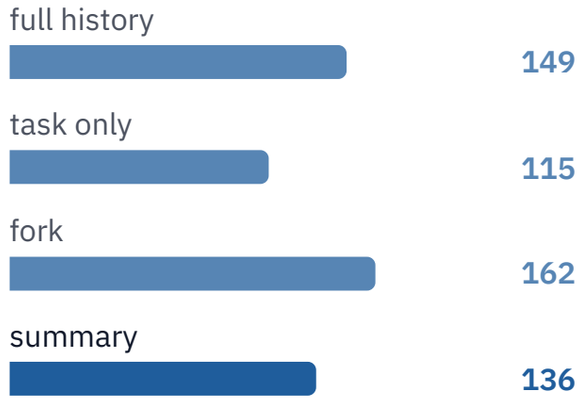
THE PRODUCER HAS

- brief
- pitch : golden
- pitch : shield
- the 8 rules
- bug report
- summary : 7 colours chosen

THE ARTIST SEES ON THIS CALL

- the 8 rules

INPUT TOKENS, BUS DEPOT DEMO




Summary : the Artist works apart and returns one short message. Any condition it leaves out is lost to the Producer.

SpiralTrain developer courses

# Handoffs

- A handoff passes control to another agent, such as from the Designer to the Coder
- A subgraph is a graph used as one node of a larger graph
- A Designer inside a subgraph reaches the Coder in the parent graph with `Command` :

```
def handoff(state):
    return Command(
        goto="coder",
        graph=Command.PARENT,
        update={"messages": state["messages"]},
    )
```

- `Command.PARENT` targets the closest parent graph, and `goto` names a node there
- Bus depot demo : a child graph hands over to `manuals`

demo02 · subgraph-communication

SpiralTrain developer courses

# Parent Message Update

- `add_messages` merges the child’s messages into the parent’s list by id
- The Coder’s list then contains the Designer’s notes
- Bus depot demo : the receiver’s list contained the stock note, and the written answer still omitted it
- A handoff made by a tool call must answer that call with a `ToolMessage`
- The module 05 transfer tool refuses a call made beside other tools
- Neither rule applies to this node transfer, which has no pending tool call

SpiralTrain developer courses

# Shared Keys

- A subgraph added as a node shares every key that appears in both schemas
- The Artist's subgraph returns `colours` to the parent
- `working_note` exists only on the child schema
- The parent result does not contain `working_note`
- Bus depot demo : `part_no` and `stock` cross, `working_note` stays behind

demo02b · shared-keys

# Transformed Input and Output

- A wrapper node translates when parent and child use different keys :

```
def mapped_art(state):
    child_input = {"game": state["title"]}
    child_output = art_graph.invoke(child_input)
    return {"palette": child_output["colours"]}
```

- The child input has no `publisher` field
- Leaving a field out of model input does not hide logs or credentials
- Bus depot demo : `mapped_parts` sends `part_no` and leaves out `requester`

SpiralTrain developer courses

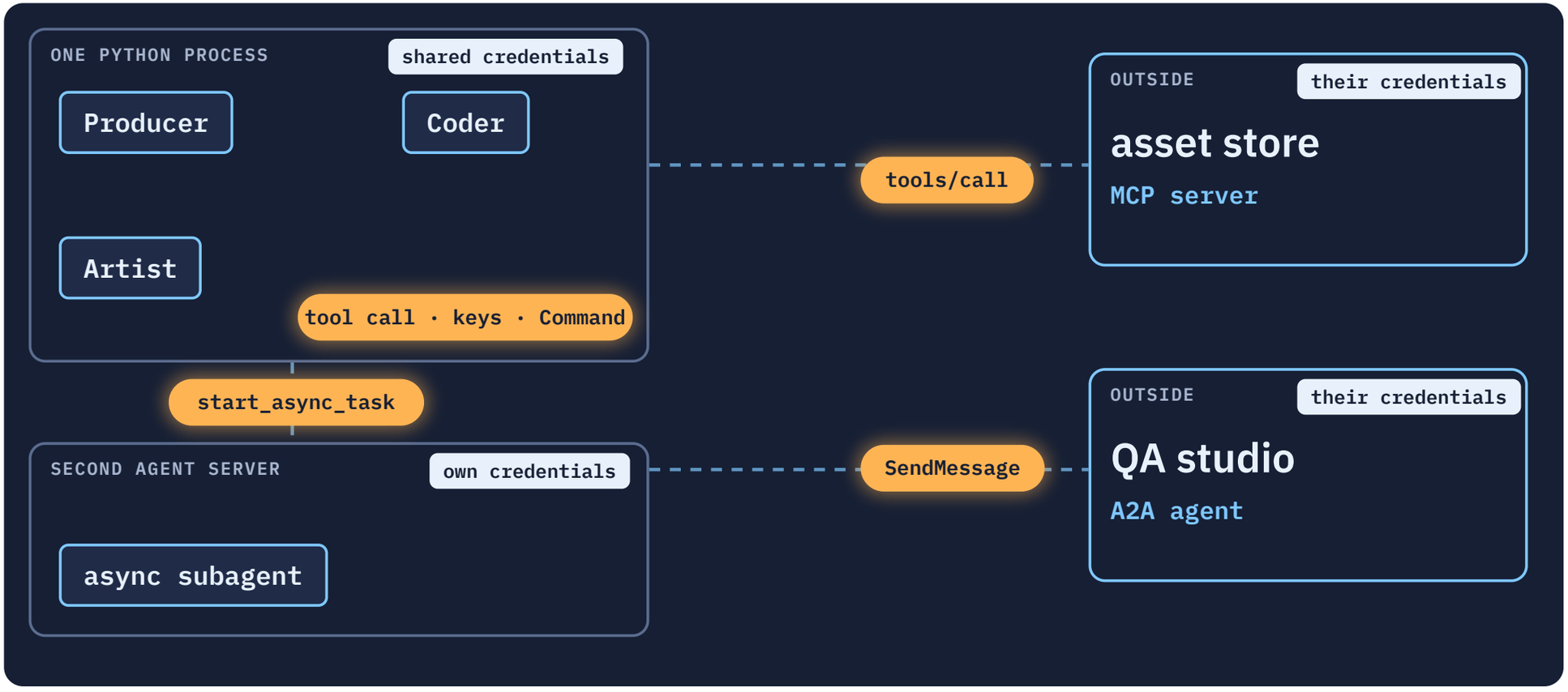
# Communication Protocols

- **The mechanism depends on what separates the two sides :**

| Boundary | Mechanism |
|---|---|
| One process | Tool call, shared keys, `Command` |
| Async subagent | Agent Protocol |
| Two servers | A2A JSON-RPC |
| Tools | MCP |

- **MCP when the caller treats an agent as a tool ; A2A when it needs tasks and a `contextId`**
- **In-process agents can access the same process credentials**
- **Separate servers can use separate storage and credentials**

# Where Each Protocol Sits




Credentials depend on the boxes. Agents in one process can reach the same credentials ; separate servers can have their own.

# The Agentic AI Foundation

- **The Agentic AI Foundation, AAIF, operates under the Linux Foundation and hosts agent standards**
- **Four standards in this course are now hosted there :**
  - MCP : how an agent reaches tools and data
  - A2A : how an agent reaches another agent, since 27 August 2026
  - Agent Skills : folders of instructions an agent loads when needed, since 22 October 2026
  - AGENTS.md : a file that tells coding agents how to work in a repository
- **Neutral governance keeps a protocol out of the control of a single vendor**
- **AAIF hosts 10 projects, among them goose, Open Responses and OpenSandbox**

SpiralTrain developer courses

# Agent Protocol

- Agent Protocol is the HTTP API for threads and runs on an Agent Server
- `POST /runs/wait` starts a run and returns its final state
- Async subagents, which work in the background on another server, use these tools :
  - `start_async_task` and `check_async_task`
  - `update_async_task`, `cancel_async_task`, `list_async_tasks`
- Deep Agents adds those tools to the agent, and the Agent Server answers their HTTP requests

SpiralTrain developer courses

# What is MCP?

- MCP, the Model Context Protocol, is an open standard for offering tools to any agent
- An MCP server wraps something useful, such as an asset store, behind one fixed set of requests
- An MCP client sits in the agent’s application and asks the server what it offers
- The model receives each tool as a name, a description and an input schema
- The tool’s code and credentials stay with the server

SpiralTrain developer courses

# MCP for Tools

- `langchain.mcp` turns the tools of an MCP server into tools for an agent :

```
async with MCPAdapter(asset_store) as adapter:
    mcp_tools = await adapter.list_tools()
    agent = create_agent(llm, tools=mcp_tools)
```

- An in-memory FastMCP server needs no port and no process
- Adapted tools are async only, so the agent runs with `ainvoke`
- Agent Server also serves each assistant as an MCP tool at `/mcp`
- The installed `mcp` package implements revision 2026-07-28, which is stateless : no sessions and no `initialize` handshake
- Bus depot demo : `part_stock` over MCP reports CE-DA-03 as stock 0 at Rietpoort

demo03c · mcp-tools

# MCP Discovery and Call




REQUESTS SO FAR `server/discover` `tools/list` `tools/call`

The result returns to the model as a tool message, and the loop goes on. The model cannot tell where the tool ran.

SpiralTrain developer courses

# Served Agents

- **`langgraph dev` runs a graph on a local web server, the Agent Server, so another process can call it**
- **`POST /runs/wait` accepts the graph id, such as `parts`**
- **Bus depot demo, saved runs :**
  - The echo server runs with the LangSmith variables removed
  - The parts server sends metadata to the configured EU endpoint
  - Both Studio banners use `smith.langchain.com`

demo03 · served-agents

SpiralTrain developer courses

# What is A2A?

- **A2A, the Agent2Agent protocol, is an open standard for giving a task to an agent in another system**
- **An outside QA studio can test Comet Snake while its own agents and prompts stay private**
- **Three objects make up the exchange :**
  - Agent card : a public JSON document that says what the agent can do and where to reach it
  - Task : one piece of work, with an id and a status
  - Artifact : a result the task produced
- **A `contextId` ties several tasks into one conversation**
- **Requests use JSON-RPC, a plain format for calling a named method over HTTP**

SpiralTrain developer courses

# Agent Cards

- The card tells a caller what one assistant does and where to send tasks
- The installed server identifies an assistant by UUID, a long generated id :

```
GET /.well-known/agent-card.json?assistant_id=<uuid>
POST /a2a/<uuid>
```

- The server rejects a graph name with "must be a UUID"
- `POST /assistants/search` returns the UUID for that graph id
- Default assistant UUIDs derive from the graph id with `uuid5`
- A2A requires a `messages` key in the input schema
- This card has no security schemes and no signature, both defined in A2A 1.0

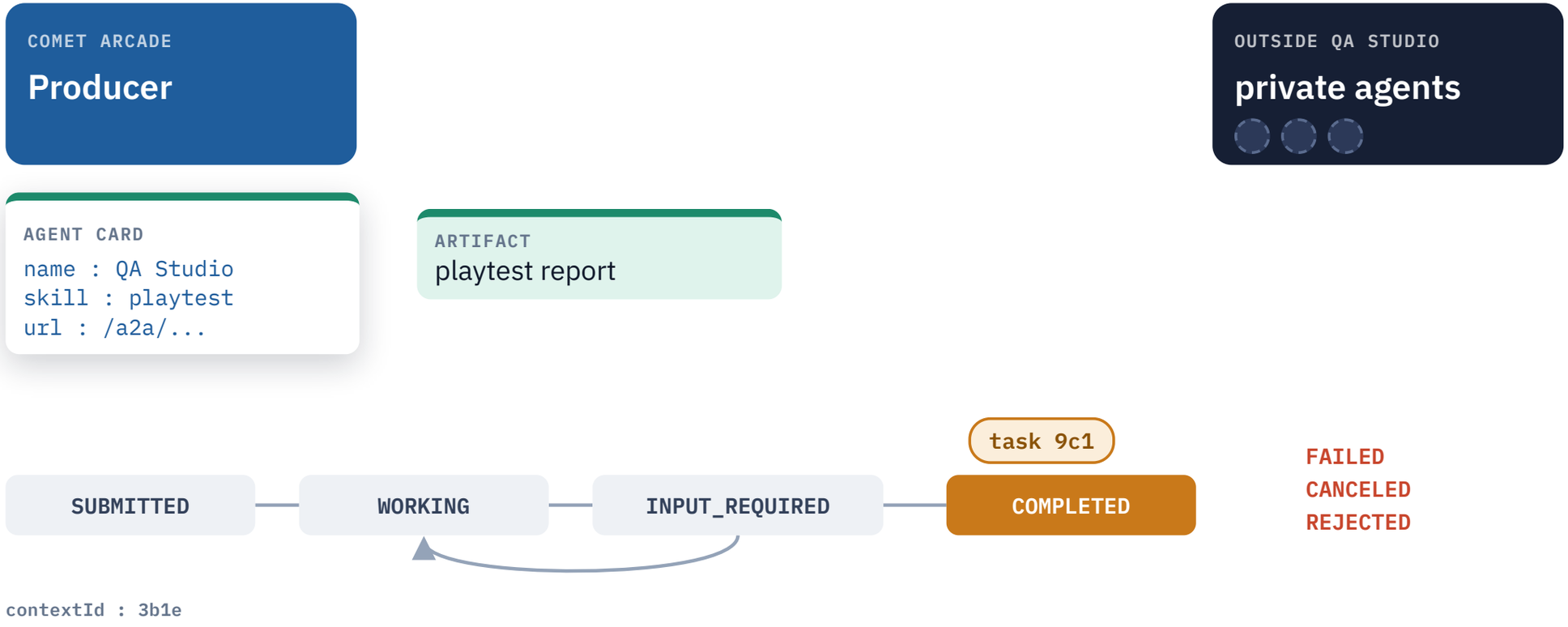
demo03b · agent-card

SpiralTrain developer courses

# A2A Tasks

- **Bus depot demo : the completed `SendMessage` response has `result.task` :**
  - `status.state` is `TASK_STATE_COMPLETED`
  - Artifacts contain the assistant text
- **`GetTask` returns the task directly, with history and no artifacts**
- **On Agent Server, `contextId` is the thread id and must be a UUID**
- **`historyScope` is an Agent Server option beside the A2A 1.0 `historyLength`**
- **Responses still contain the A2A 0.3 field `kind`**
- **`SubscribeToTask` returns -32601, method not found, although the card sets streaming**
- **`message/send`, the older method, unwraps the task and lowercases the state**

# The Life of an A2A Task




FAILED, CANCELED and REJECTED are the other final states. The next request starts a new task with the same contextId.

# Two Agents over A2A

- Bus depot demo : two processes on ports 2124 and 2125 with separate storage
- The coordinator looks up the specialist UUID and posts `SendMessage` :

```
{"jsonrpc": "2.0", "id": "<uuid>", "method": "SendMessage",
 "params": {"message": {"role": "ROLE_USER", "messageId": "<uuid>",
   "contextId": "<thread uuid>", "parts": [{"text": "Report CE-CF-02 stock."}]},
   "configuration": {"historyScope": "task"}}}
```

- Each server allocates its own task id
- A follow-up sends a new message with the same `contextId`

demo04 · two-agents-over-a2a

# Event Systems

- A stream shows a run while it happens, one event after each step
- `updates` mode delivers the values a node returned
- A namespace names the subgraph that produced an event, such as `bot_tester`
- A custom event reports progress to whoever watches the stream
- A custom event leaves the project board unchanged

SpiralTrain developer courses

# Subgraph Streaming

- `version="v2"` yields a dict with `type`, `ns` and `data`

```
for part in graph.stream(
    {"game": "Comet Snake"},
    stream_mode="updates",
    subgraphs=True,
    version="v2",
):
    print(part["ns"], part["data"])
```

- A non-empty `ns` marks an event from inside the child
- The parent update contains the keys the child returned

SpiralTrain developer courses

# v3 Projections

- `stream_events(version="v3")` **returns a** `GraphRunStream`
- **A projection is one view of that stream, such as the values**
- **The caller drives the run by iterating a projection**
- `run.subgraphs` **yields a handle as each child starts**
- **Each child handle exposes a** `values` **projection**
- **v3 is experimental in langgraph 1.2.17**

SpiralTrain developer courses

# Custom Events

- `get_stream_writer` publishes a value beside the state update

```
writer = get_stream_writer()
writer({"event": "playtest-started", "game": state["game"]})
return {"score": score}
```

- The stream asks for `custom` and `updates`
- The custom payload is not written into state
- Bus depot demo : `lookup-started` and `lookup-finished`

demo05b · custom-events

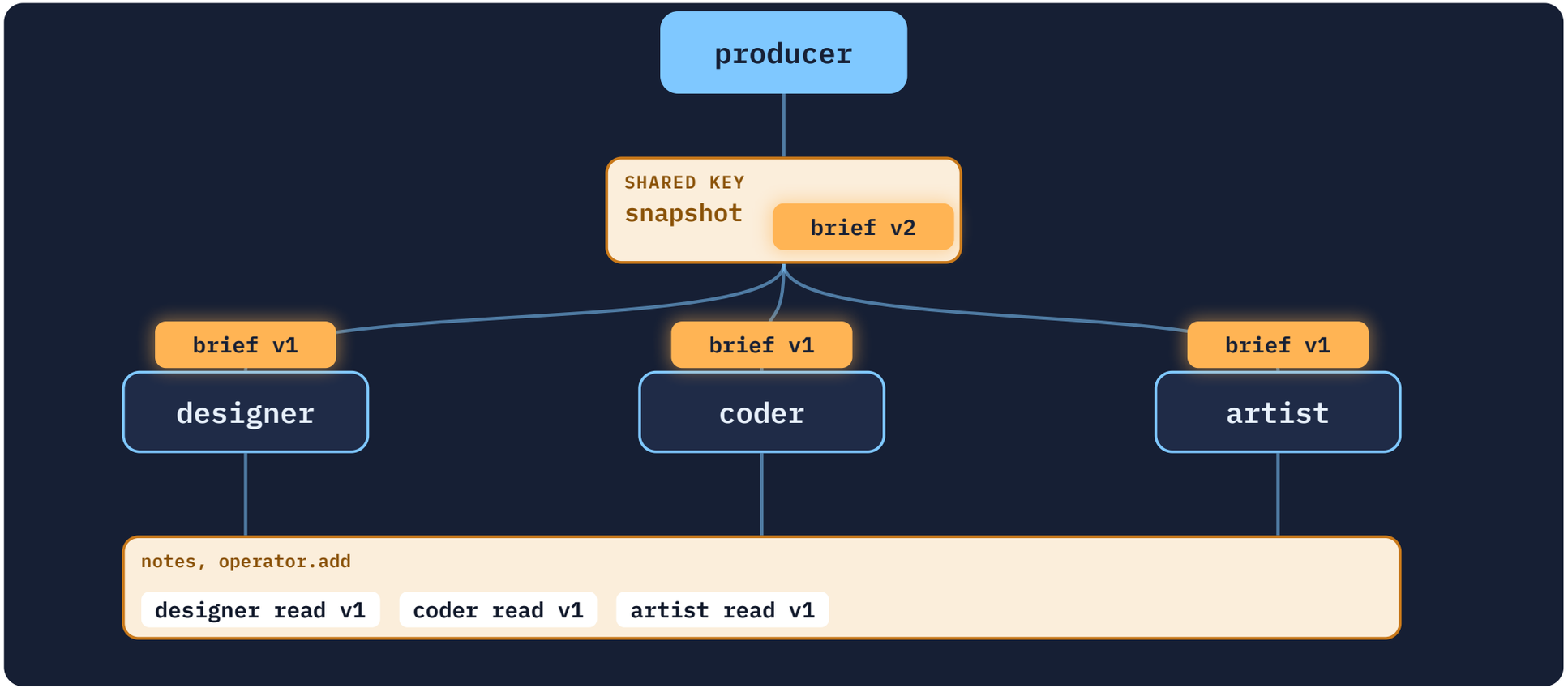
SpiralTrain developer courses

# State Broadcasting

- A broadcast publishes one value for every reader
- The Producer writes a snapshot of the brief on a shared key
- Designer, Coder and Artist each read that key in the next step
- Each returns its own note, and `operator.add` collects the notes
- All three read the value from before their step, so none sees another’s note
- Bus depot demo : seller and buyer both read stock 6 at Rietpoort

demo05c · broadcast-channel

# A Broadcast to the Crew




A later write changes the key. The readers of the finished step keep v1, and the next readers receive v2.

# Synchronization

- A join lists the nodes it waits for :

```
builder.add_edge(["rule_tester", "bot_tester"], "gate")
```

- `defer=True` delays a node until the other pending nodes finish :

```
builder.add_node("gate", gate, defer=True)
builder.add_edge("rule_tester", "gate")
builder.add_edge("bot_tester", "gate")
```

- Without `defer`, separate edges run the gate after the first Tester alone
- The gate raises unless both Testers reported

demo05d · fan-in-and-barriers

SpiralTrain developer courses

# Conflict Resolution

- One owner can run agents with opposing goals
- Yair’s crew post puts a seller bot beside a buyer bot
- Yair calls the pair "a conflict of interest i’ve chosen to ignore"
- Those bots share one computer and the same browser logins
- A studio has the same shape : one Tester says hold the build, the other says ship it
- A few lines of code pick the winner with the same rule on every run

SpiralTrain developer courses

# Conflicting Writes

- Two nodes may write the same key in one step
- A plain key accepts one value per step
- A second value raises `InvalidUpdateError`
- A list with `operator.add` keeps both verdicts
- Both verdicts stay on the board until a later step settles them
- Bus depot demo : `seller` proposes sell, `buyer` proposes retain

demo06 · conflicts

SpiralTrain developer courses

# Voting

- Separate agents vote on one proposal, each in its own model call
- Each vote keeps the role, the choice and the reason
- Code counts the votes afterwards, such as 2 ship and 1 hold
- Voters that read the same evidence can repeat the same mistake
- Bus depot demo : selling 4 of 6 fans breaks a reserve of 4, and the saved vote was 3 retain, 0 sell

demo06b · voting

SpiralTrain developer courses

# Judge and Tie-Break

- One hold vote and one ship vote make a tie
- A judge is an agent that records a decision and a reason
- Code settles it : the majority wins, and a tie holds the build

```
chosen = "ship" if counts["ship"] > counts["hold"] else "hold"
if chosen == "ship" and failed_tests > 0:
    return "hold", "failing test"
```

- A failing test overrides a majority for ship
- Bus depot demo : a tie retains stock, and code overrides two sell votes that break the reserve

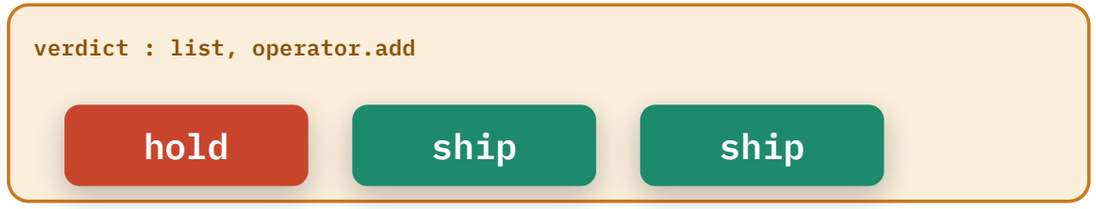
demo06c · judge-and-tie-break

# Conflict Between Two Testers

**rule Tester**
20 of 21 tests pass
`FAIL self-hit`

**Producer**

**bot Tester**
66.0 stars on average
`best score 80`




**hold : a failing test**

`bug report to the Coder`

One rule overrides the count : a failing test holds the build. The bug report goes back to the Coder.

SpiralTrain developer courses

# Seller and Buyer

- **Bus depot demo, two agents and one stock of parts :**
  - The seller proposes a quantity to release
  - The buyer proposes a quantity to keep
- **Settlement keeps at least the reserve**
- **Retention is the greater of the reserve and the buyer’s quantity**
- **The sale is capped by the seller’s quantity and the remaining surplus**
- **The result records `external_action` as false**
- **Saved run : buyer 4, seller 2, settled as sell 2, retain 4**

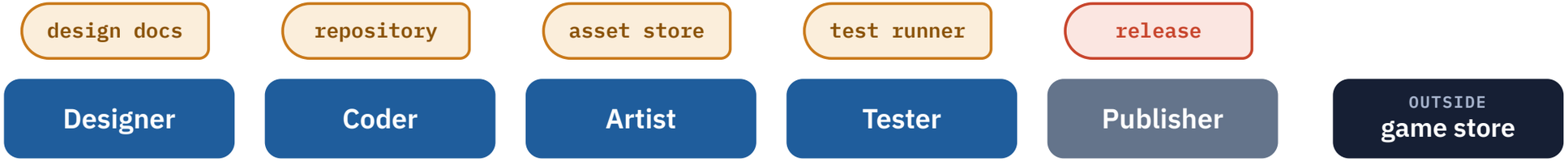
SpiralTrain developer courses

# Least Privilege

- Least privilege : each agent holds only the keys its own job needs
- Shared browser logins expose the same authenticated services to every agent
- `ToolRuntime` passes the principal, the identity of the caller, to the tool
- `context_schema=Principal` tells `create_agent` the shape of that trusted context
- The published tool schema contains `quantity` only, so the model cannot claim another identity
- `authorise` refuses when the seller’s agent tries to draft a purchase

demo07b · least-privilege-per-agent

# Which Agent Holds Which Key




The release key stays with the Publisher, a person. A prompt injected into one agent reaches one key at most.

SpiralTrain developer courses

# Summary : Agent Communication

- A direct message reaches one agent ; a shared board reaches them all
- Handoffs cross graphs with `Command` and `Command.PARENT`
- Agent Protocol and A2A connect agents across processes
- MCP lets an agent discover and call tools in memory or at `/mcp`
- MCP, A2A, Agent Skills and AGENTS.md are all hosted by one neutral body, the AAIF
- A stream shows a run live, with events from subgraphs and custom progress events
- State broadcasting publishes one snapshot for every reader
- Conflict resolution keeps both writes, then applies a vote, a judge or a tie rule
- A list edge or `defer=True` makes a node wait until both branches finish
- Least privilege gives each agent its own credentials and tools

exercise05 · Two Agents over A2A

exercise06 · Buyer and Seller

# Advanced Workflows

From LLM to Agent
Introduction to LangGraph
Graph Fundamentals
State Management
Building Agents
Multi-Agent Patterns
Agent Communication
**Advanced Workflows**
Deep Agents
Production Deployment
Real-World Applications

Advanced Workflows
Human-in-the-Loop
The interrupt Function
Resuming with Command
Approval States
Approve, Edit and Reject
Several Interrupts in One Step
Interrupts in Subgraphs
Interrupts in Subagents
Human-in-the-Loop Middleware
Gating Agent Actions
Per-Call Approval

Two Layers of Approval
Loop Detection in Graphs
A Watchdog Node
Repeated Tool Calls
Retry Strategies and Backoff
Fallback Chains
Subgraphs and Composition
Conditional Branching at Scale
Durable Execution
Resuming After a Crash
Graceful Workflow Drain
Summary : Advanced Workflows

SpiralTrain developer courses

# Advanced Workflows

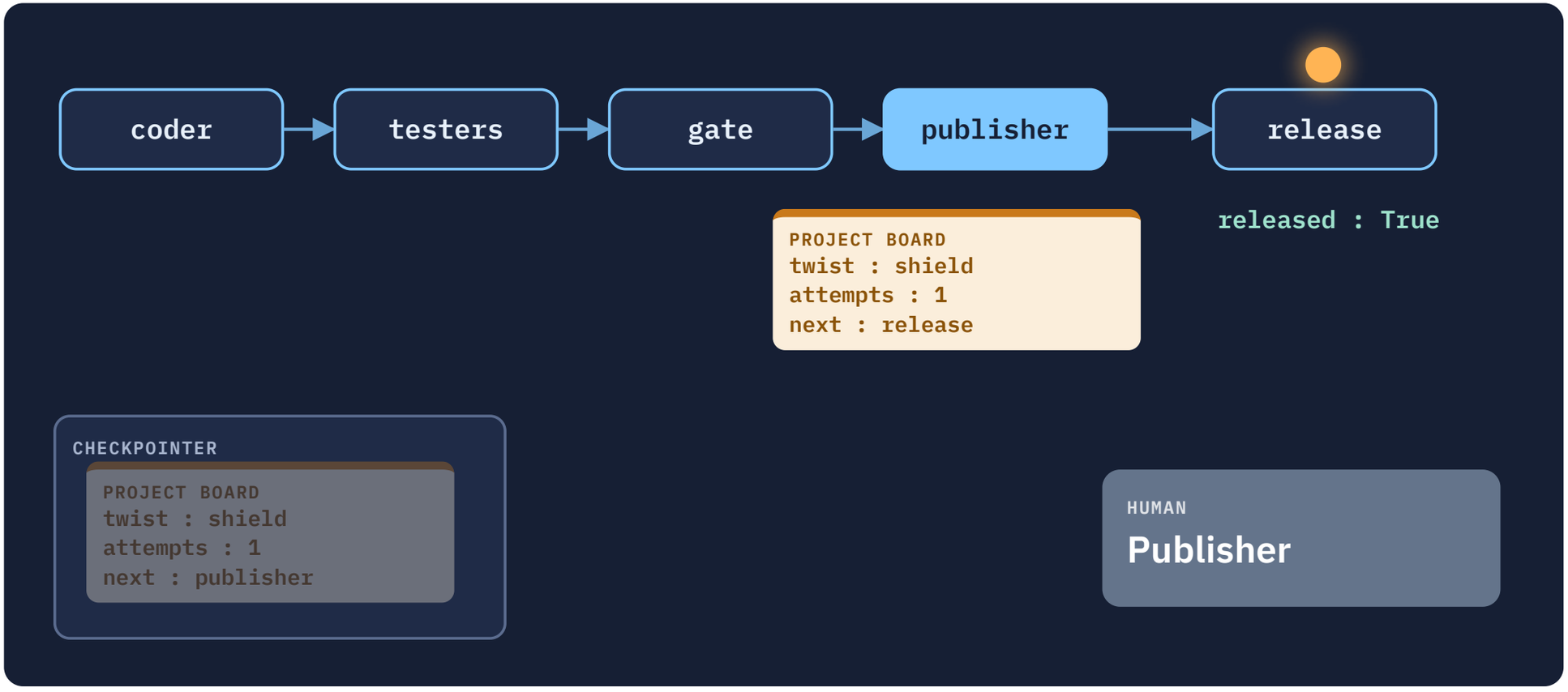
- **A workflow can pause for a human decision :**
  - The Publisher approves a release before it goes out
  - The pause also works inside a subgraph or a subagent tool
- **A workflow can bound its own loops :**
  - A watchdog node stops a fix loop that makes no progress
  - A guard blocks a repeated tool call
- **A workflow can recover from failure :**
  - Retries with backoff around a call to a remote asset store
  - A fallback chain from live data to a cached copy to a person
  - Resume from a checkpoint after a crash or a drain
- **Each team can own one subgraph of a larger workflow**

SpiralTrain developer courses

# Human-in-the-Loop

- **An agent loop can act on a wrong conclusion :**
  - Publish a build that plays badly
  - Replace the released game without sign-off
- **A human-in-the-loop workflow pauses before such an action :**
  - The Publisher, a person, sees the proposed release
  - The checkpointer stores the paused state with no time limit
  - An in-memory saver loses the pause when the process restarts
- **The graph continues only when the Publisher supplies a decision**

# A Run Paused for the Publisher




The run moves on to release. With "no" it would stop here, and released stays False.

SpiralTrain developer courses

# The interrupt Function

- `interrupt(payload)` **pauses the graph inside a node :**

```
from langgraph.types import interrupt

def publisher(state: Board):
    answer = interrupt({"notes": state["notes"], "options": ["approve", "edit", "reject"]})
    return {"action": answer["action"]}
```

- **A pause requires a checkpointer to store the thread state and the payload**
- `invoke(..., version="v2")` **returns the payload in** `out.interrupts[0].value`
- **Plain** `invoke` **returns a dict with the payload under** `"__interrupt__"`
- `out.value` **is the state**
- `graph.get_state(cfg).next` **names the paused node**

demo01 · interrupts

SpiralTrain developer courses

# Resuming with Command

- `Command(resume=value)` **continues the paused thread :**

```
from langgraph.types import Command

graph.invoke(Command(resume={"action": "approve"}), cfg, version="v2")
```

- **The paused node runs again from its first line :**
  - `interrupt(...)` now returns the resume value
  - Statements before `interrupt(...)` execute a second time
- **Side effects belong after the** `interrupt(...)` **call**

# Re-entering the Paused Node

```
def publisher(state):

    answer = interrupt("Release this build?")

    notify_team(state["notes"])

    return {"released": answer == "yes"}
```

**2**
NODE ENTRIES

**1**
NOTES SENT

The fix : side effects go after the interrupt. The note then goes out once, after the approval.

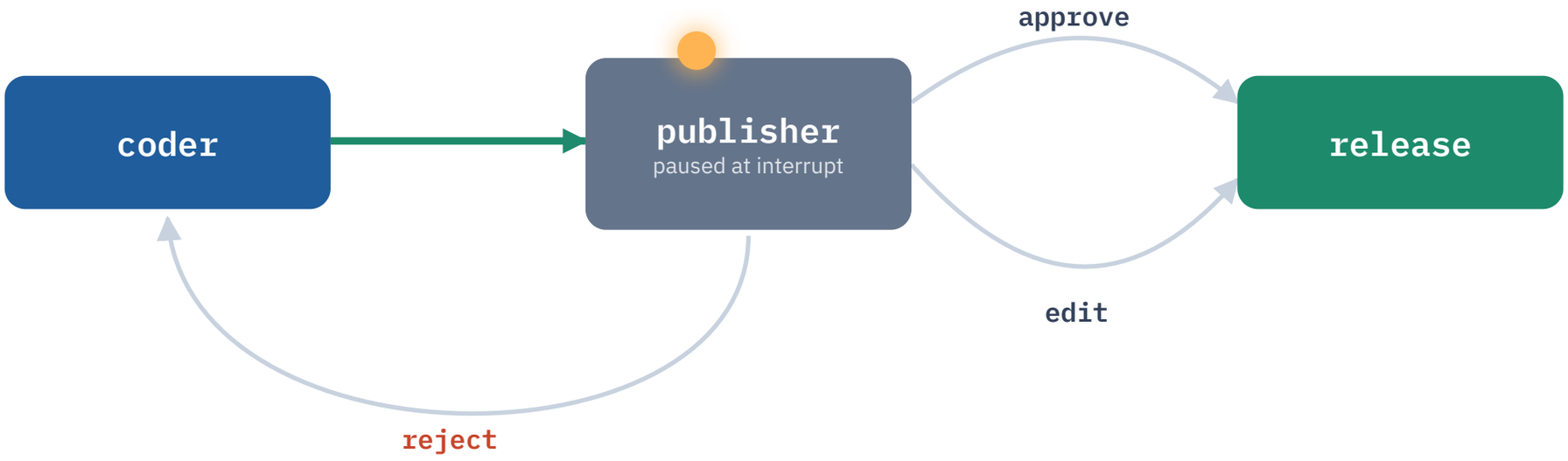
SpiralTrain developer courses

# Approval States

- **The Publisher’s choice picks the next state :**
  - Approve releases the notes unchanged
  - Edit replaces the notes, then releases
  - Reject returns feedback to the Coder
- **The run is in one of these states :**
  - Running : a node is working
  - Pending : the run waits at `interrupt`, and no process is needed
  - Resumed : `Command(resume=...)` has delivered the answer
  - Released : the build went out

SpiralTrain developer courses

# Approve or Reject a Build




The new build passes the gate and reaches the Publisher again. The cycle runs until the answer is approve or edit.

SpiralTrain developer courses

# Approve, Edit and Reject

- The Publisher node routes on the decision :

```
def publisher(state: Board) -> Command[Literal["release", "coder"]]:
    answer = interrupt({"notes": state["notes"], "options": ["approve", "edit", "reject"]})
    if answer["action"] == "approve":
        return Command(goto="release")
    if answer["action"] == "edit":
        return Command(update={"notes": answer["text"]}, goto="release")
    if answer["action"] == "reject":
        return Command(update={"feedback": answer["text"]}, goto="coder")
```

- `Command(update=..., goto=...)` updates state and routes in one return
- Edit replaces the notes without a model call
- Reject passes the Publisher's notes to `coder` for a rewrite

demo01b · approve-edit-reject

SpiralTrain developer courses

# Several Interrupts in One Step

- **Two `interrupt` calls in one node pause one after the other :**
  - LangGraph matches resume values to calls by their order
  - The order of the calls must not change between runs
- **Parallel nodes can pause in the same step :**
  - `out.interrupts` has one `Interrupt` per node, each with its own `id`
  - The resume value maps each `id` to its reply :

```
replies = {item.id: f"approved {item.value['check']}" for item in out.interrupts}
checks.invoke(Command(resume=replies), check_cfg, version="v2")
```

- **One `id` per resume also works**
- **Finished siblings do not run again**

demo01c · several-interrupts

SpiralTrain developer courses

# Interrupts in Subgraphs

- A child graph compiles without a checkpointer and mounts as a node
- The parent’s checkpointer stores the child’s state
- An interrupt in the child reaches the parent caller :
  - `parent.invoke(...)` returns it in `out.interrupts`
  - `parent.get_state(cfg, subgraphs=True)` shows the paused child node
- Resuming the parent thread delivers the reply to the child
- A release team subgraph can ask the Publisher while the crew waits above it

demo02 · interrupts-in-multi-agent

SpiralTrain developer courses

# Interrupts in Subagents

- **A subagent tool can invoke a child graph that pauses :**
  - The Producer calls the release specialist as a tool
  - The child graph reaches `interrupt` and the pause reaches the Producer
  - The Producer's graph is paused in its `tools` node
- **Resuming the Producer runs the tool function again :**
  - Code before the child invocation executes a second time
  - The child receives the reply and the tool returns its `ToolMessage`
- **Bus depot demo : two executions of the tool function for one tool call**

`demo02b · in-a-subagent`

SpiralTrain developer courses

# Human-in-the-Loop Middleware

- `HumanInTheLoopMiddleware` **names the tools that need approval :**

```
from langchain.agents.middleware import HumanInTheLoopMiddleware

policy = HumanInTheLoopMiddleware(interrupt_on={
    "publish_build": {"allowed_decisions": ["approve", "edit", "reject"]}
})
agent = create_agent(llm, tools=[publish_build], middleware=[policy],
                     checkpointer=InMemorySaver())
```

- **A call to a listed tool pauses before the tool runs**
- **Tools missing from** `interrupt_on` **run without a pause**

SpiralTrain developer courses

# Gating Agent Actions

- **The interrupt payload contains `action_requests` and `review_configs`**
- **Each action request has the tool `name` and the model's `args`**
- **`Command(resume={"decisions": [...]})` returns one decision per request**
- **Each decision has a type :**
  - `approve` runs the tool with the model's arguments
  - `edit` runs it with `edited_action["args"]`, such as another version number
  - `reject` returns an error `ToolMessage` and the tool does not run
  - `respond` returns the reviewer's text as the tool result, without running the tool
- **Bus depot demo : quantity 2 on approve, quantity 1 after the edit, nothing on reject**

SpiralTrain developer courses

# Per-Call Approval

- **LangChain 1.4.6 adds `interrupt_mode="per_call"` :**

```
gate = HumanInTheLoopMiddleware(interrupt_on={"publish_build": True},
                                interrupt_mode="per_call")
```

- **Batched mode raises one interrupt per model turn ; nothing runs until every decision arrives**
- **Per-call mode raises one interrupt per gated call :**
  - The resume maps each interrupt `id` to one decision
  - An invalid answer raises `ValidationError` and is not saved
  - An approved call runs while the others still wait

SpiralTrain developer courses

# Two Layers of Approval

- **The prompt shapes what the model proposes :**
  - A system prompt can say "After rejection, stop without retrying"
  - A prompt can ask the model to propose before a hard-to-reverse action
- **`interrupt_on` makes the pause mandatory :**
  - The middleware gates the tool, whatever the prompt says
  - A scripted model with no prompt still pauses on every gated call
- **Each layer covers a gap in the other :**
  - The gate cannot keep a model from retrying a rejected action
  - The prompt cannot stop a call the model makes anyway

SpiralTrain developer courses

# Loop Detection in Graphs

- **A cycle in a graph can run without making progress :**
  - The Coder chases a wrong diagnosis while the same test keeps failing
  - Two agents keep reversing each other’s changes
- **`recursion_limit` caps the supersteps, the rounds of node execution, in one run :**
  - Exceeding it raises `GraphRecursionError`
  - The run ends in an error, with no route to a person
- **A progress check in graph state stops a stalled run earlier**

SpiralTrain developer courses

# A Watchdog Node

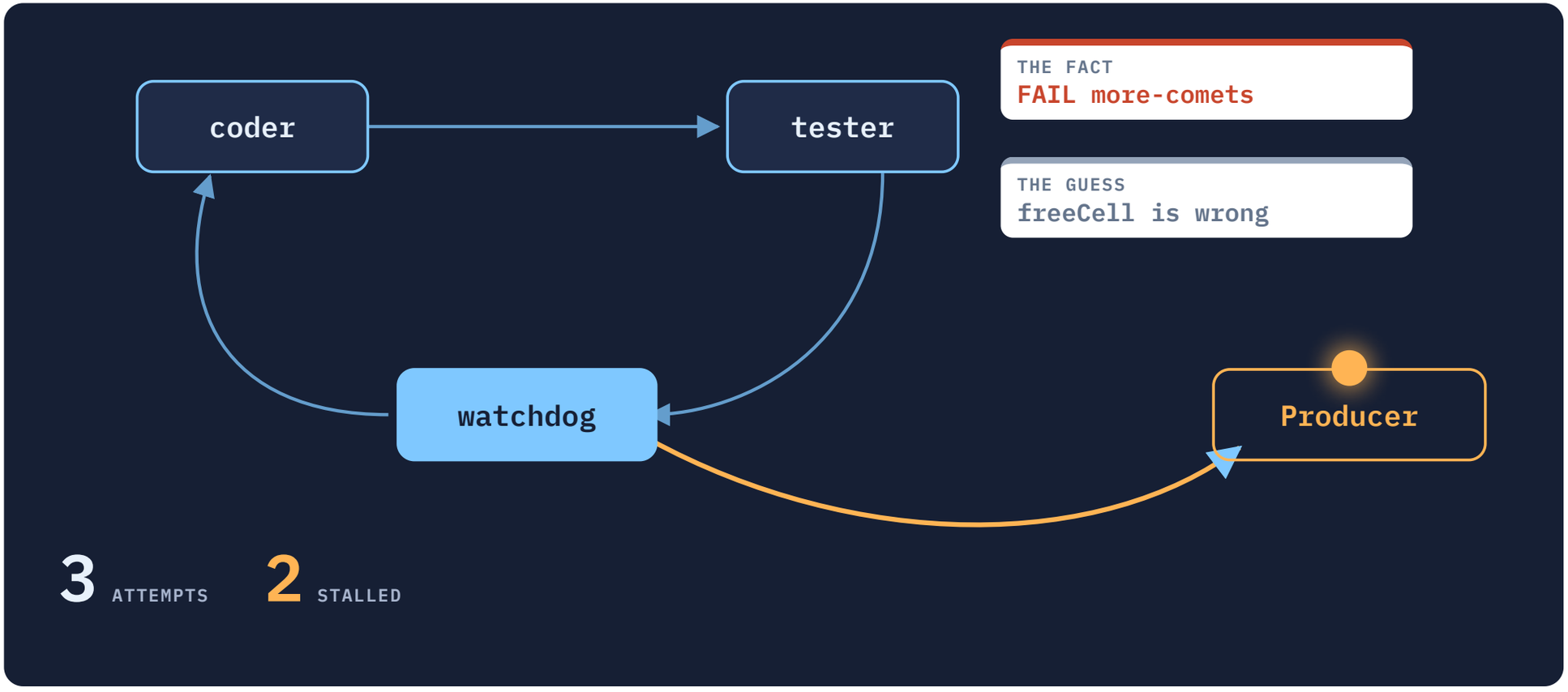
- A watchdog node counts rounds that bring no new evidence :

```
def watchdog(state: FixLoop):
    stalled = state["stalled"] + 1 if state["failing"] == state["previous"] else 0
    stop = stalled >= 2 or state["attempts"] >= 4
    return {"previous": state["failing"], "stalled": stalled,
            "status": "ask the Producer" if stop else "continue"}
```

- The watchdog runs after each round of the fix loop
- Two rounds on the same failing test, or four attempts, send the run to the Producer
- The run stops before it reaches `recursion_limit`
- Bus depot demo : the watchdog routed a stalled diagnosis to manual review on turn 2

SpiralTrain developer courses

# A Watchdog in the Fix Loop




The run leaves the loop with a clear reason. A build that fails does not ship, and a person looks at it.

SpiralTrain developer courses

# Repeated Tool Calls

- A model can repeat the same tool call on consecutive turns
- A signature `(name, sorted_args)` identifies a repeated call
- A guard node checks each call before it reaches the database :
  - The first read runs and stores its signature in `state["seen"]`
  - A repeat returns an error `ToolMessage` and does not run
- A repeat goes to a `stop` node, which ends the run with a manual-review error

SpiralTrain developer courses

# Retry Strategies and Backoff

- **`RetryPolicy` from module 04 retries a node on a transient exception, a failure that may pass by itself :**
  - Retry a timeout or an unavailable asset store
  - Fail a validation error at once ; `retry_on` selects the exceptions
- **Backoff spaces the attempts :**
  - `initial_interval` sets the first wait
  - `backoff_factor` multiplies the wait after each failure
  - `max_interval` caps the wait
  - `jitter` adds a random offset so workers do not retry in step
- **Inside `create_agent`, `ToolRetryMiddleware` retries a failing tool call**

demo05 · retries-and-fallbacks

# Backoff between Attempts




jitter adds a random offset to each wait, so the attempts spread out over time.

SpiralTrain developer courses

# Fallback Chains

- `with_fallbacks` **tries each runnable in turn on a listed exception :**

```
chain = RunnableLambda(live).with_fallbacks(
    [RunnableLambda(snapshot), RunnableLambda(manual)],
    exceptions_to_handle=(ConnectionError, LookupError)
)
```

- **Each tier has less authority than the one before :**
  - `live` reads the asset store and may clear a sprite for release
  - `snapshot` reads a cached copy and marks the answer as possibly stale
  - `manual` asks a person to check
- **A fallback answer states its source and clears nothing for release**
- **In a graph, module 04's** `error_handler` **with** `Command(goto=...)` **gives the same tiers**

demo05b · fallback-chain

SpiralTrain developer courses

# Subgraphs and Composition

- **A subgraph lets one team build and test one stage :**
  - The design team owns the pitch workflow
  - The test team owns the rule and bot workflow
- **A compiled child graph mounts as a node in the parent :**
  - Keys in both schemas pass between parent and child
  - A wrapper function maps state when the child has its own schema
- **Each child graph can be tested and traced on its own**
- **Bus depot demo : a lookup workflow and a report workflow**

demo06 · composition-and-durability

SpiralTrain developer courses

# Conditional Branching at Scale

- **A studio with many teams needs many branches :**
  - An `if/elif` chain in a router grows with every team
  - A routing table keeps the rules apart from the nodes
- `add_conditional_edges` **maps each team to a node via** `path_map` **:**

```
routes = {"puzzle": "puzzle_team", "arcade": "arcade_team", "racing": "racing_team"}
path_map = {name: name for name in set(routes.values()) | {"manual"}}
branching.add_conditional_edges("classify", lambda s: s["team"], path_map)
```

- **An unknown genre routes to the** `manual` **queue**
- **Bus depot demo :** `F-CE-110` **routes to** `electric`

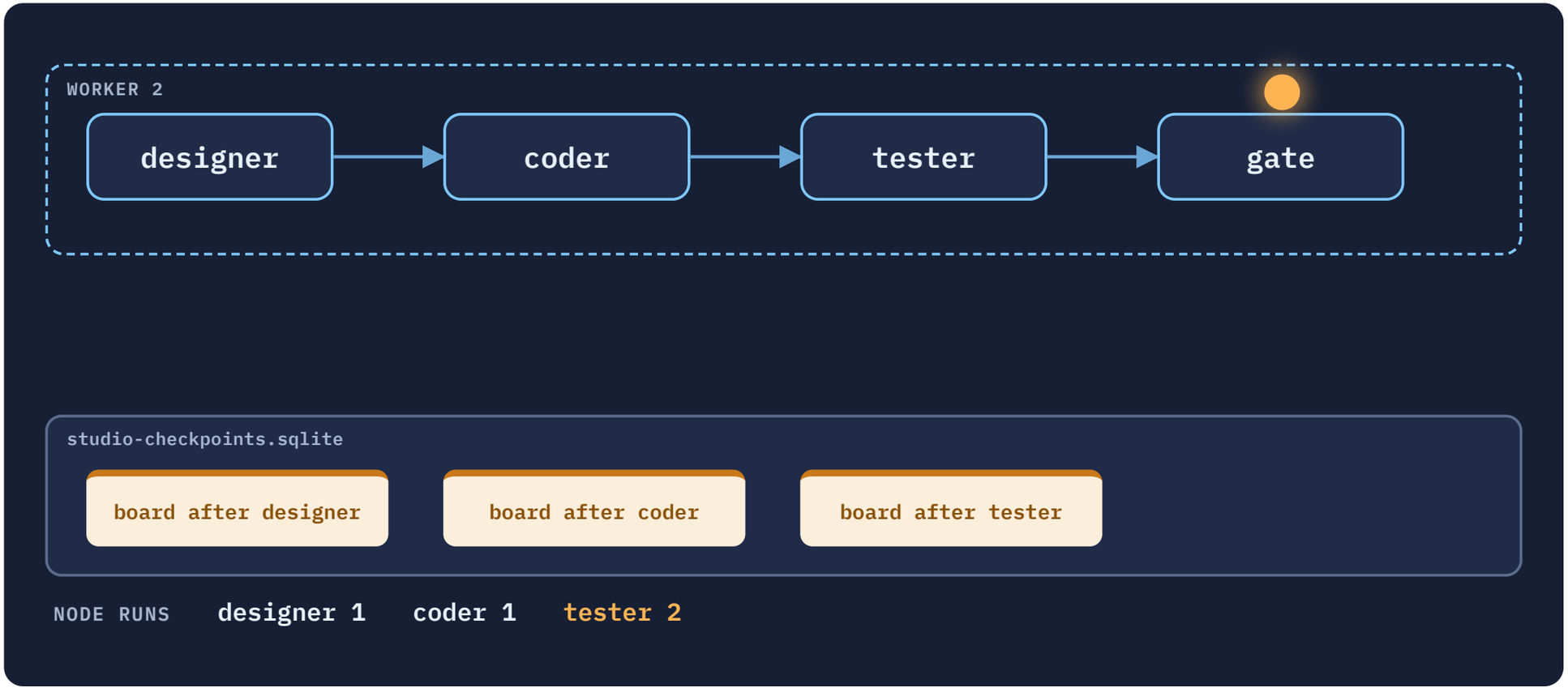
demo06b · branching-at-scale

SpiralTrain developer courses

# Durable Execution

- **Durable execution saves the project board after every step**
- **A crash at the Tester restarts at the Tester**
- **`durability` sets when the save is written :**
  - `"async"`, the default : while the next step runs
  - `"sync"` : before the next step starts
  - `"exit"` : only when the run ends
- **A crash during the Tester leaves the boards saved after Designer and Coder on disk**
- **Their model calls are not paid twice, because Designer and Coder do not run again**

# A Crash at the Tester




The run continues to the gate. A checkpoint cannot undo a write to the outside world made before the crash.

SpiralTrain developer courses

# Resuming After a Crash

- **A checkpointer preserves completed work before a node crashes :**

```
with SqliteSaver.from_conn_string("studio-checkpoints.sqlite") as saver:
    running = durable_builder().compile(checkpointer=saver)
    # Node 'coder' succeeds, node 'tester' raises RuntimeError
    running.invoke(initial, crash_cfg, durability="sync")

with SqliteSaver.from_conn_string("studio-checkpoints.sqlite") as saver:
    restarted = durable_builder().compile(checkpointer=saver)
    recovered = restarted.invoke(None, crash_cfg, durability="sync")
```

- `invoke(None, cfg)` **runs only** `tester` **; the Coder's work is not repeated**
- **Bus depot demo :** `prepare` **and** `finish`

SpiralTrain developer courses

# Graceful Workflow Drain

- **A killed worker can stop in the middle of a node**
- **A drain stops a run between nodes, for instance before a deploy**
- **`RunControl` from `langgraph.runtime` provides it :**
  - A signal handler or a node calls `request_drain(reason)`
  - The running node finishes and its checkpoint is saved
  - The run raises `GraphDrained` before the next node starts
- **`invoke(None, cfg)` on a new worker runs the remaining nodes**

SpiralTrain developer courses

# Summary : Advanced Workflows

- `interrupt` pauses a node ; `Command(resume=...)` supplies the decision
- A paused node runs again from its first line on resume
- Interrupts in subgraphs and subagent tools reach the parent caller
- `HumanInTheLoopMiddleware` pauses calls to the tools it lists, per turn or per call
- A watchdog node stops a stalled run ; a guard blocks a repeated call
- `RetryPolicy` retries with backoff ; `with_fallbacks` falls back by tier
- Subgraphs compose workflows ; a routing table maps each team to a node
- After a crash or a drain, `invoke(None, cfg)` restarts at the first unfinished node

exercise07 · Approval in a Subagent

# Deep Agents


From LLM to Agent
Introduction to LangGraph
Graph Fundamentals
State Management
Building Agents
Multi-Agent Patterns
Agent Communication
Advanced Workflows
**Deep Agents**
Production Deployment
Real-World Applications

What is an agent harness?
A harness on a game maker
Framework, runtime, harness
The agent loop
A todo list
Files in state
A task tool
create_deep_agent
The default prompt
Planning with write_todos
The virtual filesystem
Offloading large results
State backend
Filesystem backend
Store backend
Composite backend
Permissions
The interpreter
Interpreter wiring
Interpreter limits
A local shell
Symlinks and middleware overrides
Memory in AGENTS.md
Skills
Tools bound to skills
Dict subagents
CompiledSubAgent
Fork mode
Fork cost on one question
The general-purpose subagent
Dynamic subagents
A fan-out script
Recursive language models
RLM harness families
Orchestration shapes
Telling an orchestrator when to delegate
A cap on delegation
Delegation measured on gpt-6-luna
interrupt_on
Summarisation
Context limits
State across context windows
Profiles
Deep Agents and the patterns
The Claude Agent SDK
Codex
Managed Deep Agents v0.10
The Academy course
Summary : Deep Agents

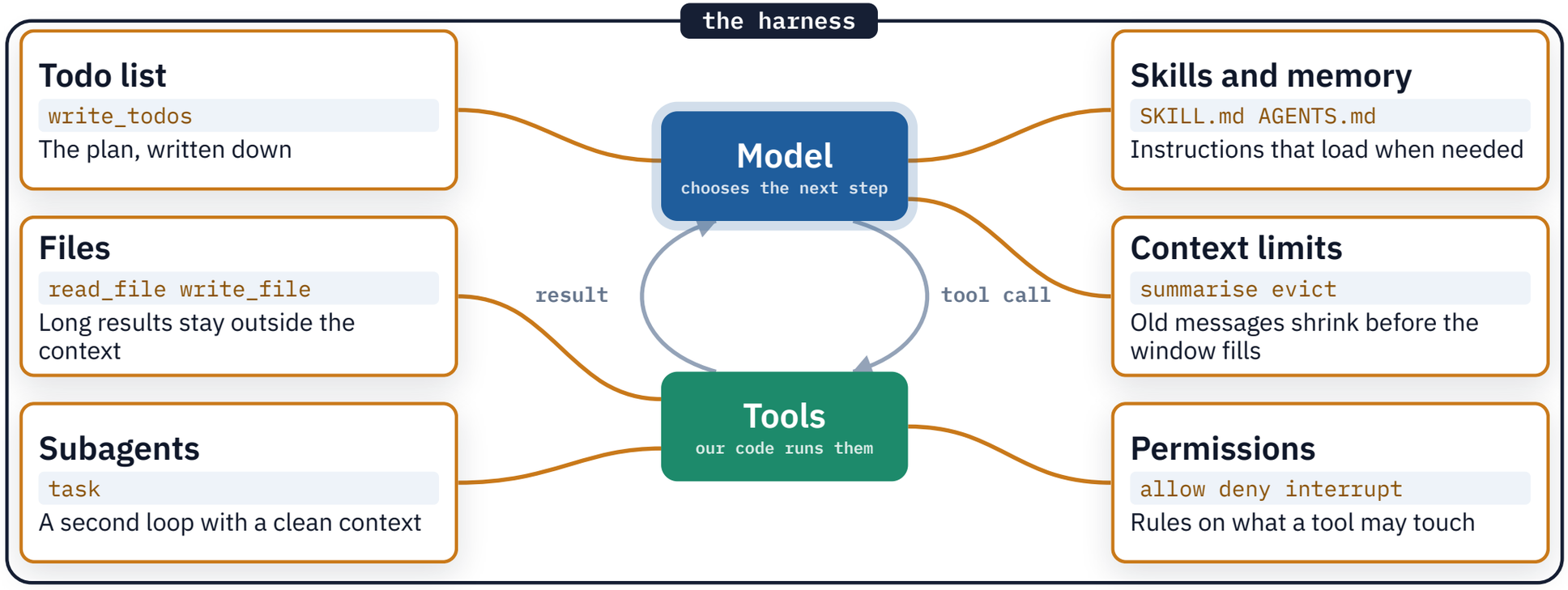
SpiralTrain developer courses

# What is an agent harness?

- **An agent is a model calling tools in a loop until the task is done**
- **A harness is all the code around that loop :**
  - It runs the tools, keeps the files, trims the context and stops the run
  - The model still chooses the next tool call
- **`create_agent` from module 04 is a small harness on the LangGraph runtime**
- **Deep Agents is a larger harness on `create_agent` :**
  - It adds a todo list, files, subagents, skills, memory and context limits
  - The caller still supplies the task, the tools and the permissions

SpiralTrain developer courses

# What a harness adds




All of this is the harness. The model in the middle still chooses every tool call.

# A harness on a game maker

| | Solo agent | Planner, generator, evaluator |
|---|---|---|
| Time | 20 minutes | 6 hours |
| Cost | $9 | $200 |
| Result | Entities appear, nothing responds to input | The entity moves and the game plays |

- **Anthropic built a retro game maker both ways :**
  - The planner turns a prompt of a few sentences into a full spec
  - The generator builds the app
  - The evaluator clicks through the running app with Playwright and grades it
- **The harness run still had a flaw : a large wall blocked the character**

# Framework, runtime, harness

- **Three layers sit under one Deep Agent :**
  - LangGraph, the runtime : runs the steps, saves checkpoints, keeps the store
  - LangChain, the framework : the model interface, tools and agent middleware
  - Deep Agents, the harness : files, the `task` tool and the rules for context
- **Middleware is code that runs around each model call or tool call**
- **The middleware stack implements the patterns of module 05**
- **The course pins `deepagents` 0.7.30, on `langchain` 1.4.6 and `langgraph` 1.2.17**

SpiralTrain developer courses

# A long build at Comet Arcade

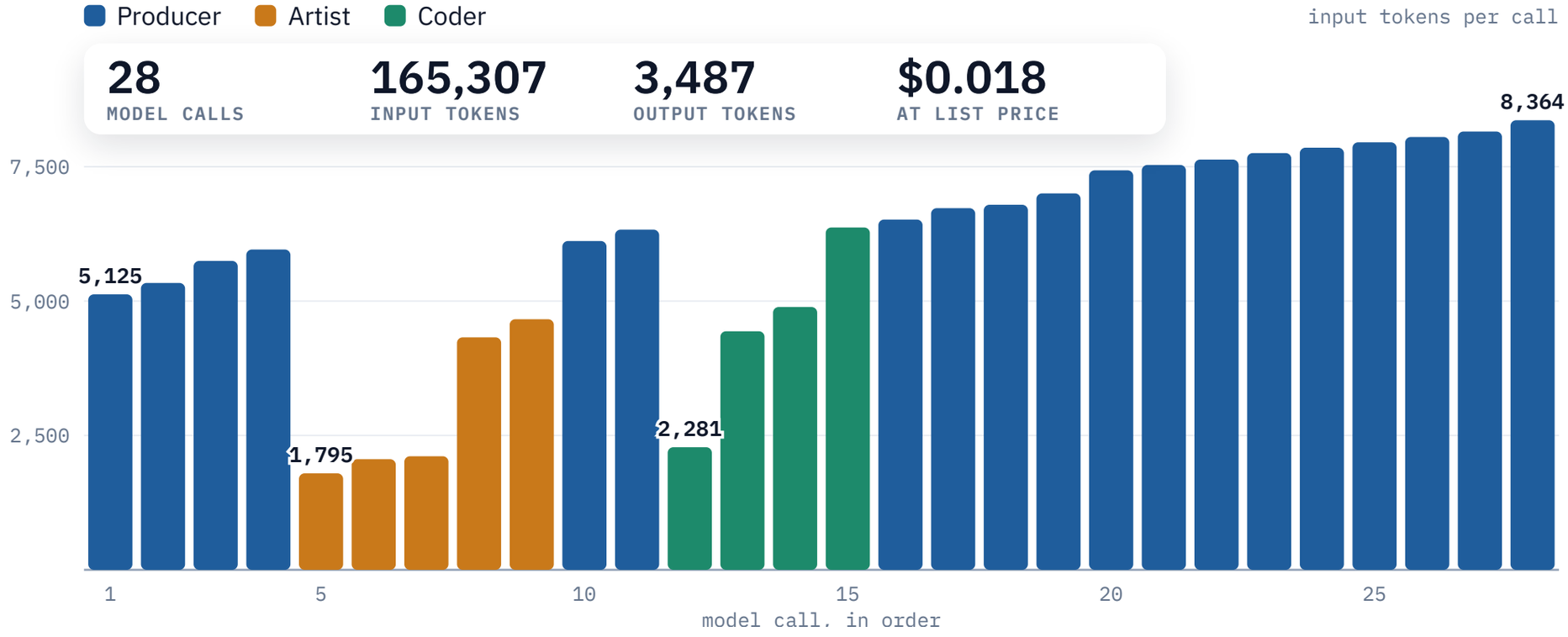
```
producer = create_deep_agent(
    model=llm, tools=[run_game_tests, bot_tick_log],
    subagents=[coder, artist],
    memory=["/AGENTS.md"], skills=["/skills/"],
    middleware=[TodoListMiddleware()],
    name="producer", system_prompt=PRODUCER)
```




- The Producer takes one brief and plans the build
- The Artist writes a theme, the Coder writes the logic
- The harness runs real tests and a bot, and the model reads the result
- 6 of 6 builds passed on `openai/gpt-6-luna`, 20 to 31 model calls, about $0.02 each

SpiralTrain developer courses

# The build call by call




Every call sends the context again. The 28 calls add up to 165,307 input tokens, about $0.02 at list price, in 92 seconds.

SpiralTrain developer courses

# The agent loop

```
plain = create_agent(llm, tools=[part_stock])
result = plain.invoke(ask(STOCK_TASK), LIMIT)
```

- **The notebooks of this module run on Veenhaven Bus, a bus company with a parts store and service manuals**
- **The model asks for a tool, our code runs it, and the model reads the result**
- **One stock lookup in demo01, on `openai/gpt-6-luna` :**
  - `part_stock` ran once
  - 2 model calls, 194 input tokens, 45 output tokens
  - The answer named 6 fans at Rietpoort
- **The callback recorded 0 malformed tool calls**

SpiralTrain developer courses

# A todo list

```
planned = create_agent(
    llm, tools=[part_stock], middleware=[TodoListMiddleware()])
```

- `TodoListMiddleware` **adds the tool** `write_todos`
- **Each call sends the full list, and every item has a status**
- **The Veenhaven stock lookup in demo01b :**
  - The agent wrote one item, called `part_stock`, then marked it completed
  - The middleware does not enforce that order
- **The list costs context on this small lookup :**
  - 4 calls and 5502 input tokens with the list
  - 2 calls and 194 input tokens without it

# A todo list in a long build

PENDING

IN_PROGRESS

COMPLETED

- [x] Read twist rules and choose a survival-focused twist
- [x] Have the artist create the game theme
- [x] Have the coder implement gameplay with the selected twist
- [x] Run twist tests and fix failures if needed
- [x] Playtest bot games and save three ending summaries

MODEL CALL 27 `bot_tick_log` `read_file × 3` `grep × 3` `/build/playtest.md`

All five are ticked. A tick is the agent's own claim ; the test report is the evidence.

SpiralTrain developer courses

# Files in state

```
class ScratchState(AgentState):
    files: Annotated[dict[str, str], merge_files]
```

- `files` is one more key in the graph state
- A reducer that merges dicts lets two tools write without losing a file
- `write_note` returns a short confirmation :
  - The note goes under `files`
  - The message list keeps only the confirmation
- The Veenhaven lookup in demo01c saved `/stock.md` and answered 6 fans at Rietpoort
- 3 model calls, 4160 input tokens, 0 malformed tool calls

SpiralTrain developer courses

# A task tool

```python
@tool
def task(description: str) -> str:
    """Delegate a self-contained Veenhaven Bus parts lookup to the parts worker."""
    result = worker.invoke(ask(description), LIMIT)
    return answer(result)
```

- The worker is a second `create_agent` with its own message list
- The parent receives one sentence
- In demo01d the worker has `part_stock`, and the parent has `task` and `write_note`
- The worker's tool results stay in the worker's messages
- 5 model calls, 4450 input tokens, 0 malformed tool calls

# create_deep_agent

```
packaged = create_deep_agent(model=llm, tools=[part_stock],
    system_prompt="Use the parts tool. Save a short finding in /stock.md, then answer.")
```

- **One call builds the parts of demo01 and more**
- **Default tools in 0.7.30 :**
  - Files : `ls` , `read_file` , `write_file` , `edit_file` , `delete` , `glob` , `grep`
  - Shell and delegation : `execute` and `task`
- **The Veenhaven lookup in demo02 :**
  - `part_stock` , then `write_file` for `/stock.md`
  - 3 model calls, 6269 input tokens, 88 output tokens

SpiralTrain developer courses

# The default prompt

- `write_todos` **is absent until** `TodoListMiddleware` **is passed**
- **Without** `system_prompt` **and a harness profile, our own prompt text is empty :**
  - `BASE_AGENT_PROMPT` has been deprecated since 0.7.0
  - Tool schemas and middleware fragments still reach the model
- **A harness profile is a set of defaults for one model name**
- **In demo02 the one-line prompt asks for the file :**
  - No `todos` key was written
  - The callback recorded 0 malformed tool calls

SpiralTrain developer courses

# Planning with write_todos

```
planner = create_deep_agent(
    model=llm, tools=[part_stock], middleware=[TodoListMiddleware()])
```

- **Planning is opt-in on this pin**
- **Three Codex harness profiles are the built-in exception :**
  - `openai:gpt-5.1-codex` , `openai:gpt-5.2-codex` , `openai:gpt-5.3-codex`
  - Each profile adds `TodoListMiddleware` because its prompt mentions todos
- `openai/gpt-6-luna` **matches no harness profile, so the demo passes the middleware itself**
- **Only the tool result confirms a completed item**
- **The Veenhaven lookup in demo02b : 4 calls, 13282 input tokens, 0 malformed tool calls**

# The virtual filesystem

```
inputs = {**ask("Read /manual.md and report the first action for F-CE-110."),
          "files": {"/manual.md": create_file_data(page)}}
```

- The agent sees paths and file tools, and a backend decides where the bytes are stored
- The default backend keeps files in the graph state and writes nothing to disk
- The notebook directory has no `manual.md` after the read
- `read_file` takes `offset` and `limit` to read a window of lines
- The seeded page in demo03 is the Veenhaven CE-12 error-code manual

SpiralTrain developer courses

# Offloading large results

- **A tool result over 20000 tokens moves to a file ; tokens are counted as characters divided by 4**
- **The model receives the path and a short preview**
- **The Veenhaven demo lowers `tool_token_limit_before_evict` to 600 :**
  - 31 manuals are 18795 characters on disk
  - The tool result was 19626 characters and went to `/large_tool_results/`
  - The model saw the path and a preview, then reported 2 sources
- **That answer is wrong because the preview is too short to count the files**

# A large result moves to a file

CONTEXT OF THE NEXT MODEL CALL

FILES OF THIS RUN, IN THE GRAPH STATE

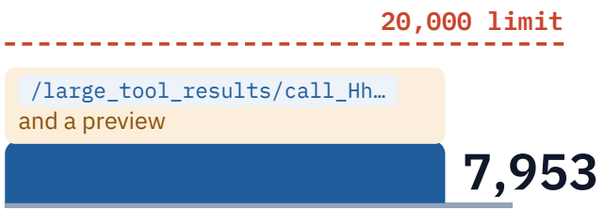
/AGENTS.md

/kit/rules.md logic.js theme.json twists.md

/build/logic.js theme.json

**/large_tool_results/call_Hh…** 289,213 characters

| | |
|---|---|
| 1091 game 2 tick 195 head 22,2 dir down | 1092 game 2 tick 196 head 23,2 dir right |
| 1121 game 2 tick 225 head 13,11 dir down | 1976 game 2 over : score 66, self |
| 3004 game 3 over : score 65, self | 896 game 1 over : score 56, self |




SECOND CALL, INPUT TOKENS




One run of a shorter log, with this limit and with eviction off : the next call reads 2,516 tokens, or 73,810.

SpiralTrain developer courses

# State backend

```
state_agent = create_deep_agent(
    model=llm, backend=StateBackend(), checkpointer=InMemorySaver())
```

- `StateBackend` is the default
- Files for a thread are stored in that thread's graph state
- The checkpointer returns them on the next turn of the same thread
- A second `thread_id` starts with no files
- `InMemorySaver` loses its data when the process ends

demo04 · backends

SpiralTrain developer courses

# Filesystem backend

```
disk = FilesystemBackend(root_dir=disk_root, virtual_mode=True)
```

- `virtual_mode=True` **maps** `/note.md` **to a file under** `root_dir`
- **A path containing** `..`**, or one that resolves outside the root, raises** `ValueError` **:**
  - `..` gives `Path traversal not allowed`
  - An absolute host path gives `... outside root directory`
- `~/x` **is an ordinary name under the root, and reads as not found**
- **These rules cover the file tools ; a shell command is outside them, as demo05 shows**

# Store backend

```
shared = StoreBackend(
    store=store, namespace=lambda _runtime: ("veen-service", "user-a"))
```

- A store keeps a file after its conversation ends, and other threads can read it
- This pin requires the `namespace` callback, which names the folder in the store
- Later invokes under one namespace see the files
- A second namespace reads the same path as missing
- `InMemoryStore` keeps the rows until the process exits
- A durable store would keep them across processes ; this demo starts none

demo04c · store

SpiralTrain developer courses

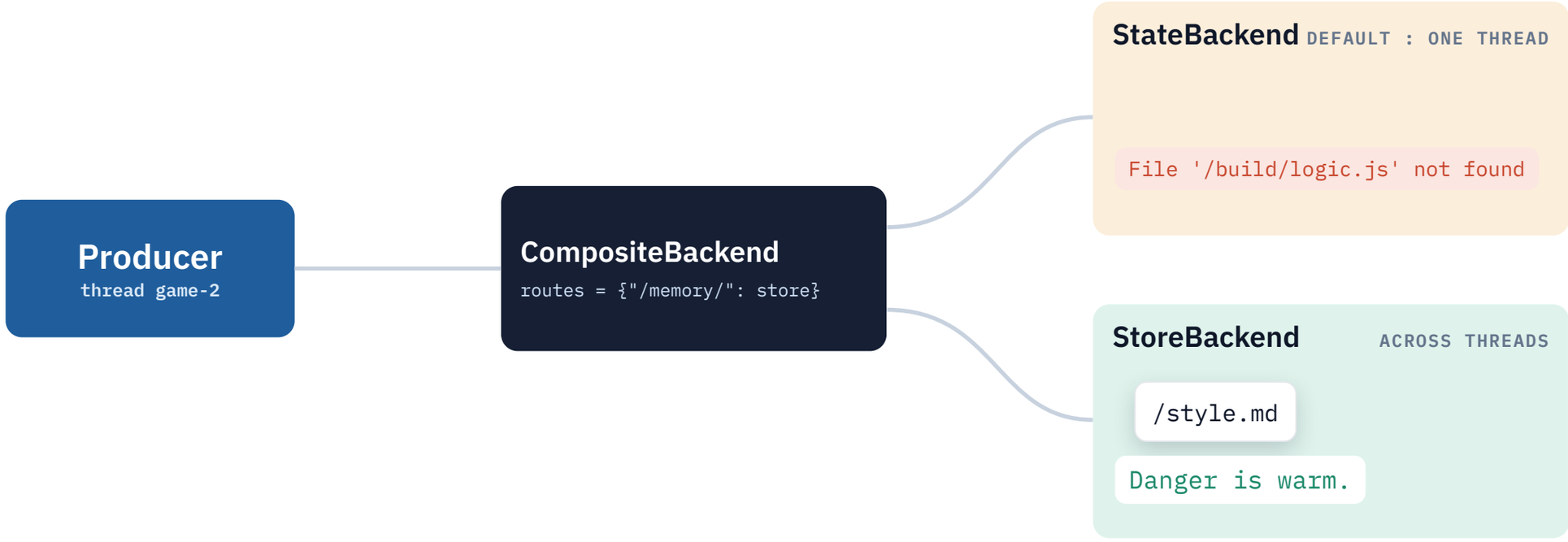
# Composite backend

```
mixed = CompositeBackend(
    default=StateBackend(), routes={"/memory/": shared})
```

- One agent uses several backends, and the path of each file selects one
- The longest matching prefix selects the backend
- `/memory/policy.md` is stored on the route as `/policy.md`
- A path with no matching prefix stays on the default, here state
- The file tools apply `permissions` ; a direct backend call skips them

demo04d · composite

# A path chooses the backend




The read of /memory/style.md reaches the store : "Danger is warm." The style guide is kept after the game.

SpiralTrain developer courses

# Permissions

```
rules = [
    FilesystemPermission(operations=["write"], paths=["/reports/**"], mode="allow"),
    FilesystemPermission(operations=["write"], paths=["/**"], mode="deny"),
]
```

- The rules are read from the top, and the first match wins
- `mode` is `allow`, `deny` or `interrupt`
- An operation is `read` or `write`
- No matching rule means the call proceeds
- A subagent inherits the list, or replaces it with its own `permissions`

demo05 · permissions-and-execution

# The first matching rule

WRITE
/build/logic.js

RULES, READ FROM THE TOP

```
operations=["write"]
paths=["/**"]
```
deny

```
operations=["write"]
paths=["/build/**"]
```
allow

deny

Swap the two rules and the same write is denied. The order of the list decides.

# The interpreter

```
analyst = create_deep_agent(model=llm, middleware=[interpreter])
```

- `CodeInterpreterMiddleware` **adds one tool,** `eval` **: the model writes JavaScript and QuickJS runs it**
- **Rows load into the script through an allowed tool, and only the result returns to the model**
- **The Veenhaven ticket question in demo05b :**
  - The script filtered and summed the rows of depot D01
  - The result was 4 tickets and 7.5 hours, and a Python check agreed
- **QuickJS has no built-in host filesystem, network, process or clock API**
- **An exposed tool still runs with the host's permissions**
- **Interpreters are a beta feature**

SpiralTrain developer courses

# Interpreter wiring

```
from langchain_quickjs import CodeInterpreterMiddleware
interpreter = CodeInterpreterMiddleware(
    ptc=[ticket_rows], subagents=False, mode="turn",
    memory_limit=16 * 1024 * 1024, timeout=2,
    max_ptc_calls=2, max_result_chars=1200)
```

- PTC, programmatic tool calling : a script calls a Python tool as `tools.ticketRows({})`
- `ptc` is the list of tools a script may call
- The import is from `langchain_quickjs`
- The raw rows reach the script without passing through a parent tool message

SpiralTrain developer courses

# Interpreter limits

- **Limits apply to different parts of the run :**
  - `memory_limit` bounds the QuickJS heap
  - `timeout` counts VM execution time ; host calls can take longer
  - `max_ptc_calls` caps `tools.*` calls in one `eval`
  - `max_result_chars` truncates result and console output separately
- **REPL state persists by `mode` :**
  - `thread` across turns, `turn` within one turn, `call` for one invocation
- **PTC calls bypass per-tool parent approval :**
  - Gate `eval` or put approval in the called agent

SpiralTrain developer courses

# A local shell

```
local = LocalShellBackend(
    root_dir=SHELL, virtual_mode=True, inherit_env=False)
```

- `execute` runs the host shell with `root_dir` as the working directory
- `virtual_mode` confines the file tools to that directory, and `execute` can still read and write outside it
- In demo05c the command `echo veenhaven` returned `veenhaven`, exit code 0
- `inherit_env=False` keeps the process environment, and its keys, out of the command
- `SHELL` is `notebooks/08-deep-agents/workspace/shell/`, gitignored with the rest of `workspace/`

demo05c · a-local-shell

SpiralTrain developer courses

# Symlinks and middleware overrides

- **Since 0.7.25 a rule is matched against the path and its symlink target :**
  - Before, `FilesystemBackend` followed a symlink after the check
  - A link to a denied file could be read
- **`permissions=` goes to the harness's own `FilesystemMiddleware`**
- **A `FilesystemMiddleware` in `middleware=` replaces it by name :**
  - On 0.7.30 the replacement has no rules
  - The denied read returned `SECRET EXPORT`
  - Passing `_permissions=rules` to the replacement restored the deny
- **0.7.31 copies the rules over and needs `langchain-core` 1.7**

demo05d · an-override-and-its-rules

SpiralTrain developer courses

# Memory in AGENTS.md

```
agent = create_deep_agent(model=llm, memory=["/AGENTS.md"])
```

- `AGENTS.md` contains standing rules for a project, in plain Markdown
- Each path is loaded at the start of the run and appended to the system prompt
- HTML comments are removed before the text goes in
- Several sources are joined in the order given
- Skill bodies stay out of that prompt until a skill is read or pinned
- The Veenhaven demo seeds one rule, the vest rule, and the answer names it

demo06 · memory-and-skills

SpiralTrain developer courses

# Skills

```
name: safety-briefing
description: Use when asked for safety rules before a depot visit.
```

- A skill is a folder of instructions the agent loads when a task needs them
- `skills=["/skills/"]` scans child folders for `SKILL.md`
- The prompt receives the name and the description
- The instructions stay in the file until `read_file`
- `name` must match the folder : lower case, digits and single hyphens, at most 64 characters
- When two sources share a name, the later source wins
- Agent Skills is an open standard that more than 70 agent platforms have integrated

# Memory and skills loading

SpiralTrain developer courses

ONE MODEL REQUEST

**system prompt**

You are the Coder at Comet Arcade ...

A game is one HTML file. Every game has a dark background and one warm danger colour. A comet is red or orange, never blue or grey.

`canvas-game` Use when writing or fixing the logic block of a canvas game.

**tools**

`read_file` `write_file` `edit_file` `play_bot`

**messages**

description of the job

## Common mistakes : Removing the tail twice before the self hit check ...

BACKEND FILES

`/AGENTS.md`

195 characters

Studio rules for every run.

`/skills/canvas-game/SKILL.md`

1,057 characters

name, description

body : pure logic, state, common mistakes

A skill can also hold back a tool. With metadata.include_tools the tool goes into the request once the skill is read.

SpiralTrain developer courses

# Tools bound to skills

- **The 7 October 2026 skills change is in this pin :**
  - `metadata.include_tools` is a space-separated list of tool names
  - Those tools are passed to `SkillsMiddleware`
  - A listed tool stays off the model request until the skill is read or named in `pinned_skills`
- **`pinned_skills` on `invoke` appends each skill body, then clears the list**
- **Set `skills_metadata` to `None` and the next run reads the files again**

```
metadata:
  include_tools: order_fan
```

SpiralTrain developer courses

# Dict subagents

```
parts = {"name": "parts", "description": "Looks up stock and location for one part number.",
         "system_prompt": "Use part_stock. Return the stock and location.", "tools": [part_stock]}
team = create_deep_agent(model=llm, subagents=[parts])  # the demo adds a system_prompt
```

- **A subagent is a dict : a name, a description, a prompt and tools**
- **The parent calls `task` with `description` and `subagent_type`**
- **The default `mode` is `isolated`, where the worker starts from the description alone**
- **A `general-purpose` subagent is added when the caller did not pass that name :**
  - It is offered the parent's tools
  - A harness profile can set `GeneralPurposeSubagentProfile(enabled=False)`

demo07 · subagents

SpiralTrain developer courses

# A subagent with a clean context

**Producer** 6,516

- brief
- write_todos
- twists.md
- write_todos
- task artist
- write_todos
- task coder : one sentence

**17,977** tokens in the worker

**+187** in the parent

SHARED FILES `/kit/rules.md` `/kit/logic.js` `/skills/canvas-game/SKILL.md` `/build/logic.js`

The Coder spent 17,977 input tokens over 4 calls. The Producer's next call grew by 187.

SpiralTrain developer courses

# CompiledSubAgent

```
compiled = {"name": "manuals-graph",
            "description": "A compiled agent for one bus error code.",
            "runnable": manuals_graph}
```

- A finished graph can be a worker without a rewrite
- `runnable` is any graph whose state includes `messages`
- The harness does not wrap it in filesystem, skills or summarisation middleware
- The parent receives the last assistant text, or `structured_response` as JSON when it is set
- `interrupt_on` from the parent is not copied onto this runnable

demo07b · compiled-subagent

SpiralTrain developer courses

# Fork mode

```
forked = {"name": "manuals-fork",
          "description": "Continues this conversation and looks up the error already named.",
          "system_prompt": "Use fault_guidance on the error code in the conversation. Return the
first action.",
          "tools": [fault_guidance], "mode": "fork"}
```

- `mode="fork"` is experimental in 0.7.30
- The worker receives the parent messages, then the task description
- Its `system_prompt` is appended to the prompt it inherited
- A fork cannot declare `skills`, and a `task` call from a fork is refused

demo07c · fork-mode

SpiralTrain developer courses

# Fork cost on one question

- **The same Veenhaven question went to a compiled worker and to this fork :**
  - Fork 8761 input tokens, compiled 4516, parent plus worker
  - The gap is the inherited history
- **Told to leave out the error code, the parent still put F-CE-110 in the description**
- **Both answers were to check the traction cooling fan**

# The general-purpose subagent

- The built-in name is `general-purpose`
- The pin’s description offers research, file search and multi-step tasks
- It is inserted when no subagent uses that name
- `enabled=False` on `GeneralPurposeSubagentProfile` stops the insertion
- With no synchronous subagent left, `create_deep_agent` does not register `task`
- Async subagents use a different middleware and stay available either way

SpiralTrain developer courses

# Dynamic subagents

- In a fan-out one task splits into many workers at once
- With dynamic subagents the agent writes that fan-out itself :
  - The workers are the subagents we configured
  - The model writes the JavaScript that dispatches them and combines the results
- The top-level `task` in a script takes a description and a `subagentType`
- Like the interpreter, this feature is in beta
- A Python loop in the application does the same with the count fixed by our code

demo08 · dynamic-subagents

SpiralTrain developer courses

# An agent-written fan-out

```
const twists = ["golden", "wormhole", "shield", "clock"];
const results = await Promise.all(twists.map((twist) => task({
  description: `Create one ... twist named "${twist}"`,
  subagentType: "designer",
  responseSchema: { twist: string, pitch: string }
})));
JSON.stringify(results);
```

[ ... ]

| | |
|---|---|
| GOLDEN | Chase the golden fruit ... |
| WORMHOLE | Slither through a wormhole ... |
| SHIELD | Grab a shield ... |
| CLOCK | Race the clock ... |

**eval** QuickJS runs the script

**Producer** tool messages : `eval`

The Producer receives one tool message from eval, with four pitches. 6 model calls, 16,577 input tokens, on Comet Arcade.

# A fan-out script

```
const codes = ["F-CE-110", "F-CE-305"];
const schema = {type: "object", properties: {
  error_code: {type: "string"}, action: {type: "string"}
}, required: ["error_code", "action"]};
const found = await Promise.all(codes.map(code => task({
  description: "First action for " + code,
  subagentType: "manuals",
  responseSchema: schema
})));
JSON.stringify(found)
```

- `responseSchema` **returns structured values for dict subagents**
- `max_subagent_concurrency=2` **caps simultaneous dispatches per REPL**

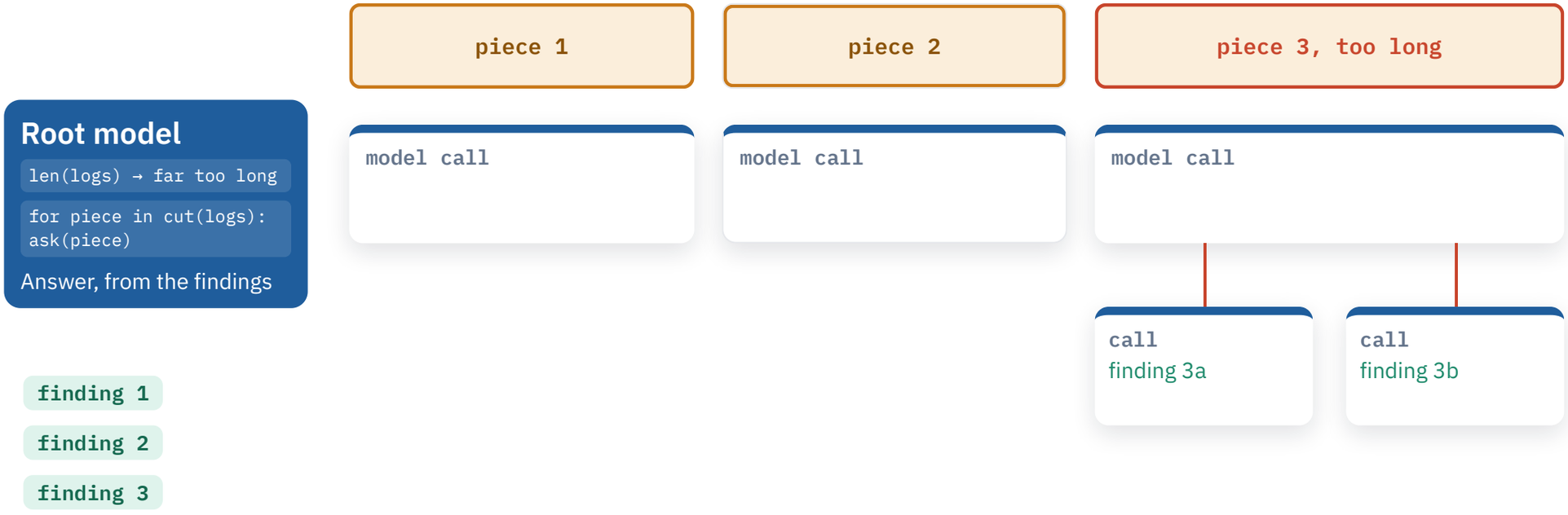
SpiralTrain developer courses

# Recursive language models

- **A recursive language model, or RLM, handles an input too long for one prompt :**
  - The input stays in a code environment as a variable
  - The root model writes code that looks at it and calls a model on the pieces
  - A piece that is still too long can be split again
- **Zhang, Kraska and Khattab, MIT CSAIL, arXiv 2512.24601**
- **On its tasks, the paper reports inputs on the order of 100 times the context window**
- **Dynamic subagents have the same shape : the script calls `task` on each piece**

demo08b · splitting-a-long-input

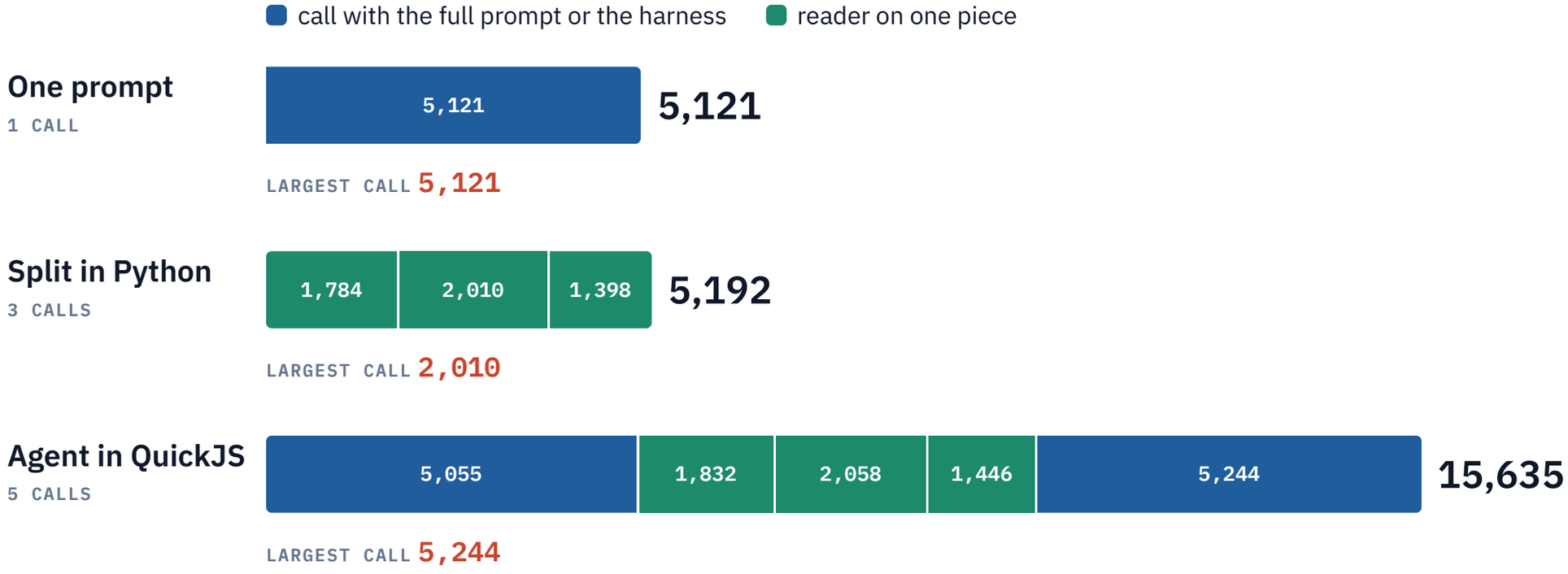
# An oversized input split in code




The root reads the findings and answers. The logs themselves never entered its context.

SpiralTrain developer courses

# Splitting manuals that fit in one prompt




These manuals fit in one prompt, so splitting them costs tokens. Splitting helps when an input is longer than the window.

SpiralTrain developer courses

# RLM harness families

- **Zhang later describes two harness families :**
  - One keeps the trajectory in the prompt
  - One orchestrates with code and sub-calls
- **QuickJS loads the chunks through PTC and calls the same model as a reader :**
  - The root writes the dispatch script and receives compact findings
  - The complete chunks stay out of the root’s model requests
- **The largest Veenhaven root call was 5244 input tokens and the largest reader 2058**
- **This run shows one level of the idea, on a small corpus**

SpiralTrain developer courses

# Orchestration shapes

- **Many units of work and a count known at run time fit a loop in code :**
  - Classify and act : one label selects one specialist
  - Fan-out and synthesise : one call per unit, then one combined answer
  - Adversarial verification : a second pass challenges the first
  - Generate and filter : keep the candidates that pass a check
  - Tournament : compare candidates and keep a winner
  - Loop until done : repeat until a condition, with an explicit cap
- **A handful of fixed roles stays a dict of subagents**

SpiralTrain developer courses

# Telling an orchestrator when to delegate

- **An orchestrator can delegate too much, and every worker costs model calls**
- **Anthropic’s guidance for Claude Opus 5 offers two controls :**
  - A delegation policy in the prompt
  - A deterministic cap on how many agents launch
- **The policy is vendor text for Claude, quoted in the demo :**

```
Delegate to a subagent only for large tasks that are genuinely
independent and parallelizable [...] If one subagent can complete
the task, use one rather than several, and keep spawn counts low.
```

demo08c · delegation-policy

SpiralTrain developer courses

# A cap on delegation

```
ToolCallLimitMiddleware(tool_name="task", run_limit=2)
```

- `ToolCallLimitMiddleware` counts the calls to one tool in a run
- demo08c allows 1 `task` call, demo08d allows 2
- A blocked call returns an error message, and the run continues
- The cap covers the parent's `task` calls ; it does not count inside a worker

SpiralTrain developer courses

# Delegation measured on gpt-6-luna

| Desk | Arm | `task` calls | Model calls | Input tokens | Complete |
|---|---|---|---|---|---|
| With bus tools | Any of the four, cap 1 | 0 | 2 | 4403 to 4555 | 3/3 |
| Specialists only | None | 2 | 6 | 11582 | 3/3 |
| Specialists only | Policy | 2 | 6 | 11669 | 2/3 |
| Specialists only | Cap of 2 | 2 | 6 | 11557 | 2/3 |
| Specialists only | Both | 2 | 6 | 11633 | 2/3 |

- Veenhaven Bus desk, medians of three runs
- With bus tools the policy adds 152 input tokens and changes no median call count
- The largest run without controls : 5 `task` calls, 12 model calls, 21795 input tokens
- The cap limited the largest capped run to 7 calls, and its blocked lookups left the stock out

`demo08d · delegation-with-specialists-only`

SpiralTrain developer courses

# interrupt_on

```
gated = create_deep_agent(
    model=llm, interrupt_on={"write_file": True}, checkpointer=InMemorySaver())
gated.invoke(Command(resume={"decisions": [{"type": "approve"}]}), config, version="v2")
```

- The run stops before the named tool and waits for a human
- `HumanInTheLoopMiddleware` inside the harness makes that pause
- A decision is `approve`, `edit`, `reject` or `respond`
- The graph needs a checkpointer, and resume continues the same thread
- A dict subagent inherits the map unless it sets `interrupt_on` itself
- A `CompiledSubAgent` does not inherit it
- `FilesystemPermission` with `mode="interrupt"` builds the same pause for matching paths

SpiralTrain developer courses

# Summarisation

- **A long run can fill the context window, the limit on what one call can read**
- **The default stack replaces old messages with a summary for the model :**
  - The old messages go to `/conversation_history/{session}.md`
  - `state["messages"]` keeps the original log
- **This model publishes no profile, so the default trigger is 170000 tokens and the last 6 messages stay**
- **The Veenhaven demo sets a trigger of 3 messages so one lookup crosses it**
- `compact_conversation` **comes from** `SummarizationToolMiddleware`**, which the default stack does not add**

demo09b · summarisation

# Old messages move out of the request

NEXT MODEL REQUEST

- summary of messages 1 to 8
- tool · message 9
- ai · message 10
- tool · message 11
- ai · message 12
- tool · message 13
- ai · message 14

/CONVERSATION_HISTORY/SESSION.MD

message 1 | message 2 | message 3 | message 4

message 5 | message 6 | message 7 | message 8

STATE["MESSAGES"]



all 14, unchanged

state["messages"] keeps the original log. Only the request to the model is shorter, and the summary can lose details.

SpiralTrain developer courses

# Context limits

- **Three mechanisms run before a full summary :**
  - A tool result over 20000 tokens is evicted to the filesystem
  - A human message over 50000 tokens is evicted the same way
  - Old `write_file` and `edit_file` arguments can be clipped at a lower threshold
- **The summary call uses the agent model**
- **Fraction triggers apply when `max_input_tokens` is on the model profile**
- **`openai/gpt-6-luna`, constructed as `ChatOpenAI` for this course, has `profile is None`**

SpiralTrain developer courses

# State across context windows

- **A task longer than one context window keeps its state outside the window :**
  - Structured status, such as a test list, goes in a JSON file
  - Free-text progress notes go in a second file
  - Messages and graph state go in the checkpointer
- **Deep Agents has a place for each :**
  - File tools write to a backend ; a store or disk backend keeps files after the thread ends
  - `memory=["/AGENTS.md"]` reloads standing rules at the start of every run
  - Summarisation writes the old messages to a file
- **A fresh start is a new `thread_id` with a prompt that names the files to read first**
- **Anthropic’s guidance for Claude says a fresh window that re-reads the files can beat compaction, its name for summarisation**

SpiralTrain developer courses

# Profiles

- **Profiles are a beta API in two registries :**
  - A provider profile changes how `resolve_model` builds the chat model
  - A harness profile changes prompts, tool visibility and extra middleware after that
- **Built-in harness keys include the three Codex specs and Claude Haiku 4.5, Sonnet 4.6 and Opus 4.7**
- **Lookup for our instance uses provider `openai` and identifier `openai/gpt-6-luna`**
- **With no matching key, the profile is empty :**
  - There is no base prompt and no system-prompt suffix
  - There are no excluded tools and no extra middleware
- **A deployment registers a profile to opt this model into planning or a shorter `task` list**

# Deep Agents and the patterns

| Harness feature | Module 05 pattern |
|---|---|
| `write_todos` | One agent ; the plan is state |
| Skills, `include_tools` | Skills, with tools scoped to the file |
| `task`, isolated | Subagents |
| `mode="fork"` | The worker sees the parent history |
| `CompiledSubAgent` | A custom workflow behind one tool |
| Fan-out in Python | A custom workflow ; the loop is code |
| `interrupt_on` | An approval gate on a tool |

- **Each feature packages a pattern built by hand on Tuesday**
- **demo10 compares both builders on one Veenhaven desk question**

`demo10 · deep-agent-or-graph`

# Graph against Deep Agent

## Lines of builder code




## Model calls




## Input tokens




`both complete` 6 fans at Rietpoort ; check the traction cooling fan

---

Both answered 6 fans at Rietpoort and the cooling-fan check, and both completeness checks passed.

SpiralTrain developer courses

# The Claude Agent SDK

- The Agent SDK runs Claude Code’s agent loop in the application’s process
- The library includes file tools, a shell, subagents, skills, memory, permissions, sessions, hooks and MCP
- Python and TypeScript are the supported libraries
- The Client SDK calls the Claude API, and the application writes the tool loop
- Managed Agents run a hosted harness, with a cloud sandbox or a self-hosted sandbox
- This course runs the Deep Agents harness in-process, on LangGraph, with the course model

SpiralTrain developer courses

# Codex

- **The Codex SDK starts and resumes a local Codex thread**
- **TypeScript package `@openai/codex-sdk` requires Node.js 18 or later**
- **Python package `openai-codex` talks to the local app server over JSON-RPC**
- **`thread.run` continues the thread ; a thread id resumes an older one**
- **Sandbox presets on the Python SDK :**
  - `read_only` reads files
  - `workspace_write` writes inside the workspace and configured roots
  - `full_access` runs without those filesystem restrictions
- **Deep Agents keeps the loop in-process and takes the model the caller built**

SpiralTrain developer courses

# Managed Deep Agents v0.10

- **Managed Deep Agents runs this harness as a hosted service on LangSmith Cloud, in public beta**
- **Version 0.10, 29 October 2026, adds two features :**
  - Browser use : `browser_handoff` gives the user the window for a sign-in and keeps the password out of messages and traces
  - Async subagents : `"async": True` runs a worker on its own thread, and the parent keeps talking until it reports back
- **Version 0.10 removes `connectors.mcp()` , the `connectors/` layout and the sandbox `scope` option ; MCP servers go in `tools/mcp.py`**
- **The Plus plan is $39 per seat per month with 1 free small serverless deployment**
- **Deployment is not on the Developer plan**

# The Academy course

- Introduction to Deep Agents, from LangChain Academy, is a free follow-up
- Six modules cover building, the execution environment, context, delegation, a capstone with async subagents and local deployment, and Managed Deep Agents
- The public repo at `github.com/langchain-ai/lca-deepagents` has no licence
- This course keeps its own wording and its own examples, Comet Arcade and Veenhaven Bus
- Their code pin is `deepagents` 0.7.0 ; this course is on 0.7.30

SpiralTrain developer courses

# Summary : Deep Agents

- A harness packages a todo list, files, skills, memory, subagents and context limits on `create_agent`
- Version 0.7 leaves the base prompt and `write_todos` to the caller, unless a profile adds them
- A large tool result moves to a file, and the model keeps a path and a preview
- File tools, permission rules and `LocalShellBackend` are the execution environment in this pin
- Skills load when needed ; `include_tools` hides a tool until the skill is in context
- Isolated subagents see the task ; fork mode also sees the parent messages
- A long input can be split in code ; `interrupt_on` and summarisation bound a long run

exercise08 · A Deep Agent

exercise09 · Offload and Measure

p. 407

# Production Deployment

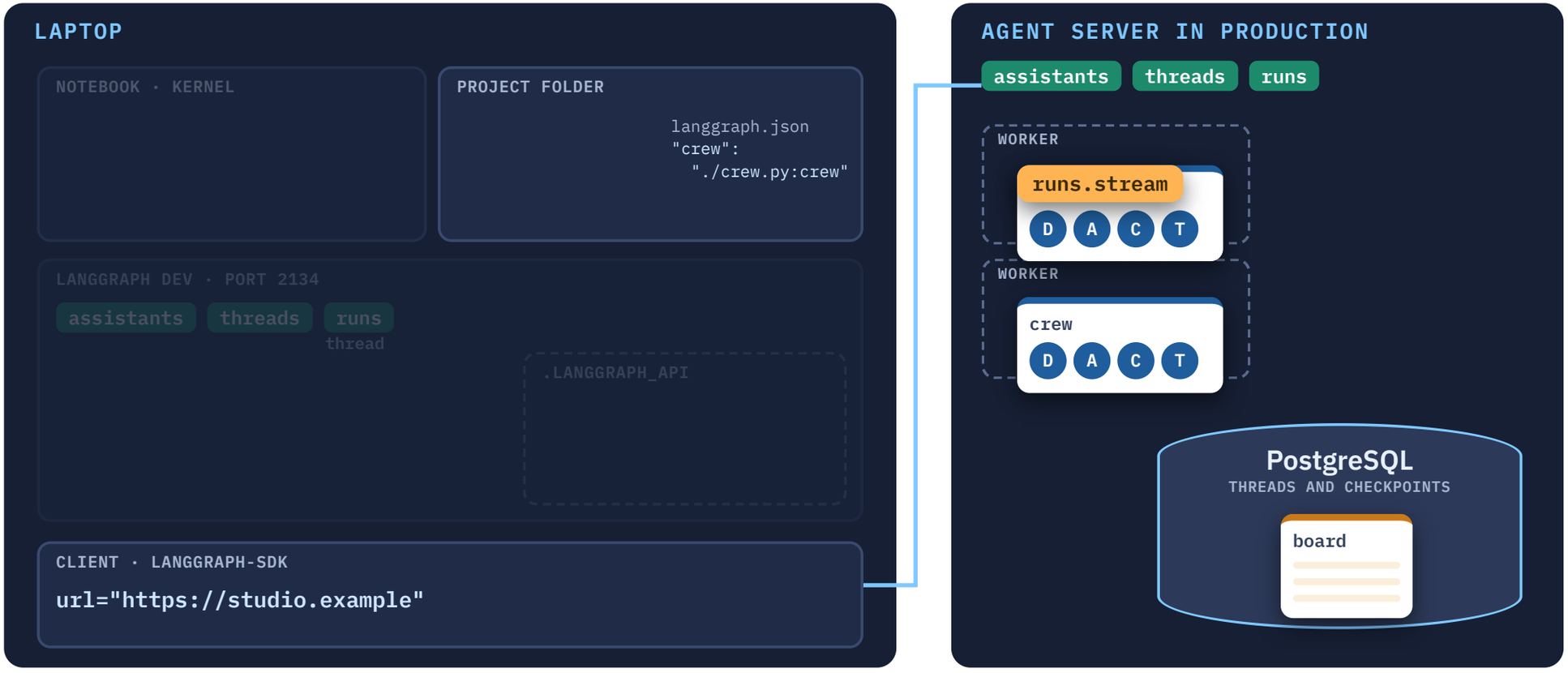

From LLM to Agent
Introduction to LangGraph
Graph Fundamentals
State Management
Building Agents
Multi-Agent Patterns
Agent Communication
Advanced Workflows
Deep Agents
**Production Deployment**
Real-World Applications

Production Agent Services
LangGraph API
Agent Server Resources
langgraph.json
langgraph dev and Studio
Threads and Runs in Practice
Streaming to Clients
Stream Modes and Subgraphs
Double-Texting and Concurrency
Persistence Backends
Checkpoint Storage and TTLs
DeltaChannel
Async Subagents
Async Task Lifecycle
Co-deployed Graphs
Firstmate Orchestration
A2A on Agent Server
Deployment Options
Managed Deep Agents
Scaling Strategies
Evaluating a Multi-Agent System
What the Comet Arcade Evals Found
Limits of the Evidence
Final Answer Evaluation
Trajectory Match
Trajectory Match Modes
Trajectory Judge
Specialist and System Evaluation
pass@k and pass^k
A LangSmith Experiment
Monitoring
Cost Optimisation
Budget Stops
Error Return Channel
Summary : Production Deployment

# Production Agent Services

- **A notebook serves one person, and its memory ends with the kernel**
- **A service has more duties :**
  - Any program can start a build over HTTP
  - The project board is still there after the server restarts
  - A score shows whether a change made the crew better
  - A budget stops a run that spends too much
- **Agent Server is the program that serves LangGraph graphs and Deep Agents over HTTP**
- **The Comet Arcade crew runs on it in this module ; demos 01 to 08 serve Veenhaven Bus graphs**

# From Laptop to Agent Server




The client code stays. Its URL changes, and a second worker takes more runs at once.

SpiralTrain developer courses

# LangGraph API

- **An API is a set of URLs that a program can call. Agent Server offers the LangGraph API**
- **Its three resources :**
  - Assistant : a graph with its default settings, such as the `crew` graph
  - Thread : one saved project board, kept across runs
  - Run : one execution of an assistant on a thread
- **`langgraph dev`, a self-hosted server and LangSmith Deployment offer the same API**
- **Clients call it through the `langgraph-sdk` package**
- **Studio is a web page that calls this API and draws the graph**

# Agent Server Resources

- An assistant binds a `graph_id` from `langgraph.json` to runtime defaults
- A thread keeps checkpointed state across runs
- A run applies input to an assistant on a thread, and returns a status and results
- Background runs support join, cancel and stream
- Follow-ups reuse the same thread id, so earlier messages stay available

SpiralTrain developer courses

# langgraph.json

- One project file tells the server which graphs to load. For the Comet Arcade crew :

```
{
  "dependencies": ["."],
  "graphs": {
    "crew": "./crew.py:crew"
  },
  "env": "../../../.env"
}
```

- `crew` is the name clients use ; `./crew.py:crew` is the compiled graph in that file
- The server adds its own checkpointer, so the graph compiles without one
- demo01 registers three Veenhaven Bus graphs the same way

SpiralTrain developer courses

# langgraph dev and Studio

- One command serves the project on the laptop :

```
uv run langgraph dev --no-browser --port 2134
```

- Studio opens against that address and draws the crew as a graph
- EU accounts add `--studio-url https://eu.smith.langchain.com`
- `/ok` answers once the server is ready for its first client
- `--n-jobs-per-worker` sets how many runs can work at once

demo01b · langgraph-dev-and-studio

SpiralTrain developer courses

# Threads and Runs in Practice

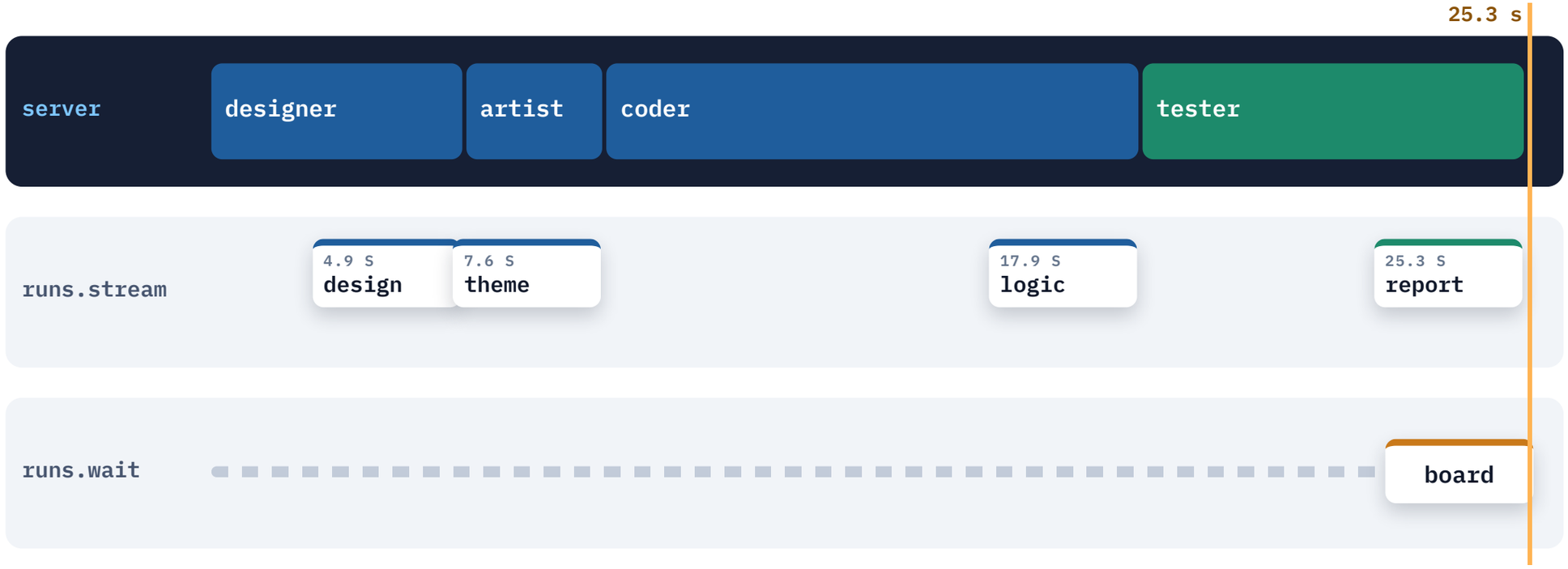
- `assistants.search` **lists the served graphs**
- `threads.create` **opens a thread and returns its id**
- `runs.wait` **starts a run and returns when it finishes**
- `threads.get_state` **reads the saved board ;** `runs.list` **lists the runs on a thread**
- **A second run on the same thread sees what the first one saved :**
  - In demo01, on the Veenhaven Bus stock graph, a follow-up recalls part CE-CF-02 without a new lookup

`demo01c · threads-and-runs`

# Streaming to Clients

- **A build takes about 25 seconds, and a client can show progress during that time**
- **Server-sent events keep one HTTP response open and deliver each event as it happens**
- **A stream mode selects the events :**
  - `updates` : what each node wrote when it finished
  - `messages` or `messages-tuple` : model text, piece by piece, with the node that wrote it
  - `values` : the full state after each step

# Streaming a Build




After 25.3 seconds the Tester reports, and runs.wait returns. Both clients took the same time, and one could show progress.

SpiralTrain developer courses

# Stream Modes and Subgraphs

- `langgraph_node` names the node that produced each message chunk
- Tool results arrive as message events too
- Clients keep AI chunks from the `model` node for text shown to a user
- `stream_subgraphs=True` adds events from nested graphs
- A list of stream modes requests tokens and node updates together
- `threads.get_state` reads the saved state after the stream ends

SpiralTrain developer courses

# Double-Texting and Concurrency

- **Double-texting : a second request arrives while a run is busy on that thread**
- `multitask_strategy` **chooses what happens :**
  - `reject` returns HTTP 409 and the first run continues
  - `enqueue` lets the first run finish, then starts the second ; it is the default
  - `interrupt` halts the first run and keeps its progress
  - `rollback` halts the first run and reverts its progress
- **Provider quotas and worker capacity both limit how many runs can work at once**

demo02b · double-texting

SpiralTrain developer courses

# Persistence Backends

- Production Agent Server keeps thread checkpoints in a database
- PostgreSQL is the default production checkpoint backend
- Local `langgraph dev` keeps development files under `.langgraph_api`
- How long threads are kept follows product and compliance requirements
- A Store keeps memory across threads, such as the studio style guide

# Checkpoint Storage and TTLs

- Shared checkpoint storage lets several worker processes serve one thread
- A TTL, a time to live, deletes expired threads or prunes older checkpoints
- Backups and access controls apply to thread data as to any customer store
- Development storage is kept across a local restart and is not a production database

demo02c · persistence-across-restarts

# DeltaChannel

- `DeltaChannel` is beta in LangGraph 1.2.17
- A list that grows by a reducer, such as the bug list, is expensive to save whole at every step
- Checkpoints store each write, with a full snapshot at intervals
- Replay applies earlier writes to rebuild the list
- Reducers must be deterministic and give the same result however writes are batched

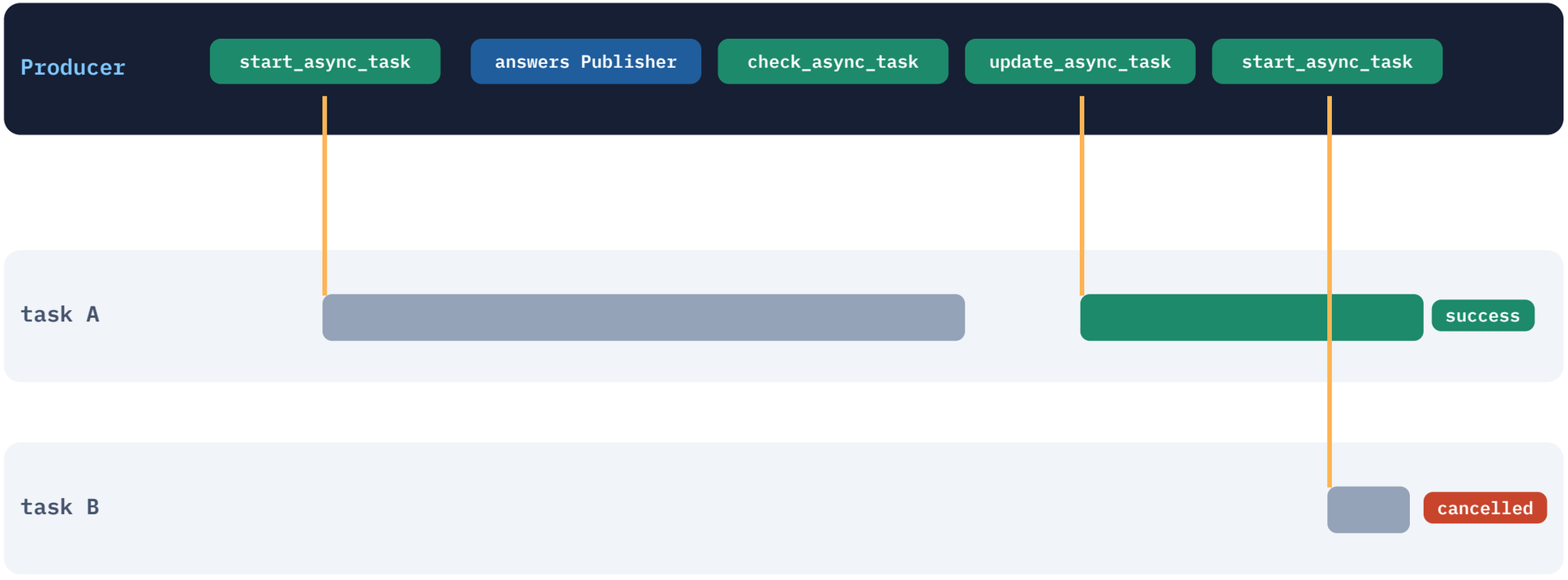
SpiralTrain developer courses

# Async Subagents

- An async subagent is a graph that works in the background while its parent keeps going
- The Producer can start a long bot test, answer the Publisher and collect the score later
- `AsyncSubAgent` adds these tools to the parent :
  - `start_async_task`, `check_async_task`, `update_async_task`
  - `cancel_async_task`, `list_async_tasks`
- A target graph on the same server omits `url` ; a remote one passes `url` and `headers`

demo03 · async-subagents

# A Background Bot Test




demo03 on Veenhaven Bus ended like this : one task finished with success, one was stopped with cancel_async_task, and list_async_tasks showed both.

SpiralTrain developer courses

# Async Task Lifecycle

- `start_async_task` launches a background run and returns a task id
- `check_async_task` reads current status and results
- `update_async_task` interrupts the current run and starts a new run on its thread
- `cancel_async_task` stops a held or running task
- `list_async_tasks` reports tracked tasks, optionally filtered by status
- The task id is the worker's thread id ; `runs.join(task_id, run_id)` waits on it
- Updates keep the task id and replace the run id

# Co-deployed Graphs

- Supervisor and worker graphs share one `langgraph.json`
- The supervisor is a Deep Agent with an `AsyncSubAgent` entry
- The worker is a compiled graph with a `messages` key
- The parent addresses the worker by `graph_id`
- In demo03 the Veenhaven Bus supervisor delegates stock research to the `researcher` graph

SpiralTrain developer courses

# Firstmate Orchestration

- `firstmate` coordinates coding agents through processes and files
- One first mate dispatches each task to a crewmate in its own git worktree
- The first mate watches, steers and cancels those tasks
- State sits on disk, and a watcher wakes the first mate on events
- Separate worktrees keep each agent’s changes apart

SpiralTrain developer courses

# A2A on Agent Server

- Every assistant exposes `/a2a/{assistant_id}`, where `assistant_id` is a UUID
- The agent card is at `/.well-known/agent-card.json?assistant_id=...`
- The binding is A2A v1.0 JSON-RPC, with send, stream, get, cancel and list tasks
- `contextId` identifies the Agent Server thread ; state needs a `messages` key
- `assistants.search` resolves graph names to assistant UUIDs
- Task history returns tool results to the caller as a `data` part
- MCP, A2A, Agent Skills and AGENTS.md all belong to one neutral body, the Agentic AI Foundation

demo04 · a2a-on-agent-server

# Deployment Options

- **LangSmith Deployment lists four kinds of hosting :**
  - Cloud : LangChain hosts the control plane, the data plane and the databases
  - Hybrid : LangChain’s control plane, Agent Servers in your infrastructure
  - Self-hosted with control plane : both planes in your infrastructure
  - Standalone server : Agent Server containers without a control plane
- **The control plane creates and updates servers ; the data plane runs them**
- **All four run the same Agent Server from the same `langgraph.json`**
- **Operations ownership, region and compliance decide the choice**

SpiralTrain developer courses

# Managed Deep Agents

- A hosted runtime in LangSmith runs the open-source Deep Agents harness
- Public beta at launch ; launch hosting was US only
- Version 0.10 of 29 October 2026 adds browser use and async subagents
- An agent is a folder :
  - `agent.py` : model and configuration, required
  - `instructions.md` , `skills/` , `tools/` , `middleware/`
  - Optional `sandbox/` , `memory.py` , `channels/` , `schedules/` , `evals/`
- CLI `mda` from the `managed-deepagents` package : `init` , `dev` , `evals` , `deploy`
- LangChain pricing, 30 October 2026 : Plus is $39 per seat per month with 1 free small serverless deployment

SpiralTrain developer courses

# Scaling Strategies

- API processes keep no state of their own and share threads through PostgreSQL
- Long runs belong in workers, and clients poll or subscribe
- Local concurrency uses `--n-jobs-per-worker`
- Production concurrency uses `N_JOBS_PER_WORKER`
- Provider quotas limit model requests across all workers
- More workers add run capacity up to database and provider limits

SpiralTrain
developer courses

# Evaluating a Multi-Agent System

- **An eval runs the agent on fixed cases and scores the result, so two versions can be compared**
- **Four levels, from the outside in :**
  - Final response : is the answer right and complete
  - Single step : did the model pick the right tool and arguments
  - Trajectory : which tools were called, in which order
  - Multi-turn : did the conversation reach its goal
- **A score per crew member shows which one failed**
- **Correct specialist results do not prove the final answer : the last step can drop a fact**

# The Eval Loop

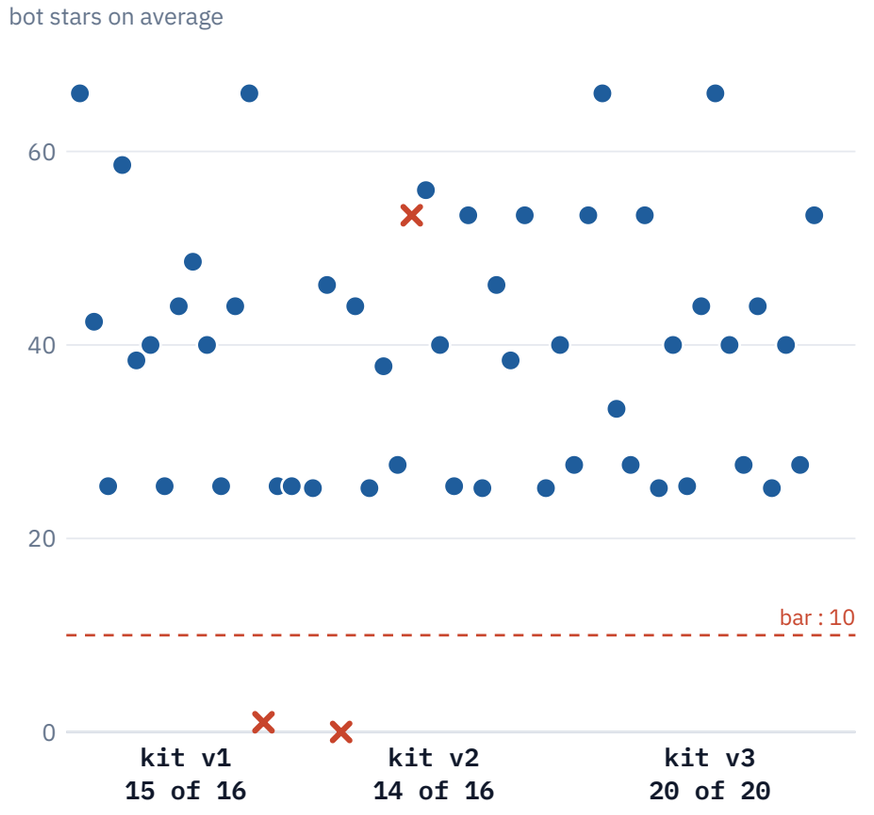
Designer | Artist | Coder

RULE TESTS · QUICKJS

21 of 21

BOT · 5 GAMES

66.0 stars




Kit v3 spells out the order of a step in rules.md : 20 of 20. The same model, with better rules.

SpiralTrain developer courses

# What the Comet Arcade Evals Found

| kit | what changed | runs | pass |
|---|---|---|---|
| v1 | first version | 16 | 15 |
| v2 | stricter `star-grows` test ; the Coder sees the raw report | 16 | 14 |
| v3 | `rules.md` spells out the order of a step | 20 | 20 |

- A pass means every rule test passed and the bot ate at least 10 stars on average over 5 games
- Every run is counted, the failed ones too
- Each failure was traced to a cause, and each cause led to a change in the kit
- All three builds that failed stopped at the cap of 3 attempts ; none shipped

SpiralTrain developer courses

# Limits of the Evidence

- **Tests prove the rules, and only opening the file proves it plays :**
  - Without the kit, 8 of 8 whole files passed the tests and 7 of 8 ran in a browser
  - The eighth called `init(0xC0MET)`, a pun that is no number, and showed a blank page
- **A model’s explanation of a bug is a guess :**
  - One Tester blamed `freeCell` and the Coder chased it for 3 attempts until the cap stopped the run
- **20 of 20 still allows a true pass rate of 86 %**
- **The same seed does not reproduce a run : seed 13 needed the fix loop once and passed first try on a repeat**
- **These numbers say nothing about a twist off the menu, which has no tests**

SpiralTrain developer courses

# Final Answer Evaluation

- A final-answer eval checks facts in the reply against a trusted source
- Code scorers compare the required facts with tool results
- Text checks miss false claims and reject valid rewordings
- demo05 takes the expected stock and depot of a Veenhaven Bus part from the database

demo05 · evaluating-agents

SpiralTrain developer courses

# Trajectory Match

- A trajectory is the list of tool calls and results in one run
- `agentevals` compares a live trajectory with a reference trajectory
- `strict` needs the same messages in the same roles, and `unordered` needs the same tool calls
- Arguments must match exactly by default
- A strict reference needs the tool result and the answer as well as the call :
  - In demo05b on the Veenhaven Bus stock graph, strict fails a short reference and passes the full one

demo05b · trajectory-match

# Trajectory Match Modes

- **Four match modes on 0.0.9 :**
  - `strict` : same message roles and matching calls per message
  - `unordered` : same tool-call coverage across messages
  - `subset` : outputs introduce no calls beyond the reference
  - `superset` : outputs include at least the reference calls
- **Strict matching ignores prose and call order within one message**
- `tool_args_match_mode` **is** `exact` **by default, or** `ignore` **,** `subset` **,** `superset`

SpiralTrain developer courses

# Trajectory Judge

- A judge is a second model that grades a run against a rubric, a written list of criteria
- `create_trajectory_llm_as_judge` gives it the task, tool calls, results and answer
- It returns a score and a short justification in `comment`
- Check a judge against human labels before it can block a release
- A judge has model biases, so read the justifications on a sample

demo05c · trajectory-judge

SpiralTrain developer courses

# Specialist and System Evaluation

- **Each specialist gets its own rubric. On the Veenhaven Bus graphs in demo05d :**
  - Parts must keep the count and depot
  - Manuals must keep the action and condition
- **A negative control proves the judge can say no : a count changed from 6 to 9 failed 3 of 3 trials**
- **A system eval checks every required fact in the combined answer**
- **A failure then points at tool coverage, specialist quality or a fact lost in synthesis**

demo05d · each-agent

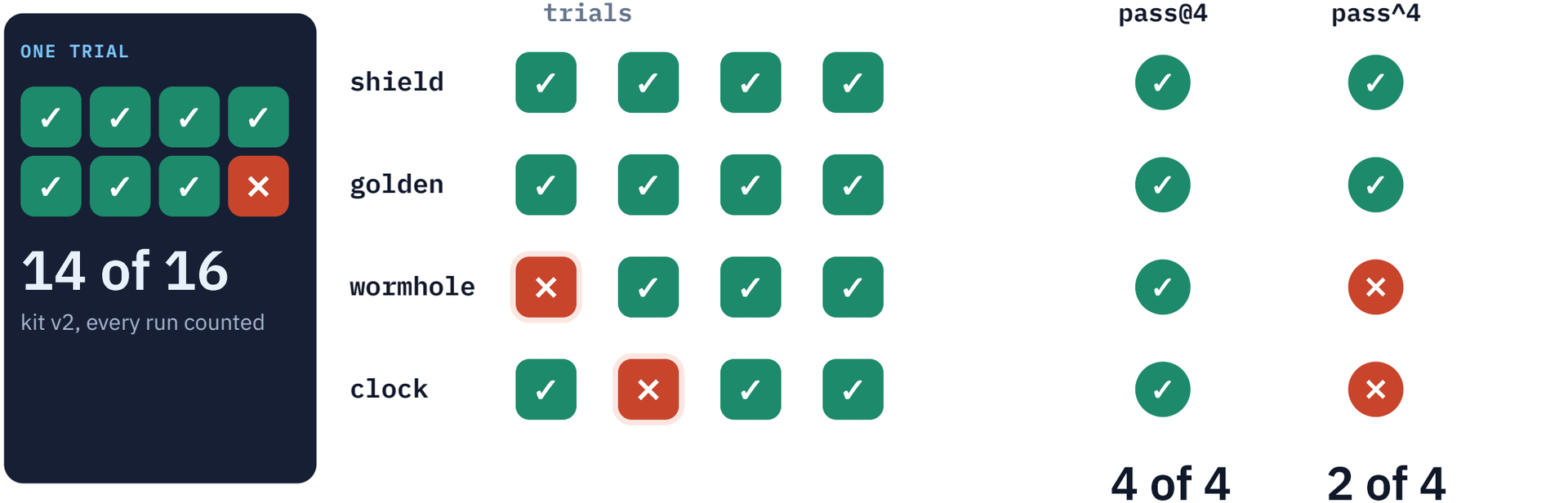
SpiralTrain developer courses

# pass@k and pass^k

- An agent varies from run to run, so one run is one sample
- Run each case k times :
  - pass@k : at least one of the k trials passes
  - pass^k : all k trials pass
- pass@k suits a tool that a person can retry ; pass^k suits an agent that must work every time
- demo05e, Veenhaven Bus, 4 cases of 3 trials : each case passed all trials or none, so pass@3 = pass^3 = 3/4
- Anthropic’s example : 75 % per trial gives pass^3 of about 42 % when trials are independent

demo05e · pass-at-k

# Rolling pass@k and pass^k




**kit v3** 20 of 20 · pass@5 4 of 4 · pass^5 4 of 4

Kit v3 ran 5 trials on each case and passed 20 of 20, so pass^5 is 4 of 4. That still allows a true rate of 86 %.

SpiralTrain developer courses

# A LangSmith Experiment

- LangSmith keeps a dataset of cases and the score of every run against it
- The trainer experiment writes a Veenhaven Bus dataset to EU LangSmith
- Code evaluators score a stock fact and `part_stock` coverage
- Account or permission failures are reported, and the local agent still runs
- Online evaluators reuse the same rubrics on sampled production traces

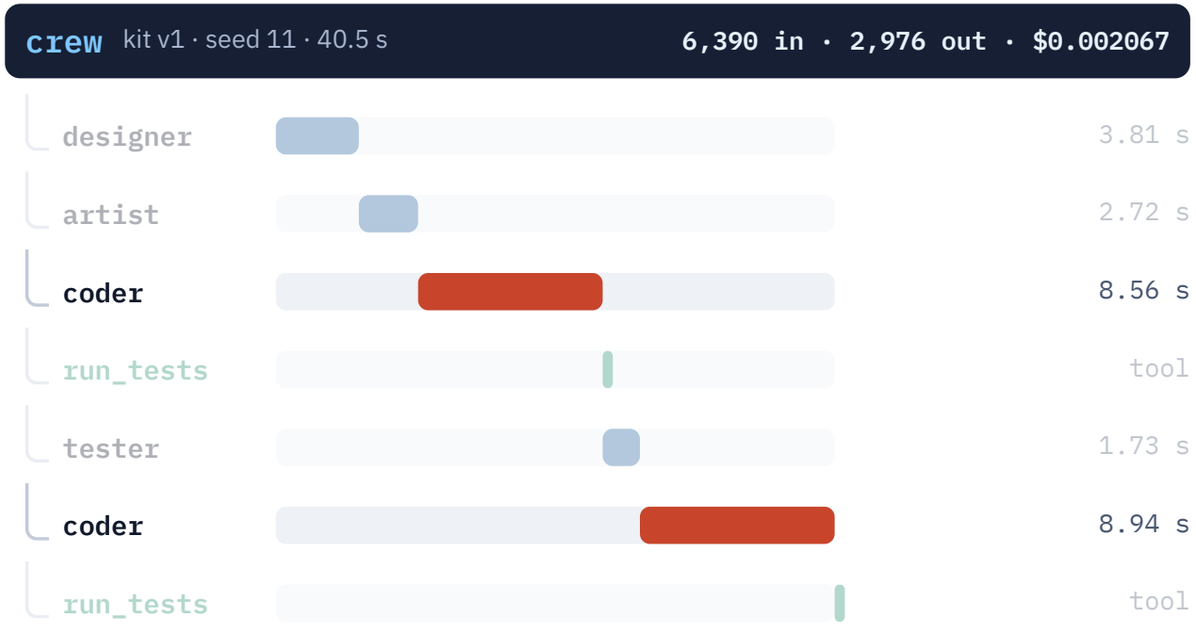
demo06 · a-langsmith-experiment

SpiralTrain developer courses

# Monitoring

- A trace is the record of one run : every model call, tool call, result and failure
- LangSmith Trajectories show one timeline across a parent and its subagents
- Online evals score a sample of production runs automatically
- Dashboards track cost per run, latency, error rate and judge scores
- Alerts fire when a quality score falls ; a sample can miss a failure

# Reading a Trace




RUN_TESTS · SENT

The new game file

RUN_TESTS · RETURNED

```
21 of 21 rule tests pass ; bot 66.0 stars
```

no tokens

Reading the costs : two Coder calls took 78 % of the money. A cheaper model or a smaller prompt would save the most here.

SpiralTrain developer courses

# Cost Optimisation

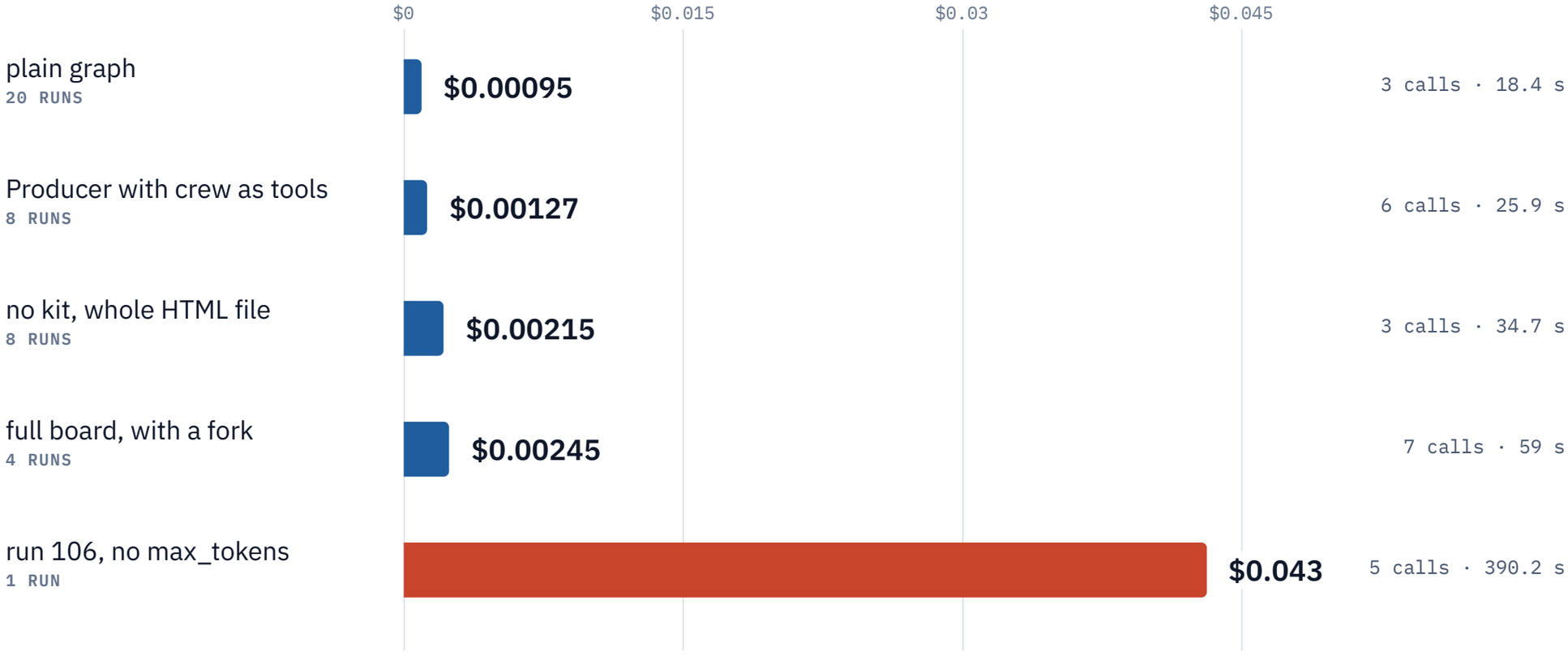
- **Each crew member can have its own model and its own token budget**
- **A compact handoff reduces input tokens :**
  - In demo07 a Veenhaven Bus synthesis read 138 tokens from a transcript and 50 from a summary
- **Provider prompt caching reuses a long stable start of a prompt :**
  - Instructions and tools go first, changing content last
  - Eligibility and retention depend on routing and the provider
  - OpenAI lists Luna cache writes at 25 % above input, so a prefix saves money from its first cached read
- **A cached prompt still sends a request and gets a new reply**
- **Usage reports separate input, cached input and output tokens**

`demo07 · cost`

`demo07b · prompt-caching`

p. 446

# What a Game Costs




Then one run without max_tokens. The axis has to stretch : run 106 alone cost about as much as 45 normal builds.

SpiralTrain developer courses

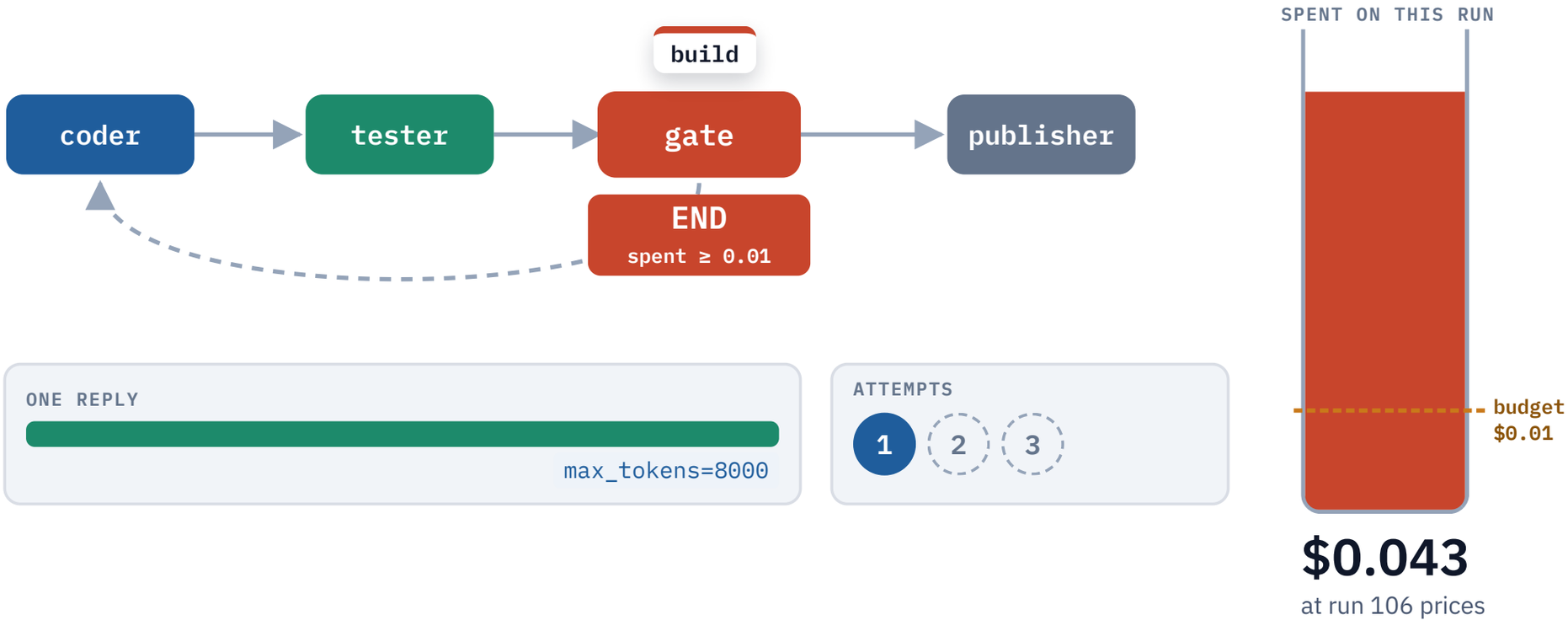
# Budget Stops

- **A guard is a limit in code that the model cannot override :**
  - `max_tokens` ends one reply. On Luna a cap of 200 ended a reply at 200 tokens
  - `ModelCallLimitMiddleware` ends a `create_agent` loop after a number of model calls
  - A gate ends the graph on attempts or on money spent :

```
def gate(board):
    if board["passed"]:
        return "publisher"
    if board["attempts"] >= 3 or board["spent"] >= 0.01:
        return END
    return "coder"
```

- **A reply cut by `max_tokens` is incomplete, so treat `finish_reason` `length` as a failure**

# The Budget Stop




The budget stop : the gate returns END and the run stops, on the first attempt, before more money goes out.

SpiralTrain developer courses

# Error Return Channel

- Prompt injection : text in a tool result that the model obeys as if it were an instruction
- Tool errors go back to the agent so it can recover, and error text can contain an injected instruction
- Fixed error codes keep untrusted text out of the reply
- demo08 changes one defence at a time on a Veenhaven Bus report tool :
  - A fixed code with the export tool kept
  - Then no export tool, and a prompt that treats codes as data
- Application code validates the stock lookup outside the agent

demo08 · injection-through-an-error

SpiralTrain developer courses

# Summary : Production Deployment

- Agent Server serves assistants, threads and runs from `langgraph.json`
- The same graph runs on a laptop and in production, and the board is kept across a restart
- Clients stream events and read saved thread state
- Async subagents work in the background ; A2A exposes assistants to other systems
- Evals score final answers, trajectories and each crew member
- pass@k and pass^k report repeated trials ; a trace shows why one failed
- Token counts guide handoffs and caching, and guards stop a run in code
- Tool error text can inject an instruction, so error messages are untrusted input

exercise10 · Evaluating a System of Agents

exercise11 · An Async Subagent on a Server

# Real-World Applications

From LLM to Agent
Introduction to LangGraph
Graph Fundamentals
State Management
Building Agents
Multi-Agent Patterns
Agent Communication
Advanced Workflows
Deep Agents
Production Deployment
**Real-World Applications**
Research Automation
Research Evidence
Customer Support
Handoffs in Support
Code Review
Pipeline Checkers
Data Analysis
Content Pipelines
Coding Agent Fleets
A Personal Crew
Firstmate Patterns
Harness or Graph
Navier-Stokes Swarm
Fermat Formalisation
The Research Team Revisited
Recursive Language Models Revisited
Multi-Agent Cost
The Capstone Desk
Summary : Real-World Applications

SpiralTrain developer courses

# Research Automation

- **A research agent turns an open question into a report with sources :**
  - It plans sub-questions, retrieves evidence and combines the findings
  - Notes and drafts go into files as the context grows
- **The Deep Agents deep-research tutorial is the current reference ;** `open_deep_research` **showed the shape and is now archived**
- **Independent sub-questions run as parallel workers, each with its own sources, as in module 05**
- **A sub-question that needs another’s answer runs after it : in demo01 the Veenhaven Bus manual search uses the fault code found in the tickets**

# Research Evidence

- **Every claim in a report needs a source that can be traced :**
  - Record the file or URL beside each finding
  - Report conflicting evidence and missing coverage
- **Files keep research notes and drafts outside the conversation, as in module 08**
- **A provenance check confirms that a source was retrieved ; a person checks that it supports the claim**

# Customer Support

- **A service agent routes a request, looks up policy and resolves or escalates :**
  - Specialists have different tools and authority
  - Irreversible actions pause for approval
- **Handoffs pass the request between specialists, as in module 05**
- **Refunds, orders and account changes wait at approval gates from module 07**

demo02 · customer-support

# Handoffs in Support

- **A handoff changes the active specialist and passes on the task :**
  - The next agent needs the request and the earlier tool results
  - Parallel tool calls can leave an unanswered call after the transfer
- **Shared state keeps the request and useful tool results across transfers, as in modules 05 and 06**
- **Each specialist returns only its new records, and the handoff appends them once**
- **Evaluation checks complete answers and valid tool histories**

SpiralTrain developer courses

# Code Review

- **Reviewers look at the same change for different risks :**
  - In demo03 one reviews correctness and one database safety, in parallel
  - Each reports every finding with a severity
- **A severity filter in code decides what reaches the judge, and the judge merges duplicates**
- **A checker in code confirms the seeded defects are among the accepted findings, and rejects an empty review**
- **Parallel review with a judge is the exploration pattern from module 05**

demo03 · code-review

# Pipeline Checkers

- `no-mistakes` **places a pipeline of checks before changes reach** `origin` **:**
  - Stages run review, test, documentation, lint, push, pull request and CI
  - Mechanical fixes apply automatically, and changes to intent wait for approval
- **Defined pass conditions make it a maker-checker workflow from module 05**
- **Human approval controls consequential changes, as in module 07**

SpiralTrain developer courses

# Data Analysis

- **An analytics agent turns a question into a query and an explanation :**
  - The SQL query, returned rows and explanation stay visible together
  - The database connection allows reading only
- `PRAGMA query_only` **makes SQLite reject writes on the connection**
- **A code check compares every number in the explanation with the returned rows, as in the checker pattern from module 05**
- **The Deep Agents data-analysis tutorial gives an agent CSV files and a sandbox from module 08**

SpiralTrain developer courses

# Content Pipelines

- **A content pipeline turns source material into reviewed output :**
  - In demo04b it retrieves an approved Veenhaven Bus procedure
  - It drafts a bulletin and checks the required details before editorial review
- **Retrieval, drafting and validation form a fixed workflow from module 05**
- **An editor approves publication ; in a served pipeline the gate is an interrupt from module 07**

demo04b · content-pipeline

SpiralTrain developer courses

# Coding Agent Fleets

- **A fleet gives each coding task its own worktree, a separate checkout of the repository :**
  - One orchestrator dispatches work and receives events
  - Each worker has a brief, a session and a checkout it can write to
- **Isolated subagents return findings to the orchestrator, as in modules 05 and 06**
- **Merge decisions use the approval flow from module 07**
- **Tests written by a coding agent need the same review as its code :**
  - An agent can hard-code values that pass only the given tests
  - A prompt can forbid it, and a reviewer or a held-out test catches it
- **firstmate supports macOS and Linux**

demo05 · firstmate

# A Personal Crew

- **Monday’s first example : Yair deployed about 30 single-job bots behind one first mate :**
  - Bots cover taxes, vehicle status, portfolio, inbox and app administration
  - One first mate routes each request to the right bot
- **"they all share one computer with the same browser logins, and nothing gets sent without my ok"**
- **The course now has a name for each part :**
  - Single-job workers with short tool lists behind one orchestrator : module 05
  - A seller and a buyer for one owner : conflict resolution, module 06
  - "my ok" : an approval gate, module 07
  - Shared logins : the case for per-agent credentials, module 08

# Firstmate Patterns

| firstmate component | course pattern |
| --- | --- |
| first mate and crewmates | orchestrator with isolated subagents, module 05 |
| task worktree and session | context isolation and backend, module 08 |
| watcher and event files | event driven execution, module 06 |
| durable briefs and state | checkpointer and `Store`, modules 03 and 06 |
| merge escalation | interrupt and approval, module 07 |

- **Files and processes do the coordinating and saving that modules 05 to 08 do with graphs**

SpiralTrain developer courses

# Harness or Graph

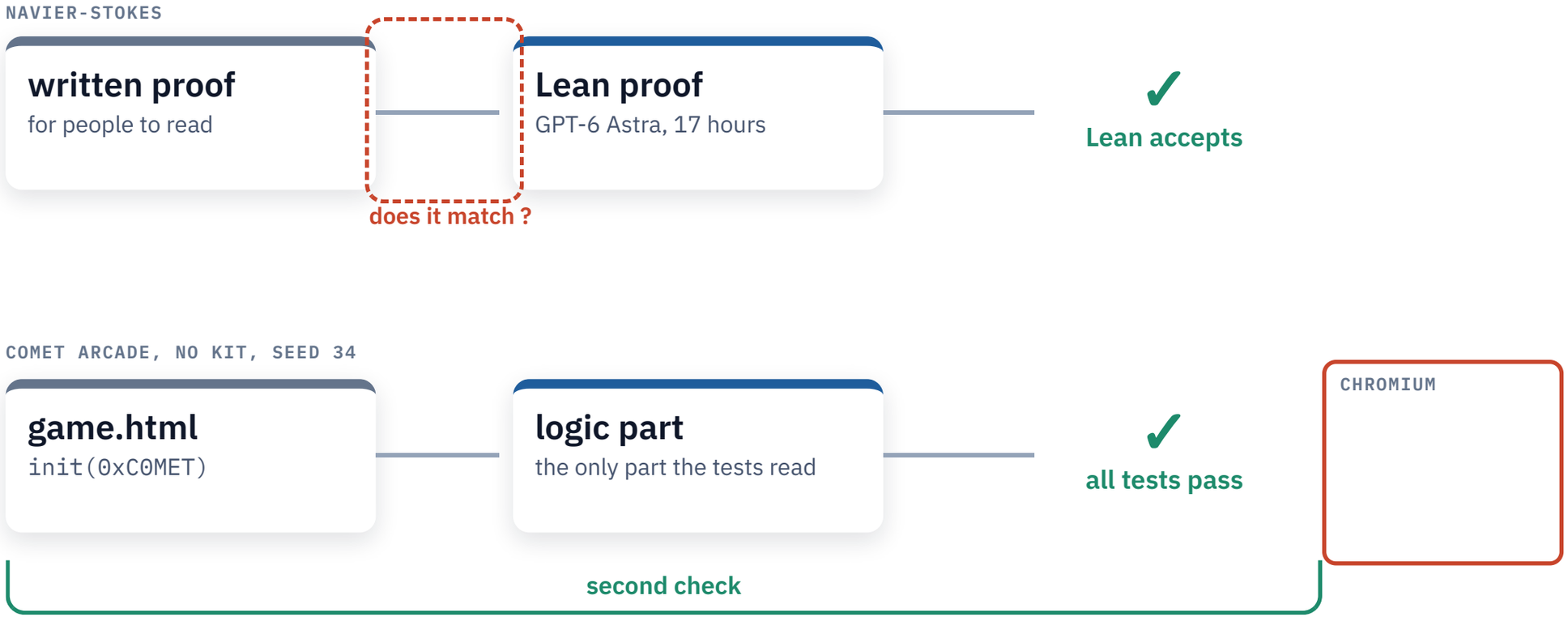
- **A coding harness fits work tied to a repository and long-lived sessions :**
  - Git worktrees, shell tools, watchers and files are its runtime
  - The model works on existing files and developer tools directly
- **A graph fits application-owned state and explicit transitions :**
  - Typed schemas, reducers and checkpointers define the boundaries
  - Conditional edges, loops and interrupts control the flow
- **Both can coordinate isolated workers, as in modules 05 and 08**
- **The choice depends on whether the application or the coding session controls state and the next step**

SpiralTrain developer courses

# Navier-Stokes Swarm

- **Monday’s second example : OpenAI reports a coordinated search on an internal model :**
  - Groups explored different problem variants and proof or disproof directions
  - The winning group had on the order of 10,000 concurrent agents
  - Codex later shared findings between groups, and the model was updated during the run
- **OpenAI reports a Lean formalisation : GPT-6 Astra wrote the Lean proof in 17 more hours**
- **Clay says the process is "deliberately unhurried"**
- **A preprint says the Lean proof does not match the written proof ; neither shows the result wrong**
- **Module 05 has the same two roles : workers explore in parallel, and a checker judges the result**

# What a Proof Checker Checks




A checker tests what it was given, so check the translation too : a second reader of the proof, a browser for the game.

# Fermat Formalisation

- **Anthropic reports formalising an existing proof of Fermat's Last Theorem :**
  - Dozens of parallel agents proved 30,300 theorems over 11 days
  - The final development used 29,500 proved theorems across 13 million lines of Lean
  - Lean's type checker rejected invalid proof steps mechanically
- **Early attempts failed when agents lost track of the project's state**
- **A shared graph of theorem dependencies coordinates the parallel work, as in modules 05 and 06**

# The Research Team Revisited

- **Monday’s third example : Anthropic’s lead agent with parallel research subagents**
- **The result stands, from June 2025 :**
  - 90.2 % better than a single Opus 4 agent on an internal research evaluation
  - About 15 times the tokens of a chat
- **The research agent of demo01 has the same shape : plan, workers, synthesis**
- **Anthropic’s advice of 22 October 2026 goes the other way for some work : fold a subagent back into the main loop when it adds little**
- **OpenAI’s mouse-brain run is a rare test at fixed resources : over 700 references with many agents, 11 with one**

SpiralTrain developer courses

# Recursive Language Models Revisited

- Monday’s fourth example : a model that explores a long input in code
- The input stays in a Python REPL, and the model calls fresh copies of itself on parts
- Module 08 ran one level of the idea in demo08b, on 31 Veenhaven Bus manuals :
  - The root model writes a dispatch script, and readers get the chunks
  - The complete chunks stay out of the root’s requests
- Those manuals fit in one call, so the split cost more : 15,635 input tokens against 5,121
- The method pays off on inputs too long for one call, which the paper reports at about 100 times the window

SpiralTrain developer courses

# Multi-Agent Cost

- **Monday’s last example : the bill for a swarm**
- **Multi-agent systems spend far more tokens than a chat :**
  - Anthropic measured about 15 times the tokens of a chat for its research system
  - Token usage alone explained 80 % of the variance in its BrowseComp results
- **OpenAI reports about 130 billion output tokens for Navier-Stokes ; Anthropic about six billion for Fermat**
- **swyx cited an estimate of about $40 million at public prices ; OpenAI reports no bill**
- **Alex Zhang stated "95% of the swarm is entirely useless", and hedged at once that OpenAI’s setup may differ ; no source measures the waste**
- **A single-agent baseline, as in modules 05 and 09, tests whether more workers justify their cost**

SpiralTrain developer courses

# Opening Examples and Patterns




Six of the nine patterns ran on Comet Arcade in this course. The same parts build the examples from Monday.

SpiralTrain developer courses

# One Shape at Every Scale




About ten orders of magnitude. The larger runs share one shape : someone makes, and something other than the maker checks.

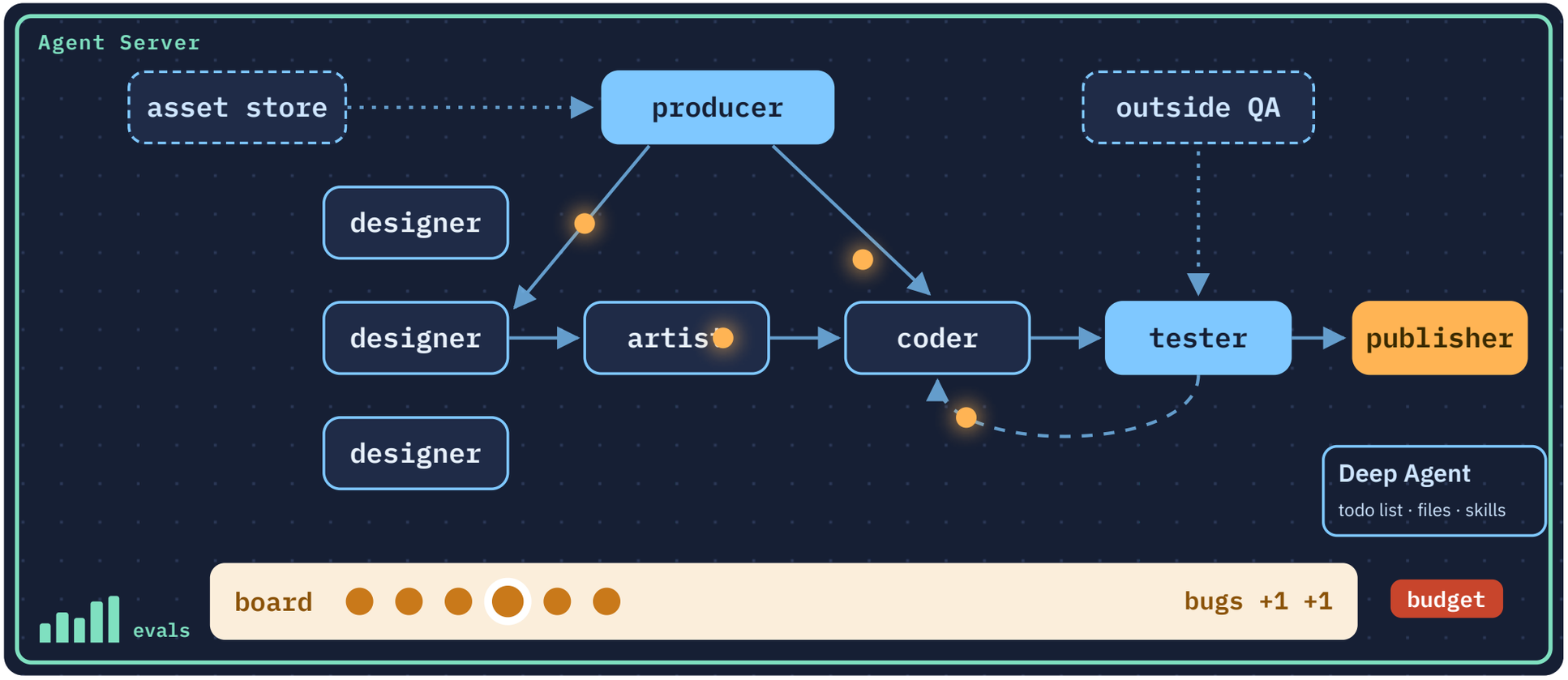
SpiralTrain developer courses

# The Capstone Desk

- **The Aurelia service desk compares a graph with a Deep Agent on one dataset :**
  - Stage 3 builds routed diagnosis, a judge, A2A communication and approval in a subagent
  - Stage 4 builds a Deep Agent with skills for procedures and specialists as subagents
- **Modules 05 to 08 supply the graph patterns, skills and subagents of both stages**
- **Stage 4 compares both designs on fact screens, tokens and counted code lines**
- **Stage 5 scores the served Deep Agent on eight cases, an injection among them**

SpiralTrain developer courses

# What You Can Build Now



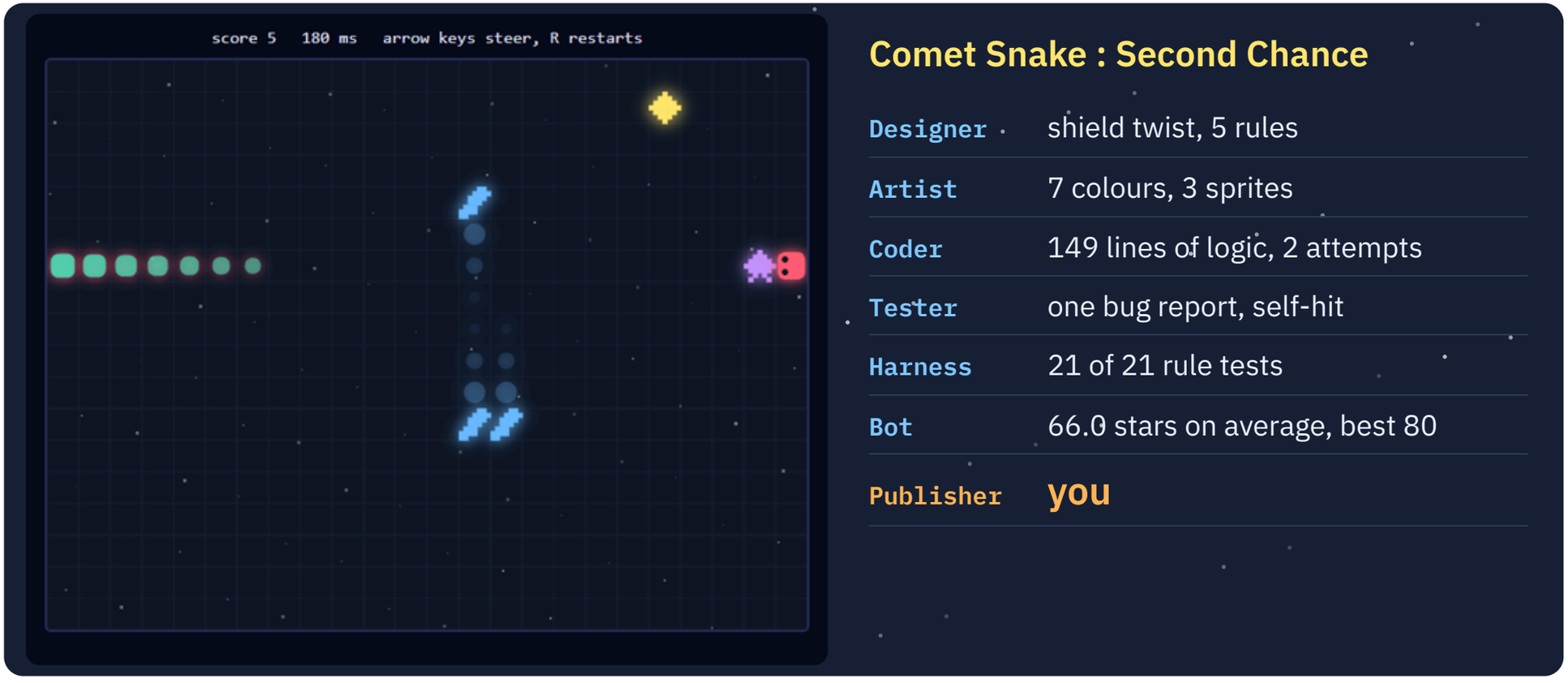

A crew that plans, builds, checks itself, asks before it ships and reports what it cost. It works on research, support, code and data as well.

SpiralTrain developer courses

# Summary : Real-World Applications

- Research agents split questions, retrieve evidence and cite findings
- Support agents hand off the task and pause before consequential actions
- Reviewers report every finding with a severity ; code filters them, a judge merges them and a checker confirms the result
- Analysis and content pipelines keep queries and sources visible, and check figures in code
- Coding fleets use isolated workers, durable state and events
- Lean and code checks reject invalid model output mechanically. A checker tests what it was given, so check the translation too
- Multi-agent runs cost many times the tokens of a chat, so compare quality and cost with a simpler baseline

# The Finished Comet Snake




5 model calls, 40.5 seconds, $0.002. Your turn : click the game and take the arrow keys, then build your own.